

최종보고서 별책 — 전체 구현 코드

서울대학교 핀테크 전문가 과정 「통계, 데이터 사이언스」 팀 프로젝트 — Lending Club 신용평가 / Sharpe Ratio 최적화

- **범위:** 저장소 `src/` 전체 — 32개 파일, 8,360줄
- **본책:** `outputs/reports/final_report.md` (부록 C가 이 별책을 가리킨다)
- 이 문서는 `src/utils/export_code_appendix.py` 가 생성한다. 손으로 고치지 말고 코드를 고친 뒤 다시 생성한다.

목차

구분	파일	줄
공통 유틸	src/utils/config.py	164
공통 유틸	src/utils/logger.py	13
전처리	src/preprocessing/loader.py	287
전처리	src/preprocessing/preprocessor.py	465
전처리	src/preprocessing/export_split_manifest.py	310
전처리	src/preprocessing/export_shared_dataset.py	291
전처리	src/preprocessing/label_pre_post_by_rule.py	68
전처리	src/preprocessing/fetch_treasury_gs1m.py	192
전처리	src/preprocessing/fetch_macro_cpi.py	113
전처리	src/preprocessing/fetch_macro_initial_claims.py	139
전처리	src/preprocessing/fetch_macro_unemployment_rate.py	90
전처리	src/preprocessing/fetch_macro_yield_spread.py	119
분석 — 본 파이프라인	src/analysis/model.py	354
분석 — 본 파이프라인	src/analysis/realized_return.py	514
분석 — 본 파이프라인	src/analysis/realized_return_cashflow.py	347
분석 — 본 파이프라인	src/analysis/oof_diagnostics.py	348
분석 — 본 파이프라인	src/analysis/sharpe_optimizer.py	759
분석 — 본 파이프라인	src/analysis/final_evaluation.py	285
분석 — 본 파이프라인	src/analysis/second_test_evaluation.py	315
분석 — 재현·검증	src/analysis/preprocessing_validation.py	403
분석 — 재현·검증	src/analysis/t2_contribution_reassessment.py	158
분석 — 재현·검증	src/analysis/term_split_comparison.py	215
분석 — 재현·검증	src/analysis/missing_scheme_comparison.py	234
분석 — 재현·검증	src/analysis/macro_indicator_screening.py	169
분석 — 재현·검증	src/analysis/auc_sample_filter_comparison.py	95
분석 — 재현·검증	src/analysis/model_comparison.py	779
분석 — 재현·검증	src/analysis/realized_return_spec_check.py	276
분석 — 재현·검증	src/analysis/realized_return_sensitivity.py	212
분석 — 재현·검증	src/analysis/hpr_realized_return.py	101
분석 — 재현·검증	src/analysis/excluded_audit.py	86

구분	파일	줄
분석 — 재현·검증	src/analysis/oracle_benchmarks.py	156
시각화	src/viz/plots.py	303
합계	32개 파일	8,360

공통 유틸

경로·설정·로깅 — 모든 스크립트가 여기를 거친다.

src/utils/config.py

config/config.yaml 로더

"""`config/config.yaml` 로더 - 경로·분할비율·seed의 단일 출처.

왜 필요한가:

분할비율(6:2:2)·seed·경로를 스크립트마다 직접 써 넣으면 값이 두 곳에 존재하게 되고,
한쪽만 고쳤을 때 조용히 어긋난다. 파이프라인 코드는 이 모듈을 통해서만 읽는다.

쓰는 법:

```
from utils.config import load_config, repo_root      # 스크립트 직접 실행 시
cfg = load_config()
cfg.split.train, cfg.split.validation, cfg.split.test  # 0.6 / 0.2 / 0.2
cfg.random_seed.default                               # 42
cfg.paths.data_processed / "파일명.csv"              # 절대경로로 해석됨

rf = cfg.require("risk_free_rate.value")  # 미확정(null) 항목은 명시적으로 실패시킨다
```

적용 범위:

****모형 학습·threshold 결정 등 본 파이프라인 코드**가 대상이다.**
src/analysis/ 의 탐색·검증용 비교 스크립트는 6:2:2가 아닌 자체 분할(2분할)을 쓰는데,
이는 처리 방식 간 AUC 상대 비교가 목적이라 의도된 차이다 - 그 스크립트들까지
이 설정에 맞출 필요는 없다 (config/config.yaml 주석 참고).

확인:

```
python src/utils/config.py      # 해석된 설정을 출력
```

"""

```
from dataclasses import dataclass
from pathlib import Path
from typing import Any
```

```
import yaml
```

```
REPO_ROOT = Path(__file__).resolve().parents[2]
CONFIG_PATH = REPO_ROOT / "config" / "config.yaml"
```

```
def repo_root() -> Path:
```

```
    """저장소 루트 절대경로. 스크립트가 parents[2]를 각자 계산하지 않도록 여기서 제공한다."""
```

```
    return REPO_ROOT
```

```
@dataclass(frozen=True)
```

```
class Paths:
```

```
    data_raw: Path
    data_processed: Path
    outputs_reports: Path
    outputs_figures: Path
```

```
@dataclass(frozen=True)
```

```
class Split:
```

```
    train: float
    validation: float
    test: float
```

```
@dataclass(frozen=True)
```

```

class RandomSeed:
    default: int
    stability_check_range: tuple[int, int]

@dataclass(frozen=True)
class RiskFreeRate:
    """무위험수익률 설정.

    2026-07-29 회의에서 참고논문 방식이 확정됐다(decision_log.md #18) - 거절한 대출의 자본은
    '발행시점(issue_d)에 대출 만기와 만기를 맞춘 미국채'에 투자했다고 가정한다.
    **고정 상수가 아니므로 value는 계속 None이다.** 미확정이라서 비어 있는 것이 아니라,
    채울 단일 값이 존재하지 않는 방식이다. issue_d x term 매칭 로직으로 처리한다.
    """

    value: float | None
    method: str | None = None
    series: dict[int, str] | None = None

@dataclass(frozen=True)
class Config:
    paths: Paths
    split: Split
    random_seed: RandomSeed
    risk_free_rate: RiskFreeRate
    _raw: dict[str, Any]

    def require(self, dotted_key: str) -> Any:
        """미확정(null) 설정을 조용히 기본값으로 대체하지 않고 명시적으로 실패시킨다.

        예: cfg.require("risk_free_rate.value") - 팀이 Rf를 확정하기 전에는
        RuntimeError를 내서, 임의의 5% 같은 값이 결과에 섞여 들어가는 것을 막는다.
        """

        node: Any = self._raw
        for part in dotted_key.split("."):
            if not isinstance(node, dict) or part not in node:
                raise KeyError(f"config.yaml에 '{dotted_key}' 항목이 없습니다.")
            node = node[part]
        if node is None:
            raise RuntimeError(
                f"config.yaml의 '{dotted_key}'가 아직 미확정(null)입니다. "
                f"팀에서 확정한 뒤 값을 채워야 합니다 - 임의의 기본값으로 대체하지 않는다."
            )
        return node

    def load_config(config_path: Path | None = None) -> Config:
        """config/config.yaml을 읽어 검증된 Config로 반환한다.

        paths는 저장소 루트 기준 상대경로로 적혀 있으므로 절대경로로 해석해서 돌려준다.
        """

        path = config_path or CONFIG_PATH
        if not path.exists():
            raise FileNotFoundError(f"설정 파일이 없습니다: {path}")

        with path.open(encoding="utf-8") as f:

```

```

raw = yaml.safe_load(f)

p = raw["paths"]
paths = Paths(
    data_raw=REPO_ROOT / p["data_raw"],
    data_processed=REPO_ROOT / p["data_processed"],
    outputs_reports=REPO_ROOT / p["outputs_reports"],
    outputs_figures=REPO_ROOT / p["outputs_figures"],
)

s = raw["split"]
split = Split(train=s["train"], validation=s["validation"], test=s["test"])
total = split.train + split.validation + split.test
if abs(total - 1.0) > 1e-9:
    raise ValueError(f"split 비율의 합이 1이 아닙니다: {total} ({s})")

r = raw["random_seed"]
lo, hi = r["stability_check_range"]
random_seed = RandomSeed(default=r["default"], stability_check_range=(lo, hi))

return Config(
    paths=paths,
    split=split,
    random_seed=random_seed,
    risk_free_rate=RiskFreeRate(
        value=raw["risk_free_rate"]["value"],
        method=raw["risk_free_rate"].get("method"),
        series=raw["risk_free_rate"].get("series"),
    ),
    _raw=raw,
)

if __name__ == "__main__":
    cfg = load_config()
    print(f"저장소 루트: {repo_root()}")
    print("\n[경로] (모두 존재해야 정상)")
    for name, path in vars(cfg.paths).items():
        mark = "0" if path.exists() else "X 없음"
        print(f" {name:18s} {path.relative_to(REPO_ROOT)} [{mark}]")
    print("\n[분할]")
    print(f" train/validation/test = {cfg.split.train}/{cfg.split.validation}/{cfg.split.test}")
    print("\n[seed]")
    print(f" default={cfg.random_seed.default} 안정성검증범위={cfg.random_seed.stability_check_range}")
    print("\n[무위험수익률]")
    print(f" method={cfg.risk_free_rate.method} series={cfg.risk_free_rate.series}")
    if cfg.risk_free_rate.value is None:
        print(" value=None - 방식은 확정됐고(decision_log.md #18) 고정 상수가 아니라서 비어 있다.")
        print(" issue_d x term 매칭으로 처리한다. require()로 읽어 쓰지 않는다.")
    else:
        print(f" value={cfg.risk_free_rate.value}")

```

src/utils/logger.py

공통 로깅 설정 유틸리티.

```
""" 공통 로깅 설정 유틸리티. """
```

```
import logging
```

```
def get_logger(name: str) -> logging.Logger:
```

```
    logger = logging.getLogger(name)
```

```
    if not logger.handlers:
```

```
        handler = logging.StreamHandler()
```

```
        handler.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s"))
```

```
        logger.addHandler(handler)
```

```
        logger.setLevel(logging.INFO)
```

```
    return logger
```

전처리

원본 로딩부터 분할·공유 데이터셋 생성까지.

src/preprocessing/loader.py

원본 대출 CSV 로딩 · 분석 표본 필터 · 피처 컬럼 선택.

""" 원본 대출 CSV 로딩 · 분석 표본 필터 · 피쳐 컬럼 선택.

본 파이프라인(`src/preprocessing/AGENTS.md`)의 입구다. 여기서 결정되는 것은 셋이다.

1. **어떤 행을 쓰는가** - `filter\_analysis\_sample()` → **723,563건** (`decision\_log.md` #16)
2. **어떤 열을 쓰는가** - `select\_feature\_columns()` → 사전(pre-approval) 변수 중 식별자·자유서술 제외
3. **원본을 건드리지 않는가** - 모든 열기는 읽기 전용이다 (`AGENTS.md` 「원본 데이터 취급」)

⚠ `data/raw/`에 쓰기 모드로 접근하는 함수를 이 모듈에 추가하지 않는다.

=====

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
try: # 스크립트 직접 실행과 패키지 임포트 양쪽 지원
```

```
    from utils.config import load_config, repo_root
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
    import sys
```

```
    sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
    from utils.config import load_config, repo_root
```

```
RAW_FILENAME = "lending_club_2020_train.csv"
```

```
SECOND_TEST_FILENAME = "lending_club_2020_test_2nd.csv"
```

```
DICTIONARY_FILENAME = "variable_dictionary_byGJ.xlsx"
```

```
# 표본 필터 (decision_log.md #16, 2026-07-29 회의 확정)
```

```
KEEP_STATUS = ("Fully Paid", "Charged Off")
```

```
CREDIT_POLICY_PREFIX = "Does not meet the credit policy. Status:"
```

```
MATURITY_CUTOFF = "2020-04" # issue_d + term ≤ 2020-04 (만기 + 버퍼 6개월)
```

```
EXPECTED_SAMPLE_SIZE = 723_563
```

```
# =====
```

```
# 변수 사전 오버라이드
```

```
# =====
```

```
# ⚠ `variable_dictionary_byGJ.xlsx`의 `is_pre_approval` 컬럼이 확정 사항과 어긋나 있다.
```

```
# 시트는 아래 8개를 사후(0) 또는 미라벨(NaN)로 두고 있는데, `decision_log.md` #1은 이들을
```

```
# **사전 변수로 재분류**했고 #17 ③은 `grade`·`sub_grade`·`int_rate`를 **피쳐로 투입**하기로
```

```
# 확정했다. 시트를 그대로 믿으면 LC 조건변수가 전부 빠져 Lean 스펙(AUC 0.68대)이 되어버린다.
```

```
#
```

```
# 시트 개정은 `decision_log.md`의 열린 실행 항목이다("변수분류 시트 개정",
```

```
# `preprocessing_crosscheck_kgj.md` 10절). 개정되면 이 표는 비워도 된다 -
```

```
# `verify_dictionary_overrides()`가 불필요해진 항목을 알려준다.
```

```
PRE_APPROVAL_OVERRIDES: dict[str, str] = {
```

```
    "grade": "#1 재분류 (시트 미라벨) · #17 ③ 투입 확정",
```

```
    "sub_grade": "#1 재분류 (시트 미라벨) · #17 ③ 투입 확정",
```

```
    "int_rate": "#1 재분류 (시트 사후) · #17 ③ 투입 확정",
```

```
    "installment": "#1 재분류 (시트 사후)",
```

```
    "funded_amnt": "#1 재분류 (시트 사후)",
```

```
    "funded_amnt_inv": "#1 재분류 (시트 사후)",
```

```
    "issue_d": "#1 재분류 (시트 사후)",
```

```
    "initial_list_status": "#1 재분류 (시트 사후)",
```

```

}

# 사전 변수이지만 피쳐로 쓰지 않는 것. 누수와는 다른 이유다.
# - 식별자: 행을 특정할 뿐 예측력이 아니다. `id`는 시트에서 사전(1)으로 라벨돼 있어 명시 제한한다.
# - 자유서술: 카디널리티가 수만~수십만이라 범주형으로 넣으면 사실상 식별자가 된다.
NON_FEATURE_PRE_APPROVAL: dict[str, str] = {
    "id": "식별자",
    "member_id": "식별자 (원본에 없음)",
    "url": "식별자 (대출 ID를 그대로 포함)",
    "emp_title": "자유서술 (고유값 수십만)",
    "title": "자유서술 (차주 작성)",
    "desc": "자유서술 (원본에 없음)",
}

# 사전 변수이고 식별자·자유서술도 아니지만 **팀 결정으로 피쳐에서 뺀 것**.
# 위 두 목록과 이유가 다르다 - 저 둘은 "원리상 피쳐가 아니다", 이쪽은 "실측해 보니 모형을
# 깎는다"다. 그래서 목록을 합치지 않고 따로 둔다(사유가 섞이면 나중에 판단 근거를 잃는다).
#
# `zip_code`: 911개 범주의 고카디널리티 변수로, XGBoost가 Train의 칸별 부도율을 암기한다.
# 같은 재분할에 `main` / `main - zip_code`를 짝지어 돌린 검증에서 제외 쪽이 **24 seed 전부
# 우세**했고(재투자 가정 2중 × 랭킹 기준 3중 = 6조합 모두 24/24), 국제·`q_score` 기준
# 짝지은 차이는  $\Delta \text{Sharpe} \approx +0.0122 \times (sd\ 0.0020)$ 였다.
# 근거: `outputs/reports/model_comparison_kgj.md`, `outputs/model_comparison_stability.csv`.
EXCLUDED_BY_DECISION: dict[str, str] = {
    "zip_code": "고카디널리티(911범주) 과적합 - 짝지은 검증 24/24 제외 우세,  $\Delta \text{Sharpe} +0.0122$ ",
}

TARGET_COLUMN = "loan_status"

def raw_path() -> Path:
    """원본 대출 CSV 경로. 1.2GB이며 git에 없다 - 팀 공유 채널에서 받는다."""
    return load_config().paths.data_raw / RAW_FILENAME

def second_test_path() -> Path:
    """**2nd Test CSV** 경로 (1,170,198행 × 141열, 850MB, git 미추적).

    이름의 `2nd`는 **매니페스트 6:2:2의 Test(20%, 144,713건)와 다른 집합**임을 뜻한다 -
    둘을 같은 "Test"로 부르면 어느 쪽 Sharpe인지 구분되지 않는다.

    `RAW_FILENAME` (train)과 **id가 한 건도 겹치지 않는 별도 파일**이며 컬럼 구성은 동일하다.
    `filter_analysis_sample(..., verify=False)`를 통과하면 481,833건이 남는다
    (부도율 16.20%) - `EXPECTED_SAMPLE_SIZE`는 train 기준이므로 `verify=True`로 부르지 않는다.
    """
    return load_config().paths.data_raw / SECOND_TEST_FILENAME

def dictionary_path() -> Path:
    return load_config().paths.data_processed / DICT_FILENAME

# -----
# 변수 사전
# -----

def load_variable_dictionary(xlsx_path: Path | None = None) -> pd.DataFrame:
    """변수 사전 + 사전/사후 라벨을 로드한다 (`decision_log.md` #9 - 단일 원본)."""

```

```

path = xlsx_path or dictionary_path()
if not path.exists():
    raise FileNotFoundError(f"변수 사전이 없습니다: {path}")
d = pd.read_excel(path)
d["LoanStatNew"] = d["LoanStatNew"].astype(str).str.strip()
return d

def pre_approval_columns(dictionary: pd.DataFrame | None = None) -> list[str]:
    """사전(pre-approval) 변수 목록. 시트 라벨에 `PRE_APPROVAL_OVERRIDES`를 얻은 결과다."""
    d = dictionary if dictionary is not None else load_variable_dictionary()
    labelled = set(d.loc[d["is_pre_approval"] == 1, "LoanStatNew"])
    return sorted(labelled | set(PRE_APPROVAL_OVERRIDES))

def verify_dictionary_overrides(dictionary: pd.DataFrame | None = None) -> list[str]:
    """오버라이드 중 시트가 이미 반영한 항목을 돌려준다.

    비어 있지 않으면 시트가 아직 개정 전이라는 뜻이고, 전부 반영되면 결과가 오버라이드
    전체와 같아진다 - 그때 `PRE_APPROVAL_OVERRIDES`를 비울 수 있다.
    """
    d = dictionary if dictionary is not None else load_variable_dictionary()
    already = set(d.loc[d["is_pre_approval"] == 1, "LoanStatNew"])
    return sorted(set(PRE_APPROVAL_OVERRIDES) & already)

def select_feature_columns(
    raw_columns: list[str],
    dictionary: pd.DataFrame | None = None,
    apply_decisions: bool = True,
) -> tuple[list[str], dict[str, str]]:
    """피쳐로 쓸 컬럼과, 제외된 사전 변수의 사유를 함께 돌려준다.

    사후 변수는 애초에 `pre_approval_columns()`에 없으므로 여기서 다시 거르지 않는다
    (누수 방지 목록은 전부 사후로 라벨돼 있음을 `test_leakage_columns_are_post()`가 확인한다).

    `apply_decisions=True` (기본)면 `EXCLUDED_BY_DECISION`도 뺀다. **비교 실험에서만 False로
    둔다** - 제외 전후를 나란히 돌려야 하는 `model_comparison.py`가 그 경우다. 본 파이프라인은
    기본값을 쓴다.
    """
    pre = pre_approval_columns(dictionary)
    present = [c for c in pre if c in raw_columns]
    dropped = dict(NON_FEATURE_PRE_APPROVAL)
    if apply_decisions:
        dropped |= EXCLUDED_BY_DECISION

    excluded: dict[str, str] = {}
    for col in pre:
        if col not in raw_columns:
            excluded[col] = "원본에 없음"
        elif col in dropped:
            excluded[col] = dropped[col]

    features = [c for c in present if c not in dropped]
    return features, excluded

```

```

# -----
# 원본 로딩
# -----

def load_raw_loans(
    usecols: list[str] | None = None,
    nrows: int | None = None,
    csv_path: Path | None = None,
) -> pd.DataFrame:
    """원본 대출 CSV를 **읽기 전용**으로 로드한다.

    전수 1,755,295행 × 141열이다. `usecols`로 필요한 열만 받는 것을 권한다 -
    전체를 올리면 수 GB를 쓴다.
    """
    path = csv_path or raw_path()
    if not path.exists():
        raise FileNotFoundError(
            f"원본 대출 데이터가 없습니다: {path}\n"
            "1.2GB라 git에 없습니다 - 팀 공유 채널(Google Drive 등)에서 받아 두세요."
        )
    return pd.read_csv(path, usecols=usecols, nrows=nrows, low_memory=False)

def normalize_loan_status(loan_status: pd.Series) -> pd.Series:
    """`Does not meet the credit policy. Status:` 접두사를 떼어 상태를 통일한다.

    접두사가 붙은 건도 각각 `Fully Paid` / `Charged Off`와 동일하게 취급한다
    (`src/preprocessing/AGENTS.md` 행 필터링 규칙).
    """
    return (
        loan_status.astype(str)
        .str.replace(CREDIT_POLICY_PREFIX, "", regex=False)
        .str.strip()
    )

def parse_term_months(term: pd.Series) -> pd.Series:
    """`36 months` → `36`."""
    return term.astype(str).str.extract(r"(\d+)")[0].astype(float)

def filter_analysis_sample(df: pd.DataFrame, verify: bool = True) -> pd.DataFrame:
    """분석 표본 필터 - **723,563건** (`decision_log.md` #16).

    두 조건의 교집합이다.

    1. **상태**: `Fully Paid` / `Charged Off`만 남긴다 (접두사 정규화 후).
       `Current`·`Late`·`In Grace Period`·`Default`·`Issued`는 완결건이 아니라 제외된다.
    2. **만기 + 버퍼 6개월**: `issue_d + term ≤ 2020-04`.
       스냅샷(2020-10) 시점에 만기가 도래하지 않은 완결건은 조기부도가 과대표집돼
       부도율이 8.5%p 높다.

    `Default` 268건(만기 내 113건)은 상각 미확정 단계라 **제외한다** -
    `decision_log.md` #24 ① 정정(2026-07-31). K=50·2nd Test 등 공표된 모든 수치가
    이 기준(723,563건)으로 산출됐다. 합치면 표본이 723,676건이 된다.
    """
    status = normalize_loan_status(df[TARGET_COLUMN])
    term_months = parse_term_months(df["term"])

```

```

issue = pd.to_datetime(df["issue_d"], format="%b-%Y", errors="coerce")

# 연·월 산술로 절대 월 서수를 만든다. Period dtype을 거치면 NaT가 int 최솟값으로
# 공개져 **만기 조건을 통과해버리므로** 쓰지 않는다.
maturity_ord = (issue.dt.year * 12 + issue.dt.month) + term_months
cutoff = pd.Timestamp(MATURITY_CUTOFF)
cutoff_ord = cutoff.year * 12 + cutoff.month

keep = status.isin(KEEP_STATUS) & (maturity_ord <= cutoff_ord) & issue.notna()
out = df.loc[keep].copy()
out[TARGET_COLUMN] = status.loc[keep]

if verify and len(out) != EXPECTED_SAMPLE_SIZE:
    raise ValueError(
        f"표본 건수가 확정값과 다릅니다: {len(out):,} != {EXPECTED_SAMPLE_SIZE:,}. "
        "원본 파일이나 필터 정의가 바뀐 것입니다 - decision_log.md #16을 확인하세요. "
        "표본/부분 로딩으로 의도적으로 다르게 돌릴 때만 verify=False를 씁니다."
    )
return out

def make_target(loan_status: pd.Series) -> pd.Series:
    """이진 타겟 - `Charged Off` = 1(부도), `Fully Paid` = 0."""
    normalized = normalize_loan_status(loan_status)
    unknown = set(normalized.unique()) - set(KEEP_STATUS)
    if unknown:
        raise ValueError(f"표본에 완결 상태가 아닌 값이 있습니다: {sorted(unknown)}")
    return (normalized == "Charged Off").astype("int8")

def validate_schema(df: pd.DataFrame, required_columns: list[str]) -> None:
    """필수 컬럼 존재 여부를 검증하고, 없으면 무엇이 빠졌는지 알려준다."""
    missing = [c for c in required_columns if c not in df.columns]
    if missing:
        raise ValueError(f"필수 컬럼 {len(missing)}개가 없습니다: {missing}")

if __name__ == "__main__":
    print(f"저장소 루트: {repo_root()}")

    d = load_variable_dictionary()
    already = verify_dictionary_overrides(d)
    print(f"\n변수 사전 {len(d)}행")
    print(f"시트 라벨 사전(1): {(d['is_pre_approval'] == 1).sum()} "
          f"사후(0): {(d['is_pre_approval'] == 0).sum()} "
          f"미라벨(NaN): {d['is_pre_approval'].isna().sum()}")
    if already:
        print(f"✅ 시트가 이미 반영한 오버라이드: {already}")
        print("→ 시트가 개정된 항목입니다. PRE_APPROVAL_OVERRIDES에서 빼도 됩니다.")
    else:
        print(f"⚠️ 시트 미개정 - 오버라이드 {len(PRE_APPROVAL_OVERRIDES)}건을 코드에서 적용 중")
        for col, why in PRE_APPROVAL_OVERRIDES.items():
            print(f"    {col:22s} {why}")

    head = load_raw_loans(nrows=5)
    features, excluded = select_feature_columns(list(head.columns), d)
    print(f"\n피처 {len(features)}개")

```

```
print(f" 제외된 사전 변수 {len(excluded)}개:")
for col, why in sorted(excluded.items()):
    print(f"         {col:22s} {why}")
```

src/preprocessing/preprocessor.py

피쳐 테이블 구성 — dtype 정리와 6:2:2 분할.

"""피쳐 테이블 구성 - dtype 정리와 6:2:2 분할.

`src/preprocessing/AGENTS.md`의 확정 규칙이 그대로 적용되는 대상이다. 핵심은 ****하지 않는 일****이 많다는 것이다.

- ****결측은 전부 NaN 그대로 둔다**** (#13 ③·#17). 대체도 결측더미도 없다 - XGBoost가 native 처리한다.
(구 뼈대에 있던 `add\_missing\_dummies()`는 이 확정으로 폐기됐다.)
- ****표준화·로그변환·구간화·캡핑을 하지 않는다**** (#17). 트리는 값의 순서만 쓴다.
`dti` 999 같은 특수값도 원본 그대로 둔다 - 캡핑하면 진짜 100인 행과 뭉개진다.
- ****파생(비율) 변수를 만들지 않는다**** - 목록이 아직 미확정이다(`src/preprocessing/AGENTS.md` 말미).

따라서 여기서 하는 일은 ****문자열로 저장된 수치를 수치로 되돌리는 것**** 뿐이다.

`' 7.97%'` → `'7.97'`, `' 36 months'` → `'36'`, `'Sep-2005'` → 월 서수. 값의 순서를 바꾸지 않는 단조 변환이라 위 금지 사항과 충돌하지 않는다.

범주형은 pandas `category` dtype으로 두고 XGBoost `enable\_categorical=True`로 넘긴다 - 원-핫으로 펼쳐지 않으므로 여기서도 정보를 더하거나 빼지 않는다.

"""

```
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

try:
    from preprocessing.loader import (
        TARGET_COLUMN,
        filter_analysis_sample,
        load_raw_loans,
        make_target,
        parse_term_months,
        select_feature_columns,
    )
    from utils.config import load_config
except ModuleNotFoundError: # pragma: no cover
    import sys

    sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
    from preprocessing.loader import (
        TARGET_COLUMN,
        filter_analysis_sample,
        load_raw_loans,
        make_target,
        parse_term_months,
        select_feature_columns,
    )
    from utils.config import load_config
```

값 끝에 `%`가 붙어 문자열로 저장된 수치형

PERCENT_COLUMNS = ("int_rate", "revol_util")

`'Sep-2005'` 형식으로 저장된 월 단위 날짜

```

MONTH_COLUMNS = (
    "issue_d",
    "earliest_cr_line",
    "sec_app_earliest_cr_line",
    "last_credit_pull_d",
)

# 순서가 있는 범주 - 문자열 정렬이 의미 순서와 다르므로 명시한다.
EMP_LENGTH_ORDER = (
    "< 1 year", "1 year", "2 years", "3 years", "4 years", "5 years",
    "6 years", "7 years", "8 years", "9 years", "10+ years",
)

# 실현수익률(`R`·`rf`·`XR`)을 나중에 결합할 때 쓰는 열. **X에 넣지 않는다.**
META_COLUMNS = ("id", "issue_d", "term", "funded_amnt", "int_rate")

def parse_percent(series: pd.Series) -> pd.Series:
    """`7.97%` → `7.97`. 파싱 불가능한 NaN으로 둔다(대체하지 않는다)."""
    if pd.api.types.is_numeric_dtype(series):
        return series
    return pd.to_numeric(
        series.astype(str).str.replace("%", "", regex=False).str.strip(),
        errors="coerce",
    )

def parse_month(series: pd.Series) -> pd.Series:
    """`Sep-2005` → 월 서수(정수). 순서를 보존하는 단조 변환이다.

    연·월로 쪼개지 않는다 - 쪼개면 트리가 같은 해 12월과 다음 해 1월을 먼 값으로 보게 된다.
    """
    if pd.api.types.is_numeric_dtype(series):
        return series
    parsed = pd.to_datetime(series, format="%b-%Y", errors="coerce")
    # `.dt.to_period("M")`은 Period dtype이라 Int64로 바로 캐스팅되지 않는다(pandas 2.3).
    # 연·월 산술이 같은 순서를 주면서 NaT를 <NA>로 보존한다.
    return (parsed.dt.year * 12 + parsed.dt.month).astype("Int64")

def coerce_dtypes(df: pd.DataFrame) -> pd.DataFrame:
    """문자열로 저장된 수치·날짜를 되돌리고, 나머지 object를 category로 만든다."""
    out = df.copy()

    for col in PERCENT_COLUMNS:
        if col in out.columns:
            out[col] = parse_percent(out[col])

    if "term" in out.columns:
        out["term"] = parse_term_months(out["term"])

    for col in MONTH_COLUMNS:
        if col in out.columns:
            out[col] = parse_month(out[col])

    if "emp_length" in out.columns:
        out["emp_length"] = pd.Categorical(

```

```

        out["emp_length"].astype("object"), categories=EMP_LENGTH_ORDER, ordered=True
    )

    for col in out.columns:
        if out[col].dtype == "object":
            out[col] = out[col].astype("category")

    return out

def build_feature_table(
    nrows: int | None = None,
    verify_sample: bool = True,
    apply_decisions: bool = True,
    csv_path: Path | None = None,
) -> tuple[pd.DataFrame, pd.Series, pd.DataFrame]:
    """원본 → `(X, y, meta)`.


`meta`는 threshold_Sharpe 단계에서 실현수익률을 결합할 때 쓰는 열이다.



**`X`에는 넣지 않는다** - 붙여두면 실수로 학습에 들어간다



(`src/preprocessing/AGENTS.md` '누수 방지').



`nrows`를 주면 원본 앞부분만 읽으므로 표본 건수 검증은 건너뛴다 - 디버깅 전용이다.



`apply_decisions=False`면 loader.EXCLUDED_BY_DECISION` (현재 `zip_code`)을 **빼지 않고** 돌려준다 - 제의 전후를 나란히 비교해야 하는 model_comparison.py` 전용이다.



본 파이프라인은 기본값을 쓴다.



`csv_path`는 train 원본 대신 다른 CSV를 같은 파이프라인으로 읽을 때만 준다



(loader.second_test_path()의 2nd Test). 표본 건수 검증은 train 기준이므로 그때는 verify_sample=False`를 함께 준다.


    """
    head = load_raw_loans(nrows=5, csv_path=csv_path)
    features, _ = select_feature_columns(list(head.columns), apply_decisions=apply_decisions)

    needed = sorted(set(features) | set(META_COLUMNS) | {TARGET_COLUMN, "issue_d", "term"})
    available = [c for c in needed if c in head.columns]

    raw = load_raw_loans(usecols=available, nrows=nrows, csv_path=csv_path)
    sample = filter_analysis_sample(raw, verify=verify_sample and nrows is None)

    y = make_target(sample[TARGET_COLUMN])

    meta = sample[[c for c in META_COLUMNS if c in sample.columns]].copy()
    meta["term"] = parse_term_months(meta["term"])
    meta["int_rate"] = parse_percent(meta["int_rate"])
    meta["issue_d"] = parse_month(meta["issue_d"])

    X = coerce_dtypes(sample[features])
    return X, y, meta

# -----
# 분할 체계 - 셋이 공존한다
# -----
# : `{scheme: (train, validation, test)}`. **비율을 코드에 넣어 놓지 않는다.**
# :

```

```

#: - `"6_2_2"` - 1차 데이터 안에서 Test 20%를 떼는 원래 체계(#13 @·#18). `config.yaml`이
#: 단일 출처이며 여기 적힌 값은 그 대조용이다.
#: - `"7_3"` - **1차 데이터 전량을 Train/Validation으로 쓰고 Test는 별도 파일**
#: (`lending_club_2020_test_2nd.csv`, 481,833건)로 삼는 체계(#30, 2026-07-31 확정).
#: 6:2:2 대비 Train +16.7%(434,137 → 506,494) · Validation +50.0%(144,713 → 217,069).
#: - `"8_2"` - 7:3과 같이 Test를 별도 파일로 두되 Train을 더 준다(#32, 2026-07-31).
#: Train 578,850 / Validation 144,713. 7:3 대비 Train +14.3% · Validation -33.3%다 -
#: Validation이 6:2:2와 같은 크기로 돌아가므로 `τ*` 표본오차와 승자의 저주가 함께 커진다.
#: 그 대가를 알고 고른 것이며 근거는 이슈 #32에 있다.
#:
#: ⚠ 체계를 늘릴 때는 `SPLIT_FUNCTIONS`(`export_split_manifest.py`)도 함께 채운다 -
#: 여기에만 추가하면 매니페스트를 만들 수 없다.
SPLIT_SCHEMES: dict[str, tuple[float, float, float]] = {
    "6_2_2": (0.6, 0.2, 0.2),
    "7_3": (0.7, 0.3, 0.0),
    "8_2": (0.8, 0.2, 0.0),
}

DEFAULT_SCHEME = "6_2_2"

def scheme_ratios(scheme: str) -> tuple[float, float, float]:
    """분할 체계의 `(train, validation, test)` 비율. 알 수 없는 이름이면 예외다.

    `"6_2_2"`는 `config.yaml`을 읽어 돌려준다 - 그쪽이 단일 출처이므로 값이 어긋나면
    **조용히 넘어가지 않고** 예외로 알린다.
    """
    if scheme not in SPLIT_SCHEMES:
        raise ValueError(f"알 수 없는 분할 체계: {scheme!r} (가능: {sorted(SPLIT_SCHEMES)})")
    if scheme != DEFAULT_SCHEME:
        return SPLIT_SCHEMES[scheme]

    cfg = load_config().split
    ratios = (cfg.train, cfg.validation, cfg.test)
    if not np.isclose(ratios, SPLIT_SCHEMES[scheme]).all():
        raise ValueError(
            f"`config.yaml`의 split {ratios}이 '{scheme}' 정의 {SPLIT_SCHEMES[scheme]}와 다르다 - "
            "둘 중 어느 쪽이 맞는지 정하고 맞춰라."
        )
    return ratios

def split_train_validation(
    X: pd.DataFrame,
    y: pd.Series,
    meta: pd.DataFrame | None = None,
    seed: int | None = None,
    scheme: str = "7_3",
) -> dict[str, tuple]:
    """**Test 칸이 없는** 체계의 2분할 - `"7_3"`(#30)과 `"8_2"`(#32)가 여기로 온다.

    1차 데이터(723,563건) 전량을 Train/Validation으로 가른다. Test는 이 표본 안에서
    떼지 않고 **별도 파일**(`loader.second_test_path()`)을 쓴다.

    비율은 `scheme_ratios(scheme)`에서 온다 - **비율을 이 함수에 박지 않는다.** 체계를
    늘릴 때 여기를 고치지 않아도 되게 하려는 것이다(#32에서 8:2를 넣을 때 `"7_3"`이
    하드코딩돼 있어 실제로 걸렸던 부분이다).
    """

```

부도율 16.2%가 한쪽에 치우치지 않도록 `stratify`를 건다.

"""

```
seed = load_config().random_seed.default if seed is None else seed
_, val_share, test_share = scheme_ratios(scheme)
if test_share:
    raise ValueError(
        f'"{scheme}"은 Test 칸이 있는 체계다({test_share:.0%}) - "
        "split_train_validation()이 아니라 split_6_2_2()를 쓴다."
    )
```

```
keys = pd.Series(X.index, index=X.index)
train_keys, val_keys = train_test_split(
    keys, test_size=val_share, random_state=seed, stratify=y
)
```

```
def take(part: pd.Series) -> tuple:
    idx = part.index
    if meta is None:
        return X.loc[idx], y.loc[idx]
    return X.loc[idx], y.loc[idx], meta.loc[idx]

return {"train": take(train_keys), "validation": take(val_keys)}
```

```
def split_6_2_2(
    X: pd.DataFrame,
    y: pd.Series,
    meta: pd.DataFrame | None = None,
    seed: int | None = None,
) -> dict[str, tuple]:
    """랜덤 6:2:2 분할 (`config/config.yaml`이 단일 출처, `decision_log.md` #13 ④).
```

`train\_test\_split`을 두 번 적용한다 - 전체를 80/20으로 나눠 Test를 떼고, 남은 80%를 75/25로 갈라 Train 60% / Validation 20%를 만든다.

부도율이 16.2%로 한쪽에 치우칠 수 있어 두 단계 모두 `stratify`를 건다.

시간순 분할은 #13 ④에서 실측 기각됐다(Train의 T2 관측률이 0%가 됨).

"""

```
cfg = load_config()
seed = cfg.random_seed.default if seed is None else seed

val_share_of_rest = cfg.split.validation / (cfg.split.train + cfg.split.validation)

keys = pd.Series(X.index, index=X.index)
rest_keys, test_keys = train_test_split(
    keys, test_size=cfg.split.test, random_state=seed, stratify=y
)
train_keys, val_keys = train_test_split(
    rest_keys,
    test_size=val_share_of_rest,
    random_state=seed,
    stratify=y.loc[rest_keys.index],
)
```

```
def take(part: pd.Series) -> tuple:
    idx = part.index
```

```

    if meta is None:
        return X.loc[idx], y.loc[idx]
    return X.loc[idx], y.loc[idx], meta.loc[idx]

return {
    "train": take(train_keys),
    "validation": take(val_keys),
    "test": take(test_keys),
}

def load_split_manifest(
    seed: int | None = None, scheme: str = DEFAULT_SCHEME
) -> pd.Series:
    """`id` → `split` 매니페스트를 읽는다. 인덱스는 `id` (문자열) 다.

    매니페스트는 `src/preprocessing/export_split_manifest.py`가 만든다. 없으면 예외다 -
    **임의로 seed 분할로 대체하지 않는다.** 조용히 다른 분할로 넘어가면 팀원 간 Test가
    어긋나는데 아무도 모르게 된다.

    `scheme`은 `SPLIT_SCHEMES`의 키( `"6_2_2"`, `"7_3"`, `"8_2"` )이며 **파일이 다르다** -
    서로 덮어쓰지 않는다.
    """

    from preprocessing.export_split_manifest import manifest_path

    cfg = load_config()
    seed = cfg.random_seed.default if seed is None else seed
    path = manifest_path(seed, scheme=scheme)
    if not path.exists():
        raise FileNotFoundError(
            f"분할 매니페스트가 없다: {path}\n"
            f"먼저 `python src/preprocessing/export_split_manifest.py --scheme {scheme}`로 "
            "만들거나, 팀 저장소에서 받아라."
        )
    m = pd.read_csv(path, dtype={"id": str})
    return m.set_index("id")["split"]

def split_from_manifest(
    X: pd.DataFrame,
    y: pd.Series,
    meta: pd.DataFrame,
    seed: int | None = None,
    unlock_test: bool = False,
    scheme: str = DEFAULT_SCHEME,
) -> dict[str, tuple]:
    """**매니페스트 기준** 6:2:2 분할 - 팀원 전원이 동일한 분할을 쓰는 경로.

    `split_6_2_2()` (seed 기반)와 달리 `id`로 매칭하므로 **원본 CSV의 행 순서·pandas·sklearn
    버전과 무관**하다. 왜 그게 필요한지는 `export_split_manifest.py` docstring 참고 -
    요약하면 `train_test_split`은 위치를 셔플하므로 행 순서가 다른 분할이 달라지는데
    723,563건 검증은 건수만 보므로 **조용히 어긋난다.**

    ⚠️ **Test는 기본적으로 반환하지 않는다.** `unlock_test=True`를 명시해야 나온다.
    실제로 `parts["test"]`를 잡는 일을 막기 위한 것이다 - Test는 모형·threshold가 전부
    확정된 뒤 **단 1회** 적용한다( `src/analysis/AGENTS.md` ).

```

반환 행 순서를 `id`로 고정한다 - 분할만 맞춰선 부족하다

분할 집합이 같아도 **행 순서가 다르면 K-fold 배정이 달라진다.**

`StratifiedKFold(shuffle=True, random\_state=seed)`도 `train\_test\_split`처럼 **위차**를 셔플하기 때문이다. 그러면 팀원마다 00F PD가 미세하게 달라지고, 그 PD로 만든 분위 경계·칸별 통계표· τ 가 조금씩 어긋난다.

그래서 각 split을 **`id` 오름차순으로 정렬해** 돌려준다. 이러면 00F fold까지 `(id 집합, seed)`만으로 결정되어 **원본 파일의 행 순서와 완전히 무관**해진다.

⚠ 이 정렬 때문에 `split\_6\_2\_2()`로 낸 기존 수치와 **소수 넷째 자리 수준의 차야**가 생긴다 (분할 자체는 동일하고 fold 구성만 바뀐다). 파이프라인을 이 함수로 넘긴 뒤에는 산출물을 한 번 다시 만들어야 한다.

"""

```
manifest = load_split_manifest(seed, scheme=scheme)
ids = meta["id"].astype(str)
```

```
assigned = ids.map(manifest)
missing = int(assigned.isna().sum())
```

```
if missing:
    raise RuntimeError(
        f"매니페스트에 없는 id가 {missing:,}건이다 - 표본 필터가 매니페스트 생성 시점과 "
        "다르다. 필터를 고치기 전에 왜 달라졌는지부터 확인하라(#16)."
```

```
def take(name: str) -> tuple:
    idx = ids[assigned == name].sort_values(kind="mergesort").index
    return X.loc[idx], y.loc[idx], meta.loc[idx]
```

```
parts = {"train": take("train"), "validation": take("validation")}
n_test = int((assigned == "test").sum())
```

```
if not n_test:
    # 7:3 - Test 칸이 아예 없다. 최종 평가는 별도 파일(2nd Test)로 한다(#30).
    if unlock_test:
        raise ValueError(
            f"'{scheme}' 매니페스트에는 Test 칸이 없다 - unlock_test는 쓸 수 없다. "
            "최종 평가는 `loader.second_test_path()`(2nd Test)로 한다(#30)."
```

```
        )
    return parts

if unlock_test:
    print(f"⚠ Test set을 열었다 ({n_test:,}건). 모형·threshold 확정 후 **1회만** 쓴다 - "
          "여기서 무언가를 고르거나 조정하면 규칙 위반이다.")
    parts["test"] = take("test")
else:
    print(f" Test {n_test:,}건은 잠긴 상태다 (unlock_test=True로 열 수 있다).")
return parts
```

```
def resplit_train_validation(
    X: pd.DataFrame,
    y: pd.Series,
    meta: pd.DataFrame,
    seed: int,
    manifest_seed: int | None = None,
    scheme: str = DEFAULT_SCHEME,
```

```

) -> dict[str, tuple]:
    """**Test를 고정한 채** Train+Validation 풀만 재분할한다 (#18의 K=50 반복용).

    `README.md`의 규정 그대로다 - *Test set은 그대로 고정해두고, 남은 Train+Validation 풀에서
    매번 다른 방식으로 Train/Validation을 재분할해 모형 학습 → threshold 탐색을 K=50회 반복*.*.

    ⚠ `split_6_2_2(seed=k)`를 반복에 쓰면 **안 된다.** 그건 전체를 다시 80/20으로 갈라
    **Test 구성까지 매번 바꾼다** - seed마다 다른 대출이 Test에 들어가므로 "Test를 고정해두고"가
    깨지고, 어떤 seed의 Test 건이 다른 seed의 Train에 들어가 사실상 Test가 오염된다.

    풀 안의 Validation 비율은 체계마다 다르다.

    | scheme | 풀 | Validation 비율 | Train / Validation |
    | --- | --- | --- | --- |
    | `6_2_2` | 매니페스트의 비Test 80% | `0.2/(0.6+0.2)` = **0.25** | 434,137 / 144,713 |
    | `7_3` | **표본 전량** (Test 칸 없음) | `0.3/(0.7+0.3)` = **0.30** | 506,494 / 217,069 |
    | `8_2` | **표본 전량** (Test 칸 없음) | `0.2/(0.8+0.2)` = **0.20** | 578,850 / 144,713 |

    `7_3`·`8_2`에서도 Test가 흔들리지 않는다 - Test가 애초에 이 표본 밖(별도 파일)이기
    때문이다(#30·#32).

    부도율이 16.2%로 치우칠 수 있어 `stratify`를 건다.
    """
    parts = split_from_manifest(X, y, meta, seed=manifest_seed, scheme=scheme)

    # 풀의 **순서를 `id`로 고정한다.** `train_test_split`이 위치를 서플하므로, 순서가 팀원마다
    # 다르면 같은 seed로도 다른 Train/Validation이 나온다. 원본 행번호로 정렬하면 원본 파일의
    # 행 순서에 다시 의존하게 되므로 `id` 기준이어야 한다.
    pool_ids = pd.concat([parts["train"][2]["id"], parts["validation"][2]["id"]]).astype(str)
    pool_idx = pool_ids.sort_values(kind="mergesort").index

    tr_share, va_share, _ = scheme_ratios(scheme)
    val_share_of_pool = va_share / (tr_share + va_share)
    keys = pd.Series(pool_idx, index=pool_idx)
    train_keys, val_keys = train_test_split(
        keys,
        test_size=val_share_of_pool,
        random_state=seed,
        stratify=y.loc[pool_idx],
    )

    def take(part: pd.Series) -> tuple:
        idx = part.index
        return X.loc[idx], y.loc[idx], meta.loc[idx]

    return {"train": take(train_keys), "validation": take(val_keys)}

if __name__ == "__main__":
    X, y, meta = build_feature_table()
    print(f"[피쳐 테이블] X={X.shape} y={y.shape} meta={meta.shape}")
    print(f"부도율: {y.mean():.4%}")

    print(f"\n[dtype 분포]\n{X.dtypes.astype(str).value_counts().to_string()}")

    cat_cols = X.select_dtypes(include="category").columns
    print(f"\n[범주형 {len(cat_cols)}개] 고유값 수")
    for c in cat_cols:

```

```

print(f"   {c:26s} {X[c].nunique():>6,}")

nan_share = X.isna().mean().sort_values(ascending=False)
print(f"\n[결측률 상위 5]\n{(nan_share.head(5) * 100).round(2).to_string()}")
print(f"   결측 없는 컬럼: {(nan_share == 0).sum()}개 / 전체 {len(nan_share)}개")

parts = split_6_2_2(X, y, meta)
print("\n[6:2:2 분할]")
for name, (Xp, yp, _) in parts.items():
    print(f"   {name:11s} n={len(Xp):>8,}   ({len(Xp) / len(X):.1%})   부도율 {yp.mean():.4%}")

```

src/preprocessing/export_split_manifest.py

6:2:2 분할 매니페스트 생성

"""6:2:2 분할 매니페스트 생성 - 팀원 전원이 **동일한 분할**을 쓰게 만든다.

왜 필요한가 - seed만으로는 부족하다

`split\_6\_2\_2()`는 `config.yaml`의 seed(42)와 비율로 `train\_test\_split`을 두 번 돌린다.
같은 입력이면 같은 결과가 나오지만, **같은 입력**의 조건이 생각보다 까다롭다.**

`train\_test\_split`은 행의 **위치(position)**를 셔플한다. 따라서 원본 CSV의 **행 순서가 다르면 완전히 다른 분할**이 나온다. 그런데 `filter\_analysis\_sample()`의 723,563건 검증은 **건수만 보므로 그대로 통과한다** - 어긋난 것을 아무도 모른 채 진행된다.

행 순서가 달라지는 경로는 실제로 있다.

- 원본을 엑셀·Numbers로 열었다 저장 (`AGENTS.md`가 금지하는 이유 중 하나)
- 팀 공유 채널에서 다른 시점에 받은 파일
- 정렬해서 저장한 사본

그리고 팀원이 각자 다른 seed로 돌리면 **한 사람의 Test 건이 다른 사람의 Train에 들어간다**.
개인 기준으로는 규칙을 지켰는데 **팀 전체로 보면 Test가 오염된다**.

해법 - `id` 기준 매니페스트

`id` (대출 고유번호)는 표본 723,563건에서 **전부 고유하고 결측이 없다** (실측).
그래서 `id` → `split` 표를 한 번 만들어 git에 올리면

- **행 순서와 무관하다** - 위치가 아니라 `id`로 매칭한다
- **pandas·sklearn 버전과 무관하다** - 난수를 다시 뽑지 않는다
- **원본 1.2GB 없이도 공유된다** - 매니페스트만 있으면 "어느 건이 Test인지" 알 수 있다
- **감사 가능하다** - 언제 어떤 seed로 만든 분할인지 기록이 남는다

⚠ 원본은 여전히 각자 받아야 한다. 매니페스트는 **분할 정의**만 공유하는 것이다.

실행: `/opt/anaconda3/bin/python src/preprocessing/export\_split\_manifest.py`
"""

```
from __future__ import annotations
```

```
import hashlib
```

```
from functools import partial
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
try:
```

```
    from preprocessing.preprocessor import (
        DEFAULT_SCHEME, SPLIT_SCHEMES, build_feature_table, scheme_ratios,
        split_6_2_2, split_train_validation,
    )
```

```
    from utils.config import load_config, repo_root
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
    import sys
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from preprocessing.preprocessor import (
    DEFAULT_SCHEME, SPLIT_SCHEMES, build_feature_table, scheme_ratios,
    split_6_2_2, split_train_validation,
```

```

)
from utils.config import load_config, repo_root

#: 체계별 분할 함수. `7_3`(#30)·`8_2`(#32)는 Test 칸을 만들지 않는다.
SPLIT_FUNCTIONS = {
    "6_2_2": split_6_2_2,
    "7_3": partial(split_train_validation, scheme="7_3"),
    "8_2": partial(split_train_validation, scheme="8_2"),
}

#: 체계별 split 이름 - 출처 카드·요약 출력 순서를 결정한다.
SPLIT_NAMES = {
    "6_2_2": ("train", "validation", "test"),
    "7_3": ("train", "validation"),
    "8_2": ("train", "validation"),
}

def manifest_path(seed: int, scheme: str = DEFAULT_SCHEME) -> Path:
    """`data/processed/split_manifest_6_2_2_seed20260730.csv.gz`.

    seed와 **분할 체계**를 파일명에 남긴다 - 다른 seed·체제로 만든 분할이 같은 파일을
    덮어쓰면 어느 것이 쓰였는지 추적할 수 없다. `7_3`이면 `split_manifest_7_3_seed...`다.
    """
    if scheme not in SPLIT_SCHEMES:
        raise ValueError(f"알 수 없는 분할 체계: {scheme!r} (가능: {sorted(SPLIT_SCHEMES)})")
    return load_config().paths.data_processed / f"split_manifest_{scheme}_seed{seed}.csv.gz"

def build_manifest(
    seed: int | None = None, scheme: str = DEFAULT_SCHEME
) -> tuple[pd.DataFrame, dict]:
    """`(매니페스트, 요약)`. 매니페스트는 `id`·`split` 두 열이다.

    `id`를 문자열로 둔다 - 정수로 캐스팅하면 선행 0이 사라지거나 dtype이 환경마다 달라진다.

    `scheme="7_3"`이면 **Test 칸이 없는** 매니페스트가 나온다 - 최종 Test는 별도 파일
    (`loader.second_test_path()`)이기 때문이다(#30).
    """
    cfg = load_config()
    seed = cfg.random_seed.default if seed is None else seed

    X, y, meta = build_feature_table()
    parts = SPLIT_FUNCTIONS[scheme](X, y, meta, seed=seed)

    rows = []
    for name in SPLIT_NAMES[scheme]:
        _, y_part, meta_part = parts[name]
        rows.append(
            pd.DataFrame({"id": meta_part["id"].astype(str), "split": name, "target": y_part})
        )
    manifest = pd.concat(rows).sort_values("id", kind="mergesort").reset_index(drop=True)

    if manifest["id"].duplicated().any():
        raise RuntimeError("id가 중복됐다 - 매니페스트를 쓸 수 없다.")
    if len(manifest) != len(X):

```

```

raise RuntimeError(f"매니페스트 {len(manifest):,}건 ≠ 표본 {len(X):,}건")

summary = {
    "seed": seed,
    "scheme": scheme,
    "n_total": len(manifest),
    "checksum": split_checksum(manifest),
    "shares": (manifest["split"].value_counts(normalize=True) * 100).round(3).to_dict(),
    "counts": manifest["split"].value_counts().to_dict(),
    "default_rate": (manifest.groupby("split")["target"].mean() * 100).round(4).to_dict(),
}

return manifest, summary

```

```
def split_checksum(manifest: pd.DataFrame) -> str:
```

"""분할 정의의 SHA-256. **행 순서에 무관하도록** `id`로 정렬한 뒤 계산한다.

팀원끼리 "같은 분할을 쓰고 있는가"를 이 한 줄로 대조할 수 있다.

"""

```

ordered = manifest.sort_values("id", kind="mergesort")
payload = "\n".join(f"{i},{s}" for i, s in zip(ordered["id"], ordered["split"]))
return hashlib.sha256(payload.encode("utf-8")).hexdigest()

```

```
def write_source_card(path: Path, summary: dict) -> Path:
```

"""짜이 되는 `.source.md` 출처 카드 (`docs/macro\_indicators\_spec.md` 2.8 규칙)."""

```

card = path.with_suffix("").with_suffix(".source.md")
sha = hashlib.sha256(path.read_bytes()).hexdigest()
counts, shares, dr = summary["counts"], summary["shares"], summary["default_rate"]
scheme = summary.get("scheme", DEFAULT_SCHEME)
tr, va, te = scheme_ratios(scheme)

```

```
if not te:
```

Test 칸이 없는 체계 - `7\_3` (#30) · `8\_2` (#32). 비율은 scheme_ratios에서 오므로
체계를 늘려도 이 분기를 고치지 않는다.

```

label = scheme.replace("_", ":")
issue = "#32" if scheme == "8_2" else "#30"
title = f"**Train/Validation {label} 분할 정의**"
how = (

```

```

    f"- 비율: train {tr} / validation {va} - **Test 칸이 없다**\n"
    "- 방법: `train_test_split` 한 번. `stratify`를 건다(부도율 16.2%).\n"
    "- **최종 Test는 별도 파일**이다 - `data/raw/lending_club_2020_test_2nd.csv`\n"
    f"    (481,833건, train 원본과 `id` 교집합 0건). `decision_log.md` {issue}."
)

```

```
test_note = (
```

```

    "## ⚠️ Test 취급\n"
    "- 이 매니페스트에는 **Test 칸이 없다.** `unlock_test=True`를 주면 예외가 난다.\n"
    "- 최종 평가는 `loader.second_test_path()`의 2nd Test로 **단 1회** 한다.\n"
    f"- K=50 반복은 `resplit_train_validation(..., scheme=\"{scheme}\")`을 쓴다 -\n"
    f"    표본 전량을 {tr:.0%}/{va:.0%}로 다시 가르며, Test는 애초에 이 표본 밖이라\n"
    "    흔들리지 않는다."
)

```

```
else:
```

```
title = "**Train/Validation/Test 6:2:2 분할 정의**"
```

```

how = (
    f"- 비율: `config/config.yaml`의 `split` (train {tr} / validation {va} / test {te})\n"
    "- 방법: `train_test_split`을 두 번 - 전체를 80/20으로 갈라 Test를 떼고, 남은 80%를\n"

```

```

    " 75/25로 Train/Validation으로 나눈다. 두 단계 모두 `stratify`를 건다(부도율 16.2%)."
```

)

```

test_note = (
    """ 🚨 Test 취급\n
    "- **Test는 기본적으로 반환되지 않는다.** `split_from_manifest(..., unlock_test=True)`로\n
    " 명시해야 나오고, 그때 경고를 출력한다.\n"
    "- Test는 **모형·threshold가 전부 확정된 뒤 단 1회** 적용한다\n"
    " (`src/analysis/AGENTS.md` `"Test set으로 모형을 재조정하지 않는다")\n"
    "- 랭킹 기준 3종 비교처럼 **여러 안을 고르는 작업에 Test를 쓰면 규칙 위반**이다 -\n"
    " 그 비교는 Validation에서 끝낸다."
)

rows = "\n".join(
    f"| {n} | {counts.get(n, 0):,} | {shares.get(n, 0)} | {dr.get(n, 0)} |"
    for n in SPLIT_NAMES[scheme]
)

card.write_text(
    f"""# {path.name} - 출처 카드

## 무엇인가
Lending Club 분석 표본 **{summary['n_total'],}건**(`decision_log.md` #16)의
{title}다. `id`와 `split`, 그리고 대조용 `target`을 담는다.

## 왜 있는가
seed만 공유하면 분할이 재현되지 않을 수 있다 - `train_test_split`은 **행의 위치**를 셔플하므로
원본 CSV의 행 순서가 다른 분할이 나오고, 723,563건 검증은 건수만 보므로 **조용히
어긋난다.** 이 파일은 `id` 기준이라 행 순서·라이브러리 버전과 무관하다.

## 어떻게 만들었나
- 생성 스크립트: `src/preprocessing/export_split_manifest.py --scheme {scheme}`
- seed: **{summary['seed'],}** (`config/config.yaml`의 `random_seed.default`)
{how}
- 입력: `data/raw/lending_club_2020_train.csv` → `filter_analysis_sample()` (만기 + 버퍼 6개월)

## 검증
| split | 건수 | 비율(%) | 부도율(%) |
| --- | ---: | ---: | ---: |
{rows}

- **분할 체크섬(SHA-256)**: `{summary['checksum']}`
  → 팀원끼리 같은 분할을 쓰는지 이 값으로 대조한다.
  `python src/preprocessing/export_split_manifest.py --scheme {scheme} --verify`
- 파일 SHA-256: `{sha}`
- 부도율이 `{len(SPLIT_NAMES[scheme])}`개 split에서 소수점 둘째 자리까지 맞는지 확인한다(`stratify` 정상 동작 근거).

## 쓰는 법
```python
from preprocessing.preprocessor import build_feature_table, split_from_manifest

X, y, meta = build_feature_table()
parts = split_from_manifest(X, y, meta, scheme="{scheme}")
X_tr, y_tr, meta_tr = parts["train"]
...

`split_6_2_2()`/`split_train_validation()`(seed 기반)를 직접 쓰지 말고 이 함수를 쓴다.

{test_note}
""",

```

```

 encoding="utf-8",
)
 return card

def parse_seed_arg(argv: list[str]) -> int | None:
 """`--seed 20260730` 형태를 읽는다. 없으면 `None` (= config의 기본 seed).

 seed를 CLI로 받는 이유는 **새 분할을 뺄 때 기존 파일을 덮지 않기 위해서**다 -
 파일명에 seed가 들어가므로 다른 seed는 다른 파일이 되고, 어느 분할로 낸 산출물인지
 추적된다.
 """
 if "--seed" not in argv:
 return None
 i = argv.index("--seed")
 if i + 1 >= len(argv):
 raise SystemExit("--seed 뒤에 값을 적어라. 예: --seed 20260730")
 return int(argv[i + 1])

def parse_scheme_arg(argv: list[str]) -> str:
 """`--scheme 7_3` 형태를 읽는다. 없으면 `6_2_2` .

 체계도 파일명에 들어가므로 두 매니페스트가 **서로 덮어쓰지 않는다** (#30).
 """
 if "--scheme" not in argv:
 return DEFAULT_SCHEME
 i = argv.index("--scheme")
 if i + 1 >= len(argv):
 raise SystemExit(f"--scheme 뒤에 값을 적어라. 가능: {sorted(SPLIT_SCHEMES)}")
 scheme = argv[i + 1]
 if scheme not in SPLIT_SCHEMES:
 raise SystemExit(f"알 수 없는 분할 체계: {scheme!r} (가능: {sorted(SPLIT_SCHEMES)})")
 return scheme

def main() -> None:
 import sys

 verify_only = "--verify" in sys.argv
 seed = parse_seed_arg(sys.argv)
 scheme = parse_scheme_arg(sys.argv)

 print("표본 구성 중... (원본 1.2GB 로딩 - 수 분 걸린다)")
 manifest, summary = build_manifest(seed=seed, scheme=scheme)

 print(f"\n[분할 요약] scheme={scheme} seed={summary['seed']} 총 {summary['n_total']:,}건")
 for name in SPLIT_NAMES[scheme]:
 print(f" {name:11s} {summary['counts'].get(name, 0):>8,}건 "
 f"{summary['shares'].get(name, 0):>7}% 부도를 {summary['default_rate'].get(name, 0):>7}%")
 if not scheme_ratios(scheme)[2]:
 print(" test (없음) - 최종 Test는 lending_club_2020_test_2nd.csv (#30·#32)")
 print(f"\n 분할 체크섬 {summary['checksum']}")

 path = manifest_path(summary["seed"], scheme=scheme)
 if verify_only:
 if not path.exists():

```

```

 raise SystemExit(f"매니페스트가 없다: {path}\n먼저 --verify 없이 실행해 생성하라.")
existing = pd.read_csv(path, dtype={"id": str})
old = split_checksum(existing)
same = old == summary["checksum"]
print(f" 기존 파일 체크섬 {old}")
print(f"\n 판정: {'✅' if old == summary['checksum'] else '❌'} 일치 - 같은 분할을 쓰고 있다." if same else '❌ 불일치!')
if not same:
 raise SystemExit(
 "분할이 어긋났다. 원본 CSV의 행 순서나 표본 필터가 달라졌을 수 있다.\n"
 "매니페스트를 새로 만들지 말고 **먼저 원인을 찾아라** - 이미 이 분할로 낸\n"
 "산출물이 전부 무효가 된다."
)
return

manifest.to_csv(path, index=False, compression="gzip")
card = write_source_card(path, summary)
size_mb = path.stat().st_size / 1024**2
print(f"\n산출물")
print(f" {path.relative_to(repo_root())} ({size_mb:.2f} MB, gzip)")
print(f" {card.relative_to(repo_root())}")
print("\n→ 이 두 파일을 git에 커밋하면 팀원 전원이 동일한 분할을 쓴다.")
print(" 팀원은 원본을 각자 받은 뒤 --verify로 자기 분할이 같은지 확인한다.")

if __name__ == "__main__":
 main()

```

## src/preprocessing/export\_shared\_dataset.py

팀 공유용 전처리 데이터셋 내보내기

""" 팀 공유용 전처리 데이터셋 내보내기 – Slack에 올릴 parquet 4종을 만든다.

## 왜 필요한가

팀원 전원이 원본 1.2GB를 갖고 있지만, \*\*각자 전처리를 돌리면 결과가 같린다.\*\*  
표본 필터·dtype 파싱·분할이 조금씩 어긋나도 건수 검증(723,563건)은 통과하므로  
조용히 다른 데이터로 작업하게 된다. 전처리까지 끝난 파일을 한 번 만들어 공유하면  
그 경로가 막힌다.

## 왜 CSV가 아니라 parquet인가

`preprocessor.coerce\_dtypes()`는 object를 전부 `category`로, `emp\_length`를 \*\*순서형\*\*  
category로, 월 날짜를 `Int64` (결측 보존)로 만든다. CSV로 내리면 이게 전부 사라진다.

특히 위험한 건 \*\*파일을 Test/비Test로 쪼갤 때\*\*다. 팀원이 CSV를 받아 다시  
`astype("category")`하면 \*\*자기 파일에 등장한 값만\*\* 카테고리가 되므로, Test에만 있는  
희귀 범주가 비Test 파일에 없으면 \*\*같은 문자열이 두 파일에서 다른 정수 코드\*\*를 받는다.  
XGBoost(`enable\_categorical=True`)는 코드로 학습하니 최종 Test 적용 때 모델이 다른 값을  
보게 된다 – 예러 없이 조용히 틀린다.

그래서 이 스크립트는 \*\*전체 표본에서 category dtype을 확정한 뒤 잘라서\*\* 저장한다.  
pandas는 슬라이싱해도 미사용 카테고리를 지우지 않으므로 두 파일의 코드가 일치한다.  
parquet은 pandas 메타데이터로 category·ordered·`Int64`를 그대로 보존한다.

## 산출물 (`data/processed/shared/`)

파일	내용
`trainval_features.parquet`	비Test 80% – 피쳐 + `target` + meta + `split` (train/validation)
`test_features.parquet`	Test 20% – **최종 1회 평가 전까지 열지 않는다**
`trainval_outcome.parquet`	비Test 사후변수 (실현수익률·조기상환 보증용)
`test_outcome.parquet`	Test 사후변수
`columns.json`	어느 열이 피쳐/meta/타겟인지 – 읽을 때 `X`를 복원하는 근거
`README_공유데이터.md`	사용법·체크섬·Test 취급 규칙

⚠️ \*\*사후변수(`outcome`)를 피쳐 파일에 합치지 않는다\*\* – 합치면 누수다  
(`src/preprocessing/AGENTS.md` 「누수 방지」). `id`로 조인해서 threshold·Sharpe 단계에서만 쓴다.

실행: `PYTHONPATH=src python src/preprocessing/export\_shared\_dataset.py`  
"""

```
from __future__ import annotations
```

```
import hashlib
```

```
import json
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
try:
```

```
 from preprocessing.preprocessor import META_COLUMNS, build_feature_table, load_split_manifest
```

```
 from utils.config import load_config, repo_root
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
 import sys
```

```
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```

from preprocessing.preprocessor import META_COLUMNS, build_feature_table, load_split_manifest
from utils.config import load_config, repo_root

```

```

SHARED_DIRNAME = "shared"

```

```

사후(post-approval) 컬럼 - `realized_return.load_outcome_frame()`이 쓰는 것과 같은 목록이다.

```

```

OUTCOME_COLUMNS = (
 "id", "loan_status", "term", "issue_d", "funded_amnt", "installment", "int_rate",
 "total_pymnt", "recoveries", "collection_recovery_fee",
 "last_pymnt_amnt", "last_pymnt_d",
)

```

```

def shared_dir() -> Path:
 d = load_config().paths.data_processed / SHARED_DIRNAME
 d.mkdir(parents=True, exist_ok=True)
 return d

```

```

def sha256_of(path: Path) -> str:
 h = hashlib.sha256()
 with path.open("rb") as f:
 for chunk in iter(lambda: f.read(1 << 20), b''):
 h.update(chunk)
 return h.hexdigest()

```

```

def build_shared_table(seed: int | None = None) -> tuple[pd.DataFrame, dict]:
 """피쳐 + 타겟 + meta + `split` 한 장. **자르기 전** 상태다.

```

```

 category dtype을 여기서 확정하는 것이 핵심이다 - 자른 뒤에 각자 만들면 코드가 어긋난다.
 """

```

```

 X, y, meta = build_feature_table()

```

```

 shared = X.copy()
 # meta 중 X에 없는 열만 덧붙인다. 겹치는 열(`issue_d`, `term`, `int_rate`, `funded_amnt`)은
 # 같은 파싱을 거쳐 값이 동일하므로 중복 저장할 이유가 없다.
 meta_only = [c for c in META_COLUMNS if c in meta.columns and c not in shared.columns]
 for col in meta_only:
 shared[col] = meta[col]

```

```

 shared["target"] = y.astype("int8")

```

```

 manifest = load_split_manifest(seed)
 assigned = meta["id"].astype(str).map(manifest)
 if assigned.isna().any():
 raise RuntimeError(
 f"매니페스트에 없는 id가 {int(assigned.isna().sum()):,}건이다 - "
 "표본 필터가 매니페스트 생성 시점과 다르다(#16)."
)

```

```

 shared["split"] = assigned.astype("category")

```

```

 layout = {
 "features": list(X.columns),
 "meta_only": meta_only,
 "target": "target",
 }

```

```

 "split": "split",
 "outcome": list(OUTCOME_COLUMNS),
 }
 return shared, layout

```

```
def load_outcome_table() -> pd.DataFrame:
```

```

 """사후변수 테이블 - 원본에서 표본 필터만 적용해 그대로 가져온다.

```

```

 파싱하지 않고 원본 형태로 둔다. `realized_return.py`가 자기 방식으로 파싱하므로
 여기서 미리 손대면 두 곳에 파싱 규칙이 생긴다.
 """

```

```

 from preprocessing.loader import filter_analysis_sample, load_raw_loans

```

```

 raw = load_raw_loans(usecols=list(OUTCOME_COLUMNS))
 return filter_analysis_sample(raw)

```

```
def export(seed: int | None = None) -> dict[str, Path]:
```

```

 out = shared_dir()

```

```

 print("표본 구성 중... (원본 1.2GB 로딩 - 수 분 걸린다)")

```

```

 shared, layout = build_shared_table(seed)

```

```

 print(f" 피쳐 {len(layout['features'])}개 + meta {len(layout['meta_only'])}개, 총 {len(shared):,}건")

```

```

 print("사후변수 로딩 중...")

```

```

 outcome = load_outcome_table()

```

```

 outcome_by_id = outcome.set_index(outcome["id"].astype(str))

```

```

 is_test = shared["split"] == "test"

```

```

 written: dict[str, Path] = {}

```

```

 for name, mask in (("trainval", ~is_test), ("test", is_test)):

```

```

 part = shared.loc[mask]

```

```

 path = out / f"{name}_features.parquet"

```

```

 part.to_parquet(path, index=False, compression="zstd")

```

```

 written[f"{name}_features"] = path

```

```

 ids = part["id"].astype(str)

```

```

 opath = out / f"{name}_outcome.parquet"

```

```

 outcome_by_id.loc[ids].reset_index(drop=True).to_parquet(

```

```

 opath, index=False, compression="zstd"

```

```

)

```

```

 written[f"{name}_outcome"] = opath

```

```

 print(f" {name:9s} {len(part):>8,}건 부도율 {part['target'].mean():.4%}")

```

```

 cols_path = out / "columns.json"

```

```

 cols_path.write_text(json.dumps(layout, ensure_ascii=False, indent=2), encoding="utf-8")

```

```

 written["columns"] = cols_path

```

```

 written["readme"] = write_readme(out, shared, layout, written, seed)

```

```

 return written

```

```
def write_readme(
```

```

 out: Path, shared: pd.DataFrame, layout: dict, written: dict[str, Path], seed: int | None

```

```

) -> Path:
 cfg = load_config()
 seed = cfg.random_seed.default if seed is None else seed
 counts = shared["split"].value_counts()

 def row(key: str) -> str:
 p = written[key]
 return (f"| `{p.name}` | {p.stat().st_size / 1024**2:.1f} MB | "
 f"`{sha256_of(p)[:16]}...` |")

 path = out / "README_공유데이터.md"
 path.write_text(
 f"""# 팀 공유 데이터셋 - 전처리 완료본

```

원본 1.2GB를 각자 전처리하면 결과가 같으므로, \*\*전처리까지 끝낸 파일\*\*을 공유한다.

받은 뒤 `filter\_analysis\_sample()`을 다시 돌리지 않는다 - 이미 적용돼 있다.

```

- 표본: **{len(shared):,}건** (만기 + 버퍼 6개월, `decision_log.md` #16)
- 분할: 6:2:2, seed **{seed}** - train {counts.get('train', 0):,} /
 validation {counts.get('validation', 0):,} / test {counts.get('test', 0):,}
- 생성 스크립트: `src/preprocessing/export_shared_dataset.py`

```

## 파일

```

| 파일 | 크기 | SHA-256 (앞 16자) |
| --- | ---: | --- |
{row('trainval_features')}
{row('trainval_outcome')}
{row('test_features')}
{row('test_outcome')}
{row('columns')}

```

## ⚠️ Test 파일은 열지 않는다

`test\_\*.parquet` 2종은 \*\*모형.threshold가 전부 확정된 뒤 단 1회\*\* 적용한다

(`src/analysis/AGENTS.md`). 랭킹 기준을 고르거나 threshold를 조정하는 데 쓰면 규칙 위반이다 - 그 비교는 Validation에서 끝낸다.

작업은 `trainval\_\*.parquet`만으로 한다.

## 쓰는 법

```

```python
from preprocessing.export_shared_dataset import load_shared

X, y, meta, split = load_shared("trainval")

X_tr, y_tr = X[split == "train"], y[split == "train"]
X_val, y_val = X[split == "validation"], y[split == "validation"]
```

```

`split` 컬럼은 \*\*기준 1회 실행(base run)\*\* 용이다. K=50 안정성 검증처럼 Train/Validation을 매번 다시 나누는 작업은 이 컬럼을 무시하고 비Test 80% 전체를 풀로 쓴다  
(`resplit\_train\_validation()`과 같은 취급).

## 사후변수 (`\*\_outcome.parquet`)

`total\_pymnt`·`recoveries`·`last\_pymnt\_d` 등 **\*\*결과변수\*\***다. 실현수익률·조기상환 보정 (B팀 1순위, #20)에 쓴다.

⚠ **\*\*피처에 합치지 않는다 - 누수다.\*\*** `id`로 조인해 threshold·Sharpe 단계에서만 쓴다.

## dtype 주의

범주형은 `category` dtype이고, **\*\*카테고리 목록은 전체 표본 기준으로 확정\*\***돼 있다 (trainval/test 파일의 코드가 일치한다). CSV로 변환해 다시 읽으면 이 보장이 깨지므로 **\*\*parquet 그대로 쓴다.\*\***

```
"""
 ,
 encoding="utf-8",
)
 return path
```

```
def load_shared(part: str = "trainval") -> tuple[pd.DataFrame, pd.Series, pd.DataFrame, pd.Series]:
 """공유 parquet -> `(X, y, meta, split)`.
```

`columns.json`으로 피처/meta를 갈라 돌려주므로 **\*\*X`에 `target`·`split`·`id`가 섞이지 않는다.\*\*** 원본 1.2GB 없이 파이프라인에 바로 쏴줄 수 있는 진입점이다.

`part="test"`는 최종 1회 평가 전까지 쓰지 않는다.

```
"""
```

```
out = shared_dir()
layout = json.loads((out / "columns.json").read_text(encoding="utf-8"))
df = pd.read_parquet(out / f"{part}_features.parquet")
```

```
if part == "test":
 print(f"⚠ Test set을 열었다 ({len(df):,}건). 모형·threshold 확정 후 **1회만** 쓴다.")
```

```
X = df[layout["features"]]
y = df[layout["target"]]
meta = df[[c for c in META_COLUMNS if c in df.columns]]
return X, y, meta, df[layout["split"]]
```

```
def load_shared_outcome(part: str = "trainval") -> pd.DataFrame:
 """공유 사후변수 테이블. `id`로 `load_shared()` 결과와 조인해 쓴다."""
 return pd.read_parquet(shared_dir() / f"{part}_outcome.parquet")
```

```
def main() -> None:
```

```
 import sys
```

```
 seed = None
```

```
 if "--seed" in sys.argv:
 seed = int(sys.argv[sys.argv.index("--seed") + 1])
```

```
 written = export(seed)
```

```
 print("\n산출물")
```

```
 total = 0.0
```

```
 for key, path in written.items():
```

```
 mb = path.stat().st_size / 1024**2
```

```
 total += mb
```

```
 print(f" {path.relative_to(repo_root())} ({mb:.1f} MB)")
```

```
print(f"\n 합계 {total:.1f} MB")
print("\n→ git 미추적이다(.gitignore). Slack 등 팀 채널로 공유한다.")
print(" 받는 사람은 README_공유데이터.md의 SHA-256으로 같은 파일인지 대조한다.")
```

```
if __name__ == "__main__":
 main()
```

## src/preprocessing/label\_pre\_post\_by\_rule.py

---

variable\_dictionary\_byGJ.xlsx에      is\_pre\_approval      라벨을      규칙      기반으로      붙여  
variable\_labels\_rule\_based.xlsx로 저장.

```
"""
```

`variable_dictionary_byGJ.xlsx`에 `is_pre_approval` 라벨을 규칙 기반으로 붙여 `variable_labels_rule_based.xlsx`로 저장.

라벨링 기준 (LendingClub 신용평가 모형에서 흔히 쓰이는 leakage 방지 기준):

- 0 (사후): 대출 실행 후 시간이 지나야(상황/연체가 진행돼야) 값이 생기는 변수.

- 즉 대출 상태/상황/연체/회수 이력, `hardship/settlement`(연체 후 구제 프로그램) 관련 필드.

- 1 (사전): 대출 신청 시점(및 그 이전 신용이력 조회 결과)에 이미 알 수 있는 변수.

- `grade/sub_grade/int_rate/installment/funded_amnt/funded_amnt_inv/issue_d/initial_list_status`는

- 대출 발행(`origination`) 시점에 함께 확정되며, 신규 심사 대상 데이터에도 이미 채워져 들어오므로

- 사전 변수로 간주한다 (근거: ``src/preprocessing/AGENTS.md`` 참고).

```
"""
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
다른 스크립트와 동일하게 저장소 루트 기준으로 경로를 잡는다 - 어느 디렉터리에서 실행해도 동작한다.
```

```
REPO_ROOT = Path(__file__).resolve().parents[2]
```

```
PROCESSED = REPO_ROOT / "data" / "processed"
```

```
SRC = PROCESSED / "variable_dictionary_byGJ.xlsx"
```

```
DST = PROCESSED / "variable_labels_rule_based.xlsx"
```

```
SHEET = "Sheet1"
```

```
SRC에는 세 AI가 매긴 라벨과 그 합의 결과(is_pre_approval)가 이미 들어 있다.
```

```
이 스크립트는 규칙 기반으로 라벨을 독립적으로 다시 매기므로, 비교 대상이 오염되지
```

```
않도록 기존 라벨 컬럼은 떼어내고 변수 사전 원본 컬럼만 남긴 상태에서 시작한다.
```

```
EXISTING_LABEL_COLS = [
```

```
 "gemini_label", "vscode_label", "claude_label", "is_pre_approval", "우수사례 판단",
```

```
]
```

```
대출 실행 후 시간이 지나야 발생·확정되는 변수(사후, 0)
```

```
POST_APPROVAL_VARS = {
```

```
 # 대출 상태/상황 이력 (편당 이후 계속 갱신됨)
```

```
 "loan_status", "out_prncp", "out_prncp_inv",
```

```
 "total_pymnt", "total_pymnt_inv", "total_rec_prncp", "total_rec_int",
```

```
 "total_rec_late_fee", "recoveries", "collection_recovery_fee",
```

```
 "last_pymnt_d", "last_pymnt_amnt", "next_pymnt_d", "last_credit_pull_d",
```

```
 "last_fico_range_high", "last_fico_range_low", "pymnt_plan",
```

```
 # 상황곤란(hardship) 프로그램 - 대출 실행 후 연체/곤란 발생 시에만 생김
```

```
 "hardship_flag", "hardship_type", "hardship_reason", "hardship_status",
```

```
 "deferral_term", "hardship_amount", "hardship_start_date",
```

```
 "hardship_end_date", "payment_plan_start_date", "hardship_length",
```

```
 "hardship_dpd", "hardship_loan_status",
```

```
 "orig_projected_additional_accrued_interest",
```

```
 "hardship_payoff_balance_amount", "hardship_last_payment_amount",
```

```
 # 자금 지급 방식 및 채무 정산(settlement) - 대출 실행/연체 이후 발생
```

```
 "disbursement_method",
```

```
 "debt_settlement_flag", "debt_settlement_flag_date",
```

```
 "settlement_status", "settlement_date", "settlement_amount",
```

```
 "settlement_percentage", "settlement_term",
```

```
}
```

```
df = pd.read_excel(SRC, sheet_name=SHEET).drop(columns=EXISTING_LABEL_COLS, errors="ignore")
```

```
unknown = POST_APPROVAL_VARS - set(df["LoanStatNew"])
```

```
if unknown:
```

```
 raise ValueError(f"variable_dictionary_byGJ.xlsx에 없는 변수명이 POST_APPROVAL_VARS에 있습니다: {unknown}")
```

```
df["is_pre_approval"] = df["LoanStatNew"].apply(
 lambda x: 0 if x in POST_APPROVAL_VARS else 1
)

df.to_excel(DST, index=False)

counts = df["is_pre_approval"].value_counts().sort_index()
print(f"저장 완료: {DST.relative_to(REPO_ROOT)}")
print(f"전체 변수 수: {len(df)}")
print(f"사후(0) 변수 개수: {counts.get(0, 0)}")
print(f"사전(1) 변수 개수: {counts.get(1, 0)}")
```

## src/preprocessing/fetch\_treasury\_gs1m.py

---

실현수익률의 재투자·역할인에 사용할 월별 GS1M을 FRED에서 수집한다.

""" 실현수익률의 재투자·역할인에 사용할 월별 GS1M을 FRED에서 수집한다.

대상 시리즈

- GS1M: Market Yield on U.S. Treasury Securities at 1-Month Constant Maturity, Quoted on an Investment Basis
- 원출처: Board of Governors of the Federal Reserve System, H.15
- 주기/단위: 월별(영업일 평균), 연율 %, 계절조정 없음
- 기간: 2007-07 ~ 2025-09 (219개월)

실행

python src/preprocessing/fetch\_treasury\_gslm.py

"""

```
from __future__ import annotations
```

```
import hashlib
```

```
import io
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
import requests
```

```
SERIES = "GS1M"
```

```
START = "2007-07-01"
```

```
END = "2025-09-01"
```

```
EXPECTED_ROWS = 219
```

```
REPO_ROOT = Path(__file__).resolve().parents[2]
```

```
OUT_PATH = (
```

```
 REPO_ROOT
```

```
 / "data"
```

```
 / "processed"
```

```
 / "us_treasury_GS1M_monthly_2007-07_to_2025-09.csv"
```

```
)
```

```
CROSS_CHECK_PATH = (
```

```
 REPO_ROOT
```

```
 / "data"
```

```
 / "processed"
```

```
 / "us_treasury_GS3_GS5_monthly_2007-06_to_2020-09.csv"
```

```
)
```

```
DOWNLOAD_URL = (
```

```
 "https://fred.stlouisfed.org/graph/fredgraph.csv"
```

```
 f"?id={SERIES}&cosd={START}&coed={END}"
```

```
)
```

```
현재 FRED/H.15 값에 넉넉한 허용범위를 둔 사건 기반 점검.
```

```
SANITY_CHECKS = [
```

```
 ("2007-07-01", 4.70, 4.90, "금융위기 이전 단기금리 4%대"),
```

```
 ("2008-12-01", 0.00, 0.10, "금융위기 이후 제로금리 근접"),
```

```
 ("2018-12-01", 2.25, 2.50, "2015~2018 금리 정상화"),
```

```
 ("2020-04-01", 0.05, 0.20, "코로나 충격 이후 재차 제로금리 근접"),
```

```
 ("2023-10-01", 5.40, 5.70, "2022~2023 긴축기의 고금리"),
```

```
 ("2025-09-01", 4.10, 4.40, "필요 범위 마지막 달"),
```

]

```
def fetch() -> pd.DataFrame:
 """FRED CSV를 내려받아 검증 전 DataFrame으로 반환한다."""
 response = requests.get(
 DOWNLOAD_URL,
 timeout=30,
 headers={"User-Agent": "LendingClub-Team-Project/1.0"},
)
 response.raise_for_status()

 frame = pd.read_csv(io.StringIO(response.text), na_values=["."])
 if frame.shape[1] != 2:
 raise ValueError(f"예상하지 못한 FRED 컬럼 구조: {frame.columns.tolist()}")

 frame.columns = ["observation_date", SERIES]
 frame["observation_date"] = pd.to_datetime(
 frame["observation_date"], errors="raise"
)
 frame[SERIES] = pd.to_numeric(frame[SERIES], errors="coerce")
 return frame

def validate(frame: pd.DataFrame) -> None:
 """저장 전에 기간·연속성·결측·중복·사건값을 검증한다."""
 if len(frame) != EXPECTED_ROWS:
 raise ValueError(
 f"행 수가 {EXPECTED_ROWS}가 아닙니다: {len(frame)}행"
)

 if frame["observation_date"].duplicated().any():
 raise ValueError("observation_date 중복이 있습니다")

 expected_dates = pd.date_range(START, END, freq="MS")
 if not frame["observation_date"].reset_index(drop=True).equals(
 pd.Series(expected_dates, name="observation_date")
):
 raise ValueError("월이 연속적이지 않거나 기간·정렬이 예상과 다릅니다")

 missing = int(frame[SERIES].isna().sum())
 if missing:
 raise ValueError(f"{SERIES} 결측이 {missing}개 있습니다")

 if (frame[SERIES] < 0).any():
 bad = frame.loc[frame[SERIES] < 0, ["observation_date", SERIES]]
 raise ValueError(f"{SERIES} 음수값을 확인해야 합니다:\n{bad}")

 failed = []
 for date, lower, upper, note in SANITY_CHECKS:
 value = frame.loc[
 frame["observation_date"] == pd.Timestamp(date), SERIES
].iloc[0]
 passed = lower <= value <= upper
 print(
 f" [{ 'OK' if passed else '실패' }] {date[:7]} "
 f"{SERIES}={value:.2f}% "
)
```

```

 f"기대 {lower:.2f}~{upper:.2f}) - {note}"
)
 if not passed:
 failed.append(date)

 if failed:
 raise ValueError(
 f"사건 기반 값 검증 실패: {failed}. 시리즈와 최신값을 확인하세요."
)

def cross_check_existing_treasury(frame: pd.DataFrame) -> None:
 """기존 GS3·GS5 파일과 월 키 및 대표 수익률곡선 구간을 교차 검증한다."""
 if not CROSS_CHECK_PATH.exists():
 print("기존 GS3·GS5 파일이 없어 교차 검증은 건너뛴니다.")
 return

 longer = pd.read_csv(CROSS_CHECK_PATH, parse_dates=["observation_date"])
 overlap = frame.merge(longer, on="observation_date", how="inner")
 expected_overlap = len(pd.date_range(START, "2020-09-01", freq="MS"))

 if len(overlap) != expected_overlap:
 raise ValueError(
 f"GS3·GS5 교차검증 월 수 불일치: "
 f"{len(overlap)} != {expected_overlap}"
)
 if overlap[[SERIES, "GS3", "GS5"]].isna().any().any():
 raise ValueError("GS1M·GS3·GS5 교차검증 구간에 결측이 있습니다")

 april_2020 = overlap.loc[
 overlap["observation_date"] == pd.Timestamp("2020-04-01")
].iloc[0]
 if not april_2020[SERIES] < april_2020["GS3"] < april_2020["GS5"]:
 raise ValueError(
 "2020-04 대표 구간에서 GS1M < GS3 < GS5 관계가 성립하지 않습니다"
)

 print(
 f" [OK] 기존 GS3·GS5와 {expected_overlap}개월 날짜 완전 매칭, "
 "2020-04 만기구조 교차검증 통과"
)

def sha256(path: Path) -> str:
 digest = hashlib.sha256()
 with path.open("rb") as file:
 for chunk in iter(lambda: file.read(1024 * 1024), b""):
 digest.update(chunk)
 return digest.hexdigest()

def main() -> None:
 frame = fetch().sort_values("observation_date").reset_index(drop=True)
 validate(frame)
 cross_check_existing_treasury(frame)

 output = frame.copy()

```

```

output["observation_date"] = output["observation_date"].dt.strftime(
 "%Y-%m-%d"
)
output.to_csv(OUT_PATH, index=False)

stats = output[SERIES].describe()
print(f"\n저장 완료: {OUT_PATH.relative_to(REPO_ROOT)}")
print(
 f"행 수: {len(output)} "
 f"기간: {output['observation_date'].iloc[0]} ~ "
 f"{output['observation_date'].iloc[-1]}"
)
print(f"결측: {int(output[SERIES].isna().sum())}개")
print(
 f"{SERIES}: min {stats['min']:.2f}% / max {stats['max']:.2f}% / "
 f"mean {stats['mean']:.3f}% / std {stats['std']:.3f}%"
)
print(f"SHA-256: {sha256(OUT_PATH)}")

if __name__ == "__main__":
 main()

```

## src/preprocessing/fetch\_macro\_cpi.py

CPI(CPIAUCSL) 월별 시계열과 YoY 인플레이션율을 FRED에서 받아 저장한다

""CPI(CPIAUCSL) 월별 시계열과 YoY 인플레이션율을 FRED에서 받아 저장한다. (이슈 #4)

규격은 docs/macro\_indicators\_spec.md 를 따른다.

- 시리즈: CPIAUCSL (CPI for All Urban Consumers: All Items, 월별, 계절조정(SA), 단위 지수 1982-84=100)
- 기간: 2007-01 ~ 2020-09 (165개월)
- 컬럼: observation\_date, CPIAUCSL, cpi\_yoy\_pct

YoY 계산을 위해 \*\*2006-01부터 받아\*\* 12개월 전 대비 변화율을 구한 뒤 2007-01 이후만 저장한다(스펙 3.4 권장안). 2007-01부터 받으면 2007년 12개월치 YoY가 전부 결측이 되지만, 이 방식은 결측 0으로 떨어진다.

지수 레벨(CPIAUCSL) 자체는 단조증가라 피쳐로서 의미가 약하다. 실제 모델에 쓸 후보는 cpi\_yoy\_pct 쪽이지만, 판단은 결합 단계로 미루고 둘 다 저장한다.

발표시차를 미리 적용하지 않는다 - 원시 시계열을 발표 기준월 그대로 저장하고, lag 적용 여부는 거시지표 결합 단계에서 팀이 일괄 결정한다(스펙 2.6).

실행:

```
python src/preprocessing/fetch_macro_cpi.py
"""
import io
from pathlib import Path

import pandas as pd
import requests

SERIES = "CPIAUCSL"
YoY 12개월치를 확보하려고 저장 시작월(2007-01)보다 1년 앞에서 받는다.
FETCH_START = "2006-01-01"
START, END = "2007-01-01", "2020-09-01"
EXPECTED_ROWS = 165

URL = (
 "https://fred.stlouisfed.org/graph/fredgraph.csv"
 f"?id={SERIES}&cosd={FETCH_START}&coed={END}"
)

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_PATH = REPO_ROOT / "data" / "processed" / "macro_cpi_monthly_2007-01_to_2020-09.csv"

스펙 2.9 값 검증 - 알려진 사건과 대조한다. (기준월, 컬럼, 최솟값, 최댓값, 설명)
기대값은 "지금 FRED에서 받으면 나오는 값" 기준으로 적는다(스펙 2.9 주의사항).
SANITY_CHECKS = [
 ("2008-07-01", "cpi_yoy_pct", 5.0, 6.0, "유가 급등기 인플레이션 정점(+5.5%)"),
 ("2009-07-01", "cpi_yoy_pct", -2.5, -1.5, "금융위기 후 디플레이션 저점(-2.0%)"),
 ("2020-05-01", "cpi_yoy_pct", -0.5, 0.8, "코로나 수요 급감으로 인플레이션 거의 소멸"),
 ("2007-01-01", SERIES, 195.0, 210.0, "지수 레벨(1982-84=100 기준) 타당 범위"),
 ("2020-09-01", SERIES, 250.0, 270.0, "기간 말 지수 레벨"),
]

def main() -> None:
 # pandas.read_csv(URL)을 직접 쓰지 않고 requests를 거치는 이유:
 # python.org 빌드 Python은 CA 인증서가 설치돼 있지 않아 표준 urllib이
 # SSLCertVerificationError로 실패한다. requests는 certifi 번들을 쓴다.
```

```

resp = requests.get(URL, timeout=30)
resp.raise_for_status()

FRED는 결측을 ".", "으로 내려주므로 NaN으로 받는다 (스펙 2.4).
df = pd.read_csv(io.StringIO(resp.text), na_values=["."])
df.columns = ["observation_date", SERIES]
df["observation_date"] = pd.to_datetime(df["observation_date"])
df = df.sort_values("observation_date").reset_index(drop=True)

if not df["observation_date"].equals(
 pd.Series(pd.date_range(FETCH_START, END, freq="MS"))
):
 raise ValueError("받은 원자료의 월이 연속적이지 않거나 중복/누락된 달이 있습니다")
if df[SERIES].isna().any():
 raise ValueError(f"원자료에 결측이 있습니다: {df[SERIES].isna().sum()}개")

12개월 전 대비 변화를. shift(12)는 위에서 월 연속성을 검증했으므로 안전하다.
df["cpi_yoy_pct"] = ((df[SERIES] / df[SERIES].shift(12) - 1) * 100).round(3)

YoY 계산용으로만 쓴 2006년 12개월을 잘라낸다.
df = df[df["observation_date"] >= pd.Timestamp(START)].reset_index(drop=True)

if len(df) != EXPECTED_ROWS:
 raise ValueError(f"행 수가 {EXPECTED_ROWS}가 아닙니다: {len(df)}행")
if df[["CPIAUCSL", "cpi_yoy_pct"]].isna().any().any():
 raise ValueError("저장 대상 구간에 결측이 있습니다 - YoY 계산용 선행 12개월을 확인하세요")

failed = []
for date, col, lo, hi, note in SANITY_CHECKS:
 value = df.loc[df["observation_date"] == pd.Timestamp(date), col].iloc[0]
 ok = lo <= value <= hi
 print(f" [{ 'OK' if ok else '실패' }] {date[:7]} {col}={value:+.3f} (기대 {lo:+.1f}~{hi:+.1f}) - {note}")
 if not ok:
 failed.append((date, col))
if failed:
 raise ValueError(f"값 검증 실패 - 시리즈를 잘못 받았을 수 있습니다: {failed}")

df["observation_date"] = df["observation_date"].dt.strftime("%Y-%m-%d")
df.to_csv(OUT_PATH, index=False)

deflation = df[df["cpi_yoy_pct"] < 0]
print(f"\n저장 완료: {OUT_PATH.relative_to(REPO_ROOT)}")
print(f"행 수: {len(df)} 기간: {df['observation_date'].iloc[0]} ~ {df['observation_date'].iloc[-1]}")
print(f"결측: {df[[SERIES, 'cpi_yoy_pct']].isna().sum().sum()}개")
for col in [SERIES, "cpi_yoy_pct"]:
 s = df[col].describe()
 print(
 f" {col:<12} min {s['min']:+.3f} / max {s['max']:+.3f} / "
 f"mean {s['mean']:+.3f} / std {s['std']:.3f}"
)
print(f"\nYoY 음수(디플레이션) 개월: {len(deflation)}개월")
if len(deflation):
 print(deflation[["observation_date", "cpi_yoy_pct"]].to_string(index=False))

```

```
if __name__ == "__main__":
 main()
```

## src/preprocessing/fetch\_macro\_initial\_claims.py

---

신규 실업수당청구(ICSA) 주간 시계열을 FRED에서 받아 월별로 집계해 저장한다

""" 신규 실업수당청구(ICSA) 주간 시계열을 FRED에서 받아 월별로 집계해 저장한다. (이슈 #2)

규격은 docs/macro\_indicators\_spec.md 를 따른다.

- 시리즈: ICSA (Initial Claims, 주간, 계절조정(SA), 단위 명(건))
- 기간: 2007-01 ~ 2020-09 (월별 165개월 / 주간 717주)
- 산출물 2개:

macro\_initial\_claims\_raw\_weekly\_2007-01\_to\_2020-09.csv (주간 원자료, 717행)

macro\_initial\_claims\_monthly\_2007-01\_to\_2020-09.csv (월별 집계, 165행)

주간 -> 월별 집계 방식 (스펙 3.2):

ICSA의 관측일은 그 주의 \*\*토요일(week ending)\*\* 이다. week ending 날짜가 속한 달로 주를 배정한 뒤 \*\*그 달에 속한 주간 관측치의 평균\*\*을 취한다.

합계(sum)가 아니라 평균(mean)을 쓰는 이유: 한 달에 속하는 주가 4주인 달이 108개, 5주인 달이 57개로 갈린다. 합계로 집계하면 5주 달의 값이 구조적으로 약 25% 부풀려져 실제 청구 수준과 무관한 계절적 톱니가 생긴다.

발표시차를 미리 적용하지 않는다 - 원시 시계열을 발표 기준주/월 그대로 저장하고, lag 적용 여부는 거시지표 결합 단계에서 팀이 일괄 결정한다(스펙 2.6).

실행:

```
python src/preprocessing/fetch_macro_initial_claims.py
"""
import io
from pathlib import Path

import pandas as pd
import requests

SERIES = "ICSA"
START, END = "2007-01-01", "2020-09-30"
EXPECTED_WEEKS = 717
EXPECTED_MONTHS = 165

URL = (
 "https://fred.stlouisfed.org/graph/fredgraph.csv"
 f"?id={SERIES}&cosd={START}&coed={END}"
)

REPO_ROOT = Path(__file__).resolve().parents[2]
PROCESSED = REPO_ROOT / "data" / "processed"
OUT_WEEKLY = PROCESSED / "macro_initial_claims_raw_weekly_2007-01_to_2020-09.csv"
OUT_MONTHLY = PROCESSED / "macro_initial_claims_monthly_2007-01_to_2020-09.csv"

스펙 2.9 값 검증 - 알려진 사건과 대조한다. (기준월, 최솟값, 최댓값, 설명)
SANITY_CHECKS = [
 ("2007-01-01", 250_000, 400_000, "금융위기 이전 평시 수준(주당 30만 건대)"),
 ("2009-03-01", 550_000, 750_000, "금융위기 청구 피크 구간"),
 ("2019-12-01", 190_000, 280_000, "코로나 직전 역사적 최저 수준"),
 ("2020-04-01", 4_000_000, 5_500_000, "코로나 급등 피크(주당 400만 건대 평균)"),
]

def main() -> None:
 # pandas.read_csv(URL)을 직접 쓰지 않고 requests를 거치는 이유:
 # python.org 빌드 Python은 CA 인증서가 설치돼 있지 않아 표준 urllib이
```

```

SSLCertVerificationError로 실패한다. requests는 certifi 번들을 쓴다.
resp = requests.get(URL, timeout=30)
resp.raise_for_status()

FRED는 결측을 "."으로 내려주므로 NaN으로 받는다 (스펙 2.4).
weekly = pd.read_csv(io.StringIO(resp.text), na_values=["."])
weekly.columns = ["observation_date", SERIES]
weekly["observation_date"] = pd.to_datetime(weekly["observation_date"])
weekly = weekly.sort_values("observation_date").reset_index(drop=True)

if len(weekly) != EXPECTED_WEEKS:
 raise ValueError(f"주간 행 수가 {EXPECTED_WEEKS}가 아닙니다: {len(weekly)}행")
if weekly[SERIES].isna().any():
 raise ValueError(f"주간 원자료에 결측이 있습니다: {weekly[SERIES].isna().sum()}개")

관측일이 전부 토요일(week ending)인지 확인 - 집계 규칙의 전제다.
weekdays = set(weekly["observation_date"].dt.day_name())
if weekdays != {"Saturday"}:
 raise ValueError(f"관측일이 토요일이 아닌 행이 있습니다: {weekdays}")

week ending 날짜가 속한 달로 재정 후 월평균.
monthly = (
 weekly.set_index("observation_date")[SERIES]
 .resample("MS")
 .mean()
 .rename(SERIES)
 .reset_index()
)

if len(monthly) != EXPECTED_MONTHS:
 raise ValueError(f"월별 행 수가 {EXPECTED_MONTHS}가 아닙니다: {len(monthly)}행")
if not monthly["observation_date"].equals(
 pd.Series(pd.date_range("2007-01-01", "2020-09-01", freq="MS"))
):
 raise ValueError("월이 연속적이지 않거나 중복/누락된 달이 있습니다")
if monthly[SERIES].isna().any():
 raise ValueError("주가 하나도 지정되지 않은 달이 있습니다")

failed = []
for date, lo, hi, note in SANITY_CHECKS:
 value = monthly.loc[monthly["observation_date"] == pd.Timestamp(date), SERIES].iloc[0]
 ok = lo <= value <= hi
 print(
 f" [{ 'OK' if ok else '실패' }] {date[:7]} {SERIES}={value:,.0f} "
 f" (기대 {lo:,}~{hi:,}) - {note}"
)
 if not ok:
 failed.append(date)
if failed:
 raise ValueError(f"값 검증 실패 - 시리즈를 잘못 받았을 수 있습니다: {failed}")

월평균은 정수 건수의 평균이라 소수가 생긴다. 원자료 단위(명)에 맞춰 반올림한다.
monthly[SERIES] = monthly[SERIES].round().astype("int64")

for df, path in ((weekly, OUT_WEEKLY), (monthly, OUT_MONTHLY)):
 out = df.copy()
 out["observation_date"] = out["observation_date"].dt.strftime("%Y-%m-%d")

```

```

out.to_csv(path, index=False)

weeks_per_month = weekly.groupby(w weekly["observation_date"].dt.to_period("M")).size()
peak_week = weekly.loc[weekly[SERIES].idxmax()]

print(f"\n저장 완료: {OUT_WEEKLY.relative_to(REPO_ROOT)} ({len(weekly)}행, 주간 원자료)")
print(f"\n저장 완료: {OUT_MONTHLY.relative_to(REPO_ROOT)} ({len(monthly)}행, 월평균)")
print(
 f"\n한 달에 속한 주 수: "
 f"{weeks_per_month.value_counts().sort_index().to_dict()} (4주/5주 달 개수)"
)
print(
 f"\n주간 최대: {peak_week['observation_date']:%Y-%m-%d} 마감 주 "
 f"{peak_week[SERIES]:,.0f}건"
)
print(f"\n월별 기간: {monthly['observation_date'].iloc[0]} ~ {monthly['observation_date'].iloc[-1]}")
print(f"\n결측: {monthly[SERIES].isna().sum()}개")
stats = monthly[SERIES].describe()
print(
 f"min {stats['min']:.0f} / max {stats['max']:.0f} / "
 f"mean {stats['mean']:.0f} / std {stats['std']:.0f}"
)

if __name__ == "__main__":
 main()

```

## src/preprocessing/fetch\_macro\_unemployment\_rate.py

실업률(UNRATE) 월별 시계열을 FRED에서 받아 data/processed/에 저장한다

""" 실업률(UNRATE) 월별 시계열을 FRED에서 받아 data/processed/에 저장한다. (이슈 #1)

규격은 docs/macro\_indicators\_spec.md 를 따른다.

- 시리즈: UNRATE (Unemployment Rate, 월별, 계절조정(SA), 단위 %)
- 기간: 2007-01 ~ 2020-09 (165개월)
- 컬럼: observation\_date(YYYY-MM-01), UNRATE

발표시차를 미리 적용하지 않는다 - 원시 시계열을 발표 기준월 그대로 저장하고,  
lag 적용 여부는 거시지표 결합 단계에서 팀이 일괄 결정한다(스펙 2.6).

실행:

```
python src/preprocessing/fetch_macro_unemployment_rate.py
"""
import io
from pathlib import Path

import pandas as pd
import requests

SERIES = "UNRATE"
START, END = "2007-01-01", "2020-09-01"
EXPECTED_ROWS = 165

URL = (
 "https://fred.stlouisfed.org/graph/fredgraph.csv"
 f"?id={SERIES}&cosd={START}&coed={END}"
)

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_PATH = REPO_ROOT / "data" / "processed" / "macro_unemployment_rate_monthly_2007-01_to_2020-09.csv"

스펙 2.9 값 검증 - 알려진 사건과 대조한다. (기준월, 최솟값, 최댓값)
SANITY_CHECKS = [
 ("2007-01-01", 4.4, 4.8, "금융위기 이전 완전고용 수준"),
 ("2009-10-01", 9.8, 10.2, "금융위기 실업률 피크(약 10%)"),
 ("2019-12-01", 3.4, 3.8, "코로나 직전 50년래 최저 수준"),
 ("2020-04-01", 14.0, 15.0, "코로나 급등 피크(약 14.7%)"),
]

def main() -> None:
 # pandas.read_csv(URL)을 직접 쓰지 않고 requests를 거치는 이유:
 # python.org 빌드 Python은 CA 인증서가 설치돼 있지 않아 표준 urllib이
 # SSLCertVerificationError로 실패한다. requests는 certifi 번들을 쓰므로
 # 별도 설정 없이 팀원 환경에서도 동일하게 동작한다.
 resp = requests.get(URL, timeout=30)
 resp.raise_for_status()

 # FRED는 결측을 "."으로 내려주므로 NaN으로 받는다 (스펙 2.4).
 df = pd.read_csv(io.StringIO(resp.text), na_values=["."])
 df.columns = ["observation_date", SERIES]
 df["observation_date"] = pd.to_datetime(df["observation_date"])
 df = df.sort_values("observation_date").reset_index(drop=True)

 if len(df) != EXPECTED_ROWS:
 raise ValueError(f"행 수가 {EXPECTED_ROWS}가 아닙니다: {len(df)}행")
```

```

if df["observation_date"].min() != pd.Timestamp(START):
 raise ValueError(f"시작월 불일치: {df['observation_date'].min():%Y-%m}")
if df["observation_date"].max() != pd.Timestamp(END):
 raise ValueError(f"종료월 불일치: {df['observation_date'].max():%Y-%m}")
if not df["observation_date"].equals(
 pd.Series(pd.date_range(START, END, freq="MS"))
):
 raise ValueError("월이 연속적이지 않거나 중복/누락된 달이 있습니다")

failed = []
for date, lo, hi, note in SANITY_CHECKS:
 value = df.loc[df["observation_date"] == pd.Timestamp(date), SERIES].iloc[0]
 ok = lo <= value <= hi
 print(f" [{ 'OK' if ok else '실패' }] {date[:7]} {SERIES}={value} (기대 {lo}~{hi}) - {note}")
 if not ok:
 failed.append(date)
if failed:
 raise ValueError(f"값 검증 실패 - 시리즈를 잘못 받았을 수 있습니다: {failed}")

df["observation_date"] = df["observation_date"].dt.strftime("%Y-%m-%d")
df.to_csv(OUT_PATH, index=False)

print(f"\n저장 완료: {OUT_PATH.relative_to(REPO_ROOT)}")
print(f"행 수: {len(df)} 기간: {df['observation_date'].iloc[0]} ~ {df['observation_date'].iloc[-1]}")
print(f"결측: {df[SERIES].isna().sum()}개")
stats = df[SERIES].describe()
print(
 f"min {stats['min']:.1f} / max {stats['max']:.1f} / "
 f"mean {stats['mean']:.3f} / std {stats['std']:.3f}"
)

if __name__ == "__main__":
 main()

```

## src/preprocessing/fetch\_macro\_yield\_spread.py

10년-2년 국채 금리차를 FRED에서 받아 저장한다

"""10년-2년 국채 금리차를 FRED에서 받아 저장한다. (이슈 #3)

규격은 docs/macro\_indicators\_spec.md 를 따른다.

- 시리즈: GS10 (10-Year Treasury Constant Maturity Rate), GS2 (2-Year), 둘 다 월별, 단위 %
- 기간: 2007-01 ~ 2020-09 (165개월)
- 컬럼: observation\_date, GS10, GS2, spread\_10y2y (= GS10 - GS2)

월별 시리즈(DGS10/DGS2/T10Y2Y)를 월평균 내지 않고 월별 GS 계열을 쓰는 이유:

이미 확보한 무위험수익률 파일(us\_treasury\_GS3\_GS5\_monthly\_\*.csv)이 같은 GS 계열이라  
만기 구조가 일관되게 맞고, 추가 가공 단계가 없어 재현이 단순하다.

다만 월평균은 짧은 역전을 지워버린다 - 2019년 10y-2y 역전은 일별 기준 단 3일  
(2019-08-27~29, 최저 -0.04) 이었고 월평균으로는 2019-08이 +0.06으로 양수다.  
자세한 내용은 outputs/reports/macro\_yield\_spread.md 참고.

발표시차: 이 지표는 시장가격이라 사실상 실시간이다(스펙 2.6의 발표시차 이슈가  
거의 없는 유일한 지표). 그래도 다른 지표와 규격을 맞추기 위해 시차는 적용하지 않는다.

실행:

```
python src/preprocessing/fetch_macro_yield_spread.py
"""
import io
from pathlib import Path

import pandas as pd
import requests

SERIES = ["GS10", "GS2"]
START, END = "2007-01-01", "2020-09-01"
EXPECTED_ROWS = 165

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_PATH = (
 REPO_ROOT / "data" / "processed" / "macro_yield_spread_10y2y_monthly_2007-01_to_2020-09.csv"
)

스펙 2.9 값 검증 - 알려진 사건과 대조한다. (기준월, 컬럼, 최솟값, 최댓값, 설명)
SANITY_CHECKS = [
 ("2007-02-01", "spread_10y2y", -0.30, -0.05, "금융위기 직전 장단기 금리 역전"),
 ("2010-02-01", "spread_10y2y", 2.60, 3.00, "위기 후 완화정책으로 곡선 최대 급경사"),
 ("2019-08-01", "spread_10y2y", -0.05, 0.20, "2019년 곡선 평탄화(월평균은 0 근처 양수)"),
 ("2020-09-01", "GS10", 0.50, 0.90, "코로나 이후 초저금리 - 10년물 1% 미만"),
 ("2007-01-01", "GS10", 4.50, 5.00, "금융위기 이전 정상 금리 수준"),
]
```

def fetch(series: str) -> pd.DataFrame:

"""FRED에서 시리즈 하나를 받아 DataFrame으로 반환한다.

여러 시리즈를 `id=GS10,GS2` 처럼 한 번에 받는 URL은 cosd/coed(기간)를 무시하고  
일부 컬럼을 빈 값으로 내려주는 경우가 있어, 시리즈별로 따로 받아 병합한다.

pandas.read\_csv(URL)을 직접 쓰지 않고 requests를 거치는 이유: python.org 빌드  
Python은 CA 인증서가 없어 표준 urllib이 SSLCertVerificationError로 실패한다.

"""

url = (

```

 "https://fred.stlouisfed.org/graph/fredgraph.csv"
 f"?id={series}&cosd={START}&coed={END}"
)
resp = requests.get(url, timeout=30)
resp.raise_for_status()

FRED는 결측을 "."으로 내려주므로 NaN으로 받는다 (스펙 2.4).
df = pd.read_csv(io.StringIO(resp.text), na_values=["."])
df.columns = ["observation_date", series]
df["observation_date"] = pd.to_datetime(df["observation_date"])

if len(df) != EXPECTED_ROWS:
 raise ValueError(f"{series}: 행 수가 {EXPECTED_ROWS}가 아닙니다 ({len(df)}행)")
if df[series].isna().any():
 raise ValueError(f"{series}: 결측이 {df[series].isna().sum()}개 있습니다")
return df

def main() -> None:
 df = fetch(SERIES[0])
 for series in SERIES[1:]:
 df = df.merge(fetch(series), on="observation_date", how="outer")

 df = df.sort_values("observation_date").reset_index(drop=True)
 if not df["observation_date"].equals(
 pd.Series(pd.date_range(START, END, freq="MS"))
):
 raise ValueError("월이 연속적이지 않거나 중복/누락된 달이 있습니다")

 # 원자료가 소수 둘째 자리이므로 차이도 둘째 자리로 맞춘다.
 # (부동산수점 밸런이 4.76-4.88 = -0.12000000000000011 처럼 나오는 것을 막는다)
 df["spread_10y2y"] = (df["GS10"] - df["GS2"]).round(2)

 failed = []
 for date, col, lo, hi, note in SANITY_CHECKS:
 value = df.loc[df["observation_date"] == pd.Timestamp(date), col].iloc[0]
 ok = lo <= value <= hi
 print(f" [{ 'OK' if ok else '실패' }] {date[:7]} {col}={value:+.2f} (기대 {lo:+.2f}~{hi:+.2f}) - {note}")
 if not ok:
 failed.append((date, col))
 if failed:
 raise ValueError(f"값 검증 실패 - 시리즈를 잘못 받았을 수 있습니다: {failed}")

 df["observation_date"] = df["observation_date"].dt.strftime("%Y-%m-%d")
 df.to_csv(OUT_PATH, index=False)

 inverted = df[df["spread_10y2y"] < 0]
 print(f"\n저장 원료: {OUT_PATH.relative_to(REPO_ROOT)}")
 print(f"행 수: {len(df)} 기간: {df['observation_date'].iloc[0]} ~ {df['observation_date'].iloc[-1]}")
 print(f"결측: {df[['GS10', 'GS2', 'spread_10y2y']].isna().sum().sum()}개")
 for col in ["GS10", "GS2", "spread_10y2y"]:
 s = df[col].describe()
 print(
 f" {col:<12} min {s['min']:+.2f} / max {s['max']:+.2f} / "
 f"mean {s['mean']:+.3f} / std {s['std']:.3f}"
)
 print(f"\n금리차 역전(음수) 개월: {len(inverted)}개월")

```

```
if len(inverted):
 print(inverted[["observation_date", "GS10", "GS2", "spread_10y2y"]].to_string(index=False))

if __name__ == "__main__":
 main()
```

## 분석 — 본 파이프라인

---

부도확률 모형부터 최종 평가까지, 보고서 결과를 직접 만드는 코드.

**src/analysis/model.py**

---

부도확률(PD) 예측 모형

"" 부도확률(PD) 예측 모형 - XGBoost 단일 · K-fold 00F.

모형은 XGBoost 하나다 ( `decision\_log.md` #17 ①, 2026-07-29 회의 확정). 로지스틱 회귀는 폐기됐고, 그에 따라 결측 대체·표준화·더미화도 함께 폐기됐다(#13 ③·#17).

## 왜 00F(out-of-fold)인가

구조 A`(#20)는 PD 분위별 칸에 실현수익을 통계를 채운다. 이때 분위기를 만드는 PD가 같은 데이터로 학습한 모형의 in-sample 예측이면 과적합된 PD로 칸을 나누게 되어, 칸별 통계가 실제보다 잘 분리된 것처럼 보인다. 그래서 Train 안에서 K-fold를 돌려 자기 fold를 학습에 쓰지 않은 예측(00F)으로 분위 경계를 만든다.

- PD 분위 경계는 Train 00F PD로 만들고 Validation·Test에 그대로 적용한다. 재분위 금지 - 재분위하면 threshold가 "PD 얼마 이하"가 아니라 "그 표본의 상위 몇 %"가 되어 이전할 수 없다 ( `src/analysis/AGENTS.md` ).
- fold 모델은 Train의 `(K-1)/K`로 학습하고 최종 모델은 Train 전체로 학습하므로 PD 스케일이 미세하게 다르다. 이 차이가 분위별 인원을 10%에서 밀어내는데, 얼마나 밀리는지는 `oof\_diagnostics.py`의 진단 C-3이 쟁다.

## 확률보정(isotonic) - 왜 필요한가

구조 A`는 `p`를 순위가 아니라 확률 값으로 쓴다.

...

$E[XR_i] = (1 - \hat{p}_i) \cdot XR_{\text{정상},i} + \hat{p}_i \cdot \mu_{\text{부도},d(i)}$

...

`p`가 0.10인데 실제 부도율이 0.13인 구간이 있으면, 그 오차 0.03이 `(mu\_정상 - mu\_부도)` (대략 20%p)를 곱해 `E[XR]`에 60bp의 편향으로 그대로 들어간다. `E[XR]` 수준이 1~3%인 것을 감안하면 무시할 크기가 아니다. AUC로는 이 오류가 전혀 안 잡힌다 - AUC는 순위만 본다.

- 보정은 Train 00F PD로 학습한다 ( `fit\_calibrator` ). in-sample 예측으로 학습하면 과적합된 매핑을 배우게 되어 보정 자체가 무의미해진다 - 00F를 만드는 이유와 같다.
- 학습 비용이 추가로 들지 않는다. 이미 만든 `pd\_oof`가 곧 isotonic의 학습 데이터다.
- ⚠️ isotonic은 비감소(non-decreasing) 계단함수라 평탄구간(plateau)에서 동순위를 만든다. "단조변환이니 분위 배정이 그대로"는 정확히는 틀리다 - 수십만 개 PD가 수백 개 값으로 뭉치고, 분위 경계가 평탄구간 안에 떨어지면 그 구간 전체가 한 칸으로 몰린다.
- 그래서 PD의 두 역할을 나눈다 ( 근거는 진단 C-4):  
 \* 분위 경계·배정과 승인선 점수는 보정 전 PD, `E[XR]`·`Var[XR]`의 `p`만 보정 후 PD를 쓴다. 분위는 칸을 묶는 도구라 순위만 필요하고 촘촘한 쪽이 정확히 10%씩 나뉜다. 확률 값이 필요한 곳은 `E[XR]` 하나다.
- fold 모델(Train 80%)로 만든 보정을 최종 모델(Train 100%) 예측에 적용하므로 PD 스케일이 미세하게 다르다. 그 차이는 C-3이 이미 쟁다(분위 인원 이탈 최대 0.46%p).

## AUC 사용 범위

성능 보고와 처리 방식 간 상대 비교에는 써도 되지만, 승인/거절 threshold를 정하는 데는 쓰지 않는다 ( `src/analysis/AGENTS.md` ). threshold는 오직 Sharpe로 정한다.  
⚠️ 보정은 AUC를 (거의) 바꾸지 않는다 - 순위 지표라서 그렇다. 보정의 효과는 AUC가 아니라 Brier·ECE로 봐야 한다.

⚠️ 기준선은 확정 표본(#16, 723,563건) 실측 0.70대다. 문서에 남아 있는 "0.71대"는 #16 만기필터 확정 이전 값이라 인용하지 않는다 - 같은 피쳐·모델로 필터만 빼면 0.7283이 나와 \*2.1%p가 오롯이 필터 효과임이 확인됐다 ( `outputs/reports/oof\_diagnostics\_kgj.md` ). Lean 0.68은 이미 확정 표본 기준이므로(#13 ⑤), 조건변수 효과는 0.68 → 0.70이다.

""

```

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.isotonic import IsotonicRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold
from xgboost import XGBClassifier

try:
 from utils.config import load_config
except ModuleNotFoundError: # pragma: no cover
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from utils.config import load_config

#: OOF fold 수 - **모든 호출부가 이 기본값 하나를 공유한다** (#32, 2026-07-31).
#:
#: `compute_oof()`는 `n_folds`를 인자로 받지만, 호출부 5곳(`sharpe_optimizer`·
#: `final_evaluation`·`second_test_evaluation`·`oof_diagnostics`·이 모듈의 `__main__`)은
#: **일부러 넘기지 않는다.** 앞의 셋이 서로 다른 fold 수를 쓰면 K=50에서 뽑은 승자를
#: 다른 모델로 재현해 2nd Test에 적용하게 되는데, **예외가 나지 않아 조용히 틀린다.**
#: 상수 하나를 공유하면 그 어긋남이 구조적으로 불가능해진다.
#:
#: 5 → 3으로 내린 근거 (8:2 · seed 0 실험, 이슈 #32):
#:
#: | k | fold AUC | 분위 인원 이탈 | ECE | Sharpe |
#: | ---: | ---: | ---: | ---: | ---: |
#: | 2 | 0.70696 | **0.904%p** ❌ | 0.535%p | 0.2046 |
#: | 3 | 0.70928 | 0.479%p ✅ | 0.356%p | 0.2050 |
#: | 5 | 0.71004 | 0.361%p ✅ | 0.330%p | 0.2051 |
#:
#: 진단 C-1의 이전 가능성 기준은 **분위 인원 이탈 0.60%p 이내**이며 k=2는 이를 위반한다
#: (term 60m - 표본의 13.9%뿐이라 먼저 깨진다). k=3은 통과하면서 5-fold 대비 학습을
#: 33% 줄인다. K=50 분실행 기준 82분 → 55분.
DEFAULT_N_FOLDS = 3

def default_params(seed: int | None = None) -> dict:
 """XGBoost 하이퍼파라미터.

 `enable_categorical=True` + `tree_method="hist"`로 범주형과 NaN을 native 처리한다 -
 원-핫도, 결측 대체도 하지 않는다는 확정(#13 @·#17)을 코드 수준에서 지키는 부분이다.

 ⚠️ `scale_pos_weight`를 쓰지 않는다. 부도율 16.2%는 극단적 불균형이 아니고, 가중치를
 걸면 예측값이 확률이 아니게 되어 $E[XR] = (1-\hat{p}) \cdot \dots + \hat{p} \cdot \dots$ 에 그대로 쓸 수 없다(#20).
 `p`를 순위가 아니라 **확률 값**으로 쓰기 때문이다.
 """

 cfg = load_config()
 return {
 "n_estimators": 600,
 "learning_rate": 0.05,
 }

```

```

 "max_depth": 6,
 "min_child_weight": 5,
 "subsample": 0.8,
 "colsample_bytree": 0.8,
 "reg_lambda": 1.0,
 "objective": "binary:logistic",
 "eval_metric": "auc",
 "tree_method": "hist",
 "enable_categorical": True,
 "max_cat_to_onehot": 1, # 항상 분할 기반 처리 - 고카디널리티에서 원-핫 폭발 방지
 "random_state": cfg.random_seed.default if seed is None else seed,
 "n_jobs": -1,
}

```

```

def train_model(
 X_train: pd.DataFrame, y_train: pd.Series, params: dict | None = None, seed: int | None = None
) -> XGBClassifier:
 """PD 모델을 학습한다."""
 model = XGBClassifier(**(params or default_params(seed)))
 model.fit(X_train, y_train, verbose=False)
 return model

```

```

def predict_default_probability(model: XGBClassifier, X: pd.DataFrame) -> pd.Series:
 """부도확률 $\hat{p}$ 를 반환한다 (양성 클래스 = `Charged Off`)."""
 proba = model.predict_proba(X)[:, 1]
 return pd.Series(proba, index=X.index, name="pd")

```

```

def compute_oof(
 X_train: pd.DataFrame,
 y_train: pd.Series,
 n_folds: int = DEFAULT_N_FOLDS,
 params: dict | None = None,
 seed: int | None = None,
 verbose: bool = True,
) -> tuple[pd.Series, list[float], pd.Series]:
 """Train 안에서 K-fold를 돌려 OOF PD를 만든다.

 Returns

 (pd_oof, fold_auc, fold_index)
 `pd_oof`는 Train과 같은 인덱스 길이를 갖는다.
 """
 cfg = load_config()
 seed = cfg.random_seed.default if seed is None else seed

 skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=seed)
 pd_oof = pd.Series(np.nan, index=X_train.index, name="pd_oof")
 fold_index = pd.Series(-1, index=X_train.index, name="fold", dtype="int8")
 fold_auc: list[float] = []

 for k, (tr, va) in enumerate(skf.split(X_train, y_train)):
 X_tr, y_tr = X_train.iloc[tr], y_train.iloc[tr]
 X_va, y_va = X_train.iloc[va], y_train.iloc[va]

```

```

model = train_model(X_tr, y_tr, params=params, seed=seed)
p = predict_default_probability(model, X_va)

pd_oof.iloc[va] = p.to_numpy()
fold_index.iloc[va] = k
auc = roc_auc_score(y_va, p)
fold_auc.append(auc)
if verbose:
 print(f" fold {k + 1}/{n_folds} n_va={len(va):>7,} AUC={auc:.5f}")

if pd_oof.isna().any():
 raise RuntimeError("OOF 예측에 결측이 남았습니다 - fold 분할을 확인하세요.")

return pd_oof, fold_auc, fold_index

확률보정 (isotonic)

def fit_calibrator(pd_oof: pd.Series, y_train: pd.Series) -> IsotonicRegression:
 """Train **OOF** PD로 isotonic 보정을 학습한다.

 `E[XR]`이 `p`를 확률 값으로 쓰므로 보정 오차가 `E[XR]`에 직접 들어간다(모듈 docstring).
 OOF를 쓰는 것이 요점이다 - in-sample 예측으로 맞추면 과적합된 매핑을 배운다.

 `out_of_bounds="clip"`은 Validation·Test에서 Train OOF 범위를 벗어난 PD가 나올 때
 양 끝값으로 자른다. 부도확률이므로 `[0, 1]`을 벗어나서는 안 된다.

 ⚠ Platt(로지스틱) 대신 isotonic을 쓰는 이유: 표본이 43만 건(Train)이라 isotonic의
 과적합 위험이 낮고, Platt은 시그모이드 모양을 강제해 XGBoost의 왜곡을 다 못 펴기 때문이다.
 """
 return IsotonicRegression(y_min=0.0, y_max=1.0, increasing=True, out_of_bounds="clip").fit(
 pd_oof.to_numpy(dtype="float64"), y_train.to_numpy(dtype="float64")
)

def apply_calibrator(calibrator: IsotonicRegression, p_hat: pd.Series) -> pd.Series:
 """보정된 부도확률. 인덱스를 보존한다."""
 return pd.Series(
 calibrator.predict(p_hat.to_numpy(dtype="float64")),
 index=p_hat.index,
 name="pd_calibrated",
)

def calibration_table(y: pd.Series, p_hat: pd.Series, n_bins: int = 10) -> pd.DataFrame:
 """구간별 예측확률 평균 vs 실제 부도율.

 동일 폭이 아니라 동일 인원(분위) 구간으로 자른다 - PD 분포가 오른쪽으로 길게
 치우쳐 있어 동일 폭으로 자르면 상위 구간이 거의 비고, 파이프라인이 쓰는 PD 10분위와도
 기준이 어긋난다.
 """
 edges = np.unique(np.quantile(p_hat.to_numpy(dtype="float64"), np.linspace(0, 1, n_bins + 1)))
 bins = pd.cut(p_hat, bins=edges, include_lowest=True, duplicates="drop")

 frame = pd.DataFrame({"y": y, "p": p_hat, "bin": bins})
 g = frame.groupby("bin", observed=True)

```

```

out = pd.DataFrame(
 {"n": g.size(), "mean_pred": g["p"].mean(), "observed": g["y"].mean()}
)
out["gap_pp"] = (out["mean_pred"] - out["observed"]) * 100.0
return out.reset_index(drop=True).assign(bin=lambda d: d.index + 1)

def calibration_metrics(y: pd.Series, p_hat: pd.Series, n_bins: int = 10) -> dict:
 """보정 품질 지표. **AUC가 아니라 이 값들로 보정 효과를 판단한다.**

 - `brier`: $\text{mean}((\hat{p} - y)^2)$. 판별력과 보정을 함께 담는다(낮을수록 좋다).
 - `ece`: 기대보정오차 $\sum (n_{b/N}) \cdot |\text{예측평균}_b - \text{실측률}_b|$. 구간별 어긋남의 가중평균.
 - `mce`: 최대보정오차 - 최악 구간의 어긋남.
 - `bias_pp`: 전체 예측평균 - 실제 부도율. 방향(과대/과소)을 본다.
 """
 tbl = calibration_table(y, p_hat, n_bins=n_bins)
 w = tbl["n"] / tbl["n"].sum()
 gap = (tbl["mean_pred"] - tbl["observed"]).abs()
 return {
 "auc": float(roc_auc_score(y, p_hat)),
 "brier": float(np.mean((p_hat.to_numpy(dtype="float64") - y.to_numpy(dtype="float64")) ** 2)),
 "ece_pp": float((w * gap).sum() * 100.0),
 "mce_pp": float(gap.max() * 100.0),
 "bias_pp": float((p_hat.mean() - y.mean()) * 100.0),
 }

def calibration_noise_floor(
 p_hat: pd.Series, n_sim: int = 20, n_bins: int = 10, seed: int | None = None
) -> dict:
 """**완벽히 보정된 모형이라도 나오는 ECE** - 판정 기준선.

 ECE는 유한표본 잡음 때문에 0이 되지 않는다. $y \sim \text{Bernoulli}(\hat{p})$ 를 직접 생성해 재보면
 "이 표본 크기에서 보정이 완벽할 때의 ECE"가 나온다. 실측 ECE가 이 값 수준이면
 보정할 것이 없다는 뜻이고, 몇 배 크면 실제로 어긋난 것이다.

 합성 실험에서 $n=50,000$ ·10구간의 바닥값이 0.32%p였다 - 표본이 작으면 판정 기준
 0.5%p가 바닥값과 구분되지 않으므로 이 함수로 함께 보고한다.
 """
 rng = np.random.default_rng(load_config().random_seed.default if seed is None else seed)
 p = p_hat.to_numpy(dtype="float64")
 eces = [
 calibration_metrics(pd.Series(rng.binomial(1, p), index=p_hat.index), p_hat, n_bins)["ece_pp"]
 for _ in range(n_sim)
]
 return {"ece_floor_mean_pp": float(np.mean(eces)), "ece_floor_max_pp": float(np.max(eces))}

def assign_pd_quantile(
 pd_values: pd.Series,
 edges: np.ndarray,
) -> pd.Series:
 """PD 값을 **주어진 경계**로 분위에 배정한다 (1 = 최저 PD).

 경계를 인자로 받는 것이 요점이다 - Validation·Test에서 다시 분위기를 만들지 않고
 Train 00F 경계를 그대로 적용한다(`src/analysis/AGENTS.md` 재분위 금지).
 """

```

```

q = np.digitize(pd_values.to_numpy(), edges, right=True) + 1
q = np.clip(q, 1, len(edges) + 1)
return pd.Series(q, index=pd_values.index, name="pd_quantile", dtype="int8")

def make_quantile_edges(pd_train_oof: pd.Series, n_quantiles: int = 10) -> np.ndarray:
 """Train OOF PD로 분위 경계를 만든다. 내부 경계만 돌려준다(길이 `n_quantiles - 1`)."""
 qs = np.linspace(0, 1, n_quantiles + 1)[1:-1]
 return np.quantile(pd_train_oof.to_numpy(), qs)

def quantile_edges_by_term(
 pd_oof: pd.Series, term: pd.Series, n_quantiles: int = 10
) -> dict[float, np.ndarray]:
 """term별로 **따로** PD 분위 경계를 만든다 (`decision_log.md` #20 권고).

 전체 공통 분위로 자르면 **60개월·저PD 칸이 거의 빈다** - 60m는 위험군이라 고PD 구간에
 쏠리기 때문이다. #16으로 60개월 비중이 25.1% → 13.9%로 줄어 이 쏠림이 더 심해졌다.

 ⚠ 입력은 **보정 전** OOF PD다 - isotonic 평탄구간이 경계를 삼켜 칸 인원이 기운다
 (진단 C-4). 보정된 PD는 `E[XR]`의 `p`로만 쓴다.
 """
 return {
 float(t): make_quantile_edges(pd_oof[term == t], n_quantiles)
 for t in sorted(term.dropna().unique())
 }

def assign_quantile_by_term(
 pd_values: pd.Series, term: pd.Series, edges_by_term: dict[float, np.ndarray]
) -> pd.Series:
 """term별 경계로 분위기를 배정한다. Validation·Test에서 **재분위하지 않는다**."""
 out = pd.Series(np.nan, index=pd_values.index, name="pd_quantile")
 for t, edges in edges_by_term.items():
 mask = term == t
 if mask.any():
 out.loc[mask] = assign_pd_quantile(pd_values.loc[mask], edges).to_numpy()
 return out.astype("Int8")

if __name__ == "__main__":
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from preprocessing.preprocessor import build_feature_table, split_from_manifest

 print("피쳐 테이블 구성 중...")
 X, y, meta = build_feature_table()
 parts = split_from_manifest(X, y, meta)
 X_tr, y_tr, _ = parts["train"]
 X_va, y_va, _ = parts["validation"]
 print(f"Train {len(X_tr):,} / Validation {len(X_va):,} 피쳐 {X.shape[1]}개")

 print(f"\n[Train {DEFAULT_N_FOLDS}-fold OOF]")
 pd_oof, fold_auc, _ = compute_oof(X_tr, y_tr)
 print(f"fold AUC 평균 {np.mean(fold_auc):.5f} (표준편차 {np.std(fold_auc):.5f})")
 print(f"OOF 전체 AUC {roc_auc_score(y_tr, pd_oof):.5f}")

```

```

print("\n[최종 모델 - Train 전체 학습 → Validation 평가]")
final = train_model(X_tr, y_tr)
p_va = predict_default_probability(final, X_va)
print(f" Validation AUC {roc_auc_score(y_va, p_va):.5f} (확정 표본 기준 0.70대 - oof_diagnostics_kgj.md)")

print("\n[확률보정 - isotonic, Train OOF로 학습 → Validation 적용]")
calibrator = fit_calibrator(pd_oof, y_tr)
p_va_cal = apply_calibrator(calibrator, p_va)
for name, p in (("보정 전", p_va), ("보정 후", p_va_cal)):
 m = calibration_metrics(y_va, p)
 print(f" {name} AUC {m['auc']:.5f} Brier {m['brier']:.5f} "
 f"ECE {m['ece_pp']:.3f}%p MCE {m['mce_pp']:.3f}%p 편향 {m['bias_pp']:+.3f}%p")

```

**src/analysis/realized\_return.py**

---

실현수익률 · 초과수익률(XR)

```
"""실현수익률 · 초과수익률(XR) - 구조 A'의 구현 (**가정 전부 확장**, `decision_log.md` #22).
```

```
`decision_log.md` #20이 확정한 구조를 계산 가능한 형태로 옮긴 것이다.
```

```
...
```

```
E[XR_i] = (1 - $\hat{p}_i$) · XR_정상,i + $\hat{p}_i$ · mu_부도,d(i)
 ↑ 건별 계약 현금흐름 ↑ 그룹 평균 (PD 분위 × term)
```

```
...
```

```
확정 가정 3건 (2026-07-31 회의, #22 - 옛 "미확정 3건")
```

```
항목	확정값
국제 금리 기준	**◎발행시점 고정** - 단 역할 3분리: `rf`=GS3/GS5 · 계약분 재투자=◎ · 실현분 재투자·역할인=GS1M 실제경로(#22 ①)
투자자 서비스수수료	**0% 미반영** (#22 ② - 보고서 한계에 명시)
조기상환 보정	**건별 실현 현금흐름 반영** (#22 ③ - `realized_basis="cashflow"` 기본값, `normal_cell_stats()`가 칸별 `Δ_
조기상환,d`·`var_정상,d`를 추정) |
```

조기상환을 반영하지 않으면 정상상환분 `R`이 과대추정된다 - 국제 재투자 가정 아래서

조기상환은 `R`을 낮춘다(12% 대출을 12개월에 회수하면 남은 24개월을 약 2% 국제로 굴려야

한다). 실측 보정폭은 36m +1.06%p / 60m +2.24%p. 보정 전에는 `var\_정상 = 0`이라

`Var[XR]`이 부도 항만 반영했다 - 양쪽 다 해소됐다.

가정을 바꿔도 \*\*모형 재학습은 불필요하고 칸별 통계표와 threshold만 재계산\*\*하면 된다(#19·#20).

산출물 파일명의 `provisional` 라벨은 기존 산출물과의 연속성을 위해 유지한다(#22 파급).

```
손실값을 상수로 박지 않는다
```

`src/analysis/AGENTS.md`의 구현 조건이다. 부도 손실은 하드코딩된 -100%나 -45%가 아니라

\*\*데이터에서 칸별로 추정한 `mu\_부도,d`\*\*이며, 재투자 가정·수수료·보정항은 전부

`ReturnAssumptions`로 주입받는다.

```
🚧 여기 쓰는 컬럼은 전부 사후(post-approval)다
```

`total\_pymnt`·`recoveries`·`last\_pymnt\_d` 등은 \*\*결과변수 쪽\*\*이라 피쳐 테이블에 넣으면

누수다(`src/preprocessing/AGENTS.md`). 이 모듈의 산출물은 threshold·Sharpe 단계에서

`id`로 결합해 쓴다.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, replace
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
try:
```

```
 from utils.config import load_config
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
 import sys
```

```
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
 from utils.config import load_config
```

```
TREASURY_FILENAME = "us_treasury_GS3_GS5_monthly_2007-06_to_2020-09.csv"
```

```
부도 회수액 수령 시점 - 최종납입 K개월 뒤 6개월.
```

```
⚠ 이 규칙은 아직 어느 문서에도 근거가 기록돼 있지 않다(#17 작업 10, B팀).
```

```
RECOVERY_LAG_MONTHS = 6
```

```
@dataclass(frozen=True)
```

```
class ReturnAssumptions:
```

```
 """수익률 계산 가정 - **전부 주입받는다.** 상수로 박지 않는다(`src/analysis/AGENTS.md`)."""
```

```
 reinvest: str = "treasury"
```

```
 """`"treasury"`: 잔존기간 매칭 국제 재투자(#18 확정) / `"cash"`: 0% 재투자(민감도용)."""
```

```
 treasury_basis: str = "issue_fixed"
```

```
 """① 발행시점 고정 - **계약 현금흐름 쪽 확정값**(#22 ①). ③실제경로·⑥상수는 미구현.
```

```
 실현 현금흐름(`realized_basis="cashflow"`)의 재투자·역할인은 이 값과 무관하게
```

```
 GSIM 실제경로를 쓴다 - #22 ①의 역할 3분리 참고.
```

```
 """
```

```
 servicing_fee_annual: float = 0.0
```

```
 """투자자 서비스수수료 연율. **0% 확정 - 미반영**(#22 ②, 보고서 한계에 명시)."""
```

```
 prepayment_adjustment: float = 0.0
```

```
 """`contract_return()`에 더하는 **스칼라** 보정항. 기본 0을 유지한다.
```

```
 ✓ 조기상환 보정은 2026-07-31부터 **간별로 데이터에서 추정**한다
```

```
 (`normal_cell_stats()`의 `Δ_조기상환,d`) - 이 스칼라는 민감도 실험용 수동 오버라이드로만
 남겨둔다. #20이 남긴 "B팀 1순위"는 간별 추정으로 해소됐다.
```

```
 """
```

```
 realized_basis: str = "cashflow"
```

```
 """실현 `R`의 기준. `"cashflow"`: 건별 실제 현금흐름 + GSIM 실제경로 재투자(B팀 명세).
```

```
 옛 `"contract"`(정상상환을 계약대로 가정)는 FP 36개월의 66%가 조기상환이라
```

```
 36m +1.06%p / 60m +2.24%p 과대추정이었다 - 되살리지 않는다.
```

```
 """
```

```
def label(self) -> str:
```

```
 """산출물 파일명·컬럼에 남길 가정 표기. 재투자 스위치는 통계표까지 바꾼다(#18).
```

```
 ⚠ `realized_basis`가 들어 있어 **조기상환 보정 전후 산출물이 서로 뒤어쓰지 않는다.**
```

```
 """
```

```
 fee = f"fee{self.servicing_fee_annual * 100:g}pct"
```

```
 return f"provisional_{self.reinvest}_{self.treasury_basis}_{fee}_{self.realized_basis}"
```

```

```

```
국제 곡선
```

```

```

```
def load_treasury() -> pd.DataFrame:
```

```
 """월별 국제수익률(연율 %). 인덱스는 절대 월 서수(`year*12 + month`)다."""
```

```
 path = load_config().paths.data_processed / TREASURY_FILENAME
```

```
 if not path.exists():
```

```
 raise FileNotFoundError(f"국제 데이터가 없습니다: {path}")
```

```
 t = pd.read_csv(path, parse_dates=["observation_date"])
```

```
t["month_ord"] = t["observation_date"].dt.year * 12 + t["observation_date"].dt.month
return t.set_index("month_ord")[["GS3", "GS5"]].sort_index()
```

```
def issue_risk_free_rate(
 issue_month_ord: pd.Series, term_months: pd.Series, treasury: pd.DataFrame | None = None
) -> pd.Series:
```

"""발행시점 x 만기매칭 무위험수익률 `rf` (\*\*유효연율\*\*, 소수).

`decision\_log.md` #18 확정 - 36개월은 `GS3`, 60개월은 `GS5`. 고정 상수가 아니므로  
`config.yaml`의 `risk\_free\_rate.value`는 읽지 않는다(계속 `null`이다).

⚠️ \*\*FRED의 GS3·GS5는 반기복리 bond-equivalent yield다\*\* - `y/100`으로 쓰면 유효연율을  
과소평가하고, 그만큼 `XR = R - rf`가 과대평가된다. `R`은 월별 현금흐름을 굴려 만든  
유효연율이므로 같은 기준으로 맞춘다(B팀 명세 8번과 동일).

$$rf = (1 + y/200)^2 - 1$$

실측 영향은 작지만 전 건에 \*\*한 방향으로\*\* 걸린다 - 평균 +0.30bp, 최대 +6.25bp.

"""

```
t = treasury if treasury is not None else load_treasury()
series_by_term = load_config().risk_free_rate.series or {36: "GS3", 60: "GS5"}
```

```
out = pd.Series(np.nan, index=issue_month_ord.index, dtype="float64")
for term, col in series_by_term.items():
 mask = term_months == term
 if not mask.any():
 continue
 out.loc[mask] = t[col].reindex(issue_month_ord.loc[mask]).to_numpy()
return np.power(1.0 + out / 200.0, 2.0) - 1.0
```

```
def _monthly_rate(annual_rate: pd.Series | np.ndarray) -> np.ndarray:
 """연율 → 월율, `(1+r)^(1/12) - 1`."""
 return np.power(1.0 + np.asarray(annual_rate, dtype="float64"), 1.0 / 12.0) - 1.0
```

```

정상상환 - 건별 계약 현금흐름

```

```
def contract_return(
 installment: pd.Series,
 funded_amnt: pd.Series,
 term_months: pd.Series,
 reinvest_rate: pd.Series,
 assumptions: ReturnAssumptions = ReturnAssumptions(),
) -> pd.Series:
```

"""정상상환 건의 \*\*계약\*\* 실현수익률 `R\_계약` (연율).

계약대로 만기까지 매달 `installment`를 받아 잔존기간 매칭 국채에 재투자한다고 본다.  
매달 같은 금액이므로 미래가치는 연금 증가 공식으로 닫힌 형태가 된다.

$$W = \text{installment} \cdot \sum_{m=1..T} (1+i)^{(T-m)} = \text{installment} \cdot ((1+i)^T - 1) / i$$

$$R = (W / P)^{(12/T)} - 1 \quad \dots \text{\#18}$$

`i`는 월 재투자율이다. `reinvest="cash"`(0% 재투자)면 `W = installment \* T`가 되어  
#18이 "이중 부과"라고 지적한 옛 관례가 그대로 재현된다 - 민감도 병기용이다.

예) `P=10,000`, `installment=332.14` (36개월 12%), 재투자 2%:

`i=0.00165`, `W = 332.14 × 37.06 = 12,309` → `R = (1.2309)^(1/3) - 1 = \*\*7.17%\*\*`.

0% 재투자면 `W = 11,957` → `R = \*\*6.14%\*\*`. 차이 **+103bp**가 재투자 가정 효과다

(전수 실측 평균 +107.5bp와 같은 크기 - `realized\_return\_sensitivity.py`).

**\*\*정합성 검증\*\***: 대출금리 = 국채금리인 \*무위험\* 등가 대출\*을 넣으면 `XR = R - rf`가

**\*\*정확히 0.000bp\*\***로 나온다. #18이 국채 재투자자를 택한 근거가 이것이다 - 0% 재투자에서는

같은 대출이 -0.96%p로 나와 무위험 자산에 벌점이 붙는다.

"""

P = funded\_amnt.to\_numpy(dtype="float64")

A = installment.to\_numpy(dtype="float64")

T = term\_months.to\_numpy(dtype="float64")

if assumptions.reinvest == "cash":

W = A \* T

elif assumptions.reinvest == "treasury":

i = \_monthly\_rate(reinvest\_rate)

with np.errstate(divide="ignore", invalid="ignore"):

factor = np.where(i == 0, T, (np.power(1.0 + i, T) - 1.0) / i)

W = A \* factor

else:

raise ValueError(f"알 수 없는 reinvest 가정: {assumptions.reinvest!r}")

if assumptions.servicing\_fee\_annual:

W = W \* np.power(1.0 - assumptions.servicing\_fee\_annual, T / 12.0)

with np.errstate(divide="ignore", invalid="ignore"):

R = np.power(W / P, 12.0 / T) - 1.0

R = np.where(P > 0, R, np.nan)

return pd.Series(R, index=funded\_amnt.index, name="R\_contract") + assumptions.prepayment\_adjustment

# -----

# 부도 - 건별 실현 현금흐름

# -----

def realized\_return\_defaulted(

df: pd.DataFrame,

reinvest\_rate: pd.Series,

assumptions: ReturnAssumptions = ReturnAssumptions(),

) -> pd.Series:

"""부도(`Charged Off`) 건의 **\*\*실현\*\*** 수익률 `R` (연율).

LC 데이터에 월별 납입 내역이 없으므로, 총 수령액을 다음처럼 배분한다

(`realized\_return\_sensitivity.py`가 이 가정의 민감도를 쟀다 - 대안 배분과의 차이가 정상/부도 모두 50bp 이내였다).

- 마지막 납입월 `K`에 `last\_pymnt\_amnt`를 받고,

- 나머지 `C\_regular - last\_pymnt\_amnt`를 `1..K-1`에 균등 배분하고,

- 순회수액 `C\_recovery`는 `K + 6`개월에 받는다.

각 수령액을 만기 `T`까지 국채로 굴려 `W`를 만들고 `R = (W/P)^(12/T) - 1`을 쫓다.

`K + 6 > T`이면 지수가 음수가 되어 **\*\*역할안\*\***되는데, 만기 후 수령분이라 맞는 처리다.

필요 컬럼: `funded\_amnt`, `term`, `K`, `total\_pymnt`, `recoveries`,  
`collection\_recovery\_fee`, `last\_pymnt\_amnt`.

"""

```

P = df["funded_amnt"].to_numpy(dtype="float64")
T = df["term"].to_numpy(dtype="float64")
K = df["K"].to_numpy(dtype="float64")
L = df["last_pymnt_amnt"].fillna(0).to_numpy(dtype="float64")
C_reg = (df["total_pymnt"].fillna(0) - df["recoveries"].fillna(0)).to_numpy(dtype="float64")
C_rec = (
 df["recoveries"].fillna(0) - df["collection_recovery_fee"].fillna(0)
).to_numpy(dtype="float64")

if assumptions.reinvest == "cash":
 W = C_reg + C_rec
elif assumptions.reinvest == "treasury":
 i = _monthly_rate(reinvest_rate)
 one = 1.0 + i

 # K>=2: 앞선 K-1개월에 균등배분 A, 마지막 달에 L
 n_pre = np.maximum(K - 1.0, 0.0)
 A = np.where(n_pre > 0, (C_reg - L) / np.where(n_pre > 0, n_pre, 1.0), 0.0)
 with np.errstate(divide="ignore", invalid="ignore"):
 # $\sum_{t=1}^{K-1} (1+i)^{(T-t)} = (1+i)^{(T-K+1)} \cdot ((1+i)^{(K-1)} - 1) / i$
 geo = np.where(i == 0, n_pre, (np.power(one, n_pre) - 1.0) / i)
 W_pre = A * np.power(one, T - K + 1.0) * geo
 W_last = L * np.power(one, T - K)

 # K<=1: 전액을 K월에 받은 것으로 본다 (배분할 앞선 달이 없다)
 lump = C_reg * np.power(one, T - np.maximum(K, 0.0))
 W = np.where(K >= 2, W_pre + W_last, lump)

 W = W + C_rec * np.power(one, T - K - RECOVERY_LAG_MONTHS)
else:
 raise ValueError(f"알 수 없는 reinvest 가정: {assumptions.reinvest!r}")

if assumptions.servicing_fee_annual:
 W = W * np.power(1.0 - assumptions.servicing_fee_annual, T / 12.0)

with np.errstate(divide="ignore", invalid="ignore"):
 ratio = np.where(P > 0, W / P, np.nan)
 R = np.where(ratio >= 0, np.power(np.abs(ratio), 12.0 / T) - 1.0, np.nan)
return pd.Series(R, index=df.index, name="R_realized")

```

```

칸별 통계표 · $E[XR]$ · $Var[XR]$

```

```

def default_cell_stats(
 xr_defaulted: pd.Series,
 pd_quantile: pd.Series,
 term_months: pd.Series,
 min_count_mean: int = 100,
 min_count_var: int = 300,

```

```

) -> pd.DataFrame:

```

```

"""칸별(`PD` 분위 × term) 부도 초과수의 통계표 - `mu_부도,d` · `var_부도,d`.
```

``var_부도``는 표본이 작은 칸에서 불안정하다(상대오차  $\approx \sqrt{2/(n-1)}$ ) -  $n=300$ 에서 8.2%,  $n=50$ 에서 20.2%). 그래서 \*\*분산에는 평균보다 엄한 최소 표본수를 요구하고, 미달 칸은 pooled 분산으로 축소추정\*\*한다(`src/analysis/AGENTS.md`). 분산이 과소추정된 칸이 체계적으로 승인되는 선택편향을 막기 위한 것이다.

```

"""
frame = pd.DataFrame(
 {"xr": xr_defaulted, "q": pd_quantile, "term": term_months}
).dropna(subset=["xr"])

g = frame.groupby(["term", "q"], observed=True)["xr"]
stats = pd.DataFrame({"n": g.size(), "mu": g.mean(), "var": g.var(ddof=1)})

pooled_var = frame["xr"].var(ddof=1)
stats["var_raw"] = stats["var"]
stats["shrunk"] = stats["n"] < min_count_var
stats.loc[stats["shrunk"], "var"] = pooled_var
stats["mu_unreliable"] = stats["n"] < min_count_mean
stats["pooled_var"] = pooled_var
return stats

def normal_cell_stats(
 xr_contract: pd.Series,
 xr_realized: pd.Series,
 pd_quantile: pd.Series,
 term_months: pd.Series,
 min_count_mean: int = 100,
 min_count_var: int = 300,
) -> pd.DataFrame:
 """ 칸별 **조기상환 보장** `Δ` 초기상환, d`와 `var` 정상, d` - 정상상환(FP) 건에서만 만든다.

 #20이 "B팀 1순위"로 남긴 미결의 구현이다(#22 ③으로 확정). 계약대로 만기까지 갚는다고 본 `xr_contract`가
 실제(`xr_realized`)보다 얼마나 높은지를 칸별 평균으로 잡는다.

 Δ_d = mean_{i ∈ d, FP}(XR_계약, i - XR_실현, i) → 점수용 보장항
 var_정상, d = var_{i ∈ d, FP}(XR_실현, i) → Var[XR]의 정상 항

 ## 왜 칸별 실현값을 그대로 점수에 쓰지 않는가

 개별 대출이 **몇 개월에 조기상환할지는 승인 시점에 알 수 없다.** `XR_실현, i`를 점수
 `E[XR_i]`에 넣으면 미래를 보고 고르는 셈이라 누수다. 그래서 **건별 계약 현금흐름(A`의
 핵심)은 유지하고, 조기상환은 칸별 평균 보장으로만** 넣는다 - `int_rate` 산포(칸별 sd
 중앙값 2.85%p)는 보존되고 조기상환의 체계적 효과만 차감된다.

 실측 보장폭: 36개월 **+1.06%p**, 60개월 **+2.24%p** (FP 36m의 66.03%가 조기상환).

 `var_정상`은 `default_cell_stats()`와 같은 이유로 표본이 작은 칸에서 pooled로 축소한다.
 보장 전에는 이 값이 **0으로 고정**돼 있어 `q_score` 분모가 부도 항만 반영했다.
 """
 frame = pd.DataFrame({
 "gap": xr_contract - xr_realized,
 "xr": xr_realized,
 "q": pd_quantile,
 "term": term_months,
 }).dropna(subset=["gap", "xr"])

 g = frame.groupby(["term", "q"], observed=True)
 stats = pd.DataFrame({
 "n": g.size(),
 "prepay_adj": g["gap"].mean(),
 "var": g["xr"].var(ddof=1),

```

```

})

pooled_var = frame["xr"].var(ddof=1)
pooled_adj = frame["gap"].mean()
stats["var_raw"] = stats["var"]
stats["shrunk"] = stats["n"] < min_count_var
stats.loc[stats["shrunk"], "var"] = pooled_var
stats.loc[stats["n"] < min_count_mean, "prepay_adj"] = pooled_adj
stats["pooled_var"] = pooled_var
stats["pooled_adj"] = pooled_adj
return stats

def expected_excess_return(
 p_hat: pd.Series,
 xr_normal: pd.Series,
 mu_default: pd.Series,
) -> pd.Series:
 """`E[XR_i] = (1 - p̂)·XR_정상,i + p̂·mu_부도,d(i)` (#20 구조 A').

 `xr_normal`에는 **조기상환 보정이 이미 반영된 값**을 넣는다
 (`xr_contract - Δ_조기상환,d`, `normal_cell_stats()` 참고).
 """
 return (1.0 - p_hat) * xr_normal + p_hat * mu_default

def variance_excess_return(
 p_hat: pd.Series,
 xr_normal: pd.Series,
 mu_default: pd.Series,
 var_default: pd.Series,
 var_normal: pd.Series | float = 0.0,
) -> pd.Series:
 """총분산의 법칙 - **교차항을 빼뜨리지 않는다.**

$$Var[XR] = (1-p) \cdot var_정상 + p \cdot var_부도 + p(1-p) \cdot (mu_정상 - mu_부도)^2$$

 마지막 항이 지배적이다. 예시(`p=0.10`, `mu_정상=+5%/sd 3%`, `mu_부도=-40%/sd 20%`)에서
 교차항이 총분산의 **79%**를 차지한다. 빼뜨리면 sd가 15.2% → 6.9%로 축소되고
 `p(1-p)`에 비해 편향이 걸려 **랭킹 순서가 바뀐다** (#20).

 `var_정상`은 `sharpe_optimizer`가 `normal_cell_stats()`의 칸별 추정값을 주입한다(#22 ③).
 기본값 0.0은 보정 전 산출물 재현·민감도 실험용으로만 남아 있다.
 """
 gap = xr_normal - mu_default
 return (1.0 - p_hat) * var_normal + p_hat * var_default + p_hat * (1.0 - p_hat) * gap**2

def q_score(expected_xr: pd.Series, variance_xr: pd.Series) -> pd.Series:
 """`q = E[XR] / √Var[XR]` - **확정된 승인선 랭킹 기준** (#21 ①, 2026-07-31 회의).

 ⚠ 개별 대출 `q` 최대화는 포트폴리오 Sharpe 최대화와 같은 문제가 아니다.
 어느 기준을 쓰든 threshold는 **Validation 실험 XR로 계산한 실제 Sharpe** 그리드서치로
 정한다(`src/analysis/AGENTS.md`).
 """
 sd = np.sqrt(variance_xr.clip(lower=0))
 return (expected_xr / sd.replace(0, np.nan)).rename("q_score")

```

```

def build_excess_returns(
 outcome: pd.DataFrame, assumptions: ReturnAssumptions = ReturnAssumptions()
) -> pd.DataFrame:
 """ 건별 `rf` · `XR_정상` (계약) · `XR_부도` (실현) · `XR_실현` .

 `XR_정상` 은 **부도 건에도 정의된다** - "계약대로 갚았다면 얼마였을까" 라서 실현 여부와
 무관하게 계산되며, $E[XR] = (1-\hat{p}) \cdot XR_{\text{정상}} + \hat{p} \cdot \mu_{\text{부도}}$ 의 첫 항이 바로 그 값이다.

 `xr_realized` 는 그와 달리 **실제로 벌어진 결과**다 - **정상·부도 모두 건별 실제 현금흐름**
 으로 계산한다(`realized_return_cashflow.py`, B팀 명세). Sharpe는 기대값이 아니라
 이 값으로 계산한다(`src/analysis/AGENTS.md`).

 ✅ **조기상환이 반영된다** (2026-07-31). 이전에는 정상상환 건의 `xr_realized`를 계약
 현금흐름으로 두어 36m 1.06% / 60m 2.24% 과대추정이었다 - FP 36개월의
 66.03% 가 조기상환**이기 때문이다(#20 B팀 1순위 해소).

 반환 열

 `xr_normal`
 계약 기준 정상상환 XR. 점수용이며, 쓸 때는 칸별 `Δ_조기상환`을 빼서 쓴다
 (`normal_cell_stats()`). 여기서 빠지 않는 것은 보정항이 **Train에서만** 추정돼야
 하기 때문이다 - 이 함수는 Train/Validation을 모른다.
 `xr_realized`
 정상·부도 모두 **실현** 현금흐름 XR. Sharpe 계산용.
 `xr_default` / `xr_normal_realized`
 `xr_realized`를 부도 / 정상으로 각각 마스킹한 것. 칸별 통계표 산출용.
 """
 from analysis.realized_return_cashflow import (
 build_cashflow_schedule,
 realized_return_actual,
)

 rf = issue_risk_free_rate(outcome["issue_month_ord"], outcome["term"])

 r_contract = contract_return(
 installment=outcome["installment"],
 funded_amnt=outcome["funded_amnt"],
 term_months=outcome["term"],
 reinvest_rate=rf,
 assumptions=assumptions,
)
 xr_normal = r_contract - rf

 if assumptions.reinvest == "cash":
 # 민감도 병기용 0% 재투자 - 실현분도 같은 관례로 맞춘다(#18).
 r_realized = realized_return_defaulted(
 outcome, reinvest_rate=rf, assumptions=assumptions
).where(outcome["is_default"] == 1, r_contract)
 else:
 schedule = build_cashflow_schedule(outcome)
 r_realized = realized_return_actual(schedule)["R"]

 xr_realized = r_realized - rf
 is_def = outcome["is_default"] == 1

```

```

return pd.DataFrame(
 {
 "rf": rf,
 "xr_normal": xr_normal,
 "xr_default": xr_realized.where(is_def),
 "xr_normal_realized": xr_realized.where(~is_def),
 "xr_realized": xr_realized,
 "term": outcome["term"],
 "int_rate": outcome["int_rate"],
 "is_default": outcome["is_default"],
 }
)

```

```
def build_return_inputs(
```

```
 csv_path: Path | None = None, verify_sample: bool = True
```

```
) -> pd.DataFrame:
```

```
 """수익률 계산에 필요한 **사후 컬럼**을 분석 표본(723,563건)에 맞춰 로드한다.
```

```
 반환 프레임은 피쳐 테이블과 **같은 인덱스**를 갖는다 - `id`로 조인하지 않아도
```

```
 `loc`으로 정렬이 맞는다. `K`는 발행 → 최종납입 개월 수다.
```

```
 `csv_path`는 train 원본 대신 다른 CSV를 읽을 때만 준다(`loader.second_test_path()`).
```

```
 표본 건수 검증(723,563)은 train 기준이므로 `verify_sample=False`를 함께 준다.
```

```
 ⚠ 이 프레임을 모델 입력에 섞지 않는다(`src/preprocessing/AGENTS.md` 누수 방지).
```

```
 """
```

```
import sys
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from preprocessing.loader import filter_analysis_sample, load_raw_loans
```

```
cols = [
```

```
 "id", "loan_status", "term", "issue_d", "funded_amnt", "installment", "int_rate",
```

```
 "total_pymnt", "recoveries", "collection_recovery_fee",
```

```
 "last_pymnt_amnt", "last_pymnt_d",
```

```
]
```

```
raw = load_raw_loans(usecols=cols, csv_path=csv_path)
```

```
s = filter_analysis_sample(raw, verify=verify_sample)
```

```
s["term"] = s["term"].astype(str).str.extract(r"(\d+)")[0].astype(float)
```

```
s["int_rate"] = pd.to_numeric(
```

```
 s["int_rate"].astype(str).str.replace("%", "", regex=False).str.strip(), errors="coerce"
```

```
)
```

```
issue = pd.to_datetime(s["issue_d"], format="%b-%Y", errors="coerce")
```

```
last = pd.to_datetime(s["last_pymnt_d"], format="%b-%Y", errors="coerce")
```

```
s["issue_month_ord"] = issue.dt.year * 12 + issue.dt.month
```

```
s["K"] = ((last.dt.year * 12 + last.dt.month) - s["issue_month_ord"]).clip(lower=0)
```

```
s["is_default"] = (s["loan_status"] == "Charged Off").astype("int8")
```

```
return s
```

```
def cash_reinvestment(assumptions: ReturnAssumptions) -> ReturnAssumptions:
```

```
 """민감도용 0% 재투자 가정 (#18 병기 확정).
```

```
 ⚠ 스위치 하나로 끝나지 않는다 - `mu`, `var`가 `XR`에서 산출되므로 **칸별 통계표까지
```

```
다시 만든다.** 산출물 파일명·컬럼에 `label()`을 남겨 어느 가정인지 추적한다.
""
return replace(assumptions, reinvest="cash")
```

src/analysis/realized\_return\_cashflow.py

---

실현 현금흐름 기반 수익률

"""\*\*실현 현금흐름 기반 수익률\*\* - B팀(유명곤) 명세의 재현 구현.

`realized\_return.py`의 `contract\_return()`은 \*\*계약대로\*\* 만기까지 갚는다고 보고 `R`을 계산한다. 그런데 실측하면 \*\*Fully Paid 36개월 대출의 66.03%가 만기 전에 상환\*\*됐고 (K 중앙값 28개월), 그만큼 `R`이 과대추정된다 - 36m  $1.06\%$  / 60m  $2.24\%$ .  
`decision\_log.md` #20이 "B팀 1순위"로 남긴 조기상환 보정이 바로 이 항목이다.

B팀이 건별 실제 현금흐름을 재구성해 산출물을 냈고  
(`lending\_club\_realized\_return\_rf\_xr\_eligible\_2020-10.csv`, 722,353건), 이 모듈은 \*\*같은 계산을 코드로 재현\*\*한다.

## 왜 CSV를 그냥 쓰지 않고 재현하는가

B팀 CSV는 \*\*train 원본 id만\*\* 담고 있다. 최종 Test로 쓰는  
`lending\_club\_2020\_test\_2nd.csv` (481,833건)에 대응하는 행이 \*\*한 건도 없어서\*\*, CSV만으로는 Test에서 `XR`을 계산할 수 없다. 그래서 입력 CSV를 갈아 끼울 수 있는 형태로 구현하고, train에 대해서는 \*\*B팀 CSV와 대조해 일치를 확인\*\*한다(`verify\_against\_teamb()`).

## 계산 규칙 (B팀 노트북 `realized\_return\_preprocessing\_step\_by\_step.ipynb`)

월별 납입 내역이 원본에 없으므로 총액을 다음처럼 배치한다.

```
`K` (발행 → 최종납입 개월)	정규 현금흐름 배치
`K = 0`	전액을 `t=0`에
`K = 1`	전액을 `t=1`에
`K ≥ 2`	`C_regular - L`을 `t=1..K-1`에 균등, 마지막 납입액 `L`을 `t=K`에
```

- 순Recovery `C\_recovery = recoveries - collection\_recovery\_fee`는 `t = K+6`에 일시 배치.  
- 정상 납입이 없는 Recovery-only `Charged Off`는 `t=6`에 배치한다(\*\*544건\*\*).

\*\*재투자·역할인은 `GS1M` 실제 경로\*\*를 쓴다 - 수령월에는 이자를 안 주고 \*\*다음 달부터\*\* 계약만기까지 굴리며, 만기 이후 수령분(회수금)은 같은 경로로 만기 시점까지 \*\*역할인\*\*한다.

$$W(T) = \sum_m CF_m \cdot \exp(\Lambda_{\text{만기}} - \Lambda_m), \quad \Lambda = \text{누적 로그성장률}$$
$$R = (W(T) / \text{funded\_amnt})^{(12/T)} - 1$$

⚠ \*\*이 재투자 기준은 `contract\_return()`과 다르다.\*\* 저쪽은 @발행시점 고정 국채 (`rf` = GS3/GS5)를 쓰고, 여기는 @실제경로(GS1M)다 - #20 미확정 3건 중 "국채 금리 기준"에 해당한다. \*\*`rf` 자체(초과수익률의 차감항)는 양쪽 모두 발행시점 × 만기매칭 GS3/GS5로 동일하다\*\*(#18 확정). 실측 대조에서 부도 건 `XR`은 두 방식이 36m  $-0.0002$  / 60m  $+0.0034$ 로 거의 같았다 - 차이는 대부분 조기상환에서 온다.

=====

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
try:
```

```
 from utils.config import load_config
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
 import sys
```

```

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from utils.config import load_config

GS1M_FILENAME = "us_treasury_GS1M_monthly_2007-07_to_2025-09.csv"

#: 회수금 수령 시점 - 최종납입 K개월 뒤 6개월(`realized_return.RECOVERY_LAG_MONTHS`와 같은 값).
RECOVERY_LAG_MONTHS = 6

#: 원본에서 읽어야 하는 컬럼. 전부 **사후(post-approval)** 라 피쳐로 쓰지 않는다.
CASHFLOW_COLUMNS = [
 "id", "loan_status", "funded_amnt", "term", "issue_d", "last_pymnt_d",
 "total_pymnt", "last_pymnt_amnt", "recoveries", "collection_recovery_fee",
 "installment",
]

#: B팀 산출물의 검증 기대값 (train 원본 기준).
TEAMB_EXPECTED = {"eligible": 722_353, "excluded": 1_210, "recovery_only": 544}

GS1M 월수익률 경로

def load_gs1m_monthly_return() -> pd.Series:
 """`GS1M` → 월 등가수익률. 인덱스는 절대 월 서수(`year*12 + month`).

 FRED `GS1M`은 **bond-equivalent yield(%)** 라 12로 나누지 않는다. 반년 복리 관례를
 풀어 `(1 + GS1M/200)^(1/6) - 1`로 월율을 만든다(B팀 명세).
 """
 path = load_config().paths.data_processed / GS1M_FILENAME
 if not path.exists():
 raise FileNotFoundError(
 f"GS1M 데이터가 없습니다: {path}\n"
 "`python src/preprocessing/fetch_treasury_gs1m.py`로 받으세요."
)
 g = pd.read_csv(path, parse_dates=["observation_date"])
 if g["GS1M"].isna().any():
 raise ValueError("GS1M에 결측이 있습니다 - 수집 스크립트를 다시 확인하세요.")
 idx = g["observation_date"].dt.year * 12 + g["observation_date"].dt.month
 out = pd.Series(
 (1.0 + g["GS1M"].to_numpy() / 200.0) ** (1.0 / 6.0) - 1.0,
 index=pd.Index(idx, name="month_ord"), name="monthly_return",
).sort_index()
 if not out.index.equals(pd.RangeIndex(out.index[0], out.index[-1] + 1)):
 raise ValueError("GS1M 월 서수가 연속이 아닙니다 - 빠진 달이 있습니다.")
 return out

현금흐름 스케줄

def build_cashflow_schedule(raw: pd.DataFrame) -> pd.DataFrame:
 """원본 행 → 현금흐름 스케줄 + 계산 가능 여부.

 `filter_analysis_sample()`을 **이미 통과한** 프레임을 받는다 - 만가+버퍼 6개월과
 `Fully Paid`/`Charged Off` 필터는 거기서 걸린다. 여기서 추가로 거르는 것은
 현금흐름을 배치할 수 없는 행뿐이며, 사유를 `exclude_reason`에 남긴다
 """

```

```

(버리지 않고 감사 테이블로 보존한다 - `src/preprocessing/AGENTS.md`).
"""

d = raw.copy()
num = ["funded_amnt", "total_pymnt", "last_pymnt_amnt", "recoveries",
 "collection_recovery_fee", "installment"]
for c in num:
 d[c] = pd.to_numeric(d[c], errors="coerce")

d["status_norm"] = (
 d["loan_status"].astype("string")
 .str.replace(r"^Does not meet the credit policy\. Status:\s*", "", regex=True)
 .str.strip()
)
`term`·`issue_d`는 이미 파싱된 프레임(`build_return_inputs()`의 반환)으로도 들어온다.
d["term_months"] = (
 pd.to_numeric(d["term"], errors="coerce")
 if pd.api.types.is_numeric_dtype(d["term"])
 else pd.to_numeric(d["term"].astype("string").str.extract(r"(\d+)")[0], errors="coerce")
)
if pd.api.types.is_numeric_dtype(d["issue_d"]):
 issue_ord = pd.to_numeric(d["issue_d"], errors="coerce")
 issue_missing = issue_ord.isna()
else:
 issue = pd.to_datetime(d["issue_d"], format="%b-%Y", errors="coerce")
 issue_ord = issue.dt.year * 12 + issue.dt.month
 issue_missing = issue.isna()
last = pd.to_datetime(d["last_pymnt_d"], format="%b-%Y", errors="coerce")
d["issue_month_ord"] = issue_ord
last_ord = last.dt.year * 12 + last.dt.month

d["K"] = (last_ord - d["issue_month_ord"]).clip(lower=0)
d["C_regular"] = d["total_pymnt"] - d["recoveries"]
d["C_recovery"] = d["recoveries"] - d["collection_recovery_fee"]
d["L"] = d["last_pymnt_amnt"]

--- 제외 사유 (여러 개면 '|'로 잇는다) ---
reason = pd.Series("", index=d.index, dtype="object")

def flag(mask: pd.Series, text: str) -> None:
 m = mask.fillna(False).to_numpy()
 reason.values[m] = np.where(
 reason.values[m] == "", text, reason.values[m] + "|" + text
)

required = ["funded_amnt", "total_pymnt", "recoveries",
 "collection_recovery_fee", "last_pymnt_amnt"]
flag(d[required].isna().any(axis=1), "필수금액결측")
flag(d["funded_amnt"] <= 0, "funded_amnt_0이하")
flag(~d["term_months"].isin([36, 60]), "term_오류")
flag(issue_missing, "issue_d_결측")

no_last = last.isna()
d["zero_cash_R_minus1"] = (
 d["status_norm"].eq("Charged Off") & no_last
 & d["total_pymnt"].eq(0) & d["recoveries"].eq(0)
)
d["recovery_only_no_payment"] = (

```

```

 d["status_norm"].eq("Charged Off") & no_last
 & d["C_regular"].abs().le(0.01) & d["recoveries"].gt(0)
 & d["L"].abs().le(0.01)
)
flag(no_last & ~(d["zero_cash_R_minus1"] | d["recovery_only_no_payment"]),
 "last_pymnt_d_결측_시점미확정")
flag(d["L"] < 0, "last_pymnt_amnt_음수")
flag(d["K"].ge(2) & (d["L"] > d["C_regular"] + 1e-6), "L이_C_regular_초과")
flag(d["C_regular"] < -0.01, "C_regular_음수")
flag(d["C_recovery"] < -0.01, "C_recovery_음수")

d["exclude_reason"] = reason
d["cashflow_eligible"] = reason.eq("")

--- 배치 ---
K = d["K"].round()
k_ge2 = K.ge(2).fillna(False)
d["K_effective"] = K
d["regular_early_count"] = np.where(k_ge2, K - 1, 0)
d["regular_early_each"] = np.where(
 k_ge2, (d["C_regular"] - d["L"]) / np.where(k_ge2, K - 1, 1), 0.0
)
d["regular_final_t"] = K
d["regular_final_amount"] = np.where(k_ge2, d["L"], d["C_regular"])
d.loc[d["zero_cash_R_minus1"], "regular_final_amount"] = 0.0
d["recovery_t"] = (K + RECOVERY_LAG_MONTHS)
d.loc[d["recovery_only_no_payment"], "recovery_t"] = RECOVERY_LAG_MONTHS
d["recovery_amount"] = d["C_recovery"]

d["cashflow_rule"] = np.select(
 [d["zero_cash_R_minus1"], d["recovery_only_no_payment"],
 K.eq(0).fillna(False), K.eq(1).fillna(False), k_ge2],
 ["현금유입 없음", "Recovery-only (발행+6)", "K=0", "K=1", "K>=2"],
 default="확인 필요",
)

return d

```

```

W(T)와 R

def realized_return_actual(
 schedule: pd.DataFrame, gs1m: pd.Series | None = None
) -> pd.DataFrame:
 """ 현금흐름 스케줄 → `W_T`·`R` (**계약만기 등가 연율**).

 `GS1M` 누적 로그성장률의 prefix를 미리 만들어 **반복문 없이** 벡터로 계산한다.
 수령월에는 이자를 주지 않고 다음 달부터 굴린다 - 계약만기 이후 수령분은
 같은 경로로 만기 시점까지 역할인되므로 계수가 1보다 작아진다.

 `cashflow_eligible`이 아닌 행은 `W_T`·`R`이 NaN이다.
 """
 g = load_gs1m_monthly_return() if gs1m is None else gs1m
 ok = schedule["cashflow_eligible"].to_numpy()
 d = schedule.loc[ok]

 rate_min = int(g.index.min())

```

```

prefix[b] = 시작월 직전부터 b번째 달까지의 누적 로그성장률. prefix[0] = 0.
prefix = np.concatenate([[0.0], np.cumsum(np.log1p(g.to_numpy()))])
inv_cumsum = np.cumsum(np.exp(-prefix))

issue_b = (d["issue_month_ord"].astype("int64").to_numpy() - rate_min + 1)
term = d["term_months"].astype("int64").to_numpy()
k = d["K_effective"].fillna(0).astype("int64").to_numpy()
rec_t = d["recovery_t"].fillna(0).astype("int64").to_numpy()
mat_b = issue_b + term

bounds = np.concatenate([issue_b, mat_b, issue_b + k, issue_b + rec_t])
if bounds.min() < 0 or bounds.max() >= len(prefix):
 raise ValueError(
 "GS1M 경로가 대출 기간을 덮지 못합니다 - 수집 구간을 확인하세요 "
 f"(필요 월 서수 {bounds.min() + rate_min - 1}~{bounds.max() + rate_min - 1})."
)

early_n = d["regular_early_count"].to_numpy()
m = early_n > 0
t=1..K-1 각 달의 계수 합 = $\exp(\Lambda_{\text{만기}}) \cdot \sum_j \exp(-\Lambda_j)$
sum_early = np.zeros(len(d))
sum_early[m] = np.exp(prefix[mat_b[m]]) * (
 inv_cumsum[issue_b[m] + k[m] - 1] - inv_cumsum[issue_b[m]]
)

final_factor = np.exp(prefix[mat_b] - prefix[issue_b + k])
rec_factor = np.exp(prefix[mat_b] - prefix[issue_b + rec_t])

W = (d["regular_early_each"].to_numpy() * sum_early
 + d["regular_final_amount"].to_numpy() * final_factor
 + d["recovery_amount"].to_numpy() * rec_factor)
P = d["funded_amnt"].to_numpy(dtype="float64")
with np.errstate(divide="ignore", invalid="ignore"):
 R = np.power(W / P, 12.0 / term) - 1.0

out = pd.DataFrame(
 {"W_T": np.nan, "R": np.nan, "regular_final_factor": np.nan,
 "recovery_factor": np.nan},
 index=schedule.index,
)
out.loc[ok, "W_T"] = W
out.loc[ok, "R"] = R
out.loc[ok, "regular_final_factor"] = final_factor
out.loc[ok, "recovery_factor"] = rec_factor
return out

def build_realized_returns(raw: pd.DataFrame) -> pd.DataFrame:
 """`filter_analysis_sample()` 통과 프레임 → 실현 `R` + `rf` + `XR` 한 장.

 `rf`는 **발행시점 × 만기매칭 GS3/GS5**다(#18 확정) - 재투자에 쓴 GS1M과 역할이 다르다.
 `GS1M`은 현금을 굴리는 이율, `rf`는 초과수익률의 차감 기준이다.
 """
 from analysis.realized_return import issue_risk_free_rate

 sched = build_cashflow_schedule(raw)
 ret = realized_return_actual(sched)

```

```
rf = issue_risk_free_rate(sched["issue_month_ord"], sched["term_months"])
```

```
return pd.DataFrame({
 "id": sched["id"].astype(str).str.strip(),
 "status_norm": sched["status_norm"],
 "term": sched["term_months"],
 "issue_month_ord": sched["issue_month_ord"],
 "K": sched["K"],
 "funded_amnt": sched["funded_amnt"],
 "installment": sched["installment"],
 "cashflow_rule": sched["cashflow_rule"],
 "cashflow_eligible": sched["cashflow_eligible"],
 "exclude_reason": sched["exclude_reason"],
 "W_T": ret["W_T"],
 "R": ret["R"],
 "rf": rf,
 "XR": ret["R"] - rf,
 "is_default": sched["status_norm"].eq("Charged Off").astype("int8"),
})
```

```
def load_raw_for_cashflow(csv_path: Path | None = None) -> pd.DataFrame:
```

```
 """ 원본 CSV에서 현금흐름 계산에 필요한 열만 읽고 분석 표본 필터를 건다.
```

```
 `csv_path`를 주면 2nd Test 등 다른 파일을 읽는다(건수 검증은 자동으로 꺼진다).
 """
```

```
import sys
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from preprocessing.loader import filter_analysis_sample, load_raw_loans
```

```
raw = load_raw_loans(usecols=CASHFLOW_COLUMNS, csv_path=csv_path)
```

```
return filter_analysis_sample(raw, verify=csv_path is None)
```

```

B팀 산출물과의 대조

```

```
def verify_against_teamb(ours: pd.DataFrame, teamb_csv: Path) -> pd.DataFrame:
```

```
 """ 우리 재현값과 B팀 CSV를 `id`로 맞춰 대조한다.
```

```
 이 검증이 통과해야 2nd Test에 같은 코드를 돌린 결과를 믿을 수 있다 -
```

```
 train에서 일치를 확인한 뒤에만 Test로 넘어간다.
 """
```

```
b = pd.read_csv(teamb_csv, usecols=["id", "R", "rf", "XR", "W_T", "K", "cashflow_rule"],
 low_memory=False)
```

```
b["id"] = b["id"].astype(str).str.strip()
```

```
m = ours.merge(b, on="id", how="inner", suffixes=("", "_b"))
```

```
rows = []
```

```
for col in ("R", "rf", "XR", "W_T"):
```

```
 diff = (m[col] - m[f"{col}_b"]).abs()
```

```
 rows.append({
```

```
 "column": col, "n": int(diff.notna().sum()),
```

```
 "max_abs_diff": float(diff.max()),
```

```
 "mean_abs_diff": float(diff.mean()),
```

```
 "n_over_1e-6": int((diff > 1e-6).sum()),
```

```
 })
```

```
rows.append({
 "column": "cashflow_rule", "n": len(m),
 "max_abs_diff": float("nan"), "mean_abs_diff": float("nan"),
 "n_over_1e-6": int((m["cashflow_rule"] != m["cashflow_rule_b"]).sum()),
})
return pd.DataFrame(rows)
```

## src/analysis/oof\_diagnostics.py

---

OOF 파이프라인 진단 4종

""00F 파이프라인 진단 4종 - 구조 A'의 설계 선택을 실측으로 검증한다.

`decision\_log.md` #20의 구현 경로는 **PD** 분위 × term 칸별 통계표\*\*를 만들고 `q\_score`로 승인선을 긋는 것을 상정한다. 그런데 그 열개가 값을 하는지는 아직 아무도 재보지 않았다. 여기서 네 가지를 쟀다.

```
진단	묻는 것	판정				
C-1	`q_score` 랭킹이 `pd` 랭킹과 **다른가?**	`\\|p\\|`≈0.99 → 칸별 통계표가 값을 못 함, 단순 PD threshold로 복귀 (#18 전제) `\\|p\\|`<0.9 → 재배열 실제, `q_score` 채택 근거				
C-2	칸 안에서 `int_rate`가 **흘어져 있는가?**	$sd > 1\%p \rightarrow A'$ 의 건별 계산이 실효 있음 $sd \approx 0.3\%p \rightarrow$ 그룹 평균으로 충분 했다는 뜻				
C-3	Train 경계를 Validation에 적용하면 **10%씩 들어가는가?**	±2%p 안이면 무사 심하면 fold를 5 → 10으로 올린다				
C-4	XGBoost $\hat{p}$ 가 **확률로서 맞는가?** (isotonic 보정)	$ECE > 0.5\%p \rightarrow$ 보정 필수 $ECE \approx 0 \rightarrow$ 보정해도 $E[XR]$ 이 안 바뀔				
```

## C-4가 왜 다른 셋보다 앞서 실행되나

구조 A'는  $\hat{p}$ 를 **순위가 아니라 확률 값**으로 쓴다( $E[XR] = (1-\hat{p}) \cdot XR_{\text{정상}} + \hat{p} \cdot \mu_{\text{부도}}$ ). 따라서 보정을 나중에 붙이면  $E[XR]$ ·`q\_score`가 전부 바뀌어 **C-1을 다시 돌려야 한다**. 그래서 보정을 먼저 학습하고, C-1이 처음부터 보정된  $\hat{p}$ 로  $E[XR]$ 을 만든다.

## PD의 두 역할을 나눈다 (C-4가 실측으로 정한 것)

```
쓰임	어느 PD	왜
PD 분위 **경계·배정** (칸을 묶는 도구)	**보정 전**	isotonic 평탄구간이 경계를 삼켜 칸 인원이 10%에서 기운다
$E[XR]$ ·`Var[XR]`·`q_score`의 $\hat{p}$ (**확률 값**)	**보정 후**	보정 오차가 $E[XR]$ 에 직접 편향으로 들어간다
승인선 **점수**로서의 `pd`	**보정 전**	보정은 순위를 안 바꾸면서 동순위만 만든다 - 점수로는 잃을 것만 있다
```

⚠ isotonic은 비감소 **계단함수**다. "단조변환이니 분위 배정이 그대로"는 정확히는 틀리다 - 수십만 개 PD가 수백 개 값으로 뭉치고, C-4가 그 결과 칸이 얼마나 기우는지 직접 쟀다.

## C-1의 메커니즘 - 왜 뒤집힐 수 있나

PD 분위가 오르면 두 힘이 반대로 작용한다.

- $\mu_{\text{정상}}$ 은 **올라간다** - LC가 위험한 대출에 높은  $\text{int\_rate}$ 를 매기기 때문이다.
- $\mu_{\text{부도}}$ 는 내려간다.

그래서  $E[XR]$ 이 PD에 대해 **비단조**일 수 있다. 예를 들어 분위 3의 금리 프리미엄이 PD 상승분보다 크면 분위 3이 분위 1보다 앞서게 되고, 그러면 PD 순서와  $q\_score$  순서가 갈린다. 이 재배열이 실재하는지가 곧 **칸별 통계표**를 만들 이유가 있는자\*\*다.

⚠  $q\_score$ 는  $XR$ 이 필요하므로 **잠정 정약**(© 발행시점 고정 · 수수료 0% · 조기상환 보정 0)로 계산한다(`realized_return.py`). 보정이 확정되면 이 진단을 다시 돌린다 - 모형 재학습은 필요 없다.

실행: `/opt/anaconda3/bin/python src/analysis/oof_diagnostics.py``  
""

```
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.metrics import roc_auc_score
```

```

import sys

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from analysis.model import (# noqa: E402
 DEFAULT_N_FOLDS,
 apply_calibrator,
 assign_quantile_by_term,
 calibration_metrics,
 calibration_noise_floor,
 calibration_table,
 compute_oof,
 fit_calibrator,
 predict_default_probability,
 quantile_edges_by_term,
 train_model,
)
from analysis.realized_return import (# noqa: E402
 ReturnAssumptions,
 build_excess_returns,
 build_return_inputs,
 default_cell_stats,
 expected_excess_return,
 q_score,
 variance_excess_return,
)
from preprocessing.preprocessor import build_feature_table, split_from_manifest # noqa: E402
from utils.config import repo_root # noqa: E402

N_QUANTILES = 10

def hdr(title: str) -> None:
 print("\n" + "=" * 78)
 print(title)
 print("=" * 78)

`build_excess_returns`() (→ `realized_return.py`)와 `quantile_edges_by_term`()
`assign_quantile_by_term`() (→ `model.py`)은 본 파이프라인(`sharpe_optimizer.py`)도 쓰므로
각자의 정보 모듈로 옮겼다. 진단 스크립트가 정의를 들고 있으면 두 벌이 갈라진다.

def main() -> None:
 out_dir = repo_root() / "outputs"
 out_dir.mkdir(parents=True, exist_ok=True)
 assumptions = ReturnAssumptions()

 hdr("0. 데이터 구성")
 X, y, meta = build_feature_table()
 parts = split_from_manifest(X, y, meta)
 X_tr, y_tr, meta_tr = parts["train"]
 X_va, y_va, meta_va = parts["validation"]
 print(f"표본 {len(X):,} 피쳐 {X.shape[1]} Train {len(X_tr):,} Validation {len(X_va):,}")
 print(f"가정: {assumptions.label()} (조기상환 보정 {assumptions.prepayment_adjustment})")

```

```

outcome = build_return_inputs()
xr = build_excess_returns(outcome, assumptions)
print(f"수익률 입력 {len(outcome):,}건 "
 f"XR 정상 결측 {xr['xr_normal'].isna().sum():,} "
 f"XR 부도 결측(부도건 중) {xr.loc[xr['is_default'] == 1, 'xr_default'].isna().sum():,}")

hdr(f"1. Train {DEFAULT_N_FOLDS}-fold 00F PD")
pd_oof, fold_auc, _ = compute_oof(X_tr, y_tr)
print(f" fold AUC 평균 {np.mean(fold_auc):.5f} (sd {np.std(fold_auc):.5f})")
print(f" 00F 전체 AUC {roc_auc_score(y_tr, pd_oof):.5f}")

----- C-4
hdr("1-b. [C-4] 확률보정(isotonic) - $\hat{p}$ 를 확률 값으로 쓸 수 있는가")
calibrator = fit_calibrator(pd_oof, y_tr)
pd_oof_cal = apply_calibrator(calibrator, pd_oof)

flood = calibration_noise_floor(pd_oof_cal)
print(f" ECE 바닥값(완벽 보정 시) 평균 {flood['ece_floor_mean_pp']:.3f}% "
 f"최대 {flood['ece_floor_max_pp']:.3f}% ← 판정 기준선")

print("\n [Train 00F]")
m_raw = calibration_metrics(y_tr, pd_oof)
m_cal = calibration_metrics(y_tr, pd_oof_cal)
print(f" 보정 전 AUC {m_raw['auc']:.5f} Brier {m_raw['brier']:.5f} "
 f"ECE {m_raw['ece_pp']:.3f}% MCE {m_raw['mce_pp']:.3f}% 편향 {m_raw['bias_pp']:+.3f}%")
print(f" 보정 후 AUC {m_cal['auc']:.5f} Brier {m_cal['brier']:.5f} "
 f"ECE {m_cal['ece_pp']:.3f}% MCE {m_cal['mce_pp']:.3f}% 편향 {m_cal['bias_pp']:+.3f}%")
print(f" ⚠ in-sample(보정을 학습한 데이터) - 증거로 쓰지 않는다")
print(f" → 보정 효과의 증거는 아래 C-3의 **Validation** 수치다.")

print("\n [보정 전 구간별 예측 vs 실측 - 어긋난 방향]")
print(calibration_table(y_tr, pd_oof).to_string(
 index=False, float_format=lambda v: f"{v: .5f}"))

bias_bp = m_raw["ece_pp"] * 0.20 * 100
print(f" 판정: " + (
 f"ECE {m_raw['ece_pp']:.3f}% 가 바닥값 {flood['ece_floor_mean_pp']:.3f}%의 "
 f"{m_raw['ece_pp'] / max(flood['ece_floor_mean_pp'], 1e-9):.1f}배 - 실제로 어긋나 있다. "
 f"(mu_정상-mu_부도)≈20%를 곱하면 E[XR]에 약 {bias_bp:.0f}bp 편향으로 들어간다."
 if m_raw["ece_pp"] > max(0.5, 2 * flood["ece_floor_mean_pp"])
 else f"ECE {m_raw['ece_pp']:.3f}% 가 바닥값 {flood['ece_floor_mean_pp']:.3f}% 수준이다 - "
 f"보정할 어긋남이 사실상 없다. E[XR] 편향은 약 {bias_bp:.0f}bp."
))

---- 평탄구간(plateau)이 분위 배정에 무슨 일을 하는가 - 역할 분리의 근거
#
isotonic은 PAV 블록 단위 계단함수라 수십만 개의 PD가 수백 개 값으로 뭉친다. 그래서
보정된 PD로 분위기를 자르면 칸 인원이 10%에서 크게 어긋난다 - 경계가 평탄구간
안쪽에 떨어지면 그 구간 전체가 한 칸으로 물리기 때문이다.
#
해법은 **역할을 나누는 것**이다. 분위는 칸별 통계를 낼 때 쓰는 **묵는 도구**일 뿐이고,
확률 값이 필요한 곳은 `E[XR] = (1- $\hat{p}$ )·... +  $\hat{p}$ ·...` 하나다.
→ 분위 경계·배정: **보정 전 PD** (순위가 촘촘해 칸이 정확히 10%씩 나뉜다)
→ `E[XR]`·`Var[XR]`·`q_score`의 ` $\hat{p}$ `: **보정 후 PD**
isotonic이 단조라 두 PD의 순위가 (동순위를 빼면) 같으므로 이 분리는 모순이 아니다.
term_tr = xr["term"].loc[X_tr.index]
edges = quantile_edges_by_term(pd_oof, term_tr)

```

```

q_tr = assign_quantile_by_term(pd_oof, term_tr, edges)

edges_cal = quantile_edges_by_term(pd_oof_cal, term_tr)
q_cal = assign_quantile_by_term(pd_oof_cal, term_tr, edges_cal)

def worst_imbalance(q: pd.Series) -> float:
 share = (
 pd.DataFrame({"term": term_tr, "q": q})
 .groupby("term", observed=True)["q"]
 .value_counts(normalize=True) * 100
)
 return float((share - 100 / N_QUANTILES).abs().max())

moved = int((q_cal != q_tr).sum())
print(f"\n [해상도 손실] 고유 PD 값 {pd_oof.nunique():,} → {pd_oof_cal.nunique():,}개 "
 f"(isotonic PAV 블록)")
print(f" p(pd_raw, pd_cal) Spearman = {spearmanr(pd_oof, pd_oof_cal)[0]:+.6f} "
 f"-- 1이 아닌 만큼이 동순위 생성분이다")
print(f" 분위 배정이 달라지는 건수 {moved:,} / {len(q_tr):,} ({moved / len(q_tr):.3%})")
print(f" 칸 인원 10%에서의 최대 이탈: 보정 전 PD로 자르면 {worst_imbalance(q_tr):.3f}%p · "
 f"보정 후 PD로 자르면 {worst_imbalance(q_cal):.3f}%p")
print(" → 판정: " + (
 "보정 후 PD로 자르면 칸이 눈에 띄게 기운다. **분위는 보정 전 PD로 자르고 "
 "확률 값만 보정된 것을 쓴다**(역할 분리) - 위 주석 참고."
 if worst_imbalance(q_cal) > worst_imbalance(q_tr) + 0.5
 else "두 방식의 칸 균형 차이가 작다. 그래도 역할 분리가 더 안전하다 "
 "(평탄구간 폭은 표본·seed에 따라 커질 수 있다).")
))
print(" ⚠ 따라서 `pd`를 승인선 **점수**로 쓸 때도 보정 전 PD를 쓴다 - 보정은 순위를 "
 "바꾸지 않으면서 동순위만 만들므로 점수로서는 잃을 게 있고 얻을 게 없다.")

pd.DataFrame([
 {"stage": "train_oof_raw", **m_raw, **floor,
 "n_unique_pd": pd_oof.nunique(), "worst_cell_imbalance_pp": worst_imbalance(q_tr)},
 {"stage": "train_oof_calibrated_INSAMPLE", **m_cal, **floor,
 "n_unique_pd": pd_oof_cal.nunique(), "worst_cell_imbalance_pp": worst_imbalance(q_cal)},
]).to_csv(out_dir / "oof_c4_calibration.csv", index=False)

print("\n [term별 PD 분위 경계 - 보정 전 PD 기준]")
for t, e in edges.items():
 print(f" {int(t):>3}m " + " ".join(f"{v:.4f}" for v in e))

----- 칸별 통계표
hdr("2. 칸별 부도 통계표 (mu_부도 · var_부도)")
xr_tr = xr.loc[X_tr.index]
stats = default_cell_stats(
 xr_tr["xr_default"], q_tr, term_tr
)
print(stats[["n", "mu", "var", "shrunk", "mu_unreliable"]].to_string(
 float_format=lambda v: f"{v:.5f}")
)
print(f"\n pooled var = {stats['pooled_var'].iloc[0]:.5f} "
 f"축소추정된 칸 {int(stats['shrunk'].sum())}/{len(stats)}")
stats.reset_index().to_csv(
 out_dir / f"oof_default_cell_stats_{assumptions.label()}.csv", index=False)

mu_map = stats["mu"].to_dict()
var_map = stats["var"].to_dict()

```

```

keys = list(zip(term_tr, q_tr))
mu_default = pd.Series([mu_map.get(k, np.nan) for k in keys], index=X_tr.index)
var_default = pd.Series([var_map.get(k, np.nan) for k in keys], index=X_tr.index)

e_xr = expected_excess_return(pd_oof_cal, xr_tr["xr_normal"], mu_default)
v_xr = variance_excess_return(
 pd_oof_cal, xr_tr["xr_normal"], mu_default, var_default, var_normal=0.0
)
q = q_score(e_xr, v_xr)

----- C-1
hdr("3. [C-1] Spearman ρ (pd, q_score) - 설계 존폐")
ok = q.notna() & pd_oof_cal.notna() & e_xr.notna()
rho_q, _ = spearmanr(pd_oof_cal[ok], q[ok])
rho_e, _ = spearmanr(pd_oof_cal[ok], e_xr[ok])
print(f" 유효 {ok.sum():,}건 / {len(ok):,} (PD는 **보정 후** 기준 - C-4)")
print(f" ρ (pd_cal, q_score) = {rho_q:+.5f}")
print(f" ρ (pd_cal, E[XR]) = {rho_e:+.5f}")
print(f" ρ (E[XR], q_score) = {spearmanr(e_xr[ok], q[ok])[0]:+.5f}")
print(f"\n [보정 전 PD로 재보기 - 보정이 C-1 결론을 바꿨는지]")
print(f" ρ (pd_raw, q_score) = {spearmanr(pd_oof[ok], q[ok])[0]:+.5f}")
print(f" ρ (pd_raw, E[XR]) = {spearmanr(pd_oof[ok], e_xr[ok])[0]:+.5f}")

verdict = (
 "| $\rho| \geq 0.99 \rightarrow$ 랭킹이 사실상 같다. 칸별 통계표·분산추정의 대가가 0이므로 "
 "#18이 전제한 단순 PD threshold로 돌아가는 것이 정답이다."
 if abs(rho_q) >= 0.99
 else (
 "| $\rho| < 0.9 \rightarrow$ 재배열이 실패한다. q_score를 채택할 근거가 있다."
 if abs(rho_q) < 0.9
 else "0.9 ≤ | $\rho| < 0.99 \rightarrow$ 부분적 재배열. Validation Sharpe 비교로 결정해야 한다."
)
)
print(f"\n 판정: {verdict}")

print("\n [PD 분위별 평균 - 비단조성 확인]")
by_cell = pd.DataFrame(
 {"term": term_tr, "q": q_tr, "pd": pd_oof_cal, "pd_raw": pd_oof,
 "xr_normal": xr_tr["xr_normal"],
 "E_XR": e_xr, "q_score": q, "int_rate": xr_tr["int_rate"]}
).groupby(["term", "q"], observed=True).mean(numeric_only=True)
print(by_cell.to_string(float_format=lambda v: f"{v: .5f}"))
by_cell.reset_index().to_csv(
 out_dir / f"oof_c1_cell_means_{assumptions.label()}.csv", index=False)

----- C-2
hdr("4. [C-2] 칸별 int_rate 표준편차 - A' 건별 계산의 실효성")
c2 = (
 pd.DataFrame({"term": term_tr, "q": q_tr, "int_rate": xr_tr["int_rate"]})
 .groupby(["term", "q"], observed=True)["int_rate"]
 .agg(["size", "mean", "std", "min", "max"])
)
c2["range"] = c2["max"] - c2["min"]
print(c2.to_string(float_format=lambda v: f"{v: .4f}"))
med_sd = c2["std"].median()
print(f"\n 칸별 sd 중앙값 = {med_sd:.4f}%p")
print(" 판정: " + (

```

```

"sd > 1%p - 칸 안에서 금리가 충분히 흩어져 있다. A'의 건별 계산이 실패 있다."
if med_sd > 1.0
else "sd ≈ 0.3%p 수준 - 그룹 평균으로 충분했다는 뜻이다."
if med_sd < 0.5
else "sd 0.5~1%p - 중간. 건별 계산의 이득이 크지 않을 수 있다."
))
c2.reset_index().to_csv(out_dir / "oof_c2_int_rate_dispersion.csv", index=False)

----- C-3
hdr("5. [C-3] Validation 분위별 인원 분포 - 경계 이전 가능성")
final = train_model(X_tr, y_tr)
p_va = predict_default_probability(final, X_va)
print(f" Validation AUC {roc_auc_score(y_va, p_va):.5f} (확정 표본 기준 0.70대)")

Train OOF로 학습한 보정을 최종 모델(Train 100%) 예측에 적용한다 - C-4와 같은 이유로
분위 배정 전에 보정해야 한다. 여기서 Validation 보정 품질도 함께 검증된다
(Train에서 좋아진 것이 Validation으로 이전되는지).
p_va_cal = apply_calibrator(calibrator, p_va)
print(" [Validation 보정 품질]")
va_rows = []
for name, p in (("보정 전", p_va), ("보정 후", p_va_cal)):
 m = calibration_metrics(y_va, p)
 print(f" {name} AUC {m['auc']:.5f} Brier {m['brier']:.5f} "
 f"ECE {m['ece_pp']:.3f}%p MCE {m['mce_pp']:.3f}%p 편향 {m['bias_pp']:+.3f}%p")
 va_rows.append({"stage": f"validation_{raw}" if name == "보정 전" else "calibrated", **m})

c4 = pd.read_csv(out_dir / "oof_c4_calibration.csv")
pd.concat([c4, pd.DataFrame(va_rows)], ignore_index=True).to_csv(
 out_dir / "oof_c4_calibration.csv", index=False)

분위 배정은 **보정 전** PD로 한다 (C-4의 역할 분리). 보정된 `p_va_cal`은
threshold 단계에서 `E[XR]`을 만들 때 쓴다.
term_va = xr["term"].loc[X_va.index]
q_va = assign_quantile_by_term(p_va, term_va, edges)

rows = []
for t in sorted(edges):
 share_tr = (q_tr[term_tr == t].value_counts(normalize=True).sort_index() * 100)
 share_va = (q_va[term_va == t].value_counts(normalize=True).sort_index() * 100)
 for qi in range(1, N_QUANTILES + 1):
 rows.append({
 "term": int(t), "quantile": qi,
 "train_pct": round(float(share_tr.get(qi, 0.0)), 3),
 "validation_pct": round(float(share_va.get(qi, 0.0)), 3),
 "diff_pp": round(float(share_va.get(qi, 0.0) - share_tr.get(qi, 0.0)), 3),
 })
c3 = pd.DataFrame(rows)
print(c3.to_string(index=False))
worst = c3["diff_pp"].abs().max()
print(f"\n 최대 이탈 {worst:.2f}%p")
print(" 판정: " + (
 "±2%p 이내 - 무시해도 된다. Train 경계를 그대로 쓴다."
 if worst <= 2.0
 else "±2%p 초과 - fold를 5 → 10으로 올려 fold 모델과 최종 모델의 학습량 차를 줄인다."
))
c3.to_csv(out_dir / "oof_c3_quantile_share.csv", index=False)

```

```
hdr("완료 - 산출물")
for f in sorted(out_dir.glob("oof_*.csv")):
 print(f" {f.relative_to(repo_root())}")
print("\n⚠️ 모든 XR은 잠정값 기준이다 - 조기상환 보정 확정 시 다시 돌린다(모형 재학습 불필요).")

if __name__ == "__main__":
 main()
```

## src/analysis/sharpe\_optimizer.py

---

Validation set 기준 Sharpe Ratio 극대화 threshold 탐색.

"""Validation set 기준 Sharpe Ratio 극대화 threshold 탐색.

정의는 `src/analysis/AGENTS.md`, 확정 근거는 `outputs/reports/decision\_log.md` #18·#20.

## 이 모듈이 하지 않는 것

- \*\*실현수익률을 다시 정의하지 않는다.\*\* 구조 A'의 단일 구현은 `src/analysis/realized\_return.py`다  
(`contract\_return()` · `realized\_return\_defaulted()` · `expected\_excess\_return()` · `q\_score()`).  
여기서는 거기서 나온 건별 `XR`과 랭킹 점수를 \*\*받아서 정렬·탐색만\*\* 한다.
- \*\*무위험수익률을 스칼라로 받지 않는다.\*\* `rf`는 발행시점(`issue\_d`) × 만기(`term`) 매칭 국제라  
건별로 다르다(#18). `config.yaml`의 `risk\_free\_rate.value`는 계속 `null`이며 읽지 않는다.  
이미 `XR = R - rf`로 차감된 초과수익률을 입력으로 받는다.
- \*\*`int\_rate`를 수익률로 쓰지 않는다.\*\* 폐기된 관례다(#18) - `int\_rate`는 피처이자 계약  
현금흐름의 입력일 뿐이다.

## 무엇으로 Sharpe를 계산하는가 - 기대값이 아니라 실현값

점수(`E[XR]` · `q\_score`)는 \*\*승인선을 그을 때만\*\* 쓴다. Sharpe 자체는 Validation에서 실제로  
벌어진 \*\*실현 `XR`\*\*(`realized\_return.build\_excess\_returns()`의 `xr\_realized`)로 계산한다.  
기대값으로 Sharpe를 재면 모형이 자기 예측으로 자기를 채점하는 셈이다.

## 승인선 랭킹 기준 - 세 후보를 한 번에 비교한다

`pd` / `E[XR]` / `q\_score` 중 무엇으로 승인선을 그을지는 \*\*미확정\*\*이다(#5·#20).

세 기준은 같은 중간 테이블에서 \*\*정렬만 바꾸면 나오므로 추가 학습이 필요 없다.\*\*

→ Validation에서만 비교해 승자를 사전 확정한 뒤 \*\*Test는 1화\*\*다. 세 기준을 Test에서  
비교하면 "Test set으로 모형을 재조정하지 않는다" 규칙 위반이다.

⚠ `pd`를 점수로 쓸 때는 \*\*보정 전\*\* PD를 쓴다 - isotonic 보정은 순위를 바꾸지 않으면서  
동순위만 만들어(고유값 42.8만 → 115개), 점수로서는 잃을 게 있고 얻을 게 없다(진단 C-4).  
보정된 PD는 `E[XR]` · `Var[XR]`의 `p`로만 들어간다.

## 절대 Sharpe와 Δ Sharpe를 \*\*함께\*\* 보고한다

헤드라인은 \*\*Δ Sharpe = (모형) - (approve-all 대조군)\*\* 이다(#17 ②·#18) - 재투자 가치가  
양쪽을 같이 밀어올리므로 절대값에는 가정 효과가 섞인다.

Δ를 헤드라인으로 쓰는 것이 \*\*최적화 목표를 바꾸지는 않는다.\*\* approve-all Sharpe는 `τ`에  
대해 상수라 `argmax\_τ [Sharpe(τ) - 상수] = argmax\_τ Sharpe(τ)` 이기 때문이다. 프로젝트 목적  
(Sharpe 극대화)은 그대로다.

그럼에도 \*\*절대 Sharpe를 반드시 병기한다.\*\* 이유가 둘이다.

1. 수업에서 제시된 기준선이 절대 수준이다(`README.md`: "사프비율은 0.2~0.4 정도 나오면  
옳게 한 것"). Δ만 내면 그 기준과 대조할 숫자가 보고서에 없다.
2. #18은 "가정이 두 전략을 똑같이 밀어올린다"고 전제했지만 \*\*107.5bp는 균일한 평행이동이  
아니다\*\* - `int\_rate` · `term` 별로 다르게 들어가 건별 `XR`의 순서·분산을 바꾸므로 `τ`·  
자체가 움직일 수 있다. 그래서 \*\*재투자 가정 2종(국제/0%)을 모두 돌려\*\* Δ가 가정에  
안정적인지 실측한다(#18이 남긴 "검증 과제").

따라서 산출 표는 \*\*((랭킹 기준 3종 + approve-all) × (재투자 가정 2종))\*\* 이며 열은  
`τ` · 승인을 · 절대 Sharpe · Δ Sharpe다.

실행: `opt/anaconda3/bin/python src/analysis/sharpe\_optimizer.py`  
"""

```

from __future__ import annotations

import time
from pathlib import Path
from typing import Callable, Iterable

import numpy as np
import pandas as pd

try:
 from utils.config import load_config, repo_root
except ModuleNotFoundError: # pragma: no cover
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from utils.config import load_config, repo_root

Sharpe 계산

def portfolio_excess_returns(excess_returns: pd.Series, approved: pd.Series) -> pd.Series:
 """ 포트폴리오 건별 초과수익률.

 승인 건은 실현 `XR`을 그대로, **거절 건은 무위험자산에 투자했다고 보아 `XR = 0`** 이다
 (`R = rf` → `XR = R - rf = 0`). 거절 건을 표본에서 빼지 않는다 -
 빼면 승인율이 낮을수록 분모가 줄어 Sharpe가 부풀려진다.
 """
 return excess_returns.where(approved.astype(bool), 0.0)

def sharpe_ratio(portfolio_excess_returns: pd.Series) -> float:
 """ `평균(XR) / 표본표준편차(XR, ddof=1)` .

 입력이 **이미 초과수익률**이므로 여기서 다시 `rf`를 빼지 않는다(#18).
 """
 x = portfolio_excess_returns.to_numpy(dtype="float64")
 sd = x.std(ddof=1)
 return float(x.mean() / sd) if sd > 0 else float("nan")

def evaluate_threshold(
 score: pd.Series,
 excess_returns: pd.Series,
 threshold: float,
 lower_is_better: bool = True,
) -> dict:
 """ threshold 하나를 평가한다.

 `score`는 랭킹 기준(`pd` / `E[XR]` / `q_score`) 중 **하나**다 - 어느 것을 쓸지는
 미확정하므로 주입받는다(#5·#20). `lower_is_better`는 `pd`면 `True`,
 `E[XR]`·`q_score`면 `False`다.

 반환에는 **승인율을 반드시 포함한다** - Sharpe만 보고 조이면 승인율이 비현실적으로
 낮아질 수 있다(`src/analysis/AGENTS.md`).
 """
 approved = score <= threshold if lower_is_better else score >= threshold

```

```

pxr = portfolio_excess_returns(excess_returns, approved)
return {
 "threshold": float(threshold),
 "n_total": int(len(score)),
 "n_approved": int(approved.sum()),
 "approval_rate": float(approved.mean()),
 "sharpe": sharpe_ratio(pxr),
 "mean_xr": float(pxr.mean()),
 "sd_xr": float(pxr.std(ddof=1)),
 "mean_xr_approved": (
 float(excess_returns[approved].mean()) if approved.any() else float("nan")
),
}

```

```

def sharpe_curve(
 score: pd.Series, excess_returns: pd.Series, lower_is_better: bool = True
) -> pd.DataFrame:

```

"""\*\*모든\*\* 승인 컷의 Sharpe를 한 번에 계산한다 (누적합,  $O(n \log n)$ ).

격자를 임의로 찍지 않는다 - 점수로 정렬해 상위  $k$  건을 승인하는 경우를  $k=1..n$  전부 훑으므로 \*\*격자 해상도 때문에 최적점을 놓치는 일이 없다.\*\*

거절 건의 `XR`이 0이라는 점을 쓰면 누적합만으로 닫힌 형태가 나온다. 승인 `k` 건의 `XR` 합을  $S_1$ , 제곱합을  $S_2$ 라 하면 (분모는 거절 건을 포함한 전체  $n$  건이다)

평균 =  $S_1 / n$   
 $\sum (x_i - \text{평균})^2 = S_2 - S_1^2/n$       ← 거절 건은 두 합에 0으로 기여한다  
 표본분산 =  $(S_2 - S_1^2/n) / (n - 1)$

⚠ 반환 `threshold`는 그 컷의 \*\*경계 점수값\*\*이다. 동순위가 있으면 `score <= threshold`로 자른 실제 승인 건수가 `k`보다 많을 수 있어, `find\_optimal\_threshold()`는 최적 `k`를 찾은 뒤 \*\*경계값으로 다시 평가\*\*해 실제 승인율을 보고한다.

```

"""
s = score.to_numpy(dtype="float64")
x = excess_returns.to_numpy(dtype="float64")
n = len(s)
if n < 2:
 raise ValueError(f"표본이 너무 작다: n={n}")

order = np.argsort(s, kind="mergesort")
if not lower_is_better:
 order = order[::-1]
xs, ss = x[order], s[order]

s1 = np.cumsum(xs)
s2 = np.cumsum(xs**2)

mean = s1 / n
var = (s2 - s1**2 / n) / (n - 1)
with np.errstate(divide="ignore", invalid="ignore"):
 sharpe = np.where(var > 0, mean / np.sqrt(var), np.nan)

return pd.DataFrame(
 {
 "k": np.arange(1, n + 1),
 "threshold": ss,

```

```

 "approval_rate": np.arange(1, n + 1) / n,
 "mean_xr": mean,
 "sd_xr": np.sqrt(np.maximum(var, 0.0)),
 "sharpe": sharpe,
 }
)

```

```
def approve_all_sharpe(excess_returns: pd.Series) -> float:
```

```

 """대조군 - 전부 승인(#17 ②). threshold와 무관한 **상수**다.

```

```

 그래서 `argmax_τ [Sharpe(τ) - 이 값] = argmax_τ Sharpe(τ)` 이며, Δ Sharpe를 헤드라인으로
 쓰는 것은 **최적화 목표를 바꾸지 않는다** - 보고·해석의 문제일 뿐이다.
 """

```

```

 return sharpe_ratio(excess_returns)

```

```
def find_optimal_threshold(
```

```

 score: pd.Series,
 excess_returns: pd.Series,
 lower_is_better: bool = True,
) -> dict:
```

```

 """Validation 실현 `XR`로 계산한 **실제 Sharpe**를 최대화하는 `τ*`를 찾는다.

```

```

 점수의 이론값을 최대화하지 않는다 - 개별 대출 `q_score` 최대화는 포트폴리오 Sharpe
 최대화와 같은 문제가 아니다(`src/analysis/AGENTS.md`).

```

```

 절대 Sharpe와 **Δ Sharpe = (모형) - (approve-all)** 을 함께 돌려준다(#17 ②·#18).
 """

```

```

 curve = sharpe_curve(score, excess_returns, lower_is_better)

```

```

 if curve["sharpe"].isna().all():

```

```

 raise RuntimeError("Sharpe가 전부 NaN이다 - XR 분산이 0인지 확인하라.")

```

```

 best = curve.loc[curve["sharpe"].idxmax()]

```

```

 # 동순위를 포함한 실제 승인 집합으로 다시 평가한다

```

```

 out = evaluate_threshold(score, excess_returns, best["threshold"], lower_is_better)

```

```

 baseline = approve_all_sharpe(excess_returns)

```

```

 out["sharpe_approve_all"] = baseline

```

```

 out["delta_sharpe"] = out["sharpe"] - baseline

```

```

 out["k_at_optimum"] = int(best["k"])

```

```

 return out

```

```
def compare_ranking_criteria(
```

```

 scores: dict[str, tuple[pd.Series, bool]],
 excess_returns: pd.Series,
) -> pd.DataFrame:
```

```

 """랭킹 기준 여러 개를 **같은 표본·같은 실현 XR**로 비교한다 (#5·#20).

```

```

 `scores`는 `{이름: (점수 시리즈, lower_is_better)}`다. 추가 학습이 없다 - 정렬만 바꾼다.

```

```

 ⚠ 비교가 성립하려면 **세 기준이 모두 유효한 행만** 써야 한다. `q_score`는 분산이 0인
 칸에서 NaN이 되므로, 기준마다 표본이 다르면 Sharpe 차이가 기준 차이인지 표본 차이인지
 구분되지 않는다. 공통 유효 마스크는 호출자가 맞춰서 넣는다
 (`scores_for_assumptions`)가 그 일을 한다.
 """

```

```

 rows = [

```

```

 {"criterion": name, **find_optimal_threshold(s, excess_returns, lower)}
 for name, (s, lower) in scores.items()
]
 baseline = approve_all_sharpe(excess_returns)
 rows.append(
 {
 "criterion": "approve_all(대조군)",
 "threshold": float("nan"),
 "n_total": len(excess_returns),
 "n_approved": len(excess_returns),
 "approval_rate": 1.0,
 "sharpe": baseline,
 "mean_xr": float(excess_returns.mean()),
 "sd_xr": float(excess_returns.std(ddof=1)),
 "mean_xr_approved": float(excess_returns.mean()),
 "sharpe_approve_all": baseline,
 "delta_sharpe": 0.0,
 "k_at_optimum": len(excess_returns),
 }
)
 return pd.DataFrame(rows)

```

```
def append_row_csv(path: Path, row: dict) -> None:
```

"""행 하나를 CSV에 즉시 덧붙인다. 파일이 없으면 헤더를 쓴다.

K=50 반복은 seed마다 5-fold 00F + 최종 모델(=6회 학습)이라 전체가 수 시간이다.

\*\*끝까지 돌려가 결과가 나오는 구조면 중간에 끊기면 전부 날아간다\*\* - 그래서 반복마다

디스크에 남긴다. `completed\_keys()`와 짝을 이뤄 재시작을 가능하게 한다.

"""

```
path.parent.mkdir(parents=True, exist_ok=True)
```

```
pd.DataFrame([row]).to_csv(path, mode="a", header=not path.exists(), index=False)
```

```
def completed_keys(path: Path, key_cols: tuple[str, ...]) -> set[tuple]:
```

"""이미 계산된 조합의 키 집합. 없으면 빈 집합 - 중단된 실행을 이어서 돌릴 때 쓴다."""

```
if not path.exists():
```

```
 return set()
```

```
done = pd.read_csv(path)
```

```
if not set(key_cols) <= set(done.columns):
```

```
 raise ValueError(f"{path.name}에 키 컬럼 {key_cols}이 없다 - 파일을 지우고 다시 시작하라.")
```

```
return set(map(tuple, done[list(key_cols)].astype(str).to_numpy()))
```

```
def repeat_threshold_search(
```

```
 score_fn: Callable[[int], tuple[pd.Series, pd.Series, bool]],
```

```
 n_repeats: int = 50,
```

```
 verbose: bool = True,
```

```
 seeds: Iterable[int] | None = None,
```

```
 append_to: Path | None = None,
```

```
) -> pd.DataFrame:
```

"""랜덤 6:2:2 분할을 \*\*K=50회\*\* 반복해  $\tau^*$ 의 분포(평균·표준편차)를 본다 (#18).

`seeds`를 주면 `range(n\_repeats)` 대신 그 목록을 쓴다 - \*\*구간을 나눠 여러 번에 걸쳐 돌릴 수 있다.\*\* 각 반복이 seed 하나로 자족적이라(앞 결과를 뒤에서 쓰지 않는다) 어떤 순서로 쪼개 돌려도 \*\*seed 집합이 같으면 결과가 같다.\*\* 단 구간이 겹치면 같은 분할이 중복 집계되고, 빠지면 K가 줄어든다 - 합집합이 의도한 K와 같은지 확인하라.

`append\_to`를 주면 반복마다 그 CSV에 덧붙이고, **\*\*이미 있는 seed는 건너뛴다\*\***(재시작).

Validation 표본 하나로 켄 Sharpe는 "진짜 성과"가 아니라 표본에 따라 흔들리는 추정치다 – 대출 수익률이 부도 여부(베르누이)로 결정되므로 누가 Validation에 뽑혔는지에 따라 평균·표준편차가 크게 달라진다.

최종 threshold는 **\*\*반복에서 나온  $\tau$  값 자체의 평균/중앙값\*\***을 쓴다. "평균 Sharpe와 비슷한 결과를 낸 threshold 하나를 고르는" 방식보다 안정적이다 –  $\tau \rightarrow$  Sharpe 매핑이 1:1이 아니기 때문이다(`README.md`). 분포가 비대칭이면 중앙값이 낫다.

`score\_fn(seed)`는 seed로 재분할·재학습해 `(점수, 실현 XR, lower\_is\_better)`를 돌려준다.

**⚠ \*\*비용이 크다\*\*** – seed마다 5-fold 00F + 최종 모델이라 K=50이면 300회 학습이다. 칸별 통계표를 seed마다 다시 만들지 seed 1개로 고정할지는 아직 미확정이다(#20 구현경로 2).  
"""

```
done = list(range(n_repeats)) if seeds is None else list(seeds)
done = completed_keys(append_to, ("seed",)) if append_to else set()

rows = []
for seed in todo:
 if (str(seed),) in done:
 if verbose:
 print(f" seed {seed:>3} 건너뛴 (이미 계산됨)")
 continue
 score, xr, lower = score_fn(seed)
 res = find_optimal_threshold(score, xr, lower)
 row = {"seed": seed, **res}
 rows.append(row)
 if append_to:
 append_row_csv(append_to, row)
 if verbose:
 print(f" seed {seed:>3} τ ={res['threshold']:.6f} "
 f"승인율 {res['approval_rate']:.3%} "
 f"Sharpe {res['sharpe']:.4f} Δ {res['delta_sharpe']:+.4f}", flush=True)

if append_to and append_to.exists():
 return pd.read_csv(append_to) # 이전 구간까지 합친 전체를 돌려준다
return pd.DataFrame(rows)
```

```
def repeat_full_search(
 seeds: Iterable[int],
 out_path: Path,
 verbose: bool = True,
 scheme: str = "6_2_2",
) -> pd.DataFrame:
 """**본 실행** – 재분할 K회 × 랭킹기준 3종 × 재투자가정 2종의 τ 를 한 번에 낸다 (#18).
```

`repeat\_threshold\_search()`와 무엇이 다른가: 저쪽은 `score\_fn(seed)` 하나를 반복하므로 **\*\*기준마다 모형을 다시 학습\*\***한다( $3 \times 2 \times 50 = 300 \rightarrow$  실제로는 1800회 학습). 여기서는 seed당 **\*\*6회 학습(5-fold 00F + 최종 1)\*\*** 만 하고, 그 위에서 정렬만 바꿔 6조합을 만든다 – 기준·가정은 학습에 개입하지 않기 때문이다(#19). K=50이면 총 300회 학습이다.

Test는 열지 않는다. `resplit\_train\_validation()`이 매니페스트의 Train+Validation 풀만 다시 가르므로 Test 20%는 seed와 무관하게 고정된다.

seed마다 6행을 CSV에 덧붙이고, **\*\*이미 있는 `(seed, reinvest, criterion)`은 건너뛴다\*\*** –

수 시간짜리라 중단·재시작이 필요하다.

"""

**import** sys

sys.path.insert(0, str(Path(\_\_file\_\_).resolve().parents[1]))

**from** analysis.realized\_return **import** ReturnAssumptions, cash\_reinvestment

treasury = ReturnAssumptions()

assumptions = (treasury, cash\_reinvestment(treasury))

seed\_list = list(seeds)

done = completed\_keys(out\_path, ("seed", "reinvest", "criterion"))

**if** done **and** verbose:

    print(f" 이미 계산된 (seed, reinvest, criterion) {len(done)}쌍 - 건너뛰다", flush=True)

data = None

**for** seed **in** seed\_list:

    expected = {(str(seed), a.reinvest, c)

**for** a **in** assumptions

**for** c **in** ("pd (보정 전)", "E[XR]", "q\_score")}

**if** expected <= done:

**if** verbose:

        print(f"\nseed {seed}: 전부 계산됨 - 건너뛴", flush=True)

**continue**

**if** data **is** None:                   # 첫 실제 계산 때만 원본을 읽는다

**if** verbose:

        print("\n원본 로딩 (1회, 이후 seed에서 재사용)", flush=True)

    data = load\_pipeline\_data()

t0 = time.time()

**if** verbose:

    print(f"\n{'=' \* 60}\nseed {seed}", flush=True)

bundle = build\_validation\_scores(seed=seed, verbose=verbose, data=data, scheme=scheme)

auc = float(np.mean(bundle["fold\_auc"]))

**for** a **in** assumptions:

    scores, realized, audit = scores\_for\_assumptions(bundle, a)

**for** crit, (score, lower) **in** scores.items():

**if** (str(seed), a.reinvest, crit) **in** done:

**continue**

        res = find\_optimal\_threshold(score, realized, lower)

        append\_row\_csv(out\_path, {

            "seed": seed, "reinvest": a.reinvest, "assumptions": a.label(),

            "criterion": crit, "fold\_auc\_mean": auc,

            "n\_usable": audit["n\_usable"], \*\*res,

        })

**if** verbose:

        print(f" [{a.reinvest:8s}] {crit:12s} τ\*={res['threshold']:.6f} "

            f"승인율 {res['approval\_rate']:.1%} "

            f"Sharpe {res['sharpe']:.4f} Δ{res['delta\_sharpe']:+.4f}", flush=True)

**if** verbose:

    print(f" seed {seed} 완료 - {time.time() - t0:.0f}초 (fold AUC {auc:.5f})",

        flush=True)

**return** pd.read\_csv(out\_path) **if** out\_path.exists() **else** pd.DataFrame()

```

def summarize_full_repeats(df: pd.DataFrame) -> pd.DataFrame:
 """K회 반복을 (재투자자정 × 랭킹기준) 별로 요약한다.

 `τ*`는 **중앙값**이 최종 확정값이다(#18) - 분포가 비대칭이고, "가장 좋았던 seed의 τ*"를
 고르면 분할 운을 성과로 착각한다(승자의 저주).
 """
 g = df.groupby(["reinvest", "criterion"])
 out = pd.DataFrame({
 "K": g.size(),
 "tau_median": g["threshold"].median(),
 "tau_sd": g["threshold"].std(ddof=1),
 "sharpe_mean": g["sharpe"].mean(),
 "sharpe_sd": g["sharpe"].std(ddof=1),
 "sharpe_max": g["sharpe"].max(),
 "delta_mean": g["delta_sharpe"].mean(),
 "delta_sd": g["delta_sharpe"].std(ddof=1),
 "approval_mean": g["approval_rate"].mean(),
 })
 out["delta_se"] = out["delta_sd"] / np.sqrt(out["K"])
 return out.reset_index()

def summarize_repeats(repeats: pd.DataFrame) -> pd.DataFrame:
 """반복 결과 요약 - `τ*`는 **중앙값**을 우선 본다(분포가 비대칭이다)."""
 cols = ["threshold", "approval_rate", "sharpe", "sharpe_approve_all", "delta_sharpe"]
 return repeats[cols].agg(["mean", "std", "median", "min", "max"]).T

파이프라인 조립

def load_pipeline_data() -> tuple:
 """원본에서 `(X, y, meta, outcome)`을 한 번 읽는다 - **K=50 반복에서 재사용하기 위한 것**.

`build_validation_scores()`를 그냥 반복 호출하면 seed마다 1.2GB 원본을 두 번(피쳐 테이블 +
 실현수익을 입력) 다시 읽는다. seed당 2분이면 K=50에서 100분이 순수 I/O로 날아간다.
 데이터는 seed와 무관하므로(분할만 바뀐다) 밖에서 한 번 읽어 넘긴다.

 """
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from analysis.realized_return import build_return_inputs
 from preprocessing.preprocessor import build_feature_table

 X, y, meta = build_feature_table()
 return X, y, meta, build_return_inputs()

#: 파일명에 fold 수를 적지 않던 시절의 실행이 쓴 fold 수. 그때 만들어진
#: `sharpe_repeat_k50.csv`·`sharpe_repeat_k50_7_3.csv`를 계속 찾을 수 있게 하는 값이다.
LEGACY_N_FOLDS = 5

def repeat_output_path(scheme: str, n_folds: int | None = None) -> Path:
 """K=50 결과 CSV 경로. **체계·fold마다 다른 파일**이라 서로 덮어쓰지 않는다 (#30·#32).

```

`"6\_2\_2"`는 기존 이름을 유지한다 — 그 파일은 **\*\*조기상환 보정 이전\*\***(계약 현금흐름) 실행 기록이라 지금 구현으로 다시 만들면 값이 달라진다. 섞어 인용하지 않는다.

⚠ **\*\*fold 수를 파일명에 넣는 이유가 재시작 로직 때문이다.\*\*** `repeat\_full\_search()`는 이미 있는 `(seed, reinvest, criterion)`을 건너뛰는데, fold 수는 그 키에 없다. 파일명이 같으면 5-fold로 돌린 행을 3-fold 실행이 "이미 계산함"으로 건너뛰어 **\*\*두 fold의 결과가 한 파일에 섞인다.\*\*** 파일을 가르면 그 사고가 구조적으로 막힌다.

`LEGACY\_N\_FOLDS` (5) 일 때만 점미사를 붙이지 않는다 — 그 이름으로 이미 만들어진 파일이 있기 때문이다. 현행 기본값은 3이므로 `...\_8\_2\_3fold.csv`가 나온다.

"""

```
if n_folds is None:
 from analysis.model import DEFAULT_N_FOLDS

 n_folds = DEFAULT_N_FOLDS
stem = "sharpe_repeat_k50" if scheme == "6_2_2" else f"sharpe_repeat_k50_{scheme}"
if n_folds != LEGACY_N_FOLDS:
 stem += f"_{n_folds}fold"
return repo_root() / "outputs" / f"{stem}.csv"
```

```
def build_validation_scores(
 seed: int | None = None,
 verbose: bool = True,
 data: tuple | None = None,
 scheme: str = "6_2_2",
) -> dict:
 """Train으로 학습·보정하고, Validation 점수를 만들 재료를 모은다.
```

**\*\*모형은 재투자 가정과 무관하다\*\*** — 타깃이 이진 `loan\_status`라 수익률 정의가 학습에 개입하지 않는다(#19). 그래서 여기서 한 번만 학습하고, 가정별 계산은 `scores\_for\_assumptions()`가 맡는다.

`data`로 `load\_pipeline\_data()`의 결과를 넘기면 원본을 다시 읽지 않는다(K=50 반복용).

"""

```
import sys

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from analysis.model import (
 apply_calibrator,
 assign_quantile_by_term,
 compute_oof,
 fit_calibrator,
 predict_default_probability,
 quantile_edges_by_term,
 train_model,
)
from analysis.realized_return import build_return_inputs
from preprocessing.preprocessor import (
 build_feature_table,
 resplit_train_validation,
 split_from_manifest,
)

cfg = load_config()

if data is None:
```

```

X, y, meta = build_feature_table()
outcome = build_return_inputs()
else:
 X, y, meta, outcome = data
if seed is None:
 # 기준 실행 - 매니페스트 분할을 그대로 쓴다. Test는 잠긴 채로 남는다.
 parts = split_from_manifest(X, y, meta, scheme=scheme)
 seed = cfg.random_seed.default
else:
 # K=50 반복 - **Test를 고정한 채** Train+Validation 풀만 재분할한다(#18).
 parts = resplit_train_validation(X, y, meta, seed=seed, scheme=scheme)
X_tr, y_tr, _ = parts["train"]
X_va, y_va, _ = parts["validation"]
if verbose:
 print(f" 표본 {len(X):,} 피쳐 {X.shape[1]} "
 f"Train {len(X_tr):,} Validation {len(X_va):,}")

pd_oof, fold_auc, _ = compute_oof(X_tr, y_tr, seed=seed, verbose=verbose)
calibrator = fit_calibrator(pd_oof, y_tr)

final = train_model(X_tr, y_tr, seed=seed)
p_va_raw = predict_default_probability(final, X_va)
p_va_cal = apply_calibrator(calibrator, p_va_raw)

return {
 "pd_oof_raw": pd_oof,
 "p_va_raw": p_va_raw,
 "p_va_cal": p_va_cal,
 "train_index": X_tr.index,
 "validation_index": X_va.index,
 "outcome": outcome,
 "fold_auc": fold_auc,
 "edges_fn": quantile_edges_by_term,
 "assign_fn": assign_quantile_by_term,
}

```

```
def scores_for_assumptions(bundle: dict, assumptions) -> tuple[dict, pd.Series, dict]:
```

"""가정 하나에 대해 Validation 점수 3종 + 실현 `XR`을 만든다.

반환 `(scores, realized\_xr, audit)`. `scores`는 `compare\_ranking\_criteria()`에 그대로 넣는 형태이며, 세 기준 모두 유효한 \*\*공통 마스크\*\*가 이미 적용돼 있다.

⚠ 칸별 통계표(`mu\_부도`, `var\_부도`)는 \*\*Train에서만\*\* 만들고 Validation에 적용한다.

Validation 결과를 보고 만들면 누수다. 분위 경계도 재분위하지 않는다(#20).

## K=50 반복에서 통계표를 seed마다 다시 만드는가 - \*\*다시 만든다\*\* (잠정)

#20 구현경로 2가 남긴 미결이었다. 이 함수는 `bundle["train\_index"]`에서 통계표를 산출하므로, `build\_validation\_scores(seed=k)`가 `resplit\_train\_validation()`으로 Train을 새로 뽑으면 \*\*통계표도 자동으로 새 Train 기준이 된다.\*\* 이 동작을 잠정 확정으로 둔다.

근거: 통계표는 "Train에서만 만든다"는 누수 방지 규칙의 산물이다. seed마다 Train이 바뀌는데 통계표를 seed 하나로 고정하면, 그 고정된 Train에 \*\*다른 seed의 Validation 행이 섞여 있어\*\* 누수가 된다. 계산이 더 들지만(통계표는 `groupby` 하나라 학습 대비 무시할 수준) 규칙과 정합적인 쪽을 택한다. → \*\*팀 확정 시 이 문단을 근거로 올린다.\*\*

"""

```

import sys

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from analysis.realized_return import (
 build_excess_returns,
 default_cell_stats,
 expected_excess_return,
 normal_cell_stats,
 q_score,
 variance_excess_return,
)

xr = build_excess_returns(bundle["outcome"], assumptions)
tr_idx, va_idx = bundle["train_index"], bundle["validation_index"]
term_tr, term_va = xr["term"].loc[tr_idx], xr["term"].loc[va_idx]

분위 경계·배정은 **보정 전** PD로 (진단 C-4의 역할 분리)
edges = bundle["edges_fn"](bundle["pd_oof_raw"], term_tr)
q_tr = bundle["assign_fn"](bundle["pd_oof_raw"], term_tr, edges)
q_va = bundle["assign_fn"](bundle["p_va_raw"], term_va, edges)

def cell_lookup(stat: pd.Series, keys: list) -> pd.Series:
 m = stat.to_dict()
 return pd.Series([m.get(k, np.nan) for k in keys], index=va_idx)

keys = list(zip(term_va, q_va))
d_stats = default_cell_stats(xr["xr_default"].loc[tr_idx], q_tr, term_tr)
mu_default = cell_lookup(d_stats["mu"], keys)
var_default = cell_lookup(d_stats["var"], keys)

정상상환 칸별 통계표 - **조기상환 보장**과 `var_정상` (#20 B팀 1순위, 2026-07-31)
n_stats = normal_cell_stats(
 xr["xr_normal"].loc[tr_idx], xr["xr_normal_realized"].loc[tr_idx], q_tr, term_tr
)
prepay_adj = cell_lookup(n_stats["prepay_adj"], keys)
var_normal = cell_lookup(n_stats["var"], keys)

$E[XR] \cdot Var[XR]$ 의 $\hat{p}$ 는 **보정 후** PD (진단 C-4)
xr_va = xr.loc[va_idx]
xr_normal_adj = xr_va["xr_normal"] - prepay_adj
e_xr = expected_excess_return(bundle["p_va_cal"], xr_normal_adj, mu_default)
v_xr = variance_excess_return(
 bundle["p_va_cal"], xr_normal_adj, mu_default, var_default, var_normal=var_normal
)
qs = q_score(e_xr, v_xr)
realized = xr_va["xr_realized"]

ok = realized.notna() & e_xr.notna() & qs.notna() & bundle["p_va_raw"].notna()
audit = {
 "n_validation": int(len(va_idx)),
 "n_usable": int(ok.sum()),
 "n_dropped": int((~ok).sum()),
 "nan_realized_xr": int(realized.isna().sum()),
 "nan_e_xr": int(e_xr.isna().sum()),
 "nan_q_score": int(qs.isna().sum()),
}

```

```

scores = {
 "pd (보정 전)": (bundle["p_va_raw"][ok], True),
 "E[XR]": (e_xr[ok], False),
 "q_score": (qs[ok], False),
}
return scores, realized[ok], audit

```

```

def parse_seed_spec(spec: str) -> list[int]:
 """`'0-49'` , `'0-9,20'` , `'7'` 형태를 seed 목록으로 푼다."""
 seeds: list[int] = []
 for part in spec.split(","):
 part = part.strip()
 if "-" in part:
 lo, hi = part.split("-")
 seeds.extend(range(int(lo), int(hi) + 1))
 elif part:
 seeds.append(int(part))
 return sorted(dict.fromkeys(seeds))

```

```

def run_repeat(seed_spec: str, scheme: str = "6_2_2") -> None:
 """`--repeat` 경로 - K회 반복을 돌리고 요약을 출력한다."""
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from preprocessing.preprocessor import scheme_ratios

 out_path = repeat_output_path(scheme)
 seeds = parse_seed_spec(seed_spec)

 print("=" * 78)
 print(f"본 실험 - 분할체계 {scheme} · 재분할 K={len(seeds)}회 "
 f"({seed {seeds[0]}~{seeds[-1]}) × 랭킹기준 3종 × 재투자가정 2종")
 print("=" * 78)
 if scheme_ratios(scheme)[2]:
 print("Test 20%는 열지 않는다 - Train+Validation 풀만 재분할한다(#18).")
 else:
 print("Test는 이 표본 밖(2nd Test 파일)이라 재분할해도 흔들리지 않는다 (#30·#32).")
 print(f"산출(덧붙임) → {out_path.name}\n")

 df = repeat_full_search(seeds, out_path, scheme=scheme)
 if df.empty:
 print("결과가 없다.")
 return

 print("\n" + "=" * 78)
 print(f"요약 - τ는 **중앙값**이 확정값이다 (실제 완료 K는 조합별로 표시)")
 print("=" * 78)
 summary = summarize_full_repeats(df)
 print(summary.to_string(index=False, float_format=lambda v: f"{v: .5f}"))

 summary_path = out_path.with_name(out_path.stem + "_summary.csv")
 summary.to_csv(summary_path, index=False)
 print(f"\n요약 산출물 → {summary_path.name}")
 print("다음 단계: 승자 기준을 확정한 뒤 `final_evaluation.py`로 Test 1회.")

```

```

def main() -> None:
 import argparse
 import sys

 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from analysis.realized_return import ReturnAssumptions, cash_reinvestment
 from preprocessing.preprocessor import SPLIT_SCHEMES

 ap = argparse.ArgumentParser(description="Sharpe threshold 탐색")
 ap.add_argument("--repeat", metavar="SEEDS", default=None,
 help="재분할 반복 실행 (예: '0-49'). 생략하면 매니페스트 분할 1회만 돈다.")
 ap.add_argument("--scheme", default="6_2_2", choices=sorted(SPLIT_SCHEMES),
 help="분할 체계 (기본 6_2_2). 7_3·8_2는 Test를 별도 파일로 돈다 - #30·#32")
 args = ap.parse_args()

 if args.repeat:
 run_repeat(args.repeat, scheme=args.scheme)
 return

 out_dir = repo_root() / "outputs"
 out_dir.mkdir(parents=True, exist_ok=True)

 print("=" * 78)
 print("Sharpe threshold 탐색 - 랭킹 기준 3종 × 재투자 가정 2종")
 print("=" * 78)
 print("⚠ 모든 XR은 **잠정값** 기준이다 - 국제 ©발행시점 고정 · 수수료 0% · 조기상환 보정 0.")
 print(" 조기상환 보정은 정상상환분 R을 낮추므로 절대 Sharpe는 과대추정이다(#20 B팀 1순위).")

 print("\n[모형 학습 - 재투자 가정과 무관하므로 1회만 (#19)]")
 bundle = build_validation_scores(scheme=args.scheme)
 print(f" fold AUC 평균 {np.mean(bundle['fold_auc']):.5f}")

 treasury = ReturnAssumptions()
 frames = []
 for assumptions in (treasury, cash_reinvestment(treasury)):
 print("\n" + "=" * 78)
 print(f"[재투자 가정] {assumptions.label()}")
 print("=" * 78)

 scores, realized, audit = scores_for_assumptions(bundle, assumptions)
 print(f" 유효 {audit['n_usable'],} / {audit['n_validation'],}건 "
 f"(제외 {audit['n_dropped'],} - 실현XR NaN {audit['nan_realized_xr'],} · "
 f"E[XR] NaN {audit['nan_e_xr'],} · q_score NaN {audit['nan_q_score'],})")
 print(f" 실현 XR 평균 {realized.mean():+.4%} sd {realized.std(ddof=1):.4%}")

 table = compare_ranking_criteria(scores, realized)
 table.insert(0, "assumptions", assumptions.label())
 table.insert(1, "reinvest", assumptions.reinvest)
 frames.append(table)

 show = table[["criterion", "threshold", "approval_rate", "sharpe",
 "sharpe_approve_all", "delta_sharpe", "mean_xr", "sd_xr"]]
 print("\n" + show.to_string(index=False, float_format=lambda v: f"{v: .5f}"))

 result = pd.concat(frames, ignore_index=True)
 path = out_dir / "sharpe_threshold_comparison.csv"

```

```

result.to_csv(path, index=False)

print("\n" + "=" * 78)
print("Δ Sharpe의 재투자 가정 안정성 (#18이 남긴 검증 과제)")
print("=" * 78)
pivot = result.pivot_table(
 index="criterion", columns="reinvest", values=["sharpe", "delta_sharpe", "approval_rate"]
)
print(pivot.to_string(float_format=lambda v: f"{v: .5f}"))
print("\n Δ가 두 가정에서 비슷하면 #18의 전제('가정이 양쪽을 똑같이 밀어올린다')가 성립한다.")
print(" 크게 다르면 Δ도 가정에 의존하므로 헤드라인 근거를 다시 세워야 한다.")

print(f"\n산출물 → {path.relative_to(repo_root())}")
print("⚠️ 랭킹 기준 확정은 **팀 결정**이다 – 이 표는 근거를 제공한다(#5·#20).")
print(" Test는 승자를 확정한 뒤 1회만 적용한다.")

if __name__ == "__main__":
 main()

```

## src/analysis/final\_evaluation.py

---

**\*\*Test 1회 평가**

```
"""**Test 1회 평가 - 최종 Sharpe Ratio 확정.**
```

파이프라인의 마지막 단계다. 앞 단계에서 확정된 것만 받아서 적용한다.

```
무엇을 Test에 적용하는가 (2026-07-30 결정)
```

`sharpe\_optimizer.py --repeat 0-49`이 낸 \*\*50개 모델 중 Validation Sharpe가 가장 높은 모델\*\*을 그대로 Test에 적용한다.

- 50개 모델은 \*\*같은 스펙\*\*이고 매니페스트 80% 풀을 6:2로 다시 가른 \*\*분할만 다르다.\*\* 따라서 "최고 모델"은 `(seed, train 분할, 모형, 보정기, 분위경계, 칸별 통계표,  $\tau$ )` 한 묶음이며, 이 스크립트는 그 seed를 \*\*똑같이 재현\*\*해 Test에 적용한다(`resplit\_train\_validation(seed)`가 결정적이므로 재현된다).
- 최종 모형의 학습 풀은 그 seed의 \*\*Train 60%\*\* 다 - 이진 모델을 그대로 쓰기 때문이다. Train+Validation 80%로 다시 학습하면  $\tau$ 를 만든 모형과 다른 모형이 Test에 가게 된다.
- \*\*Test에서 threshold를 다시 찾지 않는다.\*\* `find\_optimal\_threshold()`를 부르지 않고 `evaluate\_threshold()`에 이진 seed의  $\tau$ 를 고정값으로 넣는다. Test에서  $\tau$ 를 재탐색하면 "Test set으로 모형을 재조정하지 않는다"(`src/analysis/AGENTS.md`) 위반이다.
- \*\*랭킹 기준은 하나만 Test에 적용한다.\*\* 세 기준(` $pd`/`E[XR]`/`q_score`)$ 을 Test에서 비교하면 규칙 위반이므로, Validation K=50에서 승자를 확정된 뒤 그 하나만 넣는다.

```
보고 시 함께 낼 것 - 선택 규칙의 낙관 편의
```

최고값 선택은 \*\*분할 운을 성과에 포함\*\*한다(승자의 저주). 그래서 산출 CSV에는 헤드라인과 함께 `median\_tau` 행을 남긴다 - 같은 Test 점수에 \*\*K=50  $\tau$  중앙값\*\*을 적용한 값이며, 추가 학습이 없다(정렬만 다르다). 두 값의 차이가 곧 선택 규칙이 만든 낙관분이다.

Validation K=50에서 절대 Sharpe는 평균 0.298 / 최대 0.304였다(제의 스펙 실측) - 최대와 평균의 간격 약 +0.006이 그 크기의 눈금이다. 리포트에서는 \*\*Validation 최고값을 Test 성과로 인용하지 않는다\*\* - Test 값이 최종 성과다.

실행:

```
python src/analysis/final_evaluation.py # 기본: q_score
python src/analysis/final_evaluation.py --criterion pd
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import sys
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
try:
```

```
 from utils.config import load_config, repo_root
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
 from utils.config import load_config, repo_root
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from analysis.model import (# noqa: E402
```

```
 apply_calibrator,
```

```
 assign_quantile_by_term,
```

```

compute_oof,
fit_calibrator,
predict_default_probability,
quantile_edges_by_term,
train_model,
)
from analysis.realized_return import ReturnAssumptions, cash_reinvestment # noqa: E402
from analysis.sharpe_optimizer import (# noqa: E402
 approve_all_sharpe,
 evaluate_threshold,
 load_pipeline_data,
 scores_for_assumptions,
)
from preprocessing.preprocessor import (# noqa: E402
 resplit_train_validation,
 split_from_manifest,
)

REPEAT_CSV = "sharpe_repeat_k50.csv"
OUT_CSV = "final_test_evaluation.csv"

랭킹 기준의 짧은 이름 → `scores_for_assumptions()`가 돌려주는 키
CRITERION_KEYS = {
 "pd": "pd (보정 전)",
 "E[XR]": "E[XR]",
 "q_score": "q_score",
}

헤드라인 재투자 가정 - #18 확정(잔존기간 매칭 국제). `cash`는 민감도.
HEADLINE_REINVEST = "treasury"

어느 모델이 이겼는가

def select_best_model(
 criterion: str,
 reinvest: str = HEADLINE_REINVEST,
 min_k: int = 50,
 scheme: str = "6_2_2",
) -> pd.Series:
 """K=50 중 **Validation Sharpe 최고** seed의 행을 돌려준다.

 `min_k`에 못 미치면 예외로 중단한다 - 반복이 덜 끝난 상태에서 "최고"를 뽑으면 아직
 돌지 않은 seed가 더 좋을 수 있고, **Test는 1회뿐이라 되돌릴 수 없다.**

 `scheme`은 어느 반복 결과를 읽을지 정한다 (`repeat_output_path()`).
 """
 from analysis.sharpe_optimizer import repeat_output_path

 path = repeat_output_path(scheme)
 if not path.exists():
 raise FileNotFoundError(
 f"K=50 결과가 없다: {path}\n"
 f"먼저 `python src/analysis/sharpe_optimizer.py --repeat 0-49 --scheme {scheme}`를 돌려라."
)
 key = CRITERION_KEYS[criterion]

```

```

df = pd.read_csv(path)
sub = df[(df["criterion"] == key) & (df["reinvest"] == reinvest)]
if sub.empty:
 raise ValueError(f"'{key}' × '{reinvest}' 결과가 없다 - 인자를 확인하라.")

k = int(sub["seed"].nunique())
if k < min_k:
 raise RuntimeError(
 f"K={k} < {min_k} - 반복이 아직 덜 끝났다. 남은 seed가 더 좋을 수 있고 "
 "Test는 1회뿐이므로 여기서 중단한다. 끝까지 돌린 뒤 다시 실행하거나 "
 "의도한 것이면 --min-k로 낮춰라."
)

best = sub.loc[sub["sharpe"].idxmax()].copy()
best["K"] = k
best["median_tau"] = float(sub["threshold"].median())
best["val_sharpe_mean"] = float(sub["sharpe"].mean())
best["val_sharpe_sd"] = float(sub["sharpe"].std(ddof=1))
best["val_sharpe_rank"] = int((sub["sharpe"] > best["sharpe"]).sum()) + 1
return best

이긴 모델 재현 → Test 예측

def rebuild_winner_for_test(
 seed: int, data: tuple, te_idx: pd.Index, verbose: bool = True
) -> dict:
 """이긴 seed의 모델을 **그대로 재현**하고 Test 점수 재료를 만든다.

 `scores_for_assumptions()`가 먹는 형태로 돌려준다 - `validation_index` 자리에 Test
 인덱스를 넣는다. 방법론(분위 경계는 Train OOF PD, 칸별 통계표는 Train에서만, `E[XR]`의
 `p`는 보정 후 PD)을 K=50 단계와 **한 줄도 다르게 하지 않가** 위한 것이다.
 """
 X, y, meta, outcome = data
 parts = resplit_train_validation(X, y, meta, seed=seed)
 X_tr, y_tr, _ = parts["train"]
 X_te = X.loc[te_idx]
 if verbose:
 print(f"\n[이긴 모델 재현] seed {seed} 학습 {len(X_tr):,}건 Test {len(X_te):,}건")

 pd_oof, fold_auc, _ = compute_oof(X_tr, y_tr, seed=seed, verbose=verbose)
 calibrator = fit_calibrator(pd_oof, y_tr)
 final = train_model(X_tr, y_tr, seed=seed)
 p_te_raw = predict_default_probability(final, X_te)
 p_te_cal = apply_calibrator(calibrator, p_te_raw)

 return {
 "pd_oof_raw": pd_oof,
 "p_va_raw": p_te_raw, # `scores_for_assumptions()`의 "적용 대상" 자리 = Test
 "p_va_cal": p_te_cal,
 "train_index": X_tr.index,
 "validation_index": te_idx,
 "outcome": outcome,
 "fold_auc": fold_auc,
 "edges_fn": quantile_edges_by_term,
 "assign_fn": assign_quantile_by_term,
 }

```

```
}
```

```
def evaluate_on_test(
 bundle: dict, best: pd.Series, criterion: str, verbose: bool = True
) -> pd.DataFrame:
 """재투자 가정 2종 × (이진 τ · K=50 중앙값 τ)를 Test에서 평가한다.

 추가 학습은 없다 - 모형은 하나고 정렬·컷만 바꾼다(#19).
 """
 key = CRITERION_KEYS[criterion]
 treasury = ReturnAssumptions()
 rows: list[dict] = []

 for assumptions in (treasury, cash_reinvestment(treasury)):
 scores, realized, audit = scores_for_assumptions(bundle, assumptions)
 score, lower = scores[key]
 base = approve_all_sharpe(realized)

 taus = {
 "winner_tau": float(best["threshold"]), # 헤드라인
 "median_tau": float(best["median_tau"]), # 선택 편의 진단용
 }
 for vname, tau in taus.items():
 res = evaluate_threshold(score, realized, tau, lower_is_better=lower)
 rows.append({
 "variant": vname,
 "reinvest": assumptions.reinvest,
 "assumptions": assumptions.label(),
 "criterion": criterion,
 "best_seed": int(best["seed"]),
 "K": int(best["K"]),
 "fold_auc_mean": float(np.mean(bundle["fold_auc"])),
 "n_test_usable": audit["n_usable"],
 "n_test_dropped": audit["n_dropped"],
 **res,
 "sharpe_approve_all": base,
 "delta_sharpe": res["sharpe"] - base,
 "val_sharpe_winner": float(best["sharpe"]),
 "val_sharpe_mean": float(best["val_sharpe_mean"]),
 "val_approval_rate": float(best["approval_rate"]),
 })
 if verbose:
 print(f" [{assumptions.reinvest:8s}] {vname:11s} τ ={{tau:.6f}} "
 f"승인율 {res['approval_rate']:.1%} "
 f"Test Sharpe {res['sharpe']:.4f} Δ {{res['sharpe'] - base:+.4f}} "
 f"(approve-all {base:.4f})")
 return pd.DataFrame(rows)

def main() -> None:
 ap = argparse.ArgumentParser(description="Test 1회 평가 - 최종 Sharpe Ratio 확정")
 ap.add_argument("--criterion", default="q_score", choices=list(CRITERION_KEYS),
 help="Validation K=50에서 확정한 승인선 랭킹 기준 (기본: q_score)")
 ap.add_argument("--reinvest", default=HEADLINE_REINVEST, choices=["treasury", "cash"],
 help="최고 모델을 고를 기준이 되는 재투자 가정 (기본: treasury - #18)")
```

```

ap.add_argument("--min-k", type=int, default=50,
 help="이 K에 못 미치면 중단한다 (기본 50 - #18)")
args = ap.parse_args()

print("=" * 78)
print("최종 Test 평가 - Test set을 여는 단 한 번의 실행")
print("=" * 78)
print(f"랭킹 기준 : {args.criterion} 선택 기준 가정 : {args.reinvest}")
print("선택 규칙 : K=50 중 **Validation Sharpe 최고** 모델을 그대로 적용")
print("⚠ XR은 **잠정 가정**이다 - 국제 ☺발행시점 고정 · 수수료 0% · 조기상환 보정 0(#20).")

best = select_best_model(args.criterion, args.reinvest, min_k=args.min_k)
print(f"\n[이진 모델] seed {int(best['seed'])} "
 f"Validation Sharpe {best['sharpe']:.4f} "
 f"(K={int(best['K'])} 중 1위, 평균 {best['val_sharpe_mean']:.4f} "
 f"sd {best['val_sharpe_sd']:.4f})")
print(f" τ* {best['threshold']:.6f} 승인을 {best['approval_rate']:.1%} "
 f"Δ {best['delta_sharpe']:+.4f}")
print(f" 참고 - K=50 τ* 중앙값 {best['median_tau']:.6f} (선택 편의 진단용으로 병기)")

print("\n[원본 로딩]")
data = load_pipeline_data()
X, y, meta, _ = data

print("\n[분할 - Test를 연다]")
parts = split_from_manifest(X, y, meta, unlock_test=True)
te_idx = parts["test"][0].index

bundle = rebuild_winner_for_test(int(best["seed"]), data, te_idx)
result = evaluate_on_test(bundle, best, args.criterion)

out_path = repo_root() / "outputs" / OUT_CSV
result.to_csv(out_path, index=False)

print("\n" + "=" * 78)
print("최종 결과")
print("=" * 78)
show = result[["variant", "reinvest", "threshold", "approval_rate", "sharpe",
 "sharpe_approve_all", "delta_sharpe", "val_sharpe_winner"]]
print(show.to_string(index=False, float_format=lambda v: f"{v: .5f}"))

head = result[(result["variant"] == "winner_tau")
 & (result["reinvest"] == HEADLINE_REINVEST)]
if not head.empty:
 h = head.iloc[0]
 print(f"\n▶ 최종 Sharpe Ratio = **{h['sharpe']:.4f}** "
 f"(approve-all {h['sharpe_approve_all']:.4f}, Δ {h['delta_sharpe']:+.4f}, "
 f"승인율 {h['approval_rate']:.1%}, seed {int(h['best_seed'])})")
 print(f" Validation에서 본 값 {h['val_sharpe_winner']:.4f} → Test {h['sharpe']:.4f} "
 f" (차이 {h['sharpe'] - h['val_sharpe_winner']:+.4f})")
 print(" ※ Validation 값은 그 분할에서 최고였던 값이다 - 최종 성과는 Test 쪽이다.")

print(f"\n산출물 → outputs/{OUT_CSV}")
print("⚠ 이 실행으로 Test는 소진됐다. 결과를 보고 모형·threshold를 바꾸면 규칙 위반이다.")

```

```
if __name__ == "__main__":
 main()
```

**src/analysis/second\_test\_evaluation.py**

---

2nd Test 평가

"""\*\*2nd Test 평가\*\* - 외부 표본(`lending\_club\_2020\_test\_2nd.csv`)에서 Sharpe를 낸다.

`final\_evaluation.py`가 매니페스트 6:2:2의 Test 20%(144,713건)에 적용한 것과 \*\*완전히 같은 모델·같은  $\tau$ \*\*를, train 파일과 `id`가 한 건도 겹치지 않는 \*\*별도 파일\*\*(481,833건)에 적용한다.

## 무엇이 `final\_evaluation.py`와 같고 무엇이 다른가

```
	`final_evaluation.py`	이 스크립트
모델	K=50 승자 seed 재현	**동일** (같은 seed·같은 Train 60%)
τ	승자 τ * 고정	**동일** (재탐색 없음)
칸별 통계표·분위 경계	승자 seed의 Train	**동일**
적용 대상	매니페스트 Test 20%	**2nd Test 파일 전체**
```

즉 \*\*모형 쪽은 한 줄도 다르지 않고 적용 대상만 바뀐다.\*\* 그래서 두 결과의 차이는 오롯이 "다른 표본에서도 재현되는가"에 대한 답이다.

## 두 가지 함정과 대응

1. \*\*범주 정렬\*\* - `coerce\_dtypes()`는 각 프레임의 값에서 category를 만든다. 2nd Test를 그냥 읽으면 category \*\*코드가 train과 어긋나\*\* XGBoost가 다른 범주로 해석한다(AUC가 조용히 무너진다). `align\_categories()`가 train 프레임의 categories를 그대로 씌운다 - train에 없던 값은 NaN이 되며, 이는 XGBoost가 native로 처리한다.
2. \*\*인덱스 충돌\*\* - 두 파일 모두 0부터 시작하는 행 인덱스라 그대로 합치면 겹친다. 2nd Test 인덱스에 `INDEX\_OFFSET`을 더해 `scores\_for\_assumptions()`가 쓰는 단일 프레임을 만든다(그 함수를 그대로 재사용하기 위한 것 - 방법론을 복제하면 갈라진다).

## `oracle\_tau` 행을 어떻게 읽는가

산출 CSV에는 \*\*2nd Test에서 다시 탐색한  $\tau$ \*\*(`oracle\_tau`)가 진단용으로 들어 있다. \*\*이 값을 성과로 인용하지 않는다\*\* - 적용 대상을 보고 고른 값이라 정의상 낙관적이다. 쓰임은 하나뿐이다: `winner\_tau`와의 차이가 작으면  $\tau$  이전이 잘 된 것이고, 크면  $\tau$ 가 분할에 과적합됐다는 뜻이다.

실행:

```
/opt/anaconda3/bin/python src/analysis/second_test_evaluation.py
```

=====

```
from __future__ import annotations
```

```
import argparse
```

```
import sys
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
try:
```

```
 from utils.config import repo_root
```

```
except ModuleNotFoundError: # pragma: no cover
```

```
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
 from utils.config import repo_root
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from analysis.final_evaluation import CRITERION_KEYS, select_best_model # noqa: E402
```

```

from analysis.model import (# noqa: E402
 apply_calibrator,
 assign_quantile_by_term,
 compute_oof,
 fit_calibrator,
 predict_default_probability,
 quantile_edges_by_term,
 train_model,
)

from analysis.realized_return import (# noqa: E402
 ReturnAssumptions,
 build_return_inputs,
 cash_reinvestment,
)

from analysis.sharpe_optimizer import (# noqa: E402
 approve_all_sharpe,
 evaluate_threshold,
 find_optimal_threshold,
 load_pipeline_data,
 scores_for_assumptions,
)

from preprocessing.loader import second_test_path # noqa: E402
from preprocessing.preprocessor import (# noqa: E402
 SPLIT_SCHEMES,
 build_feature_table,
 resplit_train_validation,
)

OUT_CSV_STEM = "second_test_evaluation"
HEADLINE_REINVEST = "treasury"

2nd Test 인덱스에 더할 값. train 원본이 1,755,295행이므로 어떤 값과도 겹치지 않는다.
INDEX_OFFSET = 10_000_000

2nd Test 로딩

def align_categories(X_ref: pd.DataFrame, X_new: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
 """`X_new`의 category 컬럼을 **`X_ref`의 categories로 다시 씌운다.**

 XGBoost는 category dtype을 **코드(정수)**로 받으므로, 두 프레임이 각자 값에서 만든
 categories를 쓰면 같은 코드가 다른 범주를 가리킨다. 학습 때 본 적 없는 값은 NaN이 되며
 (native 결측 처리로 흘러간다) - 없는 범주에 대한 분기는 애초에 학습되지 않았다.
 """
 out = X_new.copy()
 unseen: dict[str, int] = {}
 for col in X_ref.columns:
 if not isinstance(X_ref[col].dtype, pd.CategoricalDtype):
 continue
 cats = X_ref[col].cat.categories
 values = out[col].astype("object")
 n_unseen = int(values.notna().sum() - values.isin(cats).sum())
 if n_unseen:
 unseen[col] = n_unseen
 out[col] = pd.Categorical(values, categories=cats, ordered=X_ref[col].cat.ordered)
 return out, unseen

```

```

def load_second_test(X_ref: pd.DataFrame, verbose: bool = True) -> tuple:
 """2nd Test를 **같은 파이프라인**으로 읽어 `(X2, y2, meta2, outcome2)`를 돌려준다.

 표본 필터는 동일하되 건수 검증만 끈다 - `EXPECTED_SAMPLE_SIZE`는 train 기준이다.
 인덱스에는 `INDEX_OFFSET`이 더해져 있다.
 """
 path = second_test_path()
 if verbose:
 print(f" 파일 {path.name}")

 X2, y2, meta2 = build_feature_table(verify_sample=False, csv_path=path)
 outcome2 = build_return_inputs(csv_path=path, verify_sample=False)

 if list(X2.columns) != list(X_ref.columns):
 raise ValueError(
 "2nd Test의 피쳐 구성이 train과 다르다 - 컬럼 목록을 확인하라.\n"
 f" train에만: {sorted(set(X_ref.columns) - set(X2.columns))}\n"
 f" test에만 : {sorted(set(X2.columns) - set(X_ref.columns))}"
)
 if not X2.index.equals(outcome2.index):
 raise ValueError("피쳐 테이블과 수익률 입력의 인덱스가 어긋난다 - 필터가 다르게 걸렸다.")

 X2, unseen = align_categories(X_ref, X2)
 if verbose:
 print(f" 표본 {len(X2):,}건 부도율 {y2.mean():.4%} 피쳐 {X2.shape[1]}")
 if unseen:
 top = sorted(unseen.items(), key=lambda kv: -kv[1][:5])
 total = sum(unseen.values())
 print(f" ⚠️ train에 없던 범주값 {total:,}건 → NaN 처리 "
 f"('{', '.join(f'{{c}} {{n:,}}' for c, n in top)}')")
 else:
 print(" 범주값은 전부 train에서 관측된 것이다")

 idx = pd.Index(X2.index + INDEX_OFFSET, name=X2.index.name)
 for frame in (X2, meta2, outcome2):
 frame.index = idx
 y2.index = idx
 return X2, y2, meta2, outcome2

승자 재현 → 2nd Test 예측

def rebuild_winner_for_second_test(
 seed: int, data: tuple, verbose: bool = True, scheme: str = "6_2_2"
) -> dict:
 """승자 seed의 모델을 재현하고, 2nd Test 점수 재료를 `bundle` 형태로 만든다.

 `scores_for_assumptions()`가 그대로 먹도록 train·2nd Test의 `outcome`을 **하나의 프레임으로
 이어붙인다** (인덱스가 어긋나 있으므로 안전하다). 방법론(분위 경계는 Train 00F PD, 칸별
 통계표는 Train에서만, `E[XR]`의 `p`는 보정 후 PD)을 K=50·`final_evaluation`과 한 줄도
 다르게 하지 않기 위한 것이다.
 """
 X, y, meta, outcome = data
 parts = resplit_train_validation(X, y, meta, seed=seed, scheme=scheme)

```

```

X_tr, y_tr, _ = parts["train"]

print("\n[2nd Test 로딩 - 같은 전처리 파이프라인]")
X2, y2, _, outcome2 = load_second_test(X_tr, verbose=verbose)
if outcome.index.intersection(outcome2.index).size:
 raise ValueError("인덱스 오프셋이 충분하지 않다 - INDEX_OFFSET을 키워라.")

if verbose:
 print(f"\n[승자 재현] seed {seed} 학습 {len(X_tr):,}건 2nd Test {len(X2):,}건")

pd_oof, fold_auc, _ = compute_oof(X_tr, y_tr, seed=seed, verbose=verbose)
calibrator = fit_calibrator(pd_oof, y_tr)
final = train_model(X_tr, y_tr, seed=seed)
p2_raw = predict_default_probability(final, X2)
p2_cal = apply_calibrator(calibrator, p2_raw)

return {
 "pd_oof_raw": pd_oof,
 "p_va_raw": p2_raw, # "적용 대상" 자리 = 2nd Test
 "p_va_cal": p2_cal,
 "train_index": X_tr.index,
 "validation_index": X2.index,
 "outcome": pd.concat([outcome, outcome2]),
 "fold_auc": fold_auc,
 "edges_fn": quantile_edges_by_term,
 "assign_fn": assign_quantile_by_term,
 "y_second": y2,
}

def evaluate_on_second_test(
 bundle: dict, best: pd.Series, criterion: str, verbose: bool = True
) -> pd.DataFrame:
 """재투자 가정 2종 × (승자 τ^* · K=50 중앙값 τ^* · 진단용 oracle τ)를 평가한다.

 추가 학습은 없다 - 모형은 하나고 정렬·컷만 바꾼다(#19).
 """
 from sklearn.metrics import roc_auc_score

 key = CRITERION_KEYS[criterion]
 treasury = ReturnAssumptions()
 y2 = bundle["y_second"]
 auc2 = float(roc_auc_score(y2, bundle["p_va_raw"]))
 rows: list[dict] = []

 for assumptions in (treasury, cash_reinvestment(treasury)):
 scores, realized, audit = scores_for_assumptions(bundle, assumptions)
 score, lower = scores[key]
 base = approve_all_sharpe(realized)
 oracle = find_optimal_threshold(score, realized, lower)

 taus = {
 "winner_tau": float(best["threshold"]), # 헤드라인 - Validation에서 확정된 값
 "median_tau": float(best["median_tau"]), # 선택 편의 진단용
 "oracle_tau": float(oracle["threshold"]), # ⚠️ 2nd Test를 보고 고른 값 - 인용 금지
 }
 for vname, tau in taus.items():

```

```

res = evaluate_threshold(score, realized, tau, lower_is_better=lower)
rows.append({
 "variant": vname,
 "reinvest": assumptions.reinvest,
 "assumptions": assumptions.label(),
 "criterion": criterion,
 "best_seed": int(best["seed"]),
 "K": int(best["K"]),
 "fold_auc_mean": float(np.mean(bundle["fold_auc"])),
 "auc_second_test": auc2,
 "n_second_usable": audit["n_usable"],
 "n_second_dropped": audit["n_dropped"],
 **res,
 "sharpe_approve_all": base,
 "delta_sharpe": res["sharpe"] - base,
 "val_sharpe_winner": float(best["sharpe"]),
 "val_sharpe_mean": float(best["val_sharpe_mean"]),
 "val_approval_rate": float(best["approval_rate"]),
})
if verbose:
 mark = " ← 진단 전용" if vname == "oracle_tau" else ""
 print(f" [{assumptions.reinvest:8s}] {vname:11s} τ={tau:.6f} "
 f"승인율 {res['approval_rate']:.1%} "
 f"Sharpe {res['sharpe']:.4f} Δ{res['sharpe'] - base:+.4f} "
 f"(approve-all {base:.4f}){mark}")

return pd.DataFrame(rows)

def main() -> None:
 ap = argparse.ArgumentParser(description="2nd Test 평가 - 외부 표본 Sharpe")
 ap.add_argument("--criterion", default="q_score", choices=list(CRITERION_KEYS),
 help="Validation K=50에서 확정한 승인선 랭킹 기준 (기본: q_score)")
 ap.add_argument("--reinvest", default=HEADLINE_REINVEST, choices=["treasury", "cash"],
 help="승자를 고르는 기준이 되는 재투자 가정 (기본: treasury - #18)")
 ap.add_argument("--min-k", type=int, default=50, help="이 K에 못 미치면 중단 (기본 50)")
 ap.add_argument("--scheme", default="6_2_2", choices=sorted(SPLIT_SCHEMES),
 help="분할 체계 (기본 6_2_2). 8_2가 #32 현행 체계다.")
 args = ap.parse_args()
 out_csv = (f"{OUT_CSV_STEM}.csv" if args.scheme == "6_2_2"
 else f"{OUT_CSV_STEM}_{args.scheme}.csv")

 print("=" * 78)
 print("2nd Test 평가 - lending_club_2020_test_2nd.csv (train과 id 무교집합)")
 print("=" * 78)
 print(f"랭킹 기준 : {args.criterion} 선택 기준 가정 : {args.reinvest}")
 print(f"모델·τ는 final_evaluation.py와 **동일**하다 - 적용 대상만 바꾼다.")
 print(f"XR 가정(#22 확정): 국제 ©발행시점 고정 · 수수료 0% · 조기상환 건별 현금흐름 반영.")

 best = select_best_model(args.criterion, args.reinvest, min_k=args.min_k,
 scheme=args.scheme)
 print(f"\n[승자] seed {int(best['seed'])} Validation Sharpe {best['sharpe']:.4f} "
 f"K={int(best['K'])} 중 1위 τ* {best['threshold']:.6f}")

 print("\n[train 원본 로딩]")
 data = load_pipeline_data()

```

```

bundle = rebuild_winner_for_second_test(int(best["seed"]), data, scheme=args.scheme)
result = evaluate_on_second_test(bundle, best, args.criterion)
result.insert(0, "scheme", args.scheme)

out_path = repo_root() / "outputs" / out_csv
result.to_csv(out_path, index=False)

print("\n" + "=" * 78)
print("2nd Test 결과")
print("=" * 78)
show = result[["variant", "reinvest", "threshold", "approval_rate", "sharpe",
 "sharpe_approve_all", "delta_sharpe"]]
print(show.to_string(index=False, float_format=lambda v: f"{v: .5f}"))

head = result[(result["variant"] == "winner_tau")
 & (result["reinvest"] == HEADLINE_REINVEST)]
if not head.empty:
 h = head.iloc[0]
 print(f"\n▶ 2nd Test Sharpe Ratio = **{h['sharpe']:.4f}** "
 f"(approve-all {h['sharpe_approve_all']:.4f}, Δ {h['delta_sharpe']:+.4f}, "
 f"승인율 {h['approval_rate']:.1%})")
 print(f" 2nd Test AUC {h['auc_second_test']:.5f} "
 f"(승자 seed의 Train fold AUC {h['fold_auc_mean']:.5f})")
 print(f" 유효 {int(h['n_second_usable']):,}건 / 제외 {int(h['n_second_dropped']):,}건")

print(f"\n산출물 → outputs/{out_csv}")
print("⚠ `oracle_tau` 행은 2nd Test를 보고 고른 값이다 — 성과로 인용하지 않는다.")

if __name__ == "__main__":
 main()

```

## 분석 — 재현·검증

보고서 근거 표를 다시 만드는 스크립트. 수치를 의심할 때 돌린다.

**src/analysis/preprocessing\_validation.py**

변수 전처리 방식(결측치 처리 규칙) 검증 분석.

""" 변수 전처리 방식(결측치 처리 규칙) 검증 분석.

outputs/reports/preprocessing\_validation\_kgj.md 에 실린 표를 전부 재생성한다.  
문서의 숫자를 검증하거나 시트가 개정된 뒤 다시 계산할 때 이 스크립트를 돌린다.

검증 대상은 data/processed/lending\_club\_변수분류\_류성환.xlsx\_v2.numbers 에 정리된  
결측 티어(T0~T3) 분류와 티어별 처방이다.

이 스크립트는 \*\*탐색·검증\*\*용이다 – 모형 학습이나 threshold 결정과는 무관하며,  
src/analysis/AGENTS.md의 Sharpe Ratio 기반 의사결정 규칙이 적용되는 단계가 아니다.  
5절의 AUC 비교는 처리 방식 간 상대 비교가 목적이므로 AUC를 쓰지만, 승인/거절 기준을  
정하는 데 쓰지 않는다.

표본 필터링은 src/preprocessing/AGENTS.md 규칙을 따른다:

- loan\_status가 Current / Late (16-30 days) / Late (31-120 days)인 행 제외
- "Does not meet the credit policy. Status:\*" 는 접두어를 떼고 동일하게 취급

입력은 data/raw/lending\_club\_2020\_train.csv (~1.2GB)다. 이 파일은 용량 때문에  
git에 올리지 않으므로(.gitignore 참고) 팀 공유 채널에서 받아 data/raw/ 에 두고 실행한다.

실행:

```
python src/analysis/preprocessing_validation.py # 1~4, 6절 (수 분)
python src/analysis/preprocessing_validation.py --with-model # 5절 AUC 비교까지 (수십 분)
```

"""

```
import argparse
from pathlib import Path
```

```
import numpy as np
import pandas as pd
```

```
REPO_ROOT = Path(__file__).resolve().parents[2]
RAW_FILE = REPO_ROOT / "data" / "raw" / "lending_club_2020_train.csv"
```

```
EXCLUDED = ["Current", "Late (16-30 days)", "Late (31-120 days)"]
POLICY_PREFIX = "Does not meet the credit policy. Status:"
```

# 시트 「결측티어」 분류 – 검증 대상

```
T2 = [
 "il_util", "mths_since_rcnt_il", "all_util", "inq-fi", "inq_last_12m", "max_bal_bc",
 "open_acc_6m", "open_act_il", "open_il_12m", "open_il_24m", "open_rv_12m",
 "open_rv_24m", "total_bal_il", "total_cu_tl",
]
T3 = [
 "mths_since_last_record", "mths_since_recent_bc_dlq", "mths_since_last_major_derog",
 "mths_since_recent_revol_delinq", "mths_since_last_delinq", "mths_since_recent_inq",
]
T1_SAMPLE = [
 "emp_length", "mort_acc", "avg_cur_bal", "mo_sin_old_il_acct",
 "mths_since_recent_bc", "num_sats", "dti", "revol_util",
]
```

# 5절 AUC 비교에서 공통 통제로 쓰는 결측 없는 기본 피처

```
BASE_FEATURES = [
 "loan_amnt", "annual_inc", "dti", "fico_range_low", "delinq_2yrs", "inq_last_6mths",
 "open_acc", "pub_rec", "revol_bal", "total_acc", "emp_length", "term",
]
```

```

T3 중 관측값 자체는 신호가 약해 '더미만' 남겨도 되는 변수 (문서 5절 처방 ③).
mths_since_last_record는 경계선이다 - 2016~2018 구간에서는 스프레드 2.68pp라 '값도 신호'로
판정되지만 2016~2020으로 넓히면 1.82pp로 뒤집힌다. 여기서는 공격적인 쪽(더미만)으로 두고,
그래도 성능 손실이 잡음 이하임을 9절에서 확인한다.
T3_FLAT = [
 "mths_since_last_record", "mths_since_recent_bc_dlt", "mths_since_last_major_derog",
 "mths_since_recent_revol_delinq", "mths_since_last_delinq",
]

EMP_LENGTH_MAP = {
 "< 1 year": 0, "1 year": 1, "2 years": 2, "3 years": 3, "4 years": 4, "5 years": 5,
 "6 years": 6, "7 years": 7, "8 years": 8, "9 years": 9, "10+ years": 10,
}

def header(title: str) -> None:
 print("\n" + "=" * 78)
 print(title)
 print("=" * 78)

def load_loans(usecols: list[str]) -> pd.DataFrame:
 """ 원본을 읽어 전처리 규칙대로 필터링하고 부도 라벨·발행연도를 만든다. """
 need = sorted(set(usecols) | {"loan_status", "issue_d"})
 raw = pd.read_csv(RAW_FILE, usecols=need, low_memory=False)

 status = raw["loan_status"].astype(str).replace(POLICY_PREFIX, "", regex=False)
 keep = status.isin(["Fully Paid", "Charged Off"])

 df = raw[keep].copy()
 df["y"] = (status[keep] == "Charged Off").astype(int)
 df["issue_dt"] = pd.to_datetime(df["issue_d"], format="%b-%Y", errors="coerce")
 df["year"] = df["issue_dt"].dt.year
 return df

def missing_vs_default(df: pd.DataFrame, cols: list[str], label: str) -> pd.DataFrame:
 """ 결측 여부별 부도율과 2-bin IV를 계산한다 (문서 3절 표). """
 n_bad = int(df["y"].sum())
 n_good = len(df) - n_bad
 rows = []
 for c in cols:
 miss = df[c].isna()
 bad_m, n_m = int(df.loc[miss, "y"].sum()), int(miss.sum())
 good_m = n_m - bad_m

 # 결측 bin과 관측 bin 두 개로 IV를 구한다.
 iv = 0.0
 for bad, good in ((bad_m, good_m), (n_bad - bad_m, n_good - good_m)):
 p_bad, p_good = bad / n_bad, good / n_good
 iv += (p_bad - p_good) * np.log((p_bad + 1e-9) / (p_good + 1e-9))

 rows.append({
 "변수": c,
 "결측률%": miss.mean() * 100,
 "부도율_결측%": df.loc[miss, "y"].mean() * 100 if n_m else np.nan,
 "부도율_관측%": df.loc[~miss, "y"].mean() * 100,
 })

```

```

 "차이pp": (df.loc[miss, "y"].mean() - df.loc[~miss, "y"].mean()) * 100,
 "IV": iv,
 })

```

```

table = pd.DataFrame(rows).sort_values("결측률%", ascending=False)
print(f"\n----- {label} (전체 부도를 {df['y'].mean() * 100:.2f}%) -----")
print(table.to_string(index=False, float_format=lambda x: f"{x:8.3f}"))
return table

```

```

def identical_missing_groups(df: pd.DataFrame, cols: list[str]) -> list[list[str]]:
 """결측벡터가 비트 단위로 일치하는 변수들을 묶는다 (문서 5절 처방 ②)."""
 groups: list[list[str]] = []
 for c in cols:
 for g in groups:
 if df[c].isna().equals(df[g[0]].isna()):
 g.append(c)
 break
 else:
 groups.append([c])
 return groups

```

```

def section_sample(df: pd.DataFrame) -> None:
 header("1) 표본 구성 - 행 필터링 규칙 적용 후")
 print(f"완결표본 n = {len(df):,} 부도를 {df['y'].mean() * 100:.2f}%")
 print(f"Charged Off = {int(df['y'].sum()):,}")
 print("\n연도별 건수·부도를:")
 print(df.groupby("year")["y"].agg(["size", "mean"]).round(4).to_string())

```

```

def section_missrate_base(df: pd.DataFrame) -> None:
 """시트의 원본 전수 기준 결측률과 모델링 표본 기준을 대조한다 (문서 2절 ①)."""
 header("2) 결측률 기준 비교 - 시트(원본 전수) vs 모델링 표본(완결)")
 cols = T2 + T3 + ["emp_length"]
 full = pd.read_csv(RAW_FILE, usecols=cols, low_memory=False)
 cmp = pd.DataFrame({
 "원본전수%": full[cols].isna().mean() * 100,
 "완결표본%": df[cols].isna().mean() * 100,
 })
 cmp["차이pp"] = cmp["완결표본%"] - cmp["원본전수%"]
 print(cmp.round(2).to_string())

```

```

def section_t2_is_vintage(df: pd.DataFrame) -> None:
 """T2 결측더미가 신용 신호인지 발행 시점인지 판정한다 (문서 2절 ②)."""
 header("3) T2 결측더미 = 발행 시점인가")

 print("\n연도별 T2 결측률(%)")
 print(df.groupby("year")[T2].apply(lambda g: g.isna().mean() * 100).round(1).to_string())

 d_t2 = df["all_util"].isna().astype(int)
 print(f"\nD_T2 ~ 발행연도 상관 : {d_t2.corr(df['year']):+.4f}")
 print(f"D_T2 == (발행연도 < 2016) 일치율: {(d_t2 == (df['year'] < 2016).astype(int)).mean():.4f}")

 y15 = df[df["year"] == 2015]
 print("\n연도 고정(2015년 - 유일한 현재 연도) 시 잔여 신호:")

```

```

print(y15.groupby(y15["all_util"].isna())["y"].agg(["size", "mean"]).round(4).to_string())

print("\n2015년 월별 T2 결측 비율 - 수집 개시 시점:")
print(y15.groupby(y15["issue_dt"].dt.month)["all_util"].apply(
 lambda s: s.isna().mean()).round(3).to_string())

def section_first_observed(df: pd.DataFrame) -> None:
 """T3로 분류된 변수에 수집 블록 성격이 섞여 있는지 본다 (문서 2절 ㉓)."""
 header("4) 변수별 최초 관측 연월 - 수집 블록 여부")
 for c in T3 + ["il_util"]:
 first = df.loc[df[c].notna(), "issue_dt"].min()
 print(f" {c:32s} {first.strftime('%Y-%m')} if pd.notna(first) else 'NA'")

def section_missing_meaning(df: pd.DataFrame) -> None:
 """결측이 신호도 신호인지, 연도 고정 후에도 남는지 확인한다 (문서 3절)."""
 header("5) 결측의 의미 - 결측 여부별 부도율")
 missing_vs_default(df, T3, "T3 사건없음형")
 missing_vs_default(df, T2, "T2 구조적 미수집형")
 missing_vs_default(df, T1_SAMPLE, "T1 수집실패형(일부)")

 print("\n----- 연도 고정(2016~2018) 후에도 신호가 남는가 -----")
 fixed = df[df["year"].between(2016, 2018)]
 for c in T3:
 miss = fixed[c].isna()
 gap = (fixed.loc[miss, "y"].mean() - fixed.loc[~miss, "y"].mean()) * 100

 # 관측값 자체가 신호인지 - 4분위 부도율 스프레드로 본다.
 obs = fixed.loc[~miss, [c, "y"]]
 by_q = obs.groupby(pd.qcut(obs[c], 4, duplicates="drop"), observed=True)["y"].mean() * 100
 spread = by_q.max() - by_q.min()
 verdict = "값도 신호" if spread > 2 else "값은 거의 무신호(더미만 남겨도 됨)"
 print(f" {c:32s} 더미효과 {gap:+6.2f}pp | 값 4분위 스프레드 {spread:5.2f}pp -> {verdict}")

def section_post_2015(df: pd.DataFrame) -> None:
 """수집 개시 이후 남은 결측의 원인을 진단한다 (문서 4절)."""
 header("6) 2015-12 수집 개시 이후의 잔여 결측 - 원인 진단")
 post = df[df["year"] >= 2016]
 print(f"2016년 이후 완결표본 n = {len(post):,}, 부도율 {post['y'].mean() * 100:.2f}%\n")

 for c in ["il_util", "mths_since_rcnt_il", "all_util", "total_bal_il"]:
 miss = post[c].isna()
 if not miss.any():
 print(f" {c:20s} 결측 0.00% - 잔여 결측 없음")
 continue
 gap = (post.loc[miss, "y"].mean() - post.loc[~miss, "y"].mean()) * 100
 no_il = (post.loc[miss, "open_act_il"] == 0).mean() * 100
 print(f" {c:20s} 결측 {miss.mean() * 100:5.2f}% | 부도율 차이 {gap:+5.2f}pp "
 f"| 결측건 중 open_act_il==0 비율 {no_il:5.2f}%")

 print("\n-> 결측 원인이 '할부계좌 미보유'라면 중앙값 대체가 아니라 해당없음 더미가 맞다.")

def section_redundancy(df: pd.DataFrame) -> None:
 """결측더미를 몇 개까지 줄일 수 있는지 본다 (문서 5절 처방 ㉔)."""

```

```

header("7) 결측데이터 중복도 - 몇 개면 정보 손실이 0인가")
for cols, label in ((T2, "T2 14개"), (T2 + T3, "T2+T3 20개")):
 groups = identical_missing_groups(df, cols)
 print(f"\n{label} → 서로 다른 결측벡터 {len(groups)}개")
 for i, g in enumerate(groups, 1):
 print(f" 그룹{i} (결측 {df[g[0]].isna().mean() * 100:5.2f}%): {'', '.join(g)}")

```

```
def section_sheet_facts() -> None:
```

```
 """시트에 적힌 실측 수치를 재현한다 (문서 6절)."""
```

```
 header("8) 시트 기재 사실 검증")
```

```

 cols = [
 "loan_amnt", "funded_amnt", "funded_amnt_inv", "total_pymnt", "recoveries",
 "loan_status", "policy_code", "pymnt_plan", "out_prncp", "out_prncp_inv",
 "next_pymnt_d", "grade", "sub_grade", "int_rate", "hardship_flag",
 "debt_settlement_flag", "num_tl_120dpd_2m", "num_tl_30dpd", "delinq_amnt",
 "acc_now_delinq", "issue_d",
]

 fin = load_loans(cols)
 fin["int_rate"] = (fin["int_rate"].astype(str)
 .str.replace("%", "", regex=False).str.strip().astype(float))

 print(f"완결표본 n = {len(fin):,} (시트 기재 1,115,888 - 차이는 '{POLICY_PREFIX}' 포함 여부)")
 print(f"Charged Off = {int(fin['y'].sum()):,} (시트 기재 217,366)")

 print("\n상관계수 - 시트 미결사항 ① 근거")
 grade = fin["grade"].map({g: i for i, g in enumerate("ABCDEFG")})
 sub = (fin["sub_grade"].str[0].map({g: i for i, g in enumerate("ABCDEFG")}) * 5
 + fin["sub_grade"].str[1].astype(int))
 for name, a, b in [
 ("loan_amnt ~ funded_amnt", fin["loan_amnt"], fin["funded_amnt"]),
 ("funded_amnt ~ funded_amnt_inv", fin["funded_amnt"], fin["funded_amnt_inv"]),
 ("grade ~ int_rate", grade, fin["int_rate"]),
 ("sub_grade ~ int_rate", sub, fin["int_rate"]),
 ("grade ~ sub_grade", grade, sub),
]:
 print(f" {name:32s} {a.corr(b):.6f}")

 print("\n저분산 주장 - 완결표본 기준 0 초과 비율(%)")
 for c in ["num_tl_120dpd_2m", "num_tl_30dpd", "delinq_amnt", "acc_now_delinq"]:
 print(f" {c:22s} {(fin[c] > 0).mean() * 100:6.3f}%")
 print(f" policy_code 고유값 수 {fin['policy_code'].nunique()} / "
 f"pymnt_plan 고유값 {sorted(fin['pymnt_plan'].dropna().unique())}")

 print("\n사후변수 '사용불가' 주장")
 for c in ["out_prncp", "out_prncp_inv"]:
 print(f" {c:15s} 0 초과 비율 {(fin[c] > 0).mean() * 100:.4f}%")
 print(f" next_pymnt_d 비결측 비율 {fin['next_pymnt_d'].notna().mean() * 100:.4f}% (시트 0%)")

 print("\n[현금흐름 관련]")
 bad = fin[fin["y"] == 1]
 ret = bad["total_pymnt"] / bad["funded_amnt"] - 1
 print(f" 부도건 recoveries>0 {(bad['recoveries'] > 0).mean() * 100:.2f}% (시트 72.9%)")
 print(f" 부도건 total_pymnt>0 {(bad['total_pymnt'] > 0).mean() * 100:.2f}% (시트 99.6%)")
 print(f" 부도건 총수익률 평균 {ret.mean() * 100:.2f}% (시트 재검증치 -43.7%)")
 print(f" hardship_flag=Y {(fin['hardship_flag'] == 'Y').mean() * 100:.4f}% (시트 0.09%)")
 print(f" debt_settlement_flag=Y {(fin['debt_settlement_flag'] == 'Y').mean() * 100:.4f}% (시트 2.74%)")

```

```

def build_design_matrix(df: pd.DataFrame, scheme: str) -> pd.DataFrame:
 """처리 방식별 설계행렬을 만든다 (문서 5절 표)."""
 X = df[BASE_FEATURES].copy()

 if scheme == "S1_시트안": # 원본 20 + 결측더미 20
 for c in T2 + T3:
 X[c] = df[c]
 X["D_" + c] = df[c].isna().astype(int)
 elif scheme == "S2_더미없음": # 원본 20, 결측은 모델 native 처리에 위임
 for c in T2 + T3:
 X[c] = df[c]
 elif scheme == "S3_블록압축": # T2 더미 14 -> 1
 for c in T2 + T3:
 X[c] = df[c]
 X["D_T2_block"] = df["all_util"].isna().astype(int)
 for c in T3:
 X["D_" + c] = df[c].isna().astype(int)
 elif scheme == "S4_최소": # 블록압축 + 무신호 T3는 더미만
 for c in T2:
 X[c] = df[c]
 X["D_T2_block"] = df["all_util"].isna().astype(int)
 for c in T3_FLAT:
 X["D_" + c] = df[c].isna().astype(int)
 X["mths_since_recent_inq"] = df["mths_since_recent_inq"]
 X["D_mths_since_recent_inq"] = df["mths_since_recent_inq"].isna().astype(int)
 else:
 raise ValueError(f"알 수 없는 스킴: {scheme}")
 return X

def section_scheme_comparison(df: pd.DataFrame, n_sample: int = 300_000,
 n_seeds: int = 5) -> None:
 """결측 처리 방식별 성능을 비교한다 (문서 5절).

 AUC는 처리 방식 간 상대 비교 지표로만 쓴다 - 승인/거절 threshold 결정과 무관하다.
 """
 from sklearn.metrics import roc_auc_score
 from sklearn.model_selection import train_test_split
 from xgboost import XGBClassifier

 header("9) 결측 처리 방식별 성능 비교")

 d = df.copy()
 d["term"] = d["term"].astype(str).str.extract(r"(\d+)").astype(float)
 d["emp_length"] = d["emp_length"].map(EMP_LENGTH_MAP)
 d = d.sample(n=min(n_sample, len(d)), random_state=0)
 print(f"표본 {len(d):,}건 · seed 0~{n_seeds - 1} 반복\n")

 schemes = ["S1_시트안", "S3_블록압축", "S4_최소", "S2_더미없음"]
 results = {}
 for scheme in schemes:
 X = build_design_matrix(d, scheme)
 aucs = []
 for seed in range(n_seeds):
 X_tr, X_te, y_tr, y_te = train_test_split(

```

```

 X, d["y"].values, test_size=0.25, random_state=seed, stratify=d["y"].values)
 model = XGBClassifier(
 n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.8,
 colsample_bytree=0.8, eval_metric="logloss", tree_method="hist",
 n_jobs=8, random_state=seed)
 model.fit(X_tr, y_tr)
 aucs.append(roc_auc_score(y_te, model.predict_proba(X_te)[: , 1]))
 results[scheme] = np.array(aucs)
 print(f" {scheme:12s} p={X.shape[1]:3d} AUC {np.mean(aucs):.5f} "
 f"(표준편차 {np.std(aucs):.5f})")

base = results["S1_시트안"]
print(f"\n 시트안 대비 차이 (seed 쌍별) - 비교 기준: seed 간 자체 편차 {np.std(base):.5f}")
for scheme in schemes[1:]:
 print(f" {scheme:12s} Δ평균 {(results[scheme] - base).mean():+.5f}")

Out-of-time: 학습 구간에만 존재하는 결측더미가 실전에서 작동하는지 확인한다.
print("\n Out-of-time (학습 ≤2016 → 검증 2017~2019):")
tr, te = d[d["year"] <= 2016], d[d["year"].between(2017, 2019)]
print(f" 학습 n={len(tr):,} (T2 결측 {tr['all_util'].isna().mean() * 100:.1f}%) / "
 f"검증 n={len(te):,} (T2 결측 {te['all_util'].isna().mean() * 100:.1f}%)")
for scheme in schemes:
 model = XGBClassifier(
 n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.8,
 colsample_bytree=0.8, eval_metric="logloss", tree_method="hist",
 n_jobs=8, random_state=0)
 model.fit(build_design_matrix(tr, scheme), tr["y"].values)
 auc = roc_auc_score(te["y"].values,
 model.predict_proba(build_design_matrix(te, scheme))[: , 1])
 print(f" {scheme:12s} AUC {auc:.5f}")

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--with-model", action="store_true",
 help="9절 처리 방식별 AUC 비교까지 실행 (수십 분 소요)")
 args = parser.parse_args()

 if not RAW_FILE.exists():
 raise SystemExit(
 f"원본 파일이 없다: {RAW_FILE}\n"
 "용량 때문에 git에 올리지 않는다 - 팀 공유 채널에서 받아 data/raw/ 에 두고 실행한다.")

 cols = T2 + T3 + T1_SAMPLE + BASE_FEATURES + ["open_act_il"]
 df = load_loans(sorted(set(cols)))

 section_sample(df)
 section_missrate_base(df)
 section_t2_is_vintage(df)
 section_first_observed(df)
 section_missing_meaning(df)
 section_post_2015(df)
 section_redundancy(df)
 section_sheet_facts()
 if args.with_model:
 section_scheme_comparison(df)
 else:

```

```
print("\n(9절 AUC 비교는 --with-model 옵션으로 실행한다)")
```

```
if __name__ == "__main__":
 main()
```

**src/analysis/t2\_contribution\_reassessment.py**

---

T2 블록(14개) 기여도 재측정

""T2 블록(14개) 기여도 재측정 – seasoning 표본 필터 변경에 따른 재판정.

preprocessing\_validation\_kgj.md 5절의 \*T2 변수 자체를 제거하면 AUC -0.00813\* 은  
\*\*현행 표본(Current만 제외, T2 관측률 54.4%)\*에서 꽤 값이다.  
팀이 seasoning 편향 때문에 만기도래 대출만 쓰기로 하면서 T2 관측률이 35.8%로 떨어졌고,  
특히 60개월 만기 구간에서는 \*\*0.0%\*\*가 된다(60m 만기도래 = 2015년 이전 발행 vs  
T2 수집 개시 2015-12 – 구조적 배타). 따라서 옛 수치를 그대로 인용할 수 없다.

이 스크립트는 표본 정의를 바꿔가며 \*\*T2 14개 변수를 넣은 모형과 뺀 모형의 AUC 차이\*\*만  
측정한다. 판정 기준선은 seed 간 자체 편차이며, 그 이하면 "기여 없음"으로 본다.

표본 필터링은 src/preprocessing/AGENTS.md 규칙(완결건만) 위에 만기 조건을 얹는다:

만기 = issue\_d + term, 버퍼 6개월 -> 만기 <= 2020-04  
(스냅샷 2020-10에서 상각 확정까지 4~5개월 걸리므로 버퍼를 둔다)

AUC는 처리 방식 간 상대 비교용이다 – 승인/거절 threshold 결정과 무관하다  
(src/analysis/AGENTS.md).

실행:

```
python src/analysis/t2_contribution_reassessment.py
python src/analysis/t2_contribution_reassessment.py --seeds 3 # 빠르게
"""

import argparse
from pathlib import Path

import numpy as np
import pandas as pd

from preprocessing_validation import (# noqa: E402 (같은 폴더의 검증 스크립트와 정의 공유)
 BASE_FEATURES, EMP_LENGTH_MAP, POLICY_PREFIX, RAW_FILE, T2, T3, header,
)

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_CSV = REPO_ROOT / "outputs" / "t2_contribution_by_sample.csv"

만기 + 버퍼 6개월. 스냅샷(last_credit_pull_d 최종월) = 2020-10.
MATURITY_CUTOFF = pd.Period("2020-04", freq="M")

def load_loans() -> pd.DataFrame:
 """원본을 읽어 완결표본을 만들고 만기(issue_d + term)를 계산한다."""
 cols = sorted(set(T2 + T3 + BASE_FEATURES + ["loan_status", "issue_d", "term"]))
 raw = pd.read_csv(RAW_FILE, usecols=cols, low_memory=False)

 status = raw["loan_status"].astype(str).str.replace(POLICY_PREFIX, "", regex=False)
 keep = status.isin(["Fully Paid", "Charged Off"])

 df = raw[keep].copy()
 df["y"] = (status[keep] == "Charged Off").astype(int)
 df["issue_dt"] = pd.to_datetime(df["issue_d"], format="%b-%Y", errors="coerce")
 df["year"] = df["issue_dt"].dt.year
 df["term_m"] = df["term"].astype(str).str.extract(r"(\d+)").astype(int)
 df["maturity"] = df["issue_dt"].dt.to_period("M") + df["term_m"]

 df["term"] = df["term_m"].astype(float)
 df["emp_length"] = df["emp_length"].map(EMP_LENGTH_MAP)
```

```
return df
```

```
def slices(df: pd.DataFrame) -> dict[str, tuple[pd.DataFrame, list[str]]]:
```

```
 """비교할 표본 정의와 각각에서 볼 스킴.
```

```
 주 비교측은 네 표본 전부에서 동일하게 S2(결측더미 없음 - XGBoost 권고안)로 맞춘다.
```

```
 S1(시트안)은 '만가+버퍼6m 전체'에서만 추가로 돌려, 결론이 더미 스킴 선택에
```

```
 의존하지 않는다는 것만 확인한다.
```

```
 """
```

```
 matured = df[df["maturity"] <= MATURITY_CUTOFF]
```

```
 return {
```

```
 "현행(Current만 제외)": (df, ["S2_더미없음"]),
```

```
 "만가+버퍼6m 전체": (matured, ["S1_시트안", "S2_더미없음"]),
```

```
 "만가+버퍼6m 36m만": (matured[matured["term_m"] == 36], ["S2_더미없음"]),
```

```
 "만가+버퍼6m 60m만": (matured[matured["term_m"] == 60], ["S2_더미없음"]),
```

```
 }
```

```
def design(df: pd.DataFrame, with_t2: bool, dummies: bool) -> pd.DataFrame:
```

```
 """설계행렬. with_t2=False면 T2 14개 변수를 통째로 뺀다."""
```

```
 X = df[BASE_FEATURES].copy()
```

```
 value_cols = (T2 if with_t2 else []) + T3
```

```
 for c in value_cols:
```

```
 X[c] = df[c]
```

```
 if dummies:
```

```
 for c in value_cols:
```

```
 X["D_" + c] = df[c].isna().astype(int)
```

```
 return X
```

```
def measure(sample: pd.DataFrame, name: str, n_sample: int, n_seeds: int,
```

```
 schemes: list[str]) -> list[dict]:
```

```
 """한 표본 정의 안에서 T2 포함/제외 AUC를 재고, 차이를 seed 편차와 대조한다."""
```

```
 from sklearn.metrics import roc_auc_score
```

```
 from sklearn.model_selection import train_test_split
```

```
 from xgboost import XGBClassifier
```

```
 d = sample.sample(n=min(n_sample, len(sample)), random_state=0)
```

```
 t2_obs = d["all_util"].notna().mean() * 100
```

```
 print(f"\n----- {name} -----")
```

```
 print(f" 전체 n={len(sample):,} 부도율 {sample['y'].mean() * 100:.2f}% "
 f"| 분석 표본 n={len(d):,} T2 관측률 {t2_obs:.1f}%")
```

```
 rows = []
```

```
 for scheme in schemes:
```

```
 dummies = scheme == "S1_시트안"
```

```
 aucs: dict[bool, list[float]] = {True: [], False: []}
```

```
 for seed in range(n_seeds):
```

```
 for with_t2 in (True, False):
```

```
 X = design(d, with_t2=with_t2, dummies=dummies)
```

```
 X_tr, X_te, y_tr, y_te = train_test_split(
```

```
 X, d["y"].values, test_size=0.25, random_state=seed,
```

```
 stratify=d["y"].values)
```

```
 model = XGBClassifier(
```

```
 n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.8,
```

```
 colsample_bytree=0.8, eval_metric="logloss", tree_method="hist",
```

```

 n_jobs=8, random_state=seed)
 model.fit(X_tr, y_tr)
 aucs[with_t2].append(roc_auc_score(y_te, model.predict_proba(X_te)[: , 1]))

 inc, exc = np.array(aucs[True]), np.array(aucs[False])
 delta, seed_sd = (exc - inc).mean(), np.std(inc)
 rows.append({
 "표본": name, "스킴": scheme, "n": len(d), "T2관측률%": round(t2_obs, 1),
 "AUC_T2포함": round(inc.mean(), 5), "AUC_T2제외": round(exc.mean(), 5),
 "delta": round(delta, 5), "seed편차": round(seed_sd, 5),
 "판정": "기여 있음" if abs(delta) > seed_sd else "잡음 이하",
 })
 print(f" {scheme:10s} T2포함 {inc.mean():.5f} / T2제외 {exc.mean():.5f} "
 f"| Δ {delta:+.5f} (seed 편차 {seed_sd:.5f}) "
 f"→ {'기여 있음' if abs(delta) > seed_sd else '잡음 이하'}")
 return rows

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--seeds", type=int, default=5)
 parser.add_argument("--n-sample", type=int, default=300_000)
 args = parser.parse_args()

 if not RAW_FILE.exists():
 raise SystemExit(f"원본 파일이 없다: {RAW_FILE}")

 header("T2 블록 기여도 재측정 - 표본 정의별")
 print("Δ = AUC(T2 제외) - AUC(T2 포함). 음수면 T2가 기여하고 있다는 뜻이다.")
 print("판정 기준선 = seed 간 자체 편차.")

 df = load_loans()
 all_rows = []
 for name, (sample, schemes) in slices(df).items():
 all_rows += measure(sample, name, args.n_sample, args.seeds, schemes)

 out = pd.DataFrame(all_rows)
 OUT_CSV.parent.mkdir(parents=True, exist_ok=True)
 out.to_csv(OUT_CSV, index=False, encoding="utf-8-sig")
 print(f"\n\n{out.to_string(index=False)}")
 print(f"\n저장: {OUT_CSV.relative_to(REPO_ROOT)}")

if __name__ == "__main__":
 main()

```

## src/analysis/term\_split\_comparison.py

36m/60m 분리 모형 vs 통합 모형

""36m/60m 분리 모형 vs 통합 모형 - 어느 쪽이 나은가.

## 왜 재는가

만기 컷(`issue\_d + term <= 2020-04`)을 적용하면 term 구성이 크게 바뀐다.

60개월 대출은 만기가 2년 늦게 오므로 발행 상한이 2015-04까지로 밀리고

(36개월은 2017-04), 그 2년이 Lending Club 물량 급증기와 겹쳐 60m가 3분의 2 날아간다.

36개월 : 837,476 -> 623,125건 (잔존 74.4%, 비중 86.1%)

60개월 : 280,095 -> 100,438건 (잔존 35.9%, 비중 13.9%)

두 상품은 부도율도 다르다(36m 14.61% vs 60m 26.18%). 그래서 `decision\_log.md` #12 (5)는 term-vintage 교각의 완화책으로 \*\*36m/60m 분리 모형\*\*을 후보로 올려 뒀다.

이 스크립트는 그 후보를 실측으로 판정한다. \*\*같은 표본·같은 하이퍼파라미터·같은 test set\*\* 위에서 두 방식만 바꿔 비교하므로, 차이가 나면 그것은 분리 여부에서 온 것이다.

(a) 통합 : 전체로 모형 1개 학습. `term`을 피처로 투입한다

(b) 분리 : 36m/60m 각각 모형 1개씩. `term`은 그룹 내 상수라 제외한다

## 어떻게 재는가

평가는 \*\*동일한 test set\*\*에서 세 구간으로 나눠 본다. (b)의 전체 AUC는 두 모형의 예측확률을 각 행의 term에 맞춰 이어 붙여 계산한다.

- 전체 : 두 상품을 한 포트폴리오로 볼 때의 순위매김 성능
- 36개월 안에서 : 36m 부분집합에서만 쟀 성능
- 60개월 안에서 : 60m 부분집합에서만 쟀 성능 <- 분리의 기대 이익이 가장 큰 곳

판정 기준선은 늘 그렇듯 \*\*seed 간 자체 편차\*\*다. 그보다 작으면 잡음으로 본다.

다만 60m는 차이와 편차가 비슷한 크기로 나오므로 \*\*seed별 부호 일관성\*\*도 함께 본다.

## 결과 (seeds=5 기준, 2026-07-29)

| 구간       | (a) 통합  | (b) 분리  | 차이(b-a)  | seed편차  | 판정                 |
|----------|---------|---------|----------|---------|--------------------|
| 전체       | 0.68080 | 0.68033 | -0.00047 | 0.00150 | 미세하게 나뉘 (부호 5/5)   |
| 36개월 안에서 | 0.67197 | 0.67223 | +0.00025 | 0.00131 | 차이 없음              |
| 60개월 안에서 | 0.62783 | 0.62219 | -0.00564 | 0.00563 | 뚜렷하게 나뉘 (5/5 seed) |

\*\*분리는 이득이 없고, 60개월에서는 오히려 일관되게 나쁘다.\*\*

이유는 표본 크기다. 분리하면 60m 모형이 8만 건으로 학습하는데, 통합하면 58만 건을 본다.

신용위험의 기본 패턴(FICO가 낮으면 위험, dti가 높으면 위험)은 상품 종류와 무관하게

공통이므로 36m 데이터로 배운 것이 60m 예측에도 전이된다. 데이터를 7배 더 본 쪽이 이긴다.

그리고 트리 모형은 필요하면 스스로 나눈다. `term`을 피처로 넣어 두면 두 상품이 정말

다르게 행동하는 지점에서 XGBoost가 알아서 `term < 48`로 분기한다. 사람이 미리 갈라 주면

그 분기를 강제하는 대신 \*\*공통 패턴을 배울 기회를 뺏는\*\* 셈이다.

## 성능 밖의 비용 (수치로 재지 않은 부분 - 문서 8절 참조)

- \*\*threshold 탐색이 2차원이 된다.\*\* 본 과제의 판단 기준은 Sharpe Ratio이고, Sharpe는 포트폴리오 단위 지표다. 36m/60m를 따로 최적화해 합치면 전체 Sharpe가 최적이라는 보장이 없으므로 (t36, t60) 격자를 동시에 탐색해야 한다. Validation에서 맞추는 파라미터가 1개에서 2개로 늘어 과적합 위험도 커진다.
- \*\*변수중요도가 두 세트가 된다.\*\* 특히 60m 모형에서는 T2 14개가 전량 결측이라

(60m 만기도래 = 2015-04 이전 발행 vs T2 수집 개시 2015-12) 피처가 통째로 죽는다.

## ## 결론

**\*\*통합 모델을 쓴다.\*\*** 다만 학습한 뒤 예측값을 term으로 나눠 성능표를 따로 보고한다 - 모델을 나누지 않고도 "36m/60m가 다르다"는 진단은 그대로 얻을 수 있다.

주의: 이것은 **\*\*PD 모형\*\***을 나누지 않는다는 뜻이지, 수익률 계산에서 term을 무시한다는 뜻이 아니다. 60개월 대출은 5년간 현금흐름이 나오므로 IRR/Sharpe 계산식에는 만기가 반드시 들어간다 (`decision\_log.md` #4).

AUC는 처리 방식 간 상대 비교용이다 - 승인/거절 threshold 결정과 무관하다  
(`src/analysis/AGENTS.md`).

실행:

```
python src/analysis/term_split_comparison.py
python src/analysis/term_split_comparison.py --seeds 3 # 빠르게
"""

import argparse
from pathlib import Path

import numpy as np
import pandas as pd

from preprocessing_validation import (# noqa: E402 (같은 폴더의 검증 스크립트와 정의 공유)
 BASE_FEATURES, EMP_LENGTH_MAP, POLICY_PREFIX, RAW_FILE, T2, T3, header,
)

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_CSV = REPO_ROOT / "outputs" / "term_split_comparison.csv"

만기 + 버퍼 6개월. 스냅샷(last_credit_pull_d 최종월) = 2020-10.
MATURITY_CUTOFF = pd.Period("2020-04", freq="M")

결측더미 없는 권고안(preprocessing_validation_kgj.md 5절 S2). 값만 투입하고
결측은 XGBoost의 native 분기에 맡긴다.
FEATURES = BASE_FEATURES + T2 + T3

XGB_PARAMS = dict(
 n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.8,
 colsample_bytree=0.8, eval_metric="logloss", tree_method="hist", n_jobs=8,
)

def load_matured() -> pd.DataFrame:
 """완결표본에 만기 조건을 얹어 반환한다."""
 cols = sorted(set(T2 + T3 + BASE_FEATURES + ["loan_status", "issue_d", "term"]))
 raw = pd.read_csv(RAW_FILE, usecols=cols, low_memory=False)

 status = raw["loan_status"].astype(str).str.replace(POLICY_PREFIX, "", regex=False)
 keep = status.isin(["Fully Paid", "Charged Off"])

 df = raw[keep].copy()
 df["y"] = (status[keep] == "Charged Off").astype(int)
 issue_dt = pd.to_datetime(df["issue_d"], format="%b-%Y", errors="coerce")
 df["term_m"] = df["term"].astype(str).str.extract(r"(\d+)").astype(int)
 df = df[(issue_dt.dt.to_period("M") + df["term_m"]) <= MATURITY_CUTOFF].copy()
```

```

df["term"] = df["term_m"].astype(float)
df["emp_length"] = df["emp_length"].map(EMP_LENGTH_MAP)
return df

def one_seed(df: pd.DataFrame, seed: int) -> dict:
 """한 seed에서 (a)통합과 (b)분리를 같은 test set으로 평가한다."""
 from sklearn.metrics import roc_auc_score
 from sklearn.model_selection import train_test_split
 from xgboost import XGBClassifier

 train, test = train_test_split(
 df, test_size=0.2, random_state=seed, stratify=df["y"])
 tr36, tr60 = train["term_m"] == 36, train["term_m"] == 60
 te36, te60 = test["term_m"] == 36, test["term_m"] == 60

 # (a) 통합 - term을 피쳐로 투입
 unified = XGBClassifier(**XGB_PARAMS, random_state=seed)
 unified.fit(train[FEATURES], train["y"])
 p_a = unified.predict_proba(test[FEATURES])[:, 1]

 # (b) 분리 - term은 그룹 내 상수이므로 제외
 feats_split = [f for f in FEATURES if f != "term"]
 p_b = np.empty(len(test))
 for mask_tr, mask_te in ((tr36, te36), (tr60, te60)):
 model = XGBClassifier(**XGB_PARAMS, random_state=seed)
 model.fit(train.loc[mask_tr, feats_split], train.loc[mask_tr, "y"])
 p_b[mask_te.values] = model.predict_proba(test.loc[mask_te, feats_split])[:, 1]

 def auc(mask, pred):
 return roc_auc_score(test["y"][mask], pred[mask.values])

 everything = pd.Series(True, index=test.index)
 return {
 "seed": seed,
 "통합_전체": auc(everything, p_a), "분리_전체": auc(everything, p_b),
 "통합_36m": auc(te36, p_a), "분리_36m": auc(te36, p_b),
 "통합_60m": auc(te60, p_a), "분리_60m": auc(te60, p_b),
 }

def main() -> None:
 parser = argparse.ArgumentParser(description="36m/60m 분리 모형 vs 통합 모형")
 parser.add_argument("--seeds", type=int, default=5)
 args = parser.parse_args()

 if not RAW_FILE.exists():
 raise SystemExit(f"원본 파일이 없다: {RAW_FILE}")

 header("36m/60m 분리 모형 vs 통합 모형")
 print("같은 표본·같은 하이퍼파라미터·같은 test set. 판정 기준선은 seed 간 자체 편차.")

 df = load_matured()
 n36, n60 = (df["term_m"] == 36).sum(), (df["term_m"] == 60).sum()
 print(f"\n만기+버퍼6m 표본 {len(df):,}건 부도를 {df['y'].mean() * 100:.2f}%")
 for label, mask in (("36개월", df["term_m"] == 36), ("60개월", df["term_m"] == 60)):

```

```

sub = df[mask]
print(f" {label} {len(sub):>8,}건 ({len(sub) / len(df) * 100:5.1f}%) "
 f"부도를 {sub['y'].mean() * 100:5.2f}%")
if min(n36, n60) == 0:
 raise SystemExit("한쪽 term이 비어 있어 비교할 수 없다.")

res = pd.DataFrame([one_seed(df, s) for s in range(args.seeds)])

rows = []
print(f"\n{'구간':<16}{'(a) 통합':>11}{'(b) 분리':>11}{'차이(b-a)':>12}"
 f"{'seed편차':>10} 판정")
print("-" * 78)
for label, col_a, col_b in (("전체", "통합_전체", "분리_전체"),
 ("36개월 안에서", "통합_36m", "분리_36m"),
 ("60개월 안에서", "통합_60m", "분리_60m")):
 delta = res[col_b].mean() - res[col_a].mean()
 seed_sd = res[col_a].std()
 worse = int((res[col_b] < res[col_a]).sum())
 direction = "나쁨" if delta < 0 else "나음"
 # 크기가 (잡음 대비)와 부호 일관성을 분리해서 표기한다. 크기가 잡음 이하라도
 # 모든 seed에서 부호가 같으면 "우연"으로 보기 어렵기 때문이다.
 if abs(delta) > seed_sd:
 verdict = f"분리가 뚜렷하게 {direction} ({worse}/{args.seeds} seed)"
 elif worse in (0, args.seeds):
 verdict = f"분리가 미세하게 {direction} (부호 일관 {worse}/{args.seeds}, 크기는 잡음 이하)"
 else:
 verdict = "차이 없음"
 rows.append({"구간": label, "AUC_통합": round(res[col_a].mean(), 5),
 "AUC_분리": round(res[col_b].mean(), 5), "delta": round(delta, 5),
 "seed편차": round(seed_sd, 5), "분리гна쁜seed": worse,
 "판정": verdict})

 print(f"{label:<14}{res[col_a].mean():>11.5f}{res[col_b].mean():>11.5f}"
 f"{delta:>+12.5f}{seed_sd:>10.5f} {verdict}")
print("-" * 78)
print("60개월은 차이와 편차가 비슷하므로 seed별 부호 일관성으로 판정한다.")

out = pd.DataFrame(rows)
OUT_CSV.parent.mkdir(parents=True, exist_ok=True)
out.to_csv(OUT_CSV, index=False, encoding="utf-8-sig")
print(f"\nseed별 원자료:\n{res.round(5).to_string(index=False)}")
print(f"\n저장: {OUT_CSV.relative_to(REPO_ROOT)}")

if __name__ == "__main__":
 main()

```

## src/analysis/missing\_scheme\_comparison.py

2016년 이후 잔여 결측을 어떻게 채울 것인가

""2016년 이후 잔여 결측을 어떻게 채울 것인가 - NaN 유지 vs 0 대체 vs 전용 더미.

## 왜 재는가

`preprocessing\_validation\_kgj.md` 4절은 T2 결측을 시점별로 쪼개 처방했다.

~2015-11 : 미수집(vintage). 정보가 존재하지 않음 -> 결측더미를 만들지 않는다  
2015-12~ : 진짜 "할부계좌 없음" -> 성격별로 나눠 처리

그리고 2016년 이후 구간에 대해 \*\*금액·개수형은 0 대체가 옳다\*\* (계좌가 없으면 잔액도 개설건수도 진짜 0)고 권고했다. 류성환·이지희 문서는 같은 구간에 \*\*중앙값 대체\*\*를 적었다. 교차검증 문서는 이 대답을 "반드시 정할 것"으로 올렸다.

그런데 XGBoost 단일화를 전제하면 중앙값 대체 주장은 근거를 잃는다(NaN을 그대로 받으므로). 남는 질문은 \*\*"0으로 볼 것인가 NaN으로 둘 것인가"\*\* 하나인데, 그 답을 재기 전에 \*\*애초에 0을 채울 대상이 얼마나 있는가\*\*부터 확인해야 한다.

## 무엇을 확인하는가

1단계 - 만기컷 표본의 2016년 이후 발행본에서 \*\*T2 14개의 변수별 결측률\*\*을 센다.

2단계 - 처리 스킴 4개의 AUC를 비교한다. 판정 기준선은 seed 간 자체 편차다.

N : NaN 유지 (권고안 / 기준선)  
Z : 비율·시점형까지 0으로 채움 (0 대체가 유리한지)  
D : NaN 유지 + has\_no\_il\_account (전용 더미가 기여하는지)  
Za : 2016+ 모든 T2 결측을 0으로 (시트가 금지한 방식 - sanity check)

## 결과 (만기+버퍼6m 723,563건, seeds=5, 2026-07-29)

1단계가 결정적이다.

il\_util 2016+ 결측률 14.12% <- 비율형  
mths\_since\_rcnt\_il 2016+ 결측률 3.14% <- 시점형  
all\_util 2016+ 결측률 0.02%  
나머지 금액·개수형 11개 2016+ 결측률 0.01% <- 사실상 없다

\*\*"0 대체가 의미상 맞다"고 했던 금액·개수형 변수에는 채울 결측이 남아 있지 않다.\*\*

25만 건 중 약 25건이다. 실제로 결측이 남는 두 변수(`il\_util`, `mths\_since\_rcnt\_il`)는 분모가 0이라 정의 자체가 불가능한 경우이므로 0이 틀린 값이다.

2단계도 같은 방향이다.

| 스킴            | AUC     | N 대비     | 나쁜 seed             |
|---------------|---------|----------|---------------------|
| N NaN 유지      | 0.68080 | -        | - (seed 편차 0.00134) |
| Z 비율형까지 0     | 0.68074 | -0.00006 | 3/5                 |
| D NaN + 더미    | 0.68089 | +0.00009 | 2/5                 |
| Za 2016+ 전부 0 | 0.68089 | +0.00009 | 2/5                 |

최대 차이 0.00009는 seed 편차의 \*\*1/15\*\*이고 부호도 seed마다 갈린다. 네 스킴이 같다.

## 결론

\*\*전부 NaN으로 두고 아무것도 만들지 않는다.\*\*

- 성능이 같으면 더 단순한 쪽을 택한다. 의미상으로도 `0 / 0`에 0을 넣는 것은 틀렸다.

- `has\_no\_il\_account` 더미도 불필요하다. \*\*`open\_act\_il`이 이미 피쳐로 들어가 있어\*\*

트리가 `open\_act\_il < 1`로 직접 분기할 수 있기 때문이다 - 같은 정보를 두 번 넣는 셈이다.

- 이는 결측더미 전반을 만들지 않기로 한 판단(`preprocessing\_validation\_kgj.md` 5절 처방 ①, `decision\_log.md` #8의 GBM native 결측 처리 원칙)과 같은 결론이다.

정리하면 전제(XGBoost 단일)에서 결측 처리 규칙은 하나로 통일된다.

```
T2 pre-2016 -> NaN 그대로 (미수집)
T2 2016+ -> NaN 그대로 (계좌 미보유 - open_act_il이 이미 그 정보를 담고 있다)
T3 -> NaN 그대로 (사건 없음)
파생 더미 -> 만들지 않음
```

주의: 이 결론은 \*\*XGBoost를 쓴다는 전제\*\*에 의존한다. NaN을 받지 못하는 선형모형을 쓴다면 대체값과 더미가 다시 필요해진다.

AUC는 처리 방식 간 상대 비교용이다 - 승인/거절 threshold 결정과 무관하다  
(`src/analysis/AGENTS.md`).

실행:

```
python src/analysis/missing_scheme_comparison.py
python src/analysis/missing_scheme_comparison.py --seeds 3 # 빠르게
"""

import argparse
from pathlib import Path

import numpy as np
import pandas as pd

from preprocessing_validation import (# noqa: E402 (같은 폴더의 검증 스크립트와 정의 공유)
 BASE_FEATURES, EMP_LENGTH_MAP, POLICY_PREFIX, RAW_FILE, T2, T3, header,
)

REPO_ROOT = Path(__file__).resolve().parents[2]
OUT_CSV = REPO_ROOT / "outputs" / "missing_scheme_comparison.csv"

MATURITY_CUTOFF = pd.Period("2020-04", freq="M")

2016년 이후 결측이 실제로 남는 두 변수. 분모가 0이라 값이 정의되지 않는 성격이다
(il_util = 할부잔액/할부한도, mths_since_rcnt_il = 최근 할부계좌 개설 후 경과월).
RATIO_LIKE = ["il_util", "mths_since_rcnt_il"]

계좌가 없으면 값도 진짜 0인 성격 - "0 대체가 옳다"고 권고했던 대상.
AMOUNT_LIKE = [v for v in T2 if v not in RATIO_LIKE + ["all_util"]]

FEATURES = BASE_FEATURES + T2 + T3

XGB_PARAMS = dict(
 n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.8,
 colsample_bytree=0.8, eval_metric="logloss", tree_method="hist", n_jobs=8,
)

def load_matured() -> pd.DataFrame:
 """완결표본에 만기 조건을 얹고 발행연도를 붙인다."""
 cols = sorted(set(T2 + T3 + BASE_FEATURES + ["loan_status", "issue_d", "term"]))
 raw = pd.read_csv(RAW_FILE, usecols=cols, low_memory=False)

 status = raw["loan_status"].astype(str).str.replace(POLICY_PREFIX, "", regex=False)
```

```

keep = status.isin(["Fully Paid", "Charged Off"])

df = raw[keep].copy()
df["y"] = (status[keep] == "Charged Off").astype(int)
issue_dt = pd.to_datetime(df["issue_d"], format="%b-%Y", errors="coerce")
df["year"] = issue_dt.dt.year
df["term_m"] = df["term"].astype(str).str.extract(r"(\d+)").astype(int)
df = df[(issue_dt.dt.to_period("M") + df["term_m"]) <= MATURITY_CUTOFF].copy()

df["term"] = df["term_m"].astype(float)
df["emp_length"] = df["emp_length"].map(EMP_LENGTH_MAP)
return df

```

```

def report_missing(df: pd.DataFrame) -> pd.DataFrame:
 """1단계 - 2016년 이후 발행분에서 T2 변수별 결측률. 0을 채울 대상이 있는지 본다."""
 post = df["year"] >= 2016
 rows = []
 for v in T2:
 kind = ("비율형" if "util" in v else
 "시점형" if v in RATIO_LIKE else "금액·개수형")
 rows.append({"변수": v, "성격": kind,
 "전체_결측률%": round(df[v].isna().mean() * 100, 2),
 "2016+_결측률%": round(df.loc[post, v].isna().mean() * 100, 2)})
 out = pd.DataFrame(rows).sort_values("2016+_결측률%", ascending=False)
 print(f"\n2016년 이후 발행 {post.sum():,}건 ({post.mean() * 100:.1f}%) 기준\n")
 print(out.to_string(index=False))
 print("\n-> 금액·개수형은 2016년 이후 결측이 사실상 없다. 0을 채울 대상 자체가 없다.")
 return out

```

```

def design(df: pd.DataFrame, scheme: str) -> pd.DataFrame:
 """스킴별 설계행렬. 0 대체는 2016년 이후 발행분에만 적용한다.

 2015년 이전은 '미수집' 이라 0을 넣으면 옛 차주 전체를 무활동 차주로 왜곡하므로
 (시트 개정 5번), 어떤 스킴에서도 건드리지 않는다.
 """
 X = df[FEATURES].copy()
 post = (df["year"] >= 2016).values
 if scheme == "Z_비율형까지0":
 for v in RATIO_LIKE:
 X.loc[post & X[v].isna().values, v] = 0
 elif scheme == "D_NaN+더미":
 X["has_no_il_account"] = (df["open_act_il"] == 0).astype(int)
 elif scheme == "Za_2016+전부0":
 for v in T2:
 X.loc[post & X[v].isna().values, v] = 0
 return X

```

```

def compare(df: pd.DataFrame, n_seeds: int) -> pd.DataFrame:
 """2단계 - 스킴별 AUC. 기준선은 N(NaN 유지)."""
 from sklearn.metrics import roc_auc_score
 from sklearn.model_selection import train_test_split
 from xgboost import XGBClassifier

 schemes = ["N_NaN유지", "Z_비율형까지0", "D_NaN+더미", "Za_2016+전부0"]

```

```

scores: dict[str, np.ndarray] = {}
for scheme in schemes:
 X = design(df, scheme)
 aucs = []
 for seed in range(n_seeds):
 X_tr, X_te, y_tr, y_te = train_test_split(
 X, df["y"].values, test_size=0.2, random_state=seed,
 stratify=df["y"].values)
 model = XGBClassifier(**XGB_PARAMS, random_state=seed).fit(X_tr, y_tr)
 aucs.append(roc_auc_score(y_te, model.predict_proba(X_te[:, 1])))
 scores[scheme] = np.array(aucs)

base = scores["N_NaN유지"]
seed_sd = base.std()
rows = []
print(f"\n{'스킴':<18}{'AUC 평균':>11}{'N 대비':>11}{'나쁜 seed':>11} 판정")
print("-" * 68)
for scheme in schemes:
 a = scores[scheme]
 delta = a.mean() - base.mean()
 worse = int((a < base).sum())
 if scheme == "N_NaN유지":
 verdict = f"기준선 (seed 편차 {seed_sd:.5f})"
 shown_delta, shown_worse = "-", "-"
 else:
 verdict = "차이 없음" if abs(delta) <= seed_sd else "차이 있음"
 shown_delta, shown_worse = f"{delta:+.5f}", f"{worse}/{n_seeds}"
 rows.append({"스킴": scheme, "AUC": round(a.mean(), 5),
 "N대비delta": round(delta, 5), "나쁜seed": worse,
 "seed편차": round(seed_sd, 5), "판정": verdict})
 print(f"{scheme:<16}{a.mean():>11.5f}{shown_delta:>11}{shown_worse:>11} {verdict}")
print("-" * 68)
spread = max(abs(scores[s].mean() - base.mean()) for s in schemes)
print(f"스킴 간 최대 차이 {spread:.5f} = seed 편차의 {spread / seed_sd:.2f}배"
 f" -> {'구분 불가' if spread < seed_sd else '유의'}")
return pd.DataFrame(rows)

def main() -> None:
 parser = argparse.ArgumentParser(description="2016+ 잔여 결측 처리 스킴 비교")
 parser.add_argument("--seeds", type=int, default=5)
 args = parser.parse_args()

 if not RAW_FILE.exists():
 raise SystemExit(f"원본 파일이 없다: {RAW_FILE}")

 header("2016년 이후 잔여 결측 - NaN 유지 vs 0 대체 vs 전용 더미")
 df = load_matured()
 print(f"만기+버퍼6m 표본 {len(df):,}건 부도율 {df['y'].mean() * 100:.2f}%")

 header("1단계 - 0을 채울 대상이 있는가")
 miss = report_missing(df)

 header("2단계 - 스킴별 AUC")
 res = compare(df, args.seeds)

 OUT_CSV.parent.mkdir(parents=True, exist_ok=True)

```

```
pd.concat([miss.assign(구분="결측률"), res.assign(구분="AUC")],
 ignore_index=True).to_csv(OUT_CSV, index=False, encoding="utf-8-sig")
print(f"\n저장: {OUT_CSV.relative_to(REPO_ROOT)}")

if __name__ == "__main__":
 main()
```

## src/analysis/macro\_indicator\_screening.py

---

거시경제지표 4종의 부도확률 관련성 탐색 분석.

""" 거시경제지표 4종의 부도확률 관련성 탐색 분석.

outputs/reports/macro\_indicator\_selection.md 에 실린 표를 전부 재생성한다.  
문서의 숫자를 검증하거나 표본이 바뀐 뒤 다시 계산할 때 이 스크립트를 돌린다.

이 스크립트는 **\*\*탐색(screening)\*\*** 용이다 - 모형 학습이나 threshold 결정과는 무관하며,  
src/analysis/AGENTS.md의 Sharpe Ratio 기반 의사결정 규칙이 적용되는 단계가 아니다.

표본 필터링은 src/preprocessing/AGENTS.md 규칙을 따른다:

- loan\_status가 Current / Late (16-30 days) / Late (31-120 days)인 행 제외
- "Does not meet the credit policy. Status:\*" 는 접두어를 떼고 동일하게 취급

⚠ 입력이 9,000건 표본이다. 모든 분석은 원본 전수로 한다는 규칙(AGENTS.md 「데이터 규모」)의  
예외로, **\*\*문서에 이미 실린 표를 그대로 재현하기 위해\*\*** 표본 경로를 유지한다.  
거시지표를 실제로 모형에 쓰기로 하면 그때 전수로 바꾸고 문서 수치도 함께 재산출해야 한다.

실행:

```
python src/analysis/macro_indicator_screening.py
"""

import functools
from pathlib import Path

import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[2]
PROCESSED = REPO_ROOT / "data" / "processed"

LOAN_FILE = PROCESSED / "lending_club_2020_train_sample_9000.csv"
MACRO_FILES = [
 "macro_unemployment_rate_monthly_2007-01_to_2020-09.csv",
 "macro_initial_claims_monthly_2007-01_to_2020-09.csv",
 "macro_yield_spread_10y2y_monthly_2007-01_to_2020-09.csv",
 "macro_cpi_monthly_2007-01_to_2020-09.csv",
]

결과가 확정된 상태만 남긴다 (진행 중인 대출은 부도 여부를 알 수 없음).
RESOLVED = ["Fully Paid", "Charged Off", "Default"]
EXCLUDED = ["Current", "Late (16-30 days)", "Late (31-120 days)"]

INDICATORS = ["UNRATE", "ICSA", "spread_10y2y", "cpi_yoy_pct"]

def load_macro() -> pd.DataFrame:
 """거시지표 4종을 observation_date로 병합한다."""
 dfs = [
 pd.read_csv(PROCESSED / f, parse_dates=["observation_date"]) for f in MACRO_FILES
]
 return functools.reduce(
 lambda a, b: a.merge(b, on="observation_date", how="outer"), dfs
)

def load_loans() -> pd.DataFrame:
 """대출 표본을 읽어 전처리 규칙대로 필터링하고 default 라벨을 만든다."""
 raw = pd.read_csv(LOAN_FILE, usecols=["loan_status", "issue_d"], low_memory=False)
```

# 141컬럼 원본에 컬럼을 덧붙이면 단편화 경고가 나므로 필요한 것만 새로 구성한다.

```
df = pd.DataFrame(
 {
 # "Does not meet the credit policy. Status:Fully Paid" -> "Fully Paid"
 "status": raw["loan_status"].str.replace(
 "Does not meet the credit policy. Status:", "", regex=False
),
 "observation_date": pd.to_datetime(raw["issue_d"], format="%b-%Y"),
 }
)
```

```
df = df[~df["status"].isin(EXCLUDED)]
df = df[df["status"].isin(RESOLVED)].copy()
df["default"] = (df["status"] != "Fully Paid").astype(int)
df["year"] = df["observation_date"].dt.year
return df
```

```
def section(title: str) -> None:
 print(f"\n{'=' * 72}\n{title}\n{'=' * 72}")
```

```
def main() -> None:
 macro = load_macro()
 loans = load_loans()
 df = loans.merge(macro, on="observation_date", how="left")

 section("0. 표본")
 raw_n = len(pd.read_csv(LOAN_FILE, usecols=["id"], low_memory=False))
 print(f"원 표본 {raw_n:,}건 -> 결과 확정본 {len(df):,}건")
 print(f"부도 {df['default'].sum():,}건 (부도율 {df['default'].mean() * 100:.2f}%)")
 print(f"거시지표 결합 결측: {df[INDICATORS].isna().sum().sum():,}개")

 section("1. 거시지표 상호 상관 (월별 165개월 기준)")
 print(macro[INDICATORS + ["CPIAUCSL"]].corr().round(3).to_string())

 section("2. 지표 4분위별 부도율 (지표 값으로 줄 세워 4등분)")
 for col in INDICATORS:
 binned, edges = pd.qcut(df[col], 4, labels=["Q1", "Q2", "Q3", "Q4"], retbins=True)
 tbl = (
 df.assign(q=binned)
 .groupby("q", observed=True)
 .agg(
 건수=("default", "size"),
 지표최소=(col, "min"),
 지표최대=(col, "max"),
 부도건수=("default", "sum"),
 부도율=("default", "mean"),
)
)
 tbl["부도율"] = (tbl["부도율"] * 100).round(2)
 print(f"\n-- {col} (경계 {[round(e, 2) for e in edges]}) --")
 print(tbl.to_string())

 section("3. 교란 진단 ① - 거시지표가 발행연도의 대리변수인가")
 for col in INDICATORS:
 print(f" corr({col}, 발행연도) = {df[[col, 'year']].corr().iloc[0, 1]:+.3f}")
```

```

print("\n실업률 4분위 x 발행연도 구성비(%):")
q = pd.qcut(df["UNRATE"], 4, labels=["Q1", "Q2", "Q3", "Q4"])
ct = pd.crosstab(q, df["year"], normalize="index").mul(100).round(1)
print(ct.loc[:, [c for c in ct.columns if c >= 2012]].to_string())

section("4. 교란 진단 ② - seasoning(관측 편향)")
allrows = pd.read_csv(LOAN_FILE, usecols=["issue_d", "loan_status"], low_memory=False)
allrows["y"] = pd.to_datetime(allrows["issue_d"], format="%b-%Y").dt.year
s = allrows["loan_status"].str.replace(
 "Does not meet the credit policy. Status:", "", regex=False
)
tbl = allrows.assign(s=s).groupby("y").apply(
 lambda g: pd.Series(
 {
 "전체": len(g),
 "Current비중": (g["s"] == "Current").mean() * 100,
 "결과확정비중": g["s"].isin(RESOLVED).mean() * 100,
 }
),
 include_groups=False,
)
print(tbl.round(1).tail(9).to_string())

section("5. 발행연도 고정 시 (2015~2018)")
sub = df[df["year"].between(2015, 2018)].copy()
print(f"대상 {len(sub):,}건 (결과 확정 표본의 {len(sub) / len(df) * 100:.0f}%)\n")
for col in INDICATORS:
 sub["qq"] = pd.qcut(sub[col], 3, labels=["하", "중", "상"])
 r = sub.groupby("qq", observed=True)["default"].agg(["size", "mean"])
 cells = " | ".join(f"{i} {v * 100:5.2f}% (n={n:,})" for i, (n, v) in r.iterrows())
 print(f" {col:<14} {cells}")

section("6. 금리차 -0.870 분해 - '금리가 낮아졌다'는 오독 방지")
ys = pd.read_csv(
 PROCESSED / "macro_yield_spread_10y2y_monthly_2007-01_to_2020-09.csv",
 parse_dates=["observation_date"],
)
ys["year"] = ys["observation_date"].dt.year
print("연도별 평균(165개월 전체):")
print(ys.groupby("year")[["GS10", "GS2", "spread_10y2y"]].mean().round(2).to_string())
print("\n대출이 물린 2013~2019 구간의 연도 상관:")
win = ys[ys["year"].between(2013, 2019)]
for col in ["GS10", "GS2", "spread_10y2y"]:
 print(f" corr({col}, 연도) = {win[[col, 'year']].corr().iloc[0, 1]:+.3f}")
print("\n같은 상관을 어디서 재느냐에 따라 값이 달라진다:")
print(f" 165개월 전체 corr(spread, 연도) = {ys[['spread_10y2y', 'year']].corr().iloc[0, 1]:+.3f}")
print(f" 대출 표본 기준 corr(spread, 연도) = {df[['spread_10y2y', 'year']].corr().iloc[0, 1]:+.3f}")

section("7. 발행연도별 부도율 (seasoning 편향 있음 - 해석 주의)")
g = df.groupby("year").agg(건수=("default", "size"), 부도율=("default", "mean"))
g["부도율"] = (g["부도율"] * 100).round(2)
print(g.to_string())

if __name__ == "__main__":
 main()

```

문서의 "AUC 0.71대"와 실측 0.70대의 차이가 어디서 오는지 가른다.

""" 문서의 "AUC 0.71대"와 실측 0.70대의 차이가 어디서 오는지 가른다.

`decision\_log.md` #13 ⑤·#17 ③은 조건변수 투입 시 최종 모형 AUC 기준을 **0.71대**로 적고 있는데, 확정 표본(#16, 723,563건)으로 재보면 **0.70대**가 나온다. 원인 후보는 둘이었다 – 표본 필터가 다르거나, 피처·하이퍼파라미터가 다르거나.

**피처·모델·분할을 완전히 고정하고 표본 필터만 바꿔** 원인을 분리한다.

실행 결과 (2026-07-30, conda base):

|                       |             |            |             |
|-----------------------|-------------|------------|-------------|
| 완결만 (만기필터 없음)         | n=1,117,571 | 부도율 19.49% | AUC=0.72827 |
| 완결 + 만기 + 버퍼 6m (#16) | n= 723,563  | 부도율 16.21% | AUC=0.70732 |

→ **-2.1%p가 오롯이 #16 만기필터 효과다.** 문서의 0.713은 필터 없는 쪽 범위에 있다.

만기가 도래하지 않은 완결건은 **조기부도가 과대표집**돼(부도율 19.49% vs 16.21%) 부도 신호가 인위적으로 뚜렷하다. 필터로 그 편향을 제거하면 **AUC가 내려가는 것이 정상**이다 – 성능이 나빠진 게 아니라 부풀려진 값이 빠진 것이다. #16이 필터를 넣은 이유가 바로 그것이다.

성격: **탐색·검증(재현)** 스크립트다. `config.yaml`의 6:2:2가 아니라 단일 80/20 분할을 쓴다 – 두 표본을 같은 조건으로 비교하는 것이 목적이라 의도된 차이다. 여기서 쓰는 AUC는 **표본 간 상대 비교**이며 승인/거절 기준을 정하는 데 쓰지 않는다(`src/analysis/AGENTS.md`).

재현 대상: `outputs/reports/oof\_diagnostics\_kgj.md` 「문서의 "AUC 0.71대"」 절.

"""

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
from sklearn.metrics import roc_auc_score
```

```
from sklearn.model_selection import train_test_split
```

```
import sys
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from analysis.model import predict_default_probability, train_model # noqa: E402
```

```
from preprocessing.loader import (# noqa: E402
```

```
 TARGET_COLUMN,
```

```
 load_raw_loans,
```

```
 make_target,
```

```
 normalize_loan_status,
```

```
 parse_term_months,
```

```
 select_feature_columns,
```

```
)
```

```
from preprocessing.preprocessor import coerce_dtypes # noqa: E402
```

```
from utils.config import load_config, repo_root # noqa: E402
```

```
MATURITY_CUTOFF_ORD = 2020 * 12 + 4 # issue_d + term ≤ 2020-04
```

```
def main() -> None:
```

```
 cfg = load_config()
```

```
 head = load_raw_loans(nrows=5)
```

```

features, _ = select_feature_columns(list(head.columns))
needed = sorted(set(features) | {TARGET_COLUMN, "issue_d", "term"})
raw = load_raw_loans(usecols=[c for c in needed if c in head.columns])

status = normalize_loan_status(raw[TARGET_COLUMN])
term = parse_term_months(raw["term"])
issue = pd.to_datetime(raw["issue_d"], format="%b-%Y", errors="coerce")
maturity_ord = (issue.dt.year * 12 + issue.dt.month) + term

completed = status.isin(["Fully Paid", "Charged Off"]) & issue.notna()
scenarios = [
 ("완결만 (만기필터 없음)", completed),
 ("완결 + 만기 + 버퍼 6m (#16 확정)", completed & (maturity_ord <= MATURITY_CUTOFF_ORD)),
]

rows = []
for name, mask in scenarios:
 sub = raw.loc[mask]
 y = make_target(sub[TARGET_COLUMN])
 X = coerce_dtypes(sub[features])
 X_tr, X_te, y_tr, y_te = train_test_split(
 X, y, test_size=0.2, random_state=cfg.random_seed.default, stratify=y
)
 model = train_model(X_tr, y_tr)
 auc = roc_auc_score(y_te, predict_default_probability(model, X_te))
 print(f"{name:32s} n={len(sub):>9,} 부도율 {y.mean():.2%} AUC={auc:.5f}")
 rows.append({"scenario": name, "n": len(sub), "default_rate": round(float(y.mean()), 6),
 "auc": round(float(auc), 5)})

out = repo_root() / "outputs" / "auc_sample_filter_comparison.csv"
out.parent.mkdir(parents=True, exist_ok=True)
df = pd.DataFrame(rows)
df["auc_diff_vs_unfiltered"] = (df["auc"] - df["auc"].iloc[0]).round(5)
df.to_csv(out, index=False)
print(f"\n차이: {(df['auc'].iloc[1] - df['auc'].iloc[0]) * 100:+.2f}%p → {out.relative_to(repo_root())}")

if __name__ == "__main__":
 main()

```

## src/analysis/model\_comparison.py

### PD 모형 후보 교차검증

""PD 모형 후보 교차검증 - `outputs/reports/model\_comparison\_kgj.md`의 표를 재생성한다.

팀원 3인이 각자 브랜치에 만든 모형 후보를 **\*\*동일 프로토콜\*\***로 붙여 비교한다.

후보 스크립트들은 `.fit()` 1회 + Validation AUC 1회 구조라 00F·보정·threshold 단계가 없어 그대로는 비교가 성립하지 않는다 - 그래서 **\*\*스펙(피쳐 + 하이퍼파라미터)만 추출해 본 파이프라인에 이식\*\***한다. 이 파일이 그 이식 코드다.

## 성격 - 재현·검증 스크립트다

`AGENTS.md` 「자료 위치 색인」 ② 범주. **\*\*본 파이프라인이 아니다.\*\*** 다만 탐색용 비교 스크립트 4종과 달리 **\*\*`config.yaml`의 6:2:2 매니페스트를 그대로 쓴다\*\*** - 비교 대상이 "팀 표준 분할에서 어느 스펙이 나온가"이므로 자체 분할을 쓰면 질문이 달라진다. Test는 열지 않는다.

⚠ **\*\*AUC는 스펙 간 상대 비교 전용이다.\*\*** 승인/거절 기준은 Sharpe로만 정한다 (`src/analysis/AGENTS.md`). 그리고 `--only decompose`의 A행(0.731)은 **\*\*#16 필터 미적용\*\*** 값이므로 어떤 문서에도 성능으로 인용하지 않는다.

## 다섯 블록

| 블록        | 재현 대상                                     | 산출                                                                           |
|-----------|-------------------------------------------|------------------------------------------------------------------------------|
| cv        | 리포트 4절 - 6스펙 5-fold 교차검증 + Δ Sharpe       | `model_comparison_crossvalidation.csv`                                       |
| decompose | 리포트 3절 전반 - 필터·분할·early stopping·피쳐 4단 분해 | `model_comparison_ayh_decompose.csv`                                         |
| ablation  | 리포트 3절 후반 - 20개 그룹별 기여                    | `model_comparison_zip_ablation.csv`                                          |
| seed      | 리포트 4절 보강 - 학습 seed 흔들림 + 페어드 부트스트랩       | `model_comparison_seed_stability.csv`                                        |
| zipdiag   | 리포트 5절 - `zip_code` 암기 진단 (학습 없음, 수초)     | `model_comparison_zip_diagnostic.csv`<br>`model_comparison_zip_examples.csv` |

## 실행

|                                                                                        |             |
|----------------------------------------------------------------------------------------|-------------|
| <code>/opt/anaconda3/bin/python src/analysis/model_comparison.py</code>                | # 전부 (~20분) |
| <code>/opt/anaconda3/bin/python src/analysis/model_comparison.py --only cv</code>      | # 블록 하나만    |
| <code>/opt/anaconda3/bin/python src/analysis/model_comparison.py --only zipdiag</code> | # 수초        |

**\*\*전제\*\***: ③ 원본 `data/raw/lending\_club\_2020\_train.csv` (1.2GB, git 미추적) - `cv`·`decompose`가 읽는다 ④ 공유 parquet `data/processed/shared/` - `ablation`·`seed`·`zipdiag`가 읽는다 ⑤ 분할 매니페스트 (`config.yaml`의 seed 기준). scikit-learn·xgboost가 필요하다 - macOS 시스템 `python3`에는 없다(`AGENTS.md` 실행 환경).  
""

```
from __future__ import annotations
```

```
import argparse
```

```
import sys
```

```
import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.metrics import brier_score_loss, log_loss, roc_auc_score
```

```
from sklearn.model_selection import StratifiedKFold, train_test_split
```

```
from xgboost import XGBClassifier
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from analysis.model import (# noqa: E402
```

```

 apply_calibrator,
 assign_quantile_by_term,
 calibration_metrics,
 fit_calibrator,
 quantile_edges_by_term,
)
from analysis.realized_return import (# noqa: E402
 ReturnAssumptions,
 build_excess_returns,
 build_return_inputs,
 cash_reinvestment,
 default_cell_stats,
 expected_excess_return,
 q_score,
 variance_excess_return,
)
from analysis.sharpe_optimizer import (# noqa: E402
 append_row_csv,
 compare_ranking_criteria,
 completed_keys,
 find_optimal_threshold,
)
from preprocessing.export_shared_dataset import load_shared # noqa: E402
from preprocessing.loader import MATURITY_CUTOFF, raw_path # noqa: E402
from preprocessing.preprocessor import (# noqa: E402
 build_feature_table,
 load_split_manifest,
 resplit_train_validation,
 split_from_manifest,
)
from utils.config import load_config, repo_root # noqa: E402

N_FOLDS = 5
N_BOOT = 2_000
SEED_SWEEP = [42, 7, 2026, 99] # config seed는 런타임에 맨 앞으로 붙인다

비교 대상 스펙 - 하이퍼파라미터

⚠️ `random_state`는 넣지 않는다. 실행 시 `config.yaml`의 seed를 주입한다
(팀원 원본은 각각 다른 seed를 썼고, 그 차이가 곧 seed 잡음이다 - 블록 `seed` 참고).

def params_main() -> dict:
 """`src/analysis/model.py` `default_params()` (origin/main)와 동일."""
 return dict(
 n_estimators=600, learning_rate=0.05, max_depth=6, min_child_weight=5,
 subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
 objective="binary:logistic", eval_metric="auc", tree_method="hist",
 enable_categorical=True, max_cat_to_onehot=1, n_jobs=-1,
)

def params_ljh() -> dict:
 """`origin/feature/#5-xgb-model-ljh` : `run_xgboost_candidate.py` (`77d857d`).

 원본은 `eval_set=[(X_val, y_val)]`을 넘기지만 `early_stopping_rounds`가 없어 학습에

```

영향이 없다 - 여기서는 넘기지 않는다(넘기면 Validation이 학습 신호가 될 위험만 남는다).

"""

```
return dict(
 n_estimators=300, learning_rate=0.04, max_depth=4, min_child_weight=10,
 subsample=0.75, colsample_bytree=0.75, reg_alpha=0.5, reg_lambda=2.0,
 enable_categorical=True, tree_method="hist", eval_metric="logloss", n_jobs=-1,
)
```

```
def params_ayh() -> dict:
```

```
 """커밋 `fe88d6c` : `src/analysis/model.py` `train_model()`.
```

⚠ 이 코드는 브랜치 `feature/#9-setup-ayh`의 tip에 \*\*없다\*\* - 머지(`da2d94f`)가  
`model.py`를 main 쪽으로 해소하면서 소실됐다. 원문은 `git show fe88d6c:src/analysis/model.py`.  
원본의 `early\_stopping\_rounds=50`은 500트리 내내 미발동해 기여가 0으로 실측됐으므로  
(블록 `decompose`의 B행 = C행) 여기서는 뺐다.

"""

```
return dict(
 n_estimators=500, max_depth=4, learning_rate=0.05,
 subsample=0.8, colsample_bytree=0.8, tree_method="hist",
 enable_categorical=True, eval_metric="auc", n_jobs=-1,
)
```

```

```

```
비교 대상 스펙 - 안예환 피쳐 목록 (커밋 `fe88d6c`에서 전사)
```

```

```

```
출처: `data/processed/lending_club_변수분류_류성환.xlsx_v2.numbers` 「유지변수_84」 시트
(2026-07-27). 팀 단일 원본은 `variable_dictionary_byGJ.xlsx`이며 그쪽은 102개를 낸다 -
두 문서가 답하는 질문이 다르다(리포트 6절). 여기 목록은 **비교 대상 스펙의 정의**로서
값을 박아 둔다. 팀원 모듈을 import하지 않는 이유는 그 파일이 브랜치 tip에 없기 때문이다.
```

```
AYH_FEATURES = [
```

```
 # A - 핵심 대출·신청 정보 (22)
```

```
 "annual_inc", "delinq_2yrs", "dti", "emp_length", "fico_range_high", "fico_range_low",
 "funded_amnt", "funded_amnt_inv", "grade", "home_ownership", "inq_last_6mths",
 "installment", "int_rate", "loan_amnt", "mort_acc", "pub_rec", "pub_rec_bankruptcies",
 "purpose", "revol_util", "sub_grade", "term", "verification_status",
```

```
 # B - CB 상세 계좌통계, 결측 5.3% 이하 (40)
```

```
 "acc_open_past_24mths", "addr_state", "application_type", "avg_cur_bal",
 "bc_open_to_buy", "bc_util", "chargeoff_within_12_mths", "collections_12_mths_ex_med",
 "earliest_cr_line", "initial_list_status", "mo_sin_old_il_acct", "mo_sin_old_rev_tl_op",
 "mo_sin_rcnt_rev_tl_op", "mo_sin_rcnt_tl", "mths_since_recent_bc",
 "num_accts_ever_120_pd", "num_actv_bc_tl", "num_actv_rev_tl", "num_bc_sats",
 "num_bc_tl", "num_il_tl", "num_op_rev_tl", "num_rev_accts", "num_rev_tl_bal_gt_0",
 "num_sats", "num_tl_90g_dpd_24m", "num_tl_op_past_12m", "open_acc", "pct_tl_nvr_dlq",
 "percent_bc_gt_75", "revol_bal", "tax_liens", "tot_coll_amt", "tot_cur_bal",
 "tot_hi_cred_lim", "total_acc", "total_bal_ex_mort", "total_bc_limit",
 "total_il_high_credit_limit", "total_rev_hi_lim",
```

```
 # C - 결측 12.7% 이상 (20)
```

```
 "mths_since_last_record", "mths_since_recent_bc_dlq", "mths_since_last_major_derog",
 "mths_since_recent_revol_delinq", "mths_since_last_delinq", "il_util",
 "mths_since_rcnt_il", "all_util", "inq-fi", "inq_last_12m", "max_bal_bc",
 "open_acc_6m", "open_act_il", "open_il_12m", "open_il_24m", "open_rv_12m",
 "open_rv_24m", "total_bal_il", "total_cu_tl", "mths_since_recent_inq",
```

```
 # 특수 - 시점 변수 (1)
```

```
 "issue_d",
```

```
]
```

```

AYH_CATEGORICAL = [
 "grade", "sub_grade", "home_ownership", "verification_status", "purpose",
 "addr_state", "initial_list_status", "application_type",
]
AYH_EMP_LENGTH = {
 "< 1 year": 0, "1 year": 1, "2 years": 2, "3 years": 3, "4 years": 4, "5 years": 5,
 "6 years": 6, "7 years": 7, "8 years": 8, "9 years": 9, "10+ years": 10,
}
POLICY_PREFIX = "Does not meet the credit policy. Status:"

팀 102피처 중 안예환이 쓰지 않은 20개 - 블록 `ablation`의 그룹 정의
SEC_APP = [
 "sec_app_chargeoff_within_12_mths", "sec_app_collections_12_mths_ex_med",
 "sec_app_earliest_cr_line", "sec_app_fico_range_high", "sec_app_fico_range_low",
 "sec_app_inq_last_6mths", "sec_app_mort_acc", "sec_app_num_rev_accts",
 "sec_app_open_acc", "sec_app_open_act_il", "sec_app_revol_util",
]
JOINT = ["annual_inc_joint", "dti_joint", "revol_bal_joint"]
RARE_DELINQ = ["acc_now_delinq", "delinq_amnt", "num_tl_120dpd_2m", "num_tl_30dpd"]

def hdr(title: str) -> None:
 print("\n" + "=" * 78 + f"\n{title}\n" + "=" * 78, flush=True)

def out_dir() -> Path:
 d = repo_root() / "outputs"
 d.mkdir(parents=True, exist_ok=True)
 return d

def maturity_cutoff_ord() -> int:
 """`issue_d + term ≤ 2020-04`의 절대 월 서수 (#16). `loader.MATURITY_CUTOFF`가 원본."""
 c = pd.Timestamp(MATURITY_CUTOFF)
 return c.year * 12 + c.month

안예환 피처 파이프라인 (`fe88d6c`의 `_build_target` · `_build_features` 재현)

def _ayh_target(loan_status: pd.Series) -> pd.Series:
 """`Charged Off`=1 / `Fully Paid`=0, 나머지는 NaN. 신용정책 접두사를 떼고 판정한다."""
 status = loan_status.astype(str).str.replace(POLICY_PREFIX, "", regex=False)
 y = pd.Series(np.nan, index=loan_status.index)
 y[status == "Charged Off"] = 1
 y[status == "Fully Paid"] = 0
 return y

def _ayh_features(df: pd.DataFrame) -> pd.DataFrame:
 """날짜 → 파생 4개, FICO 2개 → 평균 1개. 결측은 채우지 않는다.

 83개 목적이 **84개 학습 피처**가 되는 계산: 83 - 2(날짜) + 4(파생) - 2(FICO) + 1(평균).
 """
 X = df.copy()
 issue = pd.to_datetime(X["issue_d"], format="%b-%Y", errors="coerce")

```

```

earliest = pd.to_datetime(X["earliest_cr_line"], format="%b-%Y", errors="coerce")
X["issue_year"] = issue.dt.year.astype("float64")
angle = 2 * np.pi * issue.dt.month / 12
X["issue_month_sin"] = np.sin(angle)
X["issue_month_cos"] = np.cos(angle)
X["credit_history_months"] = (
 (issue.dt.year - earliest.dt.year) * 12 + (issue.dt.month - earliest.dt.month)
).astype("float64")
X = X.drop(columns=["issue_d", "earliest_cr_line"])

X["fico_avg"] = (X["fico_range_high"] + X["fico_range_low"]) / 2
X = X.drop(columns=["fico_range_high", "fico_range_low"])

X["emp_length"] = X["emp_length"].map(AYH_EMP_LENGTH)
X["term"] = X["term"].astype(str).str.extract(r"(\d+)").astype("float64")
for c in ("int_rate", "revol_util"):
 if X[c].dtype == object:
 X[c] = X[c].astype(str).str.rstrip("%").astype("float64")
for c in AYH_CATEGORICAL:
 X[c] = X[c].astype("category")
return X

```

```

def load_ayh_raw(apply_maturity_filter: bool) -> tuple[pd.DataFrame, pd.Series, pd.Series]:

```

```

 """원본을 **읽기 전용**으로 열어 안에한 스펙 테이블을 만든다. -> `(X, y, id)`

```

```

 `apply_maturity_filter=False`가 `fe88d6c` `main()`의 실제 동작이다 - #16 필터가 빠진다.
 """

```

```

usecols = sorted(set(AYH_FEATURES) | {"loan_status", "earliest_cr_line", "id"})
raw = pd.read_csv(raw_path(), usecols=usecols, low_memory=False)
y = _ayh_target(raw["loan_status"])
keep = y.notna()

```

```

if apply_maturity_filter:
 issue = pd.to_datetime(raw["issue_d"], format="%b-%Y", errors="coerce")
 term = raw["term"].astype(str).str.extract(r"(\d+)")[0].astype("float64")
 keep &= (((issue.dt.year * 12 + issue.dt.month) + term) <= maturity_cutoff_ord())
 keep &= issue.notna()

```

```

X = _ayh_features(raw.loc[keep, AYH_FEATURES])
return X, y.loc[keep].astype(int), raw.loc[keep, "id"].astype(str)

```

```

공통 - OOF · 평가 · Sharpe

```

```

def compute_oof_local(X, y, params: dict, seed: int) -> tuple[pd.Series, list[float]]:

```

```

 """Train 안 5-fold OOF. `model.compute_oof()`와 같은 절차이나 임의 params를 받는다."""

```

```

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=seed)
oof = pd.Series(np.nan, index=X.index, name="pd_oof")
aucs: list[float] = []
for k, (tr, va) in enumerate(skf.split(X, y)):
 m = XGBClassifier(**params, random_state=seed).fit(X.iloc[tr], y.iloc[tr], verbose=False)
 p = m.predict_proba(X.iloc[va])[0, 1]
 oof.iloc[va] = p
 aucs.append(roc_auc_score(y.iloc[va], p))
print(f" fold {k + 1}/{N_FOLDS} AUC={aucs[-1]:.5f}", flush=True)

```

```

if oof.isna().any():
 raise RuntimeError("OOF에 결측이 남았다 - fold 분할을 확인하라.")
return oof, aucs

```

```

def sharpe_for(pd_oof, p_va_raw, p_va_cal, xr, tr_idx, va_idx) -> pd.DataFrame:
 """승인선 3종 + approve-all. 분위·점수는 보정 전 PD, `E[XR]`의 `p`는 보정 후 (진단 C-4).

```

간별 통계표(`mu\_부도`, `var\_부도`)는 **\*\*Train에서만\*\*** 만들어 Validation에 적용한다 - 재분위하지 않는다(#20).

```

"""
term_tr, term_va = xr["term"].loc[tr_idx], xr["term"].loc[va_idx]
edges = quantile_edges_by_term(pd_oof, term_tr)
q_tr = assign_quantile_by_term(pd_oof, term_tr, edges)
q_va = assign_quantile_by_term(p_va_raw, term_va, edges)

st = default_cell_stats(xr["xr_default"].loc[tr_idx], q_tr, term_tr)
mu_map, var_map = st["mu"].to_dict(), st["var"].to_dict()
keys = list(zip(term_va, q_va))
mu_d = pd.Series([mu_map.get(k, np.nan) for k in keys], index=va_idx)
var_d = pd.Series([var_map.get(k, np.nan) for k in keys], index=va_idx)

xr_va = xr.loc[va_idx]
e_xr = expected_excess_return(p_va_cal, xr_va["xr_normal"], mu_d)
v_xr = variance_excess_return(p_va_cal, xr_va["xr_normal"], mu_d, var_d, var_normal=0.0)
qs = q_score(e_xr, v_xr)
realized = xr_va["xr_realized"]

ok = realized.notna() & e_xr.notna() & qs.notna() & p_va_raw.notna()
return compare_ranking_criteria(
 {"pd": (p_va_raw[ok], True), "E[XR]": (e_xr[ok], False), "q_score": (qs[ok], False)},
 realized[ok],
)

```

```

블록 cv - 리포트 4절

```

```

def block_cv(seed: int) -> pd.DataFrame:
 hdr("블록 cv - 6스팩 5-fold 교차검증 + Δ Sharpe (리포트 4절)")

 # 비교 실험이므로 `zip_code`를 남긴 채 받는다 - 제의 전후를 나란히 돌리는 게 이 블록의 질문이다.
 X_team, y, meta = build_feature_table(apply_decisions=False)
 parts = split_from_manifest(X_team, y, meta)
 tr_idx = parts["train"][0].index
 va_idx = parts["validation"][0].index
 print(f"표본 {len(X_team):,} 팀 피쳐 {X_team.shape[1]} "
 f"train {len(tr_idx):,} validation {len(va_idx):,}", flush=True)

 X_ayh, y_ayh, _ = load_ayh_raw(apply_maturity_filter=True)
 if not X_ayh.index.equals(X_team.index):
 raise RuntimeError("안예한 표본 인덱스가 #16 표본과 다르다 - 필터 정의를 확인하라.")
 if not y_ayh.equals(y.astype(int)):
 raise RuntimeError("타깃 정의가 어긋난다 - 접두사 정규화를 확인하라.")
 print(f"안예한 피쳐 {X_ayh.shape[1]} (목록 {len(AYH_FEATURES)} + 파생)", flush=True)

 outcome = build_return_inputs()

```

```

xr_by = {"treasury": build_excess_returns(outcome, ReturnAssumptions()),
 "cash": build_excess_returns(outcome, cash_reinvestment(ReturnAssumptions()))}

X_nozip = X_team.drop(columns=["zip_code"])
specs = [
 ("1 main 기본 (팀102·main파라)", X_team, params_main()),
 ("2 이지희 77d857d (팀102·ljh파라)", X_team, params_ljh()),
 ("3 안예환 fe88d6c (ayh84·ayh파라)", X_ayh, params_ayh()),
 ("4 ayh파라만 (팀102·ayh파라)", X_team, params_ayh()),
 ("5 main - zip_code (팀101)", X_nozip, params_main()),
 ("6 이지희 - zip_code (팀101)", X_nozip, params_ljh()),
]

rows = []
for name, Xf, params in specs:
 print(f"\n--- {name} (피쳐 {Xf.shape[1]}) ---", flush=True)
 t0 = time.time()
 X_tr, y_tr = Xf.loc[tr_idx], y.loc[tr_idx]
 X_va, y_va = Xf.loc[va_idx], y.loc[va_idx]

 pd_oof, fold_auc = compute_oof_local(X_tr, y_tr, params, seed)
 calib = fit_calibrator(pd_oof, y_tr)
 final = XGBClassifier(**params, random_state=seed).fit(X_tr, y_tr, verbose=False)
 p_va_raw = pd.Series(final.predict_proba(X_va)[: , 1], index=X_va.index)
 p_va_cal = apply_calibrator(calib, p_va_raw)
 p_tr_in = pd.Series(final.predict_proba(X_tr)[: , 1], index=X_tr.index)

 m_raw, m_cal = calibration_metrics(y_va, p_va_raw), calibration_metrics(y_va, p_va_cal)
 auc_in, auc_oof = roc_auc_score(y_tr, p_tr_in), roc_auc_score(y_tr, pd_oof)
 row = {
 "spec": name, "n_features": Xf.shape[1],
 "cv_auc_mean": float(np.mean(fold_auc)), "cv_auc_sd": float(np.std(fold_auc, ddof=1)),
 "cv_auc_min": min(fold_auc), "cv_auc_max": max(fold_auc),
 "oof_auc": auc_oof, "auc_train_insample": auc_in, "auc_val": m_raw["auc"],
 "gap_insample": auc_in - m_raw["auc"], "gap_oof": auc_oof - m_raw["auc"],
 "ece_raw_pp": m_raw["ece_pp"], "ece_cal_pp": m_cal["ece_pp"],
 "brier_cal": m_cal["brier"],
 }
 print(f" CV {row['cv_auc_mean']:.5f} ± {row['cv_auc_sd']:.5f} "
 f"val {row['auc_val']:.5f} ECE {m_raw['ece_pp']:.3f}→{m_cal['ece_pp']:.3f}%p "
 f"Brier(보정후) {m_cal['brier']:.6f}", flush=True)

 for akey, xr in xr_by.items():
 cmp = sharpe_for(pd_oof, p_va_raw, p_va_cal, xr, tr_idx, va_idx)
 for _, r in cmp.iterrows():
 if r["criterion"] == "approve_all(대조군)":
 continue
 row[f"dS_{akey}_{r['criterion']}"] = r["delta_sharpe"]
 row[f"approv_{akey}_{r['criterion']}"] = r["approval_rate"]
 print(f" [{akey}] " + " ".join(
 f"{r['criterion']}: Δ{r['delta_sharpe']:+.4f}" for _, r in cmp.iterrows()
 if r["criterion"] != "approve_all(대조군)"), flush=True)
 print(f" ({time.time() - t0:.0f}s)", flush=True)
 rows.append(row)

return pd.DataFrame(rows)

```

```

블록 decompose - 리포트 3절 전반

def _fit_eval(X_tr, y_tr, X_va, y_va, params, early: bool, label: str, seed: int) -> dict:
 t0 = time.time()
 p = dict(params)
 if early:
 p["early_stopping_rounds"] = 50
 m = XGBClassifier(**p, random_state=seed)
 m.fit(X_tr, y_tr, eval_set=[(X_va, y_va)] if early else None, verbose=False)
 p_tr, p_va = m.predict_proba(X_tr)[: , 1], m.predict_proba(X_va)[: , 1]
 bi = getattr(m, "best_iteration", None)
 row = {
 "variant": label, "n_train": len(X_tr), "n_val": len(X_va),
 "default_rate": float(y_tr.mean()), "n_features": X_tr.shape[1],
 "early_stopping": early,
 "trees_used": (bi + 1) if (early and bi is not None) else params["n_estimators"],
 "auc_train": roc_auc_score(y_tr, p_tr), "auc_val": roc_auc_score(y_va, p_va),
 "logloss_val": log_loss(y_va, p_va), "brier_val": brier_score_loss(y_va, p_va),
 }
 print(f" [{label}] n_tr={len(X_tr):,} feat={X_tr.shape[1]} trees={row['trees_used']} "
 f"→ val AUC {row['auc_val']:.5f} ({time.time() - t0:.0f}s)", flush=True)
 return row

def block_decompose(seed: int) -> tuple[pd.DataFrame, pd.DataFrame]:
 hdr("블록 decompose - 필터·분할·early stopping·피쳐 4단 분해 (리포트 3절)")
 params = params_ayh()
 rows = []

 # A. as-committed - #16 필터 없음 · 자체 75/25 분할 · early stopping ON
 X, y, ids = load_ayh_raw(apply_maturity_filter=False)
 print(f"완결건(#16 필터 없음): {len(X):,} 부도를 {y.mean():.4%}", flush=True)
 Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed)
 rows.append(_fit_eval(Xtr, ytr, Xva, yva, params, True, "A. as-committed", seed))

 # 팀 Test 유입량 - 리포트 2절 ②
 team = ids.map(load_split_manifest())
 ayh_split = pd.Series("ayh_train", index=ids.index)
 ayh_split.loc[Xva.index] = "ayh_validation"
 overlap = pd.crosstab(ayh_split, team.fillna("#16 필터로 팀 표본에서 제외"))
 n_test = int(team.eq("test").sum())
 if "test" in overlap.columns:
 used = int(overlap.loc["ayh_train", "test"])
 print(f" ⚠ 팀 Test {n_test:,}건 중 {used:,}건({used / n_test:.1%})이 학습에 유입", flush=True)
 del X, y, Xtr, Xva, ytr, yva

 # B·C·D - #16 필터 적용
 X, y, ids = load_ayh_raw(apply_maturity_filter=True)
 print(f"완결+만기버퍼(#16 확정): {len(X):,} 부도를 {y.mean():.4%}", flush=True)
 Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed)
 rows.append(_fit_eval(Xtr, ytr, Xva, yva, params, True, "B. +#16 필터", seed))
 rows.append(_fit_eval(Xtr, ytr, Xva, yva, params, False, "C. +ES OFF", seed))
 del Xtr, Xva, ytr, yva

 team = ids.map(load_split_manifest())

```

```
tr, va = (team == "train").to_numpy(), (team == "validation").to_numpy()
print(f" 매니페스트 매칭: train {tr.sum():,} validation {va.sum():,} "
 f"미매칭 {int(team.isna().sum()):,}", flush=True)
rows.append(_fit_eval(X[tr], y[tr], X[va], y[va], params, False, "D. +팀 분할", seed))
del X, y
```

```
E - 팀 102피쳐
```

```
Xs, ys, _, sp = load_shared("trainval")
rows.append(_fit_eval(Xs[sp == "train"], ys[sp == "train"],
 Xs[sp == "validation"], ys[sp == "validation"],
 params, False, "E. +팀 피쳐", seed))
```

```
out = pd.DataFrame(rows)
out["delta_vs_prev"] = out["auc_val"].diff()
return out, overlap
```

```

블록 ablation - 리포트 3절 후반

```

```
def block_ablation(seeds: list[int]) -> pd.DataFrame:
 hdr("블록 ablation - 팀102 중 20개 그룹별 기여 (리포트 3절)")
 print("⚠ 하이퍼파라미터는 **안예한 것**을 쓴다 - '그의 스펙에서 무엇이 이득인가'가 질문이다.\n"
 " main 파라미터 기준 동일 효과는 블록 cv의 스펙1 → 스펙5에서 확인된다.", flush=True)
```

```
X, y, _, sp = load_shared("trainval")
tr, va = sp == "train", sp == "validation"
X_tr, y_tr, X_va, y_va = X[tr], y[tr], X[va], y[va]
params = params_ayh()
```

```
groups = {
 "(기준) 팀102 전체": [],
 "- zip_code": ["zip_code"],
 "- sec_app* (11)": SEC_APP,
 "- joint (3)": JOINT,
 "- policy_code": ["policy_code"],
 "- 회소연체카운터 (4)": RARE_DELINQ,
 "- 20개 전부 (=ayh)": ["zip_code", *SEC_APP, *JOINT, "policy_code", *RARE_DELINQ],
}
```

```
rows = []
for label, drop in groups.items():
 A_tr = X_tr.drop(columns=drop) if drop else X_tr
 A_va = X_va.drop(columns=drop) if drop else X_va
 aucs = [
 roc_auc_score(y_va, XGBClassifier(**params, random_state=s)
 .fit(A_tr, y_tr, verbose=False).predict_proba(A_va)[:, 1])
 for s in seeds
]
 rows.append({"variant": label, "n_features": A_tr.shape[1],
 "auc_mean": float(np.mean(aucs)), "auc_sd": float(np.std(aucs, ddof=1)),
 "auc_min": min(aucs), "auc_max": max(aucs)})
 print(f" [{label}] feat={A_tr.shape[1]} → AUC {np.mean(aucs):.5f} "
 f"(sd {np.std(aucs, ddof=1):.5f}, n_seed={len(seeds)})", flush=True)
```

```
out = pd.DataFrame(rows)
out["delta_vs_base"] = out["auc_mean"] - out.loc[0, "auc_mean"]
```

```
return out
```

```

블록 seed - 리포트 4절 보강

def block_seed(seeds: list[int]) -> pd.DataFrame:
 hdr("블록 seed - 학습 seed 흔들림 + 페어드 부트스트랩 (리포트 4절)")
 X, y, _, sp = load_shared("trainval")
 X_tr, y_tr = X[sp == "train"], y[sp == "train"]
 X_va, y_va = X[sp == "validation"], y[sp == "validation"]
 yv = y_va.to_numpy(dtype="int8")

 preds, rows = {}, []
 for name, params in (("main", params_main()), ("ljh", params_ljh())):
 for s in seeds:
 p = XGBClassifier(**params, random_state=s).fit(
 X_tr, y_tr, verbose=False).predict_proba(X_va[:, 1])
 rows.append({"model": name, "seed": s, "auc_val": roc_auc_score(yv, p)})
 print(f" {name:5s} seed={s:<9} val AUC={rows[-1]['auc_val']:.5f}", flush=True)
 if s == seeds[0]:
 preds[name] = p

 tbl = pd.DataFrame(rows)
 print("\n모델별 흔들림:\n" +
 tbl.groupby("model")["auc_val"].agg(["mean", "std", "min", "max"]).to_string())

 rng = np.random.default_rng(seeds[0])
 n = len(yv)
 d = np.empty(N_BOOT)
 for b in range(N_BOOT):
 i = rng.integers(0, n, n)
 yb = yv[i]
 d[b] = (np.nan if yb.sum() in (0, len(yb))
 else roc_auc_score(yb, preds["ljh"][i]) - roc_auc_score(yb, preds["main"][i]))
 d = d[~np.isnan(d)]
 lo, hi = np.percentile(d, [2.5, 97.5])
 p_two = 2 * min((d <= 0).mean(), (d >= 0).mean())
 print(f"\n페어드 부트스트랩 ΔAUC(ljh - main), 동일 Validation, B={len(d)}")
 print(f" 관측 Δ {roc_auc_score(yv, preds['ljh']) - roc_auc_score(yv, preds['main']):+.5f}"
 f" 95% CI [{lo:+.5f}, {hi:+.5f}] 양측 p≈{p_two:.3f}")
 return tbl
```

```

블록 zipdiag - 리포트 5절

def block_zipdiag() -> tuple[pd.DataFrame, pd.DataFrame]:
 hdr("블록 zipdiag - zip_code 암기 진단 (리포트 5절, 학습 없음)")
 X, y, _, sp = load_shared("trainval")
 tr, va = sp == "train", sp == "validation"
 base = float(y[tr].mean())

 rows, examples = [], pd.DataFrame()
 for col in ("zip_code", "grade", "addr_state", "purpose"):
 a = pd.DataFrame({"k": X.loc[tr, col].astype(str), "y": y[tr]}).groupby("k")["y"].agg(["mean", "size"])
 b = pd.DataFrame({"k": X.loc[va, col].astype(str), "y": y[va]}).groupby("k")["y"].agg(["mean", "size"])
```

```

j = a.join(b, lsuffix="_tr", rsuffix="_va", how="inner").dropna()

칸이 전부 기저 부도율이라도 표본 때문에 생기는 sd
noise = float(np.mean(np.sqrt(base * (1 - base) / j["size_tr"])))
obs = float(j["mean_tr"].std(ddof=1))
w = j["size_va"]
mt, mv = np.average(j["mean_tr"], weights=w), np.average(j["mean_va"], weights=w)
r = (np.average((j["mean_tr"] - mt) * (j["mean_va"] - mv), weights=w)
 / np.sqrt(np.average((j["mean_tr"] - mt) ** 2, weights=w)
 * np.average((j["mean_va"] - mv) ** 2, weights=w)))

관측분산 = 신호분산 + 잡음분산
signal = float(np.sqrt(max(obs ** 2 - noise ** 2, 0.0)))
rows.append({"variable": col, "n_categories": len(j),
 "median_n_train": float(j["size_tr"].median()),
 "observed_sd": obs, "noise_sd": noise, "signal_sd": signal,
 "observed_over_noise": obs / noise, "corr_train_val": float(r)})
print(f" {col:12s} 범주 {len(j):>4,} 칸당중위 {j['size_tr'].median():>7.0f} "
 f"실측sd {obs:.4f} 잡음sd {noise:.4f} 비율 {obs / noise:.2f}배 r={r:.3f}", flush=True)

if col == "zip_code":
 big = j[j["size_tr"] >= 300]
 examples = big.nlargest(5, "mean_tr").reset_index().rename(columns={"k": "zip_code"})
 examples["base_default_rate"] = base
 print(f"\n Train 부도율 상위 5개 (300건 이상) - 전체 평균 {base:.1%}")
 for _, e in examples.iterrows():
 print(f" {e['zip_code']}: Train {e['mean_tr']:.1%} (n={e['size_tr']:.0f})"
 f" → Validation {e['mean_va']:.1%} (n={e['size_va']:.0f})", flush=True)

```

```

return pd.DataFrame(rows), examples

```

```

블록 stability - `zip_code` 제외 결정의 K회 반복 안정성 (#18)

```

```

def parse_seeds(spec: str) -> list[int]:
 """`"0-24" . . "0-9,20,30-32"` → seed 목록. 구간이 겹치면 중복을 제거한다."""
 out: set[int] = set()
 for part in spec.split(","):
 part = part.strip()
 if "-" in part.lstrip("-"):
 a, b = part.split("-", 1)
 out |= set(range(int(a), int(b) + 1))
 elif part:
 out.add(int(part))
 return sorted(out)

```

```

def block_stability(seed_list: list[int], out_path: Path) -> pd.DataFrame:
 """Train+Validation 풀을 seed마다 재분할해 `τ*`·Δ Sharpe의 **분포**를 본다 (#18).

 Validation 하나로 전 Δ Sharpe는 표본에 따라 흔들리는 추정치다. `main`과
 `main - zip_code`를 **같은 재분할**에 함께 돌려 찍지는 비교(paired)를 만든다 -
 seed별 차이의 분포를 보면 +0.015가 우연인지 구조인지 갈린다.

```

- ⚠ Test는 `resplit\_train\_validation()`이 고정한다 - 재분할되는 건 BTest 80%뿐이다.
- ⚠ 칸별 통계표는 \*\*seed마다 다시 만든다\*\* (`sharpe\_optimizer.scores\_for\_assumptions` docstring의 잠정 확정 - seed 하나로 고정하면 다른 seed의 Validation이 섞여 누수다).

반복마다 CSV에 덧붙이고 이미 있는 `(spec, seed)`는 건너뛰다 - \*\*구간을 나눠 여러 번에 걸쳐 돌릴 수 있다.\*\* seed 집합이 같으면 한 번에 돌린 것과 결과가 같다.

====

```
hdr(f"블록 stability - seed {seed_list[0]}~{seed_list[-1]} ({len(seed_list)}개) 재분할 반복")
print(f"산출(덧붙임) → {out_path.name}", flush=True)
```

# 위 cv 블록과 같은 이유로 `zip\_code`를 남긴 채 받는다(찍지는 비교가 이 블록의 목적).

```
X_team, y, meta = build_feature_table(apply_decisions=False)
outcome = build_return_inputs()
xr_by = {"treasury": build_excess_returns(outcome, ReturnAssumptions()),
 "cash": build_excess_returns(outcome, cash_reinvestment(ReturnAssumptions()))}
variants = {
 "main": X_team,
 "main_nozip": X_team.drop(columns=["zip_code"]),
}
params = params_main()
```

```
done = completed_keys(out_path, ("spec", "seed"))
```

```
if done:
 print(f" 이미 계산된 (spec, seed) {len(done)}쌍 - 건너뛰다", flush=True)
```

```
for seed in seed_list:
 if all((name, str(seed)) in done for name in variants):
 print(f"\nseed {seed}: 전부 계산됨 - 건너뛴", flush=True)
 continue
```

```
t0 = time.time()
parts = resplit_train_validation(X_team, y, meta, seed=seed)
tr_idx = parts["train"][0].index
va_idx = parts["validation"][0].index
print(f"\nseed {seed} train {len(tr_idx):,} validation {len(va_idx):,}", flush=True)
```

```
for name, Xf in variants.items():
 if (name, str(seed)) in done:
 print(f" [{name}] 건너뛴", flush=True)
 continue
 X_tr, y_tr = Xf.loc[tr_idx], y.loc[tr_idx]
 X_va = Xf.loc[va_idx]
```

```
pd_oof, _ = compute_oof_local(X_tr, y_tr, params, seed)
calib = fit_calibrator(pd_oof, y_tr)
final = XGBClassifier(**params, random_state=seed).fit(X_tr, y_tr, verbose=False)
p_va_raw = pd.Series(final.predict_proba(X_va)[:, 1], index=X_va.index)
p_va_cal = apply_calibrator(calib, p_va_raw)
```

```
for akey, xr in xr_by.items():
 term_tr, term_va = xr["term"].loc[tr_idx], xr["term"].loc[va_idx]
 edges = quantile_edges_by_term(pd_oof, term_tr)
 q_tr = assign_quantile_by_term(pd_oof, term_tr, edges)
 q_va = assign_quantile_by_term(p_va_raw, term_va, edges)
 st = default_cell_stats(xr["xr_default"].loc[tr_idx], q_tr, term_tr)
 mu_map, var_map = st["mu"].to_dict(), st["var"].to_dict()
 keys = list(zip(term_va, q_va))
 mu_d = pd.Series([mu_map.get(k, np.nan) for k in keys], index=va_idx)
 var_d = pd.Series([var_map.get(k, np.nan) for k in keys], index=va_idx)
```

```

xr_va = xr.loc[va_idx]
e_xr = expected_excess_return(p_va_cal, xr_va["xr_normal"], mu_d)
v_xr = variance_excess_return(p_va_cal, xr_va["xr_normal"], mu_d, var_d,
 var_normal=0.0)

qs = q_score(e_xr, v_xr)
realized = xr_va["xr_realized"]
ok = realized.notna() & e_xr.notna() & qs.notna() & p_va_raw.notna()

for crit, (score, lower) in {
 "pd": (p_va_raw[ok], True),
 "E[XR]": (e_xr[ok], False),
 "q_score": (qs[ok], False),
}.items():
 res = find_optimal_threshold(score, realized[ok], lower)
 append_row_csv(out_path, {"spec": name, "seed": seed, "reinvest": akey,
 "criterion": crit, **res})

 if crit == "q_score":
 print(f" [{name}·{akey}] q_score τ*={res['threshold']:.4f} "
 f"승인을 {res['approval_rate']:.1%} Δ{res['delta_sharpe']:+.4f}",
 flush=True)

print(f" (seed {seed} {time.time() - t0:.0f}s)", flush=True)

return pd.read_csv(out_path)

```

```
def summarize_stability(df: pd.DataFrame) -> None:
```

```
"""seed별 Δ Sharpe 분포와 **짜지는 차이*(nozip - main)를 요약한다."""
```

```
for akey in sorted(df["reinvest"].unique()):
```

```
 for crit in ("q_score", "E[XR]", "pd"):
```

```
 sub = df[(df["reinvest"] == akey) & (df["criterion"] == crit)]
```

```
 piv = sub.pivot_table(index="seed", columns="spec", values="delta_sharpe")
```

```
 if not {"main", "main_nozip"} <= set(piv.columns):
```

```
 continue
```

```
 piv = piv.dropna()
```

```
 d = piv["main_nozip"] - piv["main"]
```

```
 print(f"\n[{akey}] · {crit} K={len(piv)}")
```

```
 print(f" main Δ Sharpe 평균 {piv['main'].mean():.4f} "
 f"sd {piv['main'].std(ddof=1):.4f} 중앙값 {piv['main'].median():.4f}")
```

```
 print(f" main_nozip Δ Sharpe 평균 {piv['main_nozip'].mean():.4f} "
 f"sd {piv['main_nozip'].std(ddof=1):.4f} 중앙값 {piv['main_nozip'].median():.4f}")
```

```
 print(f" 짜지는 차이(nozip-main) 평균 {d.mean():+.4f} sd {d.std(ddof=1):.4f} "
 f"nozip 승 {int((d > 0).sum())}/{len(d)}")
```

```
 if len(d) > 1:
```

```
 se = d.std(ddof=1) / np.sqrt(len(d))
```

```
 print(f" 표준오차 {se:.5f} → 평균/표준오차 = {d.mean() / se:.1f}배 "
 f"95% CI [{d.mean() - 1.96 * se:+.4f}, {d.mean() + 1.96 * se:+.4f}]")
```

```
 tau = sub.pivot_table(index="seed", columns="spec", values="threshold").dropna()
```

```
 print(f" τ* 중앙값 main {tau['main'].median():.4f} "
 f"nozip {tau['main_nozip'].median():.4f} "
 f"(sd {tau['main'].std(ddof=1):.4f} / {tau['main_nozip'].std(ddof=1):.4f})")
```

```

```

```
def main() -> None:
```

```
 ap = argparse.ArgumentParser(description=__doc__,
```

```
 formatter_class=argparse.RawDescriptionHelpFormatter)
```

```
 ap.add_argument("--only",
```

```

 choices=["cv", "decompose", "ablation", "seed", "zipdiag", "stability"],
 action="append", help="블록 선택 (반복 가능). 생략하면 stability 제외 전부.")
ap.add_argument("--seed", type=int, default=None, help="기본값: config.yaml의 seed")
ap.add_argument("--seeds", default="0-24",
 help="stability 블록의 재분할 seed 구간 (예: '0-24', '25-49', '0-9,20').")
args = ap.parse_args()

seed = args.seed if args.seed is not None else load_config().random_seed.default
stability는 수십 분~수 시간이라 기본 실행에 넣지 않는다 - 명시해야 돈다.
blocks = args.only or ["cv", "decompose", "ablation", "seed", "zipdiag"]
seeds = [seed, *SEED_SWEEP]
o = out_dir()
t0 = time.time()
print(f"seed={seed} 블록={blocks} 산출={o}", flush=True)

if "zipdiag" in blocks:
 diag, ex = block_zipdiag()
 diag.to_csv(o / "model_comparison_zip_diagnostic.csv", index=False)
 ex.to_csv(o / "model_comparison_zip_examples.csv", index=False)
if "ablation" in blocks:
 block_ablation(seeds[:3]).to_csv(o / "model_comparison_zip_ablation.csv", index=False)
if "seed" in blocks:
 block_seed(seeds).to_csv(o / "model_comparison_seed_stability.csv", index=False)
if "decompose" in blocks:
 dec, overlap = block_decompose(seed)
 dec.to_csv(o / "model_comparison_ayh_decompose.csv", index=False)
 overlap.to_csv(o / "model_comparison_ayh_test_overlap.csv")
if "cv" in blocks:
 block_cv(seed).to_csv(o / "model_comparison_crossvalidation.csv", index=False)
if "stability" in blocks:
 df = block_stability(parse_seeds(args.seeds), o / "model_comparison_stability.csv")
 hdr("stability 요약 - Δ Sharpe 분포와 찍지은 차이")
 summarize_stability(df)

print(f"\n완료 - {time.time() - t0:.0f}s. 리포트: outputs/reports/model_comparison_kgj.md")

if __name__ == "__main__":
 main()

```

## src/analysis/realized\_return\_spec\_check.py

실현수익률 계산 명세(팀원 제공)를 원본 전수 데이터에 적용 가능한지 점검한다.

""" 실현수익률 계산 명세(팀원 제공)를 원본 전수 데이터에 적용 가능한지 점검한다.

명세의 각 가정·예외처리·수치 주장을 `data/raw/lending_club_2020_train.csv` (1,755,295행)에 대해 실측한다. 값을 계산하는 것이 목적이 아니라, 계산이 성립하는지/몇 건이 탈락하는지를 센다.

성격: \*\*탐색·검증(재현)\*\* 스크립트다. 본 파이프라인이 아니므로 `config.yaml`을 경유하지 않는다  
(`src/analysis/AGENTS.md` 「이 폴더 스크립트의 성격 구분」).

재현 대상: `decision_log.md` #20 / 이슈 #15 코멘트 ④의 \*\*계산 가능 721,809건 · 계산 불가 1,754건\*\*,  
`AGENTS.md`의 \*\*Charged Off 평균 실현수익률 -45.19%(중앙값 -49.00%)\*.

⚠ `data/raw/`는 읽기 전용이다 - 이 스크립트는 원본을 읽기만 하고 결과는 `outputs/`에 쓴다.  
실행에 수 분 걸리며 `scikit-learn` 없이 `pandas/numpy`만 쓴다.

```
from pathlib import Path

import numpy as np
import pandas as pd

REPO = Path(__file__).resolve().parents[2]
RAW = REPO / "data/raw/lending_club_2020_train.csv"
TSY = REPO / "data/processed/us_treasury_GS3_GS5_monthly_2007-06_to_2020-09.csv"
OUT = REPO / "outputs"

COLS = [
 "loan_status", "funded_amnt", "term", "issue_d", "last_pymnt_d",
 "total_pymnt", "installment", "last_pymnt_amnt",
 "total_rec_prncp", "total_rec_int", "total_rec_late_fee",
 "recoveries", "collection_recovery_fee", "grade", "debt_settlement_flag",
]

def hdr(s):
 print("\n" + "=" * 78)
 print(s)
 print("=" * 78)

print(f"reading {RAW} ...")
df = pd.read_csv(RAW, usecols=COLS, low_memory=False)
print(f"rows={len(df):,} cols={len(df.columns):}")

----- 1. 상태
hdr("1. loan_status 분포 및 명세의 포함 대상 건수 검증")
raw_status = df["loan_status"].astype(str)
print("[원자료 loan_status]")
print(raw_status.value_counts(dropna=False).to_string())

PREFIX = "Does not meet the credit policy. Status:"
status = raw_status.str.replace(PREFIX, "", regex=False).str.strip()
df["status"] = status
print("\n[점두사 제거 후]")
print(status.value_counts(dropna=False).to_string())
print(f"\n점두사 보유 행: {raw_status.str.startswith(PREFIX).sum():,}")

n_fp = (status == "Fully Paid").sum()
n_co = (status == "Charged Off").sum()
print(f"\n명세 주장 : Fully Paid 899,745 / Charged Off 217,826 / 합 1,117,571")
```

```

print(f"실측 : Fully Paid {n_fp:,} / Charged Off {n_co:,} / 합 {n_fp + n_co:,}")
print(f"차이 : FP {n_fp - 899745:+,} / CO {n_co - 217826:+,} / 합 {n_fp + n_co - 1117571:+,}")

inc = df[status.isin(["Fully Paid", "Charged Off"])].copy()
print(f"\n계산 대상 표본: {len(inc):,}")

----- 2. 항등식
hdr("2. total_pymnt 항등식 (1센트 오차 이내 성립 주장)")
comp = (inc["total_rec_prncp"].fillna(0) + inc["total_rec_int"].fillna(0)
 + inc["total_rec_late_fee"].fillna(0) + inc["recoveries"].fillna(0))
resid = (inc["total_pymnt"].fillna(0) - comp).abs()
print(f"max |오차| = {resid.max():.6f}")
for thr in (0.01, 0.02, 0.10, 1.0):
 print(f" |오차| > {thr:>5}: {(resid > thr).sum():,}건")
print("\n전수(포함대상 외 포함) 검증:")
comp_all = (df["total_rec_prncp"].fillna(0) + df["total_rec_int"].fillna(0)
 + df["total_rec_late_fee"].fillna(0) + df["recoveries"].fillna(0))
resid_all = (df["total_pymnt"].fillna(0) - comp_all).abs()
print(f" max |오차| = {resid_all.max():.6f} / >0.01: {(resid_all > 0.01).sum():,}건")

----- 3. term
hdr("3. term 파싱")
print(inc["term"].value_counts(dropna=False).to_string())
T = inc["term"].astype(str).str.extract(r"(\d+)")[0].astype(float)
inc["T"] = T
print(f"\n파싱 실패(NaN): {T.isna().sum():,}")
print(f"고유값: {sorted(T.dropna().unique())}")

----- 4. 날짜/K
hdr("4. issue_d / last_pymnt_d 파싱과 K_i")
issue = pd.to_datetime(inc["issue_d"], format="%b-%Y", errors="coerce")
last = pd.to_datetime(inc["last_pymnt_d"], format="%b-%Y", errors="coerce")
print(f"issue_d 파싱 실패: {issue.isna().sum():,} (원자료 결측 {inc['issue_d'].isna().sum():,}")
print(f"issue_d 범위: {issue.min().date()} ~ {issue.max().date()}")
print(f"last_pymnt_d 파싱 실패: {last.isna().sum():,}")
print(f"last_pymnt_d 범위: {last.min().date()} ~ {last.max().date()}")

print("\n[last_pymnt_d 결측: 상태별] ※명세는 Charged Off 2,043건 주장")
print(inc.assign(miss=last.isna()).groupby("status")["miss"].agg(["sum", "size"]).to_string())

inc["issue"] = issue
inc["last"] = last
Ktil = (last.dt.year - issue.dt.year) * 12 + (last.dt.month - issue.dt.month)
inc["Ktil"] = Ktil
inc["K"] = Ktil.clip(lower=0)

print("\n[K_i (clip 전) 분포]")
print(f" 결측(날짜 결측) : {Ktil.isna().sum():,}")
print(f" K̃ < 0 (음수) : {(Ktil < 0).sum():,} ← clip으로 0이 되어 은폐됨")
print(f" K̃ = 0 : {(Ktil == 0).sum():,}")
print(f" K̃ = 1 : {(Ktil == 1).sum():,}")
print(f" K̃ ≥ 2 : {(Ktil ≥ 2).sum():,}")
if (Ktil < 0).sum():
 print("\n K̃<0 상태별:")
 print(inc[Ktil < 0].groupby("status").size().to_string())
 print(f" K̃<0 최소값: {Ktil.min():,}")

```

```

print("\n[K vs 계약만기 T] ※K>T = 계약만기 후 납입 (역할인 구간 필요)")
kt = inc.dropna(subset=["K", "T"])
print(f" K > T : {(kt['K'] > kt['T']).sum():,} ({(kt['K'] > kt['T']).mean():.2%})")
print(f" K > T (Fully Paid) : {(kt['K'] > kt['T']) & (kt['status'] == 'Fully Paid').sum():,}")
print(f" K > T (Charged Off): {(kt['K'] > kt['T']) & (kt['status'] == 'Charged Off').sum():,}")
print(f" (K-T) 최대: {(kt['K'] - kt['T']).max():.0f}개월")
print("\n K-T 분위수 (K>T인 건):")
over = (kt["K"] - kt["T"])[kt["K"] > kt["T"]]
print(" ", {q: round(float(over.quantile(q)), 1) for q in (0.5, 0.9, 0.99, 1.0)})

----- 5. 현금흐름 성분
hdr("5. C_regular / C_recovery / L 의 정합성")
C_reg = inc["total_pymnt"].fillna(0) - inc["recoveries"].fillna(0)
C_rec = inc["recoveries"].fillna(0) - inc["collection_recovery_fee"].fillna(0)
L = inc["last_pymnt_amnt"]
inc["C_reg"] = C_reg
inc["C_rec"] = C_rec

print(f"funded_amnt 결측<=0 : {inc['funded_amnt'].isna().sum():,} / {(inc['funded_amnt'] <= 0).sum():,}")
print(f"total_pymnt 결측 : {inc['total_pymnt'].isna().sum():,}")
print(f"recoveries 결측 : {inc['recoveries'].isna().sum():,}")
print(f"collection_recovery_fee 결측: {inc['collection_recovery_fee'].isna().sum():,}")
print(f"installment 결측 : {inc['installment'].isna().sum():,}")
print(f"last_pymnt_amnt 결측 : {L.isna().sum():,}")
print("\n last_pymnt_amnt 결측 상태별:")
print(inc.assign(m=L.isna()).groupby("status")["m"].sum().to_string())

print(f"\nC_regular < 0 : {(C_reg < 0).sum():,}")
print(f"C_recovery < 0 (fee>rec) : {(C_rec < 0).sum():,} ← 순회수익 음수")
print(f"total_pymnt == 0 : {(inc['total_pymnt'].fillna(0) == 0).sum():,} → R=-100%")
print(inc.assign(z=inc["total_pymnt"].fillna(0) == 0).groupby("status")["z"].sum().to_string())

K2 = inc["K"] >= 2
print(f"\n[K>=2 & L>C_regular] (명세: 기준계산에서 제외)")
bad_L = K2 & (L > C_reg)
print(f" 건수: {bad_L.sum():,} ({bad_L.sum() / K2.sum():.3%} of K>=2)")
print(inc[bad_L].groupby("status").size().to_string())
print(f"\n L < 0 : {(L < 0).sum():,}")
print(f" L == 0: {(L == 0).sum():,}")

----- 6. D_i 진단
hdr("6. D_i 진단지표 (명세: 표본 9,000건에서 0.9~1.1 비율 FP 84.1% / CO 83.3%)")
ok = K2 & (L <= C_reg) & L.notna() & inc["installment"].notna() & (inc["installment"] > 0)
D = ((C_reg - L) / (inc["K"] - 1)) / inc["installment"]
sub = inc[ok].assign(D=D[ok])
print(f"진단 대상: {len(sub):,}건")
for st, g in sub.groupby("status"):
 band = ((g["D"] >= 0.9) & (g["D"] <= 1.1)).mean()
 print(f" {st:<12} n={len(g):>9}, 0.9<=D<=1.1: {band:.1%} "
 f"median D={g['D'].median():.3f} p05={g['D'].quantile(.05):.3f} p95={g['D'].quantile(.95):.3f}")
print(f"\n D > 1.5 비율: {(sub['D'] > 1.5).mean():.2%} / D < 0.5 비율: {(sub['D'] < 0.5).mean():.2%}")

----- 7. 국제 커버리지
hdr("7. 국제 월수익률 커버리지 - F_i(a,T) 계산 가능 여부")
tsy = pd.read_csv(TSY, parse_dates=["observation_date"])
t_min, t_max = tsy["observation_date"].min(), tsy["observation_date"].max()
print(f"국제 파일 범위: {t_min.date()} ~ {t_max.date()} ({len(tsy)}행)")

```

```

print(f"결측: GS3 {tsy['GS3'].isna().sum()} / GS5 {tsy['GS5'].isna().sum()}")

need_max_u = np.maximum(inc["T"], inc["K"] + 6) # 필요한 마지막 달 offset
need_last = inc["issue"] + pd.to_timedelta(0, "D")
need_last = inc["issue"] + need_max_u.map(lambda u: pd.DateOffset(months=int(u)) if pd.notna(u) else pd.NaT)
inc["need_last"] = pd.to_datetime(need_last)
inc["need_first"] = inc["issue"] + pd.DateOffset(months=1)

beyond = inc["need_last"] > t_max
before = inc["need_first"] < t_min
print(f"\n필요 최종월 > 국채 최종월(2020-09): {beyond.sum():,}건 ({beyond.mean():.1%})")
print(f"필요 최초월 < 국채 최초월(2007-06): {before.sum():,}건 ({before.mean():.1%})")
print(f"둘 중 하나라도 벗어남 : {(beyond | before).sum():,}건 ({(beyond | before).mean():.1%})")
print("\n [term별 커버리지 부족]")
print(inc.assign(beyond=beyond).groupby("T")["beyond"].agg(["sum", "size", "mean"]).to_string())
print("\n [필요 최종월 초과분 분포]")
gap = ((inc["need_last"] - t_max).dt.days / 30.44)[beyond]
print(" ", {q: round(float(gap.quantile(q)), 1) for q in (0.5, 0.9, 0.99, 1.0)}, "개월 초과")
print("\n [issue_d 연도별 커버리지 부족 비율]")
print(inc.assign(beyond=beyond, yr=inc["issue"].dt.year).groupby("yr")["beyond"]
 .agg(["sum", "size", "mean"]).to_string())

----- 8. 탈락 캐스케이드
hdr("8. 제외 규칙 누적 적용 - 최종 계산 가능 건수")
n0 = len(inc)
steps = []
m = pd.Series(True, index=inc.index)
def step(name, cond):
 global m
 before_n = m.sum()
 m = m & cond
 steps.append((name, before_n - m.sum(), m.sum()))

step("issue_d 파싱 가능", inc["issue"].notna())
step("term 파싱 가능", inc["T"].notna())
step("last_pymnt_d 존재", inc["last"].notna())
step("K >= 0 (음수 아님)", inc["Ktil"] >= 0)
step("funded_amnt 유효(>0)", inc["funded_amnt"] > 0)
step("K>=2일 때 installment 존재", ~((inc["K"] >= 2) & inc["installment"].isna()))
step("K>=2일 때 last_pymnt_amnt 존재", ~((inc["K"] >= 2) & L.isna()))
step("K>=2일 때 L <= C_regular", ~((inc["K"] >= 2) & (L > C_reg)))
step("C_recovery >= 0", C_rec >= 0)
step("국채 커버리지 충족", ~(beyond | before))

print(f"시작(FP+C0): {n0:,.}\n")
for name, dropped, left in steps:
 print(f" -{dropped:>9,} {name:<32} 잔여 {left:>10,}")

OUT.mkdir(parents=True, exist_ok=True)
cascade = pd.DataFrame(steps, columns=["rule", "dropped", "remaining"])
cascade.insert(0, "step", range(1, len(cascade) + 1))
cascade.to_csv(OUT / "realized_return_spec_check_cascade.csv", index=False)
print(f"\n → {OUT / 'realized_return_spec_check_cascade.csv'}")
print(f"\n최종 계산 가능: {m.sum():,} ({m.sum()/n0:.1%} of 대상, {m.sum()/len(df):.1%} of 전수)")
print("\n 최종 표본 상태별:")
print(inc[m].groupby("status").size().to_string())
print("\n 국채 커버리지 조건을 뺀 경우:")

```

```

m2 = pd.Series(True, index=inc.index)
for c in [inc["issue"].notna(), inc["T"].notna(), inc["last"].notna(), inc["Ktil"] >= 0,
 inc["funded_amnt"] > 0, ~((inc["K"] >= 2) & inc["installment"].isna()),
 ~((inc["K"] >= 2) & L.isna()), ~((inc["K"] >= 2) & (L > C_rec)), C_rec >= 0]:
 m2 &= c
print(f" {m2.sum():,} ({m2.sum()/n0:.1%}")
print(inc[m2].groupby("status").size().to_string())

----- 9. F=1 근사 수익률
hdr("9. F=1 (채투자 무시) 근사 실현수익률 - 자릿수 sanity check")
calc = inc[m2].copy()
W = calc["C_reg"] + calc["C_rec"] # F=1 가정
ratio = W / calc["funded_amnt"]
R = ratio ** (12.0 / calc["T"]) - 1.0
calc["R"] = R
print(f"계산 건수: {len(calc):,}")
print(f"ratio(=W/P) 음수: {(ratio < 0).sum():,} → 분수거듭제곱 불능")
for st, g in calc.groupby("status"):
 print(f"\n [{st}] n={len(g):,}")
 print(f" mean R = {g['R'].mean():+.2%} median = {g['R'].median():+.2%}")
 print(f" p01={g['R'].quantile(.01):+.2%} p25={g['R'].quantile(.25):+.2%} "
 f" p75={g['R'].quantile(.75):+.2%} p99={g['R'].quantile(.99):+.2%}")
 print(f" R < -100%: {(g['R'] < -1).sum():,} R == -100%: {(g['R'] <= -0.999999).sum():,}")
 print(f" 단순 총수익률 (W/P - 1) mean = {(g['C_reg'] +
 g['C_rec']).div(g['funded_amnt']).sub(1).mean():+.2%}")

print("\n ※AGENTS.md 기록: Charged Off 217,366건 평균 실현수익률 -45.19% (중앙값 -49.00%)")
co = calc[calc["status"] == "Charged Off"]
simple = (co["C_reg"] + co["C_rec"]) / co["funded_amnt"] - 1
print(f" 본 계산 C0 단순수익률: mean {simple.mean():+.2%} / median {simple.median():+.2%} (n={len(co):,}")

print("\n [term별 연율화 효과]")
by_term = calc.groupby(["status", "T"])["R"].agg(["size", "mean", "median", "std"])
print(by_term.to_string())
by_term.reset_index().to_csv(OUT / "realized_return_spec_check_R_by_status_term.csv", index=False)
print(f"\n → {OUT / 'realized_return_spec_check_R_by_status_term.csv'}")

----- 10. 기타
hdr("10. 기타 점검")
print(f"debt_settlement_flag = Y : {(inc['debt_settlement_flag'] == 'Y').sum():,}")
print(inc.assign(ds=inc["debt_settlement_flag"] == "Y").groupby("status")["ds"].sum().to_string())
print(f"\nFully Paid인데 recoveries > 0 : {(inc['status'] == 'Fully Paid') & (inc['recoveries'] >
0)).sum():,}")
print(f"Charged Off인데 recoveries == 0 : {(inc['status'] == 'Charged Off') & (inc['recoveries'].fillna(0) ==
0)).sum():,}")
print(f" ({(inc['status'] == 'Charged Off') & (inc['recoveries'].fillna(0) == 0).mean() / (inc['status'] ==
'Charged Off').mean():.1%} of C0)")
print(f"\nK=0 건수 : {(inc['K'] == 0).sum():,}")
print(inc[inc["K"] == 0].groupby("status").size().to_string())
print(f"K=1 건수 : {(inc['K'] == 1).sum():,}")
print(inc[inc["K"] == 1].groupby("status").size().to_string())
print("\n" + "=" * 78)
print("done")

```

## src/analysis/realized\_return\_sensitivity.py

명세를 실제로 구현해 돌려보고, 세 가지 민감도를 쟀다.

""" 명세를 실제로 구현해 돌려보고, 세 가지 민감도를 쟀다.

- (A) 국채 재투자(F)를 넣는 것이  $F=1$  대비 R을 얼마나 바꾸는가
- (B) '마지막 납입액 보존 + 이전 균등배분' 가정 대신 '계약 할부금 우선 배분'을 쓰면 R이 얼마나 달라지는가 (= 월별 납입내역 부재라는 한계의 실제 비용)
- (C) 국채 시계열을 2025-09까지 연장하면(마지막 값 고정 시나리오) 몇 건이 복구되는가

성격: \*\*탐색·검증(재현)\*\* 스크립트다. 본 파이프라인이 아니므로 `config.yaml`을 경유하지 않는다  
(`src/analysis/AGENTS.md` 「이 폴더 스크립트의 성격 구분」).

재현 대상: `decision\_log.md` #18 / `AGENTS.md`의 **"국채 재투자 가정은 R을 평균 +107.5bp 올린다"**.  
`realized\_return()`은 #18 재투자 가정( $R = (W/P)^{(12/T)} - 1$ ,  $W = \sum CF_m \cdot F(m, T)$ )을  
**\*\*실측 현금흐름 기준으로\*\*** 구현한 것이다 - `use\_rates=False`가 재투자 0% 민감도 시나리오에 해당한다.

⚠ 여기서 계산하는 R은 **\*\*건별 실현수익률\*\***이다. #20 구조 A'의 정상상환분에 쓰는  
**\*\*계약 R\*\***(만기까지 납입 가정)은 `src/analysis/realized\_return.py`의 `contract\_return()`에  
구현돼 있다 - **\*\*둘의 차이가 B팀 1순위 미결인 조기상환 보정항\*\***이며, 이 스크립트가 그 차이를  
재는 재료다. `outputs/reports/handoff\_teamb\_realized\_return.md` 참고.

⚠ `data/raw/`는 읽기 전용이다 - 이 스크립트는 원본을 읽기만 하고 결과는 `outputs/`에 쓴다.

=====

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
REPO = Path(__file__).resolve().parents[2]
```

```
RAW = REPO / "data/raw/lending_club_2020_train.csv"
```

```
TSY = REPO / "data/processed/us_treasury_GS3_GS5_monthly_2007-06_to_2020-09.csv"
```

```
OUT = REPO / "outputs"
```

```
COLS = ["loan_status", "funded_amnt", "term", "issue_d", "last_pymnt_d", "total_pymnt",
 "installment", "last_pymnt_amnt", "recoveries", "collection_recovery_fee"]
```

```
def hdr(s):
```

```
 print("\n" + "=" * 78); print(s); print("=" * 78)
```

```
df = pd.read_csv(RAW, usecols=COLS, low_memory=False)
```

```
st = df["loan_status"].astype(str).str.replace(
 "Does not meet the credit policy. Status:", "", regex=False).str.strip()
```

```
d = df[st.isin(["Fully Paid", "Charged Off"])].copy()
d["status"] = st[st.isin(["Fully Paid", "Charged Off"])]
```

```
d["T"] = d["term"].astype(str).str.extract(r"(\d+)")[0].astype(int)
```

```
d["issue"] = pd.to_datetime(d["issue_d"], format="%b-%Y")
```

```
d["last"] = pd.to_datetime(d["last_pymnt_d"], format="%b-%Y", errors="coerce")
```

```
d = d[d["last"].notna()].copy()
```

```
d["K"] = (((d["last"].dt.year - d["issue"].dt.year) * 12
 + (d["last"].dt.month - d["issue"].dt.month)).clip(lower=0)).astype(int)
```

```
d["C_reg"] = d["total_pymnt"] - d["recoveries"]
```

```
d["C_rec"] = d["recoveries"] - d["collection_recovery_fee"]
```

```
d["L"] = d["last_pymnt_amnt"]
```

```
d = d[~((d["K"] >= 2) & (d["L"] > d["C_reg"]))].copy()
```

```
print(f"명세 제외규칙 적용 후(국채 커버리지 제외): {len(d):,}")
```

```
---- 달력월 인덱스 -----
```

```

d["mi_issue"] = d["issue"].dt.year * 12 + d["issue"].dt.month # 절대 월 인덱스

tsy = pd.read_csv(TSY, parse_dates=["observation_date"])
tsy["mi"] = tsy["observation_date"].dt.year * 12 + tsy["observation_date"].dt.month
MI_MIN, MI_MAX = int(tsy["mi"].min()), int(tsy["mi"].max())

def build_curves(series, extend_to):
 """월별 연율 국채금리 → 누적성장지수 G(m), 그리고 sum 1/G 용 누적합 H(m).
 extend_to 이후까지 마지막 값을 고정해 연장한다."""
 grid = np.arange(MI_MIN, extend_to + 1)
 r_ann = pd.Series(tsy.set_index("mi")[series]).reindex(grid)
 r_ann = r_ann.ffill() # 2020-09 이후 마지막 값 고정
 r_m = (1 + r_ann.values / 100.0) ** (1 / 12) - 1
 G = np.cumprod(1 + r_m) # G(m)
 H = np.cumsum(1.0 / G) # H(m) = sum_{j<=m} 1/G(j)
 return grid, G, H

MI_EXT = (2025 * 12 + 12) # 2025-12 까지 여유

def realized_return(d, allocation="spec", use_rates=True, restrict_cover=True):
 out = pd.Series(np.nan, index=d.index)
 for term, series in ((36, "GS3"), (60, "GS5")):
 sub = d[d["T"] == term]
 if not len(sub):
 continue
 grid, G, H = build_curves(series, MI_EXT)
 off = grid[0]
 def g(mi):
 return G[np.clip(mi - off, 0, len(G) - 1)]
 def h(mi):
 return H[np.clip(mi - off, 0, len(H) - 1)]

 mi0 = sub["mi_issue"].values
 K = sub["K"].values
 Tm = term
 C_reg, C_rec, L = sub["C_reg"].values, sub["C_rec"].values, sub["L"].values
 inst = sub["installment"].values

 if use_rates:
 G_T = g(mi0 + Tm)
 f_last = G_T / g(mi0 + K)
 f_rec = G_T / g(mi0 + K + 6)
 else:
 G_T = np.ones(len(sub)); f_last = np.ones(len(sub)); f_rec = np.ones(len(sub))

 W = np.zeros(len(sub))

 m0 = K == 0
 m1 = K == 1
 m2 = K >= 2

 if use_rates:
 W[m0] = C_reg[m0] * (G_T[m0] / g(mi0[m0]))
 W[m1] = C_reg[m1] * (G_T[m1] / g(mi0[m1] + 1))
 else:
 W[m0] = C_reg[m0]; W[m1] = C_reg[m1]

```

```

if allocation == "spec":
 # 이전 월 균등배분 A, 마지막 달 L
 A = np.where(m2, (C_reg - L) / np.maximum(K - 1, 1), 0.0)
 if use_rates:
 S = G_T * (h(mi0 + np.maximum(K - 1, 1)) - h(mi0)) # $\sum_{t=1}^{K-1} F(t, T)$
 else:
 S = np.maximum(K - 1, 0).astype(float)
 W[m2] = A[m2] * S[m2] + L[m2] * f_last[m2]
elif allocation == "installment":
 # 계약 할부금을 1..K-1에 채우고, 남은 금액은 마지막 달로 몰아준다
 cap = inst * np.maximum(K - 1, 0)
 pre = np.minimum(cap, np.maximum(C_reg - L, 0.0))
 A = np.where(m2, pre / np.maximum(K - 1, 1), 0.0)
 tail = np.where(m2, C_reg - L - pre, 0.0) # 초과 상환분은 마지막 달
 if use_rates:
 S = G_T * (h(mi0 + np.maximum(K - 1, 1)) - h(mi0))
 else:
 S = np.maximum(K - 1, 0).astype(float)
 W[m2] = A[m2] * S[m2] + (L[m2] + tail[m2]) * f_last[m2]
elif allocation == "lump_end":
 # 극단 비교: 정규 수령액 전부를 마지막 납입월에 수령
 W[m2] = C_reg[m2] * f_last[m2]

W = W + C_rec * f_rec
R = (W / sub["funded_amnt"].values) ** (12.0 / Tm) - 1.0
out.loc[sub.index] = R

return out

----- 커버리지
hdr("C. 국채 커버리지 - 얼마나 연장해야 하나")
need_last_mi = d["mi_issue"] + np.maximum(d["T"], d["K"] + 6)
need_max = int(need_last_mi.max())
print(f"필요한 최종 달력월: {need_max // 12}-{need_max % 12 or 12:02d} (국채 파일 최종: 2020-09)")
cover = need_last_mi <= MI_MAX
print(f"현재 파일로 계산 가능: {cover.sum():,} / {len(d):,} ({cover.mean():.1%}")
print(f"부족분 : {(~cover).sum():,} ({(~cover).mean():.1%}")
print("\n부족분 상태별:")
print(d.assign(nc=~cover).groupby("status")["nc"].agg(["sum", "size", "mean"]).to_string())
print("\n부족분 issue 연도별 비율 (모형 학습표본 편향 확인):")
print(d.assign(nc=~cover, yr=d["issue"].dt.year).groupby("yr")["nc"].agg(["sum", "size", "mean"]).tail(8).to_string())

----- A
hdr("A. 국채 채투자(F) 유무에 따른 R 차이 [커버리지 충족 799,930건]")
dc = d[cover].copy()
R_F1 = realized_return(dc, allocation="spec", use_rates=False)
R_Ftsy = realized_return(dc, allocation="spec", use_rates=True)
cmp = pd.DataFrame({"status": dc["status"], "T": dc["T"], "R_F1": R_F1, "R_tsy": R_Ftsy})
cmp["diff_bp"] = (cmp["R_tsy"] - cmp["R_F1"]) * 10000
print(cmp.groupby(["status", "T"])["R_F1", "R_tsy"].mean().to_string())
print("\n차이(bp) 요약:")
print(cmp.groupby(["status", "T"])["diff_bp"].describe()[["mean", "50%", "max"]].to_string())
print(f"\n전체 평균 차이: {cmp['diff_bp'].mean():.1f}bp |차이|>50bp 비율: {(cmp['diff_bp'].abs() > 50).mean():.1%}")

----- B
hdr("B. 월별 배분 가정 민감도 (국채 F 적용, 커버리지 충족분)")

```

```

R_inst = realized_return(dc, allocation="installment", use_rates=True)
R_lump = realized_return(dc, allocation="lump_end", use_rates=True)
sens = pd.DataFrame({"status": dc["status"], "T": dc["T"],
 "spec": R_ftsy, "installment": R_inst, "lump_end": R_lump})
print(sens.groupby(["status", "T"])["spec", "installment", "lump_end"].mean().to_string())
print("\nspec - installment (bp):")
sens["d_inst"] = (sens["spec"] - sens["installment"]) * 10000
sens["d_lump"] = (sens["spec"] - sens["lump_end"]) * 10000
print(sens.groupby("status")["d_inst", "d_lump"].agg(["mean", "median"]).to_string())
print(f"\n|spec-installment| > 50bp : {(sens['d_inst'].abs() > 50).mean():.2%}")
print(f"|spec-lump_end| > 50bp : {(sens['d_lump'].abs() > 50).mean():.2%}")

OUT.mkdir(parents=True, exist_ok=True)
g = sens.groupby(["status", "T"])
summary = pd.DataFrame({
 "n": g.size(),
 "R_reinvest_0pct": cmp.groupby(["status", "T"])["R_F1"].mean(), # 재투자 0% (민감도)
 "R_reinvest_tsy": g["spec"].mean(), # 국채 재투자 (#18 확정)
 "diff_bp": cmp.groupby(["status", "T"])["diff_bp"].mean(), # (A) 재투자 가정 효과
 "R_alloc_installment": g["installment"].mean(), # (B) 배분 가정 대안 1
 "R_alloc_lump_end": g["lump_end"].mean(), # (B) 배분 가정 대안 2
 "d_inst_bp": g["d_inst"].mean(),
 "d_lump_bp": g["d_lump"].mean(),
})
summary.reset_index().to_csv(OUT / "realized_return_sensitivity.csv", index=False)
print(f"\n → {OUT / 'realized_return_sensitivity.csv'}")

----- 최종 분포
hdr("최종 R 분포 (명세 그대로, 국채 F 적용)")
for s, g in cmp.groupby("status"):
 print(f"[{s}] n={len(g):,} mean={g['R_tsy'].mean():+.2%} median={g['R_tsy'].median():+.2%} "
 f"sd={g['R_tsy'].std():.2%} min={g['R_tsy'].min():+.2%} max={g['R_tsy'].max():+.2%}")
print(f"\n전체 mean={cmp['R_tsy'].mean():+.2%} sd={cmp['R_tsy'].std():.2%}")
print(f"R < -100% 건수: {(cmp['R_tsy'] < -1).sum():,} R > +50% 건수: {(cmp['R_tsy'] > 0.5).sum():,}")

마지막 값 고정 연장 시나리오
hdr("C-2. 국채를 2020-09 값으로 고정 연장하면 (전체 1,112,552건)")
R_all = realized_return(d, allocation="spec", use_rates=True)
print(f"계산 성공: {R_all.notna().sum():,}")
allc = pd.DataFrame({"status": d["status"], "R": R_all})
print(allc.groupby("status")["R"].agg(["size", "mean", "median", "std"]).to_string())
print("\n※ 2020-09 값 고정은 임시 가정이다. 실제로는 FRED에서 2025-09까지 받아 채워야 한다.")
print("=" * 78)

```

## src/analysis/hpr\_realized\_return.py

보고서 비교용 — 참고논문(HPR 복리 연율화) 방식 실현수익률.

""" 보고서 비교용 - 참고논문(HPR 복리 연율화) 방식 실현수익률.

참고논문(Park et al., JDQS 34-2)의 연율화 관례를 표본 필터 통과 723,563건  
\*\*전체\*\*에 그대로 적용한다. 본 파이프라인(현금흐름 재구성, `realized\_return\_cashflow.py`)  
과의 관례 차이를 실측 대조하기 위한 \*\*보고서 비교용 산출물\*\*이며, 모델 학습·threshold  
탐색 어디에도 들어가지 않는다.

```
HPR_i = (Total Amount Received_i - Loan Amount_i) / Loan Amount_i
r_i = (1 + HPR_i)^(12/T_i) - 1 (T_i = 36 | 60)
```

- 팀 확정: "IRR" 표현은 쓰지 않는다 (Slack 2026-07-31).
- 표본: `loader.filter\_analysis\_sample` 기준 723,563건 (decision\_log #16,  
Default 제의 재정정 #24 ① 반영). 80% 분할이 아니라 전체다.

실행:

```
/opt/anaconda3/bin/python src/analysis/hpr_realized_return.py
```

산출:

```
outputs/hpr_realized_return_full.csv (건별 723,563행 - **git 미추작**)
outputs/hpr_realized_return_summary.csv (상태*만기 요약 통계 - 커밋한다)
```

⚠ 건별 산출물은 반드시 `\*\_full.csv`로 둔다. `src/preprocessing/AGENTS.md`의 전수/표본  
명명 규칙이자, `.gitignore`가 전수 결과를 걸러내는 규칙이 `\*\_full.csv`이기 때문이다 -  
이름을 바꾸면 58MB CSV가 그대로 추적 대상이 된다.

=====

```
from __future__ import annotations
```

```
import sys
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
저장소의 다른 모듈과 **같은 방식**으로 경로를 잡는다 - `src/`를 sys.path에 넣고
`preprocessing...`로 import한다. 루트를 넣고 `src.preprocessing...`로 부르면 같은 파일이
별개 모듈 객체로 두 번 올라가, `loader.py`의 모듈 수준 상수(`PRE_APPROVAL_OVERRIDES` 등)가
두 번 생긴다. `src/__init__.py`도 없어 namespace package에 의존하게 된다.
```

```
REPO_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from preprocessing.loader import (# noqa: E402
 filter_analysis_sample,
 load_raw_loans,
 parse_term_months,
)
```

```
USECOLS = ["id", "loan_status", "term", "issue_d", "funded_amnt", "total_pymnt"]
OUT_DIR = REPO_ROOT / "outputs"
```

```
def main() -> None:
```

```
 df = load_raw_loans(usecols=USECOLS)
 df = filter_analysis_sample(df) # verify=True - 723,563건 아니면 즉시 실패
```

```
 term_months = parse_term_months(df["term"])
```

```
 # 참고논문 방식 그대로. total_pymnt = Total Amount Received.
```

```

hpr = (df["total_pymnt"] - df["funded_amnt"]) / df["funded_amnt"]
r_annual = (1.0 + hpr) ** (12.0 / term_months) - 1.0

out = pd.DataFrame(
 {
 "id": df["id"],
 "loan_status": df["loan_status"],
 "term_months": term_months.astype(int),
 "funded_amnt": df["funded_amnt"],
 "total_pymnt": df["total_pymnt"],
 "hpr": hpr,
 "r_annualized_hpr": r_annual,
 }
)

OUT_DIR.mkdir(exist_ok=True)
detail_path = OUT_DIR / "hpr_realized_return_full.csv"
out.to_csv(detail_path, index=False)

상태 × 만기 요약 - 보고서 표에 바로 인용할 수 있는 형태
summary = (
 out.groupby(["loan_status", "term_months"])["r_annualized_hpr"]
 .agg(n="count", mean="mean", median="median", std="std")
 .reset_index()
)

overall = (
 out.groupby("loan_status")["r_annualized_hpr"]
 .agg(n="count", mean="mean", median="median", std="std")
 .reset_index()
)

overall.insert(1, "term_months", "all")
summary = pd.concat([summary, overall], ignore_index=True)
summary_path = OUT_DIR / "hpr_realized_return_summary.csv"
summary.to_csv(summary_path, index=False)

n = len(out)
print(f"표본: {n:,}건 (필터 검증 통과)")
print(f"건별 산출: {detail_path.relative_to(REPO_ROOT)}")
print(f"요약 산출: {summary_path.relative_to(REPO_ROOT)}")
print()
print(summary.to_string(index=False))

if __name__ == "__main__":
 main()

```

## src/analysis/excluded\_audit.py

### 계산 제외 건 감사 테이블

```
"""**계산 제외 건 감사 테이블** - 현금흐름 분해 불가 건의 전수 사유 집계.
```

```
`final_report.md` 6.1과 `decision_log.md` #24 ㉔가 약속한 감사 테이블의 구현이다.
실현수익률 계산에서 제외된 건(train 1,210 · 2nd Test 836)을 **사유 × 상태 × 만기**로
집계해 한 파일로 남긴다 - 원본 행은 수정하지 않고, 제외가 모형·승인 판단과 무관한
기계적 제외임을 사후에 검증할 수 있게 한다.
```

```
실측 요지 (2026-07-31, B팀 교차검증)
```

```
두 표본 모두 제외 사유는 사실상 하나다 - **관측된 마지막 납입액(`last_pymnt_amnt`)
정규 수령 총액(`total_pymnt - recoveries`)
성립하지 않는 건. 필수금액 결측·국채 커버리지 사유는 양쪽 모두 0건이다.
```

```
표본	제외	L > C_regular	last_pymnt_amnt 음수
train (필터 723,563)	1,210	1,209	1
2nd Test (필터 481,833)	836	836	0
```

```
이 결과는 B팀 독립 구현(`실현수익률_계산_v3`)과의 교차검증에서 id 단위로 일치를
확인했다(train 7/30 · 2nd Test 7/31, R·XR 최대차 1e-15 수준).
```

```
실행 (원본 CSV가 `data/raw/`에 없으면 경로를 인자로 준다):
/opt/anaconda3/bin/python src/analysis/excluded_audit.py \
 [--train-csv 경로] [--test-csv 경로]
→ outputs/realized_return_excluded_audit.csv
"""
```

```
from __future__ import annotations
```

```
import argparse
import sys
from pathlib import Path
```

```
import pandas as pd
```

```
try:
 from utils.config import repo_root
except ModuleNotFoundError: # pragma: no cover
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from utils.config import repo_root
```

```
from analysis.realized_return_cashflow import build_cashflow_schedule, load_raw_for_cashflow
from preprocessing.loader import second_test_path
```

```
OUT_NAME = "realized_return_excluded_audit.csv"
```

```
def audit_one(csv_path: Path | None, label: str) -> pd.DataFrame:
 """표본 하나를 필터 → 현금흐름 스케줄에 태워 **제외 건만** 사유별로 집계한다."""
 raw = load_raw_for_cashflow(csv_path=csv_path)
 sched = build_cashflow_schedule(raw)
 ex = sched.loc[~sched["cashflow_eligible"]]
 table = (
 ex.groupby(["exclude_reason", "status_norm", "term_months"], observed=True)
 .size()
 .rename("n")
```

```

 .reset_index()
 .sort_values("n", ascending=False)
)
 table.columns = ["exclude_reason", "status", "term", "n"]
 table.insert(0, "sample", label)
 table.insert(1, "n_filtered", len(sched))
 print(f"[{label}] 필터 통과 {len(sched):,} / 제외 {int(table['n'].sum()):,}")
 print(table.to_string(index=False))
 return table

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
 parser.add_argument("--train-csv", type=Path, default=None,
 help="train 원본 CSV 경로 (기본: data/raw)")
 parser.add_argument("--test-csv", type=Path, default=None,
 help="2nd Test 원본 CSV 경로 (기본: data/raw)")
 args = parser.parse_args()

 tables = [
 audit_one(args.train_csv, "train"),
 audit_one(args.test_csv or second_test_path(), "2nd_test"),
]
 out_path = repo_root() / "outputs" / OUT_NAME
 pd.concat(tables, ignore_index=True).to_csv(out_path, index=False)
 print(f"\n저장: {out_path}")

if __name__ == "__main__":
 main()

```

**src/analysis/oracle\_benchmarks.py**

벤치마크·오라클 비교

"""\*\*벤치마크·오라클 비교\*\* - 외부 검증 표본에서 도달 가능한 Sharpe의 범위를 낸다.

`final\_report\_v3.md` 6.3절 표 6-2의 구현이다. 같은 외부 검증 표본·같은 실현 `XR`에 승인 규칙만 바꿔 다섯 가지를 계산한다. \*\*모형은 하나이고 재학습은 없다\*\* - 정렬과 컷만 바꾼다(#19).

```
기준	승인 규칙	쓰는 정보
`all_treasury`	아무 건도 승인하지 않는다	없음
`approve_all`	전부 승인 (대조군, #17 ②)	없음
`model_tau`	`q_score ≥ τ` (Validation에서 확정)	승인 시점 정보만
`oracle_status`	정상상환(Fully Paid) 건만 승인	**부도 여부** (사후)
`oracle_xr_pos`	실현 `XR > 0`인 건만 승인	**실현 초과수익 부호** (사후)
```

`oracle\_tau\_current`는 표 6-1의 `사후 최적  $\tau$ `와 같은 값이다 - 같은 `q\_score` 랭킹 안에서 컷만 사후에 고른 것이므로 \*\*완전예지 오라클이 아니다.\*\* 두 개념이 "oracle"이라는 한 단어로 섞여 있던 것을 갈라 놓기 위해 함께 낸다.

## 왜 필요한가

절대 Sharpe 0.2068이 높는지 낮은지는 그 자체로 판단할 수 없다. 전부 승인(0.1175)과 완전예지 오라클(1.4294) 사이의 어디인지가 판단 근거이며, 이 구간에서 모형이 회수한 몫(6.8%)의 상환을 결정하는 것이 판별력(AUC 0.71)이라는 정보 한계다.

⚠ `all\_treasury`는 승인 건이 없어 평균도 표준편차도 0이라 \*\*Sharpe가 0/0으로 정의되지 않는다\*\*(`NaN`으로 나온다). 0으로 바꿔 적지 않는다.

실행 (모형 재학습이 있어 약 10분):

```
/opt/anaconda3/bin/python src/analysis/oracle_benchmarks.py [--scheme 8_2]
→ outputs/oracle_benchmarks_8_2.csv
```

"""

```
from __future__ import annotations
```

```
import argparse
import sys
from pathlib import Path
```

```
import pandas as pd
```

```
try:
```

```
 from utils.config import repo_root
except ModuleNotFoundError: # pragma: no cover
 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
 from utils.config import repo_root
```

```
from analysis.final_evaluation import CRITERION_KEYS
```

```
from analysis.realized_return import ReturnAssumptions, cash_reinvestment
```

```
from analysis.second_test_evaluation import (
 load_pipeline_data,
 rebuild_winner_for_second_test,
 select_best_model,
)
```

```
from analysis.sharpe_optimizer import (
 evaluate_threshold,
 find_optimal_threshold,
```

```

portfolio_excess_returns,
scores_for_assumptions,
sharpe_ratio,
)

OUT_STEM = "oracle_benchmarks"
CRITERION = "q_score"

def _row(realized: pd.Series, approved: pd.Series) -> dict:
 """승인 마스크 하나를 평가한다. 분모는 거절 건을 포함한 전체 표본이다(#18)."""
 pxr = portfolio_excess_returns(realized, approved)
 return {
 "n_approved": int(approved.sum()),
 "approval_rate": float(approved.mean()),
 "mean_xr": float(pxr.mean()),
 "sd_xr": float(pxr.std(ddof=1)),
 "sharpe": sharpe_ratio(pxr),
 }

def benchmark_table(bundle: dict, best: pd.Series, criterion: str = CRITERION) -> pd.DataFrame:
 """재투자 가정 2종 × 승인 규칙 6종을 평가한다."""
 key = CRITERION_KEYS[criterion]
 y_second = bundle["y_second"]
 treasury = ReturnAssumptions()
 rows: list[dict] = []

 for assumptions in (treasury, cash_reinvestment(treasury)):
 scores, realized, audit = scores_for_assumptions(bundle, assumptions)
 score, lower = scores[key]
 y = y_second.reindex(realized.index)
 if y.isna().any():
 raise ValueError("부도 라벨이 실현 XR 인덱스에 정렬되지 않는다.")

 def cut(tau: float) -> pd.Series:
 return score <= tau if lower else score >= tau

 rules = {
 "all_treasury": pd.Series(False, index=realized.index),
 "approve_all": pd.Series(True, index=realized.index),
 "model_tau": cut(float(best["threshold"])),
 "oracle_status": y == 0, # 부도 여부를 미리 안다
 "oracle_xr_pos": realized > 0, # 실현 초과수의 부호를 미리 안다
 }

 for name, approved in rules.items():
 rows.append({
 "reinvest": assumptions.reinvest,
 "assumptions": assumptions.label(),
 "variant": name,
 "n_total": int(len(realized)),
 **_row(realized, approved),
 })

 orc = find_optimal_threshold(score, realized, lower)
 res = evaluate_threshold(score, realized, float(orc["threshold"]),
 lower_is_better=lower)

```

```

rows.append({
 "reinvest": assumptions.reinvest,
 "assumptions": assumptions.label(),
 "variant": "oracle_tau_current",
 "n_total": int(len(realized)),
 *_row(realized, cut(float(orc["threshold"]))),
 "threshold": res["threshold"],
})

print(f" [{assumptions.reinvest:8s}] 유효 {audit['n_usable']:}, }건 "
 f"제외 {audit['n_dropped']:}, }건 부도를 {float(y.mean()):.4f}")

out = pd.DataFrame(rows)
base = out[out["variant"] == "approve_all"].set_index("reinvest")["sharpe"]
out["sharpe_approve_all"] = out["reinvest"].map(base)
out["delta_sharpe"] = out["sharpe"] - out["sharpe_approve_all"]
return out

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
 parser.add_argument("--scheme", default="8_2", help="분할 체계 (기본 8_2)")
 parser.add_argument("--min-k", type=int, default=50, help="이 K에 못 미치면 중단")
 args = parser.parse_args()

 best = select_best_model(CRITERION, "treasury", min_k=args.min_k, scheme=args.scheme)
 print(f"[선정 모형] seed {int(best['seed'])} τ* {best['threshold']:.6f}")

 data = load_pipeline_data()
 bundle = rebuild_winner_for_second_test(int(best["seed"]), data, scheme=args.scheme)
 result = benchmark_table(bundle, best)
 result.insert(0, "scheme", args.scheme)

 out_path = repo_root() / "outputs" / f"{OUT_STEM}_{args.scheme}.csv"
 result.to_csv(out_path, index=False)

 show = result[["reinvest", "variant", "approval_rate", "mean_xr", "sd_xr",
 "sharpe", "delta_sharpe"]]
 print("\n" + show.to_string(index=False, float_format=lambda v: f"{v: .5f}"))
 print(f"\n저장: {out_path}")

if __name__ == "__main__":
 main()

```

## 시각화

보고서 그림 6종. 산출 CSV만 읽고 재계산하지 않는다.

**src/viz/plots.py**

최종 결과 리포트용 차트 생성

""" 최종 결과 리포트용 차트 생성 - outputs/figures/에 저장.

규칙은 `src/viz/AGENTS.md`. 핵심: 차트는 outputs/의 산출 CSV만 읽는다(재계산 금지),  
그림마다 출처 분할 체계를 파일명·각주에 표기한다, oracle\_tau는 그리지 않는다.

실행: `python src/viz/plots.py` - 전량 먹통 재생성.

```
그림	소스 CSV
fig_k50_delta_8_2_3fold.png	sharpe_repeat_k50_8_2_3fold.csv
fig_k50_tau_8_2_3fold.png	sharpe_repeat_k50_8_2_3fold.csv
fig_2ndtest_benchmark_8_2.png	second_test_evaluation_8_2.csv
fig_generalization_8_2_3fold.png	위 2종 결합
fig_cell_means_6_2_2_oof.png	oof_default_cell_stats_*.csv · oof_c1_cell_means_*.csv (6:2:2 진단)
fig_calibration_ece_6_2_2_oof.png	oof_c4_calibration.csv (6:2:2 진단)
"""
```

```
from pathlib import Path
```

```
import matplotlib as mpl
```

```
mpl.use("Agg")
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

```
ROOT = Path(__file__).resolve().parents[2]
```

```
OUTPUTS = ROOT / "outputs"
```

```
FIGURES = OUTPUTS / "figures"
```

```
---- 팔레트 (src/viz/AGENTS.md - 검증된 기본 팔레트, light) ----
```

```
INK = "#0b0b0b"
```

```
INK2 = "#52514e"
```

```
MUTED = "#898781"
```

```
GRID = "#e1e0d9"
```

```
AXIS = "#c3c2b7"
```

```
SURFACE = "#fcfcfb"
```

```
BLUE = "#2a78d6"
```

```
ORANGE = "#eb6834"
```

```
SEQ_LIGHT = "#86b6ef"
```

```
GOOD = "#006300"
```

```
ASSUMPTION_CAPTION = "가정: 국채 발행시점 고정 · 수수료 0% · 조기상환 현금흐름 반영 · 등가중"
```

```
mpl.rcParams.update({
```

```
 "font.family": ["Apple SD Gothic Neo", "AppleGothic", "sans-serif"],
```

```
 "axes.unicode_minus": False,
```

```
 "figure.facecolor": SURFACE,
```

```
 "axes.facecolor": SURFACE,
```

```
 "savefig.facecolor": SURFACE,
```

```
 "savefig.dpi": 300,
```

```
 "savefig.bbox": "tight",
```

```
 "axes.edgecolor": AXIS,
```

```
 "axes.linewidth": 0.8,
```

```
 "axes.grid": True,
```

```
 "grid.color": GRID,
```

```

"grid.linewidth": 0.6,
"xtick.color": MUTED,
"ytick.color": MUTED,
"text.color": INK,
"axes.labelcolor": INK2,
"axes.titlecolor": INK,
"axes.spines.top": False,
"axes.spines.right": False,
"font.size": 9,
"axes.titlesize": 10,
})

REINVEST_LABEL = {"treasury": "국채 재투자 (주 보고)", "cash": "0% 재투자 (민감도)"}
CRITERION_ORDER = ["E[XR]", "pd (보정 전)", "q_score"] # 아래-위 표시 순서

def _caption(fig, text: str) -> None:
 fig.text(0.01, -0.02, text, fontsize=7, color=MUTED, ha="left")

def _load_k50() -> pd.DataFrame:
 return pd.read_csv(OUTPUTS / "sharpe_repeat_k50_8_2_3fold.csv")

def plot_k50_delta() -> Path:
 """K=50 Δ Sharpe 분포 - 랭킹 기준 3종, 재투자 가정 2패널. q_score만 강조(파랑)."""
 df = _load_k50()
 fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.0), sharex=True)
 rng = np.random.default_rng(0)
 for ax, reinvest in zip(axes, ["treasury", "cash"]):
 sub = df[df["reinvest"] == reinvest]
 for i, crit in enumerate(CRITERION_ORDER):
 vals = sub.loc[sub["criterion"] == crit, "delta_sharpe"].to_numpy()
 color = BLUE if crit == "q_score" else MUTED
 y = i + rng.normal(0, 0.055, len(vals))
 ax.scatter(vals, y, s=14, color=color, alpha=0.65, linewidths=0, zorder=3)
 med = float(np.median(vals))
 ax.plot([med, med], [i - 0.22, i + 0.22], color=INK, lw=1.6, zorder=4)
 ax.annotate(f"{med:+.4f}", (med, i + 0.28), ha="center", fontsize=7.5,
 color=INK2, zorder=5)
 ax.set_yticks(range(len(CRITERION_ORDER)))
 ax.set_yticklabels(CRITERION_ORDER)
 ax.set_ylim(-0.55, len(CRITERION_ORDER) - 0.25)
 ax.set_title(REINVEST_LABEL[reinvest])
 ax.set_xlabel("Δ Sharpe (모형 - approve-all)")
 ax.grid(axis="y", visible=False)
 ax.tick_params(axis="y", colors=INK2)
 fig.suptitle("50회 재분할 반복에서 랭킹 기준 3종의 Validation Δ Sharpe 분포", x=0.01, ha="left")
 _caption(fig, f"세로선=중앙값 · 점=재분할 seed 1개 · 8:2+00F 3-fold · {ASSUMPTION_CAPTION}")
 fig.tight_layout(rect=(0, 0, 1, 0.99))
 out = FIGURES / "fig_k50_delta_8_2_3fold.png"
 fig.savefig(out)
 plt.close(fig)
 return out

def plot_k50_tau() -> Path:

```

```

"""q_score·국제 재투자자의 τ * 분포 ($K=50$) + 중앙값·승자 seed 26 τ *."""
df = _load_k50()
sub = df[(df["reinvest"] == "treasury") & (df["criterion"] == "q_score")]
taus = sub["threshold"].to_numpy()
median_tau = float(np.median(taus))
winner = sub.loc[sub["sharpe"].idxmax()]
fig, ax = plt.subplots(figsize=(6.4, 3.0))
ax.hist(taus, bins=14, color=SEQ_LIGHT, edgecolor=SURFACE, linewidth=1.2, zorder=3)
ax.axvline(median_tau, color=INK2, lw=1.4, ls=(0, (4, 2)), zorder=4)
ax.axvline(float(winner["threshold"]), color=BLUE, lw=2.0, zorder=4)
ymax = ax.get_ylim()[1]
ax.annotate(f"중앙값 τ = {median_tau:.4f}", (median_tau, ymax * 0.97),
 ha="right", va="top", fontsize=8, color=INK2, xytext=(-5, 0),
 textcoords="offset points")
ax.annotate(f"선정 모형 seed {int(winner['seed'])}\n τ * = {winner['threshold']:.4f}",
 (float(winner["threshold"]), ymax * 0.78), ha="left", va="top",
 fontsize=8, color=INK, xytext=(6, 0), textcoords="offset points")
ax.set_xlabel(" τ * (q_score 승인선, Validation Sharpe 최대점)")
ax.set_ylabel("seed 수")
ax.grid(axis="x", visible=False)
ax.set_title("50회 재분할 반복의 τ * 분포")
_caption(fig, f"국제 재투자 · q_score · 8:2+00F 3-fold · {ASSUMPTION_CAPTION}")
fig.tight_layout()
out = FIGURES / "fig_k50_tau_8_2_3fold.png"
fig.savefig(out)
plt.close(fig)
return out

```

```

def plot_second_test_benchmark() -> Path:

```

```

"""2nd Test: 모형(τ * 고정) vs approve-all - 절대 Sharpe 병기 + Δ 헤드리인."""
df = pd.read_csv(OUTPUTS / "second_test_evaluation_8_2.csv")
rows = df[df["variant"] == "winner_tau"].set_index("reinvest")
fig, ax = plt.subplots(figsize=(6.0, 3.2))
x = np.arange(2)
w = 0.32
order = ["treasury", "cash"]
model = [float(rows.loc[r, "sharpe"]) for r in order]
base = [float(rows.loc[r, "sharpe_approve_all"]) for r in order]
delta = [float(rows.loc[r, "delta_sharpe"]) for r in order]
b1 = ax.bar(x - w / 2 - 0.01, model, width=w, color=BLUE, zorder=3,
 label="모형 (q_score · τ * 고정)")
b2 = ax.bar(x + w / 2 + 0.01, base, width=w, color=MUTED, zorder=3,
 label="approve-all (대조군)")
for bars in (b1, b2):
 for rect in bars:
 ax.annotate(f"{rect.get_height():.4f}",
 (rect.get_x() + rect.get_width() / 2, rect.get_height()),
 xytext=(0, 3), textcoords="offset points",
 ha="center", fontsize=8, color=INK2)
for xi, d in zip(x, delta):
 ax.annotate(f" Δ {d:+.4f}", (xi, max(model[xi], base[xi])),
 xytext=(0, 16), textcoords="offset points",
 ha="center", fontsize=9.5, color=GOOD, fontweight="bold")
ax.set_xticks(x)
ax.set_xticklabels([REINVEST_LABEL[r] for r in order], color=INK2)
ax.set_ylabel("Sharpe (절대값. 주 성과지표는 Δ)")

```

```

ax.set_ylim(0, max(model) * 1.32)
ax.grid(axis="x", visible=False)
ax.legend(frameon=False, fontsize=8, loc="upper left")
ax.set_title("외부 검증 표본(481,833건): 모형 전략과 전부 승인 대조군")
_caption(fig, f"선정 모형 seed 26 · τ *=0.1895 고정 적용(재탐색 없음) · {ASSUMPTION_CAPTION}")
fig.tight_layout()
out = FIGURES / "fig_2ndtest_benchmark_8_2.png"
fig.savefig(out)
plt.close(fig)
return out

```

```

def plot_generalization() -> Path:
 """K=50 Validation Δ 분포 위에 승자 Validation Δ 와 2nd Test Δ 를 겹쳐 일반화를 보인다."""
 k50 = _load_k50()
 sub = k50[(k50["reinvest"] == "treasury") & (k50["criterion"] == "q_score")]
 vals = sub["delta_sharpe"].to_numpy()
 winner = sub.loc[sub["sharpe"].idxmax()]
 st = pd.read_csv(OUTPUTS / "second_test_evaluation_8_2.csv")
 st_delta = float(st[(st["variant"] == "winner_tau")
 & (st["reinvest"] == "treasury")]["delta_sharpe"].iloc[0])

 rng = np.random.default_rng(1)
 fig, ax = plt.subplots(figsize=(7.0, 2.6))
 ax.scatter(vals, rng.normal(0, 0.05, len(vals)), s=16, color=MUTED, alpha=0.6,
 linewidths=0, zorder=3, label="Validation Δ (50회 재분할)")
 mean = float(vals.mean())
 ax.plot([mean, mean], [-0.18, 0.18], color=INK, lw=1.6, zorder=4)
 ax.annotate(f"50회 평균 {mean:+.4f}", (mean, 0.22), ha="center", fontsize=8, color=INK2)
 ax.scatter([float(winner["delta_sharpe"])], [0], s=70, facecolors="none",
 edgecolors=BLUE, linewidths=1.8, zorder=5, label="선정 모형 seed 26 (Validation)")
 ax.annotate(f"선정 모형 (Validation) {float(winner['delta_sharpe']):+.4f}",
 (float(winner["delta_sharpe"]), -0.28), ha="center", fontsize=8, color=INK2)
 ax.scatter([st_delta], [0], s=90, marker="D", color=BLUE, zorder=6,
 edgecolors=SURFACE, linewidths=1.2, label="외부 검증 표본 (τ * 고정)")
 ax.annotate(f"외부 검증 표본 {st_delta:+.4f}", (st_delta, 0.34), ha="center",
 fontsize=9, color=INK, fontweight="bold")
 ax.set_ylim(-0.55, 0.62)
 ax.set_yticks([])
 ax.set_xlabel(" Δ Sharpe (모형 - approve-all)")
 ax.grid(axis="y", visible=False)
 ax.legend(frameon=False, fontsize=8, loc="upper left", ncols=1,
 bbox_to_anchor=(0.0, 1.02))
 ax.set_title("50회 반복의 Validation Δ Sharpe 분포와 외부 검증 표본 Δ Sharpe의 위치")
 _caption(fig, f"국채 재투자 · q_score · 8:2+00F 3-fold · {ASSUMPTION_CAPTION}")
 fig.tight_layout()
 out = FIGURES / "fig_generalization_8_2_3fold.png"
 fig.savefig(out)
 plt.close(fig)
 return out

```

```

def plot_cell_means() -> Path:
 """구조 A'(#20)의 칸 구조: 부도분은 PD분위 $\times$ term 그룹 평균, 정상분과 결합해 E[XR].

 출처: 6:2:2 00F 진단 산출물(#20·#21 근거 시점) - 8:2 산출물이 아니다(각주 표기).
 """
 label = "provisional_treasury_issue_fixed_fee0pct"

```

```

default_stats = pd.read_csv(OUTPUTS / f"oof_default_cell_stats_{label}.csv")
cell_means = pd.read_csv(OUTPUTS / f"oof_c1_cell_means_{label}.csv")
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8.2, 3.0))
term_color = {36.0: BLUE, 60.0: ORANGE}
for ax, df, col, title in (
 (ax1, default_stats, "mu", "부도 칸 평균 $\bar{r}_{\text{부도}}$ (그룹 평균)"),
 (ax2, cell_means, "E_XR", "결합 $E[XR] = (1-\hat{p}) \cdot XR_{\text{계약}} + \hat{p} \cdot \bar{r}_{\text{부도}}$ "),
):
 for term, color in term_color.items():
 sub = df[df["term"] == term].sort_values("q")
 ax.plot(sub["q"], sub[col], color=color, lw=2.0, marker="o", ms=4.5, zorder=3)
 ax.annotate(f"{int(term)}개월", (float(sub["q"].iloc[-1]), float(sub[col].iloc[-1])),
 xytext=(6, 0), textcoords="offset points", va="center",
 fontsize=8, color=INK2)
 ax.set_xticks(range(1, 11))
 ax.set_xlabel("PD 분위 (보정 전 PD, 만기별 10분위)")
 ax.set_title(title)
 ax.grid(axis="x", visible=False)
ax1.set_ylabel("수익률 (연율)")
fig.suptitle("PD 분위 × 만기 간별 부도 칸 평균 수익률과 결합 기대 초과수익 $E[XR]$ ",
 x=0.01, ha="left")
_caption(fig, "출처: 모형 진단 단계의 예비 분석 산출물 · 최종 분석과 분할 설정이 다르다 · "
 + ASSUMPTION_CAPTION)
fig.tight_layout(rect=(0, 0, 0.97, 0.99))
out = FIGURES / "fig_cell_means_6_2_2_oof.png"
fig.savefig(out)
plt.close(fig)
return out

```

**def** plot\_calibration\_ece() -> Path:

"""isotonic 확률보정(#21 ②)의 효과 - Validation ECE 보정 전/후.

*in-sample* 재보정 행(train\_oof\_calibrated\_INSAMPLE)은 진단 전용이라 그리지 않는다.

출처: 6:2:2 OOF 진단 산출물(진단 C-4).

"""

```

df = pd.read_csv(OUTPUTS / "oof_c4_calibration.csv").set_index("stage")
stages = [("validation_raw", "보정 전", SEQ_LIGHT), ("validation_calibrated", "보정 후", BLUE)]
fig, ax = plt.subplots(figsize=(4.6, 3.0))
for i, (stage, name, color) in enumerate(stages):
 v = float(df.loc[stage, "ece_pp"])
 ax.bar(i, v, width=0.34, color=color, zorder=3)
 ax.annotate(f"{v:.3f}%p", (i, v), xytext=(0, 4), textcoords="offset points",
 ha="center", fontsize=9, color=INK2)
ax.set_xticks(range(len(stages)))
ax.set_xticklabels([name for _, name, _ in stages], color=INK2)
ax.set_ylabel("ECE (%p) - Validation")
ax.grid(axis="x", visible=False)
ax.set_title("isotonic 보정 전과 후의 Validation 기대 보정 오차(ECE)")
_caption(fig, "출처: 모형 진단 단계의 예비 분석 산출물 · 최종 분석과 분할 설정이 다르다 · $E[XR]$ 의 $\hat{p}$ 만 보정 후 PD를 쓴다(역할 분

```

```
ALL_PLOTS = [
 plot_k50_delta,
 plot_k50_tau,
 plot_second_test_benchmark,
 plot_generalization,
 plot_cell_means,
 plot_calibration_ece,
]

def main() -> None:
 FIGURES.mkdir(parents=True, exist_ok=True)
 for fn in ALL_PLOTS:
 out = fn()
 print(f"저장 → {out.relative_to(ROOT)}")

if __name__ == "__main__":
 main()
```